국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

보안과제( ), 일반과제(o) 2018-0-01193

ICT유망기술개발지원사업(ICT R&D 바우처 지원)

빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 개발

(Development of quality verification automation framework to provide high reliability of bigdata solution)

2019. 7. 31.

㈜웨어밸리

과 학 기 술 정 보 통 신 부

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 출 문

과학기술정보통신부 장관 귀하

"빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 개발"
(총 수행기간 : 2018. 8. 1. - 2019. 7. 31.) 과제의 최종보고서 10부를 제출합니다.

2019. 7. 31.

주 관 기 관 명 : ㈜웨어밸리 (대표자) 손삼수 (인)

총괄 책임자 : 김인호 (인)

참 여 기 관 명 : 한국정보통신기술협회 (대표자) 박재문 (인)

참여 책임자 : 김현정 (인)

1. 동 보고서상의 기재 내용이 사실임을 확약하며, 만약, 사실이 아닐 경우 과학기술기본법 제 11조의2 및 정보통신·방송 연구개발사업 관리 규정 제 11조에 따라 참여제한·환수 등의 어떠한 불이익도 감수하겠습니다.
2. 정보통신·방송연구개발 관리규정 제36조에 따른 보고서 공개·활용에 동의합니다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

기술개발사업 최종보고서 초록

1. 일반현황

<table>
<tr><td colspan="2">사업명</td><td>ICT R&D 바우처 지원</td><td>기술분류</td><td>(대분류) 기반 SW·컴퓨팅
(중분류) 빅데이터 플랫폼
(소분류) 저장・처리・관리</td></tr>
<tr><td colspan="2">과제명(과제번호)</td><td colspan="3">빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 개발 (2018-0-01193)</td></tr>
<tr><td rowspan="4">주관기관</td><td>기관
(기업)명</td><td>(주)웨어밸리</td><td>설립일</td><td>2001년 1월 31일</td></tr>
<tr><td>주소</td><td colspan="3">(03925) 서울 마포구 월드컵북로 396 비즈니스타워 22층</td></tr>
<tr><td>대표자
(기관장)</td><td>손삼수</td><td>연락처</td><td>02-2132-5510</td></tr>
<tr><td>홈페이지</td><td>www.warevalley.com</td><td>Fax</td><td>02-743-4912</td></tr>
<tr><td rowspan="6">기술개발
현황</td><td>총괄책임자</td><td>김인호</td><td>연락처</td><td>02-2132-5538</td></tr>
<tr><td>실무담당자</td><td>김재석</td><td>연락처
(e-mail)</td><td>02-2132-5584
(jskim@warevalley.com)</td></tr>
<tr><td>참여기관</td><td colspan="3">한국정보통신기술협회 소프트웨어시험인증연구소</td></tr>
<tr><td rowspan="2">총사업비
(천원)</td><td rowspan="2">정부출연금</td><td colspan="2">민간부담금</td><td rowspan="2">합계</td></tr>
<tr><td>현금</td><td>현물</td></tr>
<tr><td>400,000</td><td>6,667</td><td>126,667</td><td>533,334</td></tr>
<tr><td>총개발
기간</td><td colspan="4">2018. 8. 1. ~ 2019. 7. 31.</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 개발결과 요약

키워드	테스트 자동화, 품질 검증 자동화 프레임워크, 테스트 슈트 레파지토리 (Test automation, Quality verification automation framework , Test suite repository)
핵심기술	낮은 안정성 및 신뢰도로 인한 시장 진출의 어려움을 해결하기 위한 빅데이터 분석용 DBMS의 안정성 및 신뢰도를 확보하기 위한 품질 검증 자동화 프레임워크 개발
최종목표	빅데이터 솔루션의 낮은 안정성 및 신뢰도로 인한 시장 진출의 어려움을 해결하기 위해 ㈜웨어밸리의 빅데이터 분석용 DBMS(PetaSQL)에 특화된 테스트 슈트, 테스트 자동화 엔진 및 시각화 도구 등을 제공하는 종합적인 품질 검증 자동화 프레임워크 연구개발
개발내용 및 결과	- 빅데이터 분석용 DBMS(PetaSQL)에 대한 안정성 및 신뢰성을 보장하기 위해 장애 유형별(데이터 무결성 오류, 동기화 오류, 서버 중단 등), SQL-92 표준 기반, 품질특성별(기능적합성, 성능효율성, 가용성, 신뢰성, 호환성 등) 테스트케이스 연구 개발 및 자동화를 위한 테스트 스크립트 개발 - 테스트케이스 의존도, 테스트 커버리지 등 테스트 분석 및 설계 지원, 효과적인 지적 자산 관리 및 테스트케이스 확장성, 변경성 지원을 위한 테스트 슈트 레파지토리 개발 - 빅데이터 분석용 DBMS의 핵심 기술인 메모리 관리, 동시성 제어, 프로세스 제어, 부하 등에 대한 테스트를 다양한 조건 및 환경, 의존성 분석 결과 등을 종합적으로 고려하여 자동 실행해주는 테스트 자동화 실행 엔진 개발 - 빅데이터 분석용 DBMS에 대한 테스트케이스 실행 결과(성공 및 실패 건수), 오류 발생 정보 등을 통계 및 보고서로 제공하는 테스트 결과 시각화 도구 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

기술개발배경	- 고신뢰도를 요구하는 DBMS의 경우, 소스코드를 일부만 수정하여도 테스트를 전면 다시 수행해야 하고, 매뉴얼 테스트가 불가능한 기술적인 영역이 다수 존재하기 때문에 테스트 자동화 프레임워크가 필수적임 - 본 과제는 4차 산업혁명의 핵심 분야인 빅데이터 솔루션에 대한 사업화 및 신시장 진출 지원을 목표로 하는 연구개발과제로 “빅데이터 분석 솔루션의 안정성 및 신뢰도를 확보하기 위한 품질 검증 자동화 프레임워크”를 개발하였음
핵심개발기술의 의의	최종 산출물인 빅데이터 분석용 DBMS의 안정성 및 신뢰도를 확보하기 위한 품질 검증 자동화 프레임워크는 해외 오픈소스 기반 DBMS 품질 검증 프레임워크에는 없는 One-Stop 실행 기술, 메모리 누수 로그 필터링 기술, 치명적 에러 처리 기술 등을 개발하여, 해외 제품과의 기술적 우위를 확보하였으며, 세계시장에 진출 가능하도록 타 DBMS 또는 연동 솔루션에 적용이 가능하도록 개발되었음
적용분야	- 품질 비용 감소 및 개발 기간 단축(반복적 매뉴얼 테스트, 결함 수정, 장애 등)을 위해 ㈜웨어밸리의 빅데이터 분석용 DBMS를 대상으로 품질 검증 자동화 프레임워크 적용 - ㈜웨어밸리의 빅데이터 분석용 DBMS에 “품질 검증 자동화 프레임워크”를 적용하여 제품의 안정성 및 신뢰도 확보를 통한 제품 경쟁력을 향상 및 제품의 사업화 및 신시장 진출에 기여

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 기술 및 경제적 성과

기술적 성과	- 빅데이터 분석용 DBMS의 안정성 및 신뢰도를 확보하기 위해 품질 검증 자동화 프레임워크를 개발함 - 특허 출원: 1건 - 해외논문: 1건 - 국내논문: 1건 - 프로그램 등록: 1건
경제적 성과	– 낮은 품질로 인해 발생했던 결함 수정 비용 및 유지보수 비용을 획기적으로 감소시킬 수 있고, 수작업 테스트를 자동화함에 따라 품질 관리 비용을 절감함으로서 기업의 연구개발 투자 증가 및 기업 경쟁력 향상 – ㈜웨어밸리는 보안 솔루션을 중국, 대만, 일본, 미국 등에 수출하는 수출 성공 기업으로 DBMS에 대한 국내 시장 레퍼런스 확보를 통해 해외 신시장 진출에 기여
사업화 가능성	– 마이그레이션 작업 시 테스트 프레임워크를 통해 기능 및 신뢰성을 사전에 미리 셀프로 검증할 수 있도록 제공하고, 어플리케이션 및 클라우드 사용자들을 위해 TaaS (PetaSQL Testing as a Service) 형태로 제공 – 품질 검증 자동화 프레임워크를 이용하여 품질 경쟁력을 확보 하고, 보안 제품 전용 DBMS에서 빅데이터 분석용 DBMS 시장에 진출하고, 더 나아가 외산 솔루션이 장악하고 있는 OLAP 시장으로 진출 확대

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4. 파급효과 및 기대효과

파급효과	– 주관기업의 빅데이터 분석용 DBMS의 맞춤형 프레임워크를 개발 및 적용함으로서 100여개의 중결함을 발견하였고, 이를 통해 제품 안정성 및 신뢰도 향상에 크게 기여 – 안정성 및 신뢰성을 확보한 빅데이터 솔루션을 개발함으로서, 국내 빅데이터 시장에서 외산 솔루션의 의존도를 낮추고 국내 빅데이터 산업 활성화에 기여 – 빅데이터 분석의 솔루션에 대한 안정화된 기술 확보를 통해 기존 비즈니스 모델 개선 및 신규 비즈니스 창출을 위한 도구로 활용
기대효과	– 국내 중소기업의 독자적인 기술로 개발된 고신뢰도 제품을 개발함으로서 외산 기술에 의존적인 시장 체제에서 국내 기술로 경쟁하는 체제로 전환 – 안정성 및 신뢰도를 확보한 DBMS를 상용화함에 따라 국내뿐 아니라 해외시장의 구매자 신뢰도를 확보할 수 있고, 공공기관 및 중소기업 등은 저렴한 비용의 솔루션 도입으로 구입 및 유지보수 비용 절감

5. 해당기술, 제품의 시장 현황

국내시장	– (빅데이터 분야 검증 자동화 기술) DBMS 기능 실행 및 결과 확인 방법이 DBMS마다 상이하여, 상용 또는 오픈소스 테스트 자동화 프레임워크가 활성화 되지 않음 – (국내 빅데이터 솔루션 시장) 국내 OLAP용 DBMS 시장은 외산제품이 장악하고 있는 상황으로 외산 제품의 높은 구입 및 유지보수 비용이 기업·정부의 재정 부담으로 작용하고 있고, 4차 산업혁명의 핵심 기술을 외산 기술에 의존하고 있는 실정임
해외시장	– (테스트 자동화 시장) Zion Market Reserch 보고서에 따르면 2015년~2021년 동안 전 세계 테스트 자동화 시장이 연평균 23.01% 성장할 것으로 예측함 – (해외 빅데이터 솔루션 시장) 빅데이터 분석을 위한 OLAP(On-Line Analytical Processing) 기술은 사용자가 대용량 데이터를 쉽고 다양한 관점에서 추출 및 분석할 수 있도록 지원하는 비즈니스 인텔리전스(Business Intelligence) 기술로, 빅데이터 시장에서 다차원의 복잡한 질의를 고속으로 처리하는 데이터 분석 기술로 주목받고 있음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

6. 제품 사진

< 테스트케이스 >

```python
# -*-coding:utf-8

from dbtest.query import *
from dbtest.utils import *
from dbtest.resource import *

"""
# Description :
# Scenario : .
"""

if __name__ == '__main__':
    comment("Initialization")

    query("DROP TABLE T1;", silent=True)

    start_stress(storage="100M")

    query("create table t1(c1 char(65535), c2 char(65535));")

    query("insert into t1 values(null,null),(null,null),(null,null),(null,null),(null,null);")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")

    comment("TEST")

    query("update t1 set c1 = LPAD('x',65535,'x'), c2 = LPAD('x',65535,'x');")

    comment("Finalization")

    stop_stress()
    query("DROP TABLE T1;", silent=True)
```

< 품질 검증 도구 >

```
 _______          _________  _______
|  _____ \        \__   __/ / ___  \ \_   _____/
|  |    | )  ______ |  |   / /_\  \ |  |  __)
|  |    | ) /_____/ |  |   |  ___  \|  |  \
|  /_______/         |____|\_____|__ /\___  /
-----------------------------------------------------
Welcome to 빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크(D-TAF) v1.0

Loading test suite, Please wait a few seconds .
Prog Name                                                                    Result     Memory
---- ------------------------------------------------------------------------ ---------- ----------
  1% + /testsuite/Root.suite
  1% ++ initDB.py                                                            PASS       -
  1% ++ /testsuite/Function/Function.suite
  1% +++ /testsuite/Function/Table/Table.suite
  1% ++++ /testsuite/Function/Table/BasicTable/BasicTable.suite
  1% +++++ /testsuite/Function/Table/BasicTable/CreateTable/CreateTable.suite
  1% ++++++ createTable_basic.py                                             FAIL       -
  1% ++++++ createTable_manyColumns.py                                       PASS       -
  1% ++++++ createTable_manyTables.py                                        PASS       -
  1% ++++++ createTable_manyType.py                                          PASS       -
  1% +++++ /testsuite/Function/Table/BasicTable/DeleteTable/DeleteTable.suite
  1% ++++++ delete_basic.py                                                  PASS       -
  1% ++++++ delete_iteration.py                                              PASS       -
  1% ++++++ delete_manyRows.py                                               PASS       -
  1% ++++++ delete_predicate.py                                              PASS       -
  1% ++++++ /testsuite/Function/Table/BasicTable/DeleteTable/Concurrency/Concurrency.suite
  1% +++++++ manyClients_oneTable_notConfilct.py                             PASS       -
  1% +++++++ manyClients_oneTable_Confilct.py                                FAIL       -
  2% +++++++ manyClients_manyTable_notConfilct.py                            PASS       -
  2% +++++++ manyClients_manyTable_Confilct.py                               PASS       -
  2% +++++ /testsuite/Function/Table/BasicTable/InsertTable/InsertTable.suite
  2% ++++++ insertTable_basic.py                                             PASS       -
  2% +++++ /testsuite/Function/Table/BasicTable/SelectTable/SelectTable.suite
  2% ++++++ selectTable_basic.py                                             PASS       -
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

< 시각화 도구 >



국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

기술개발사업 주요 연구성과

사업명	ICT유망기술개발지원사업(ICT R&D 바우처 지원)				
과제명	빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 개발				
주관기관명	㈜웨어밸리		설립일	2001년 1월 31일	
주소	(03925) 서울 마포구 월드컵북로 396 비즈니스타워 22층				
대표자(기관장)	손삼수		연락처	02-2132-5510	
총괄책임자	김인호		FAX	02-743-4912	
총개발기간	2018. 8. 1. ~ 2019. 7. 31.				
총사업비(백만원)	533	정부출연금	400	민간부담금	133
참여기관(책임자)	한국정보통신기술협회 (김현정 팀장)				

성과지표	세부지표			성 과	비 고
사업화 성과	매출액	개발제품	개발후 현재까지	0 억원	
			향후 3년간 매출(예상)	0 억원	
		관련제품	개발후 현재까지	8.4 억원	PetaSQL 기준
			향후 3년간 매출(예상)	30 억원	PetaSQL 기준
	시장 점유율	개발제품	개발후 현재까지	국내 : - % 국외 : - %	
			향후 3년간 매출(예상)	국내 : - % 국외 : - %	
		관련제품	개발후 현재까지	국내 : - % 국외 : - %	
			향후 3년간 매출(예상)	국내 : - % 국외 : - %	
	세계시장 경쟁력 순위	현재 제품 세계시장 경쟁력 순위		- 위	
		3년 후 제품 세계 시장경쟁력 순위(예상)		- 위	
기술적 성과	특허	국내	출원	1건	
			등록	건	
		국외	출원	건	
			등록	건	
	논문발표	국내		1건	
		국외		1건	
	표준화	국내		건	(기고/채택/반영)
		국외		건	(기고/채택/반영)
	프로그램			1건	
파급효과	고용효과	개발 전		명	
		개발 후		명	
	선진국 대비 기술수준			%	
	국산화율			%	
기타	표준 제개정, 기술이전 및 수상실적 등				

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 구체적인 연구 성과

1. 지식재산권

1)종류	2)해외출원	국내/국외	출원(등록)국	출원(등록)구분	출원(등록)번호	발명명칭	출원(등록)기관	출원(등록)일
01	1	국내	한국	특허출원	10-2019-0024162	빅데이터 분석용 DBMS를 위한 테스트 자동화 프레임워크 및 테스트 자동화 방법	특허청	2019.02.28
99	-	국내	한국	프로그램등록	C-2019-021009	빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 v1.0	한국저작권위원회	2019.07.30

1) 종류 : 01-특허, 02-실용신안, 03-디자인, 04-상표, 99-기타
2) 해외출원 여부 : 1-국내 출원, 2-PCT 해외 출원, 3-일반 해외 출원

2. 논문 게재/발표 실적

1)논문구분	학술지명	논문명	저자명	SCI구분	출판일	학술대회명	학술대회개최국	학술대회발표일
02	-	D-TAF: test automation framework compatible with various DBMS	Woosup Lim Hyunchong Kim Jaehyo Lee Oksu Song	X	2019.5.27	Proceedings of the 14th IEEE/ACM International Workshop on Automation of Software Test	Canada	2019.5.27
04	-	빅데이터 분석용 DBMS를 위한 테스트 자동화 프레임워크 설계	임우섭, 송옥수 김현정	X	2019.1.31	2019 한국소프트웨어공학 학술대회(KCSE)	한국	2019.1.29

1) 논문구분 : 01-국외전문학술지, 02-국외학술대회발표논문집, 03-국내전문학술지, 04-국내학술대회발표논문집, 05-국외기타논문집, 06-국내기타논문집

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 표준화 실적

기고서명	표준 기구명	표준명	회의명	기고자	1)구분	기고/채택/반영 일시	기고/채택/반영 번호

1) 구분 : 기고, 채택, 반영

4. 기술이전(기술료) 실적

계약명	계약 년도	대상 국가	기 징수액 (백만원)	당해년도 징수액 (백만원)	계약 체결일	실시내용	대상기관명

5. 인증/포상 실적 등 (국내 및 국외)

번호	구분	명칭	일시	국명	수여기관명	발생차수
						1차년도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

6. 사업화 계획 및 매출실적

항 목	세부 항목		성 과		
사업화 계획	사업화 소요기간(년)		1		
	소요예산(백만원)		200		
	예상 매출규모 (억원)		현재까지	3년후	5년후
			8.4	30	50
	시장점유율	단위(%)	현재까지	3년후	5년후
		국내	-	-	-
		국외	-	-	-
	향후 관련기술, 제품을 응용한 타 모델, 제품 개발계획		테스트 프레임웍을 높은 품질 신뢰도를 요구하는 자사 솔루션에 순차적으로 적용하여 품질 경쟁력을 확보하고 이를 바탕으로 시장 점유율 확대 예정		
무역수지 개선효과	(단위 : 억원)		현재	3년후	5년후
	수입대체(내수)				
	수 출				

7. 고용 창출

항목		세부 항목	성 과
고용효과	개발 전	연구인력	명
		생산인력	명
	개발 후	연구인력	명
		생산인력	명

8. 기타 성과

해당사항 없음

9. 변경이력 (있을 경우 기재)

해당사항 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

편집순서 5-요약문

국문 요약문

① 관 리 번 호	2018 - 0 - 01193			
② 과 제 명	빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 개발			
③ 사 업 명	ICT유망기술개발지원사업(ICT R&D 바우처 지원)			
④ 총 괄 책 임 자	성 명	김인호	직 위	기술이사
⑤ 주 관 기 관	㈜웨어밸리			

⑥ 연구내용 및 결과

Ⅰ. 빅데이터 솔루션의 고신뢰성 품질확보를 위한 품질 검증 자동화 프레임워크 개발

- ㈜웨어밸리의 빅데이터 시장 진출을 지원하기 위해 고신뢰성 빅데이터 분석 솔루션을 위한 품질 검증 자동화 프레임 워크 개발

Ⅱ. 연구개발의 목적 및 필요성

- 빅데이터 분석용 DBMS 수요가 증가하고 있지만, 국산 제품의 낮은 안정성 및 신뢰도로 **외산 솔루션이 국내 시장을 장악하고 있는 상황**
- 중소기업인 (㈜웨어밸리에서 빅데이터 솔루션 신뢰성 검증을 위한 기술력 확보가 어려운 상황으로 **빅데이터 분석 솔루션에 대한 품질 검증 자동화 프레임워크(D-TAF) 개발**

Ⅲ. 연구개발의 내용 및 범위

- 주관기업의 빅데이터 분석용 DBMS 분석 및 요구사항 도출

· 빅데이터 분석용 DBMS 기능 및 핵심기술 분석
· 빅데이터 분석용 DBMS 장애 발생 사례 조사
· 빅데이터 분석용 DBMS에 대한 주관기업의 품질 검증 자동화 요구사항 도출

- 빅데이터 분석용 DBMS 맞춤형 품질 검증 자동화 프레임워크 요소기술 연구

· 효율적인 테스트케이스 관리를 위한 레파지토리 구축 방법 연구
· 장애유형별, SQL-92 표준 기반, 품질특성별, 핵심요소기술별 검증 방법 연구
· 메모리 누수 등 치명 결함의 원인을 찾을 수 있는 결함 원인 분석 방법 연구

- 빅데이터 분석용 DBMS 맞춤형 품질 검증 자동화 프레임워크 설계 및 개발

· 테스트케이스 설계 및 실행 스크립트 설계 및 개발
· 테스트 슈트 레파지토리 설계 및 개발
· 테스트 자동화 관리 엔진 설계 및 개발
· 테스트 자동화 실행 엔진 설계 및 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

· 테스트 실행 결과 검증 엔진 설계 및 개발
· 테스트 결과 시각화 도구 설계 및 개발

- 품질 검증 자동화 프레임워크에 대한 사용자 매뉴얼 4종 개발
· 자동화 프레임워크 설치 매뉴얼
· 레파지토리 운영 및 테스트케이스 작성 가이드
· 테스트 자동화 도구 사용자 매뉴얼
· 트라블슈팅(Troubleshooting) 가이드

- 품질 검증 자동화 프레임워크 시범 적용 및 개선

- 빅데이터 솔루션에 대한 품질 검증 서비스 제공
· 품질 검증 자동화 프레임워크를 활용한 시스템 테스트
· 품질 개선을 위한 컨설팅 보고서 제공

Ⅳ. 연구개발결과

- 빅데이터 분석용 DBMS의 안정성 및 신뢰도를 확보하기 위한 품질 검증 자동화 프레임워크를 설계 및 개발함

Ⅴ. 연구개발결과의 활용계획

- 품질검증 자동화 프레임워크(D-TAF)를 통해 ㈜웨어밸리의 빅데이터 분석 솔루션의 신뢰도 및 안정성을 확보하고 **검증 업무를 자동화하여 시간 및 비용 절감**
- 빅데이터 분석 솔루션과 함께 판매하여 **해당 솔루션의 부가가치를 높이고**, 실행 환경 및 설정 변경, 패치 등의 시점마다 D-TAF를 통해 품질검증을 자동화함으로써 **빅데이터 분석 솔루션의 신뢰도 및 안정성 유지에 활용**
  * 단순 설정 변경 등으로 DBMS 중단 등의 치명적인 문제가 발생할 수 있기 때문에 글로벌 제품인 오라클의 경우, 제품의 안정성을 위해 DBMS 검증 자동화 도구를 DBMS 제품과 함께 판매하고 있음

⑦ 키워드

빅데이터 분석 솔루션, DBMS, 품질 검증 자동화 도구, D-TAF

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

목 차

제 1 장 서론 ····· 21
제 1 절 개발기술의 중요성 및 필요성 ····· 21
1. 빅데이터 솔루션 안정성 및 신뢰성 확보 어려움 ····· 21
가. 품질 검증에 대한 기술적 · 인력적 한계 ····· 21
나. 주관기관의 신시장 창출 의지와 연구개발 위탁 필요성 ····· 21
제 2 절 국내·외 관련 기술 및 시장의 현황 ····· 24
1. 국내·외 기술 현황 ····· 24
가. 품질 검증 자동화 기술 ····· 24
나. 국내·외 지식재산권 현황 ····· 25
다. 국내·외 표준화 현황 ····· 26
2. 국내·외 시장 현황 ····· 27
가. 품질 검증 자동화 시장 현황 ····· 27
나. 빅데이터 솔루션 시장 현황 ····· 28
다. 국내 빅데이터 산업 현실 ····· 29
제 3 절 기술개발 시 예상되는 기술적·경제적 파급효과 ····· 31
1. 기술적 파급 효과 ····· 31
2. 경제적 파급 효과 ····· 31

제 2 장 기술개발 내용 및 방법 ····· 32
제 1 절 최종 목표 및 평가 방법 ····· 32
1. 최종 목표 ····· 32
2. 평가 방법 ····· 33
제 2 절 연차별 개발 내용 및 개발범위 ····· 35
1. 1차년도(2018년~2019년) ····· 35

제 3 장 결과 및 향후계획 ····· 36
제 1 절 연구개발 결과 ····· 36
1. 연차 연구개발 추진 일정 ····· 36
2. 연차 연구개발 추진 실적 ····· 38
3. 기술개발 결과의 유형 및 무형 성과 전체를 기재 ····· 40
제 2 절 연구개발 추진 체계(각 기관/기업별 역할 및 추진 내역) ····· 183
제 3 절 시장현황 및 사업화 전망 ····· 185
1. 시장현황 ····· 185
2. 사업화 전망 ····· 188

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 4 절 사업비 사용현황 ····· 191
제 5 절 연구개발결과의 활용계획 ····· 193

부 록

1. 해외논문 ····· 194
2. 국내논문 ····· 201
3. 공인시험성적서 ····· 208
4. 특허 출원 번호 통지서 ····· 209
5. 프로그램 등록증 ····· 210

별첨 1. 자체평가서
별첨 2. 자체보안관리진단표

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

표 목 차

표 1. 주요 오픈소스 기반 테스트 자동화 프레임워크 ····· 24
표 2. 국내 지식재산권 현황 ····· 25
표 3. 국외 지식재산권 현황 ····· 26
표 4. 국내 표준화 현황 ····· 26
표 5. 국제 표준화 현황 ····· 26
표 6. OLAP용 외산 DBMS 구입 및 유지보수 예상 비용 ····· 29
표 7. 빅데이터 분석용 DBMS의 핵심 기능 리스트 ····· 42
표 8. DBMS 모듈별 주요 결함 ····· 44
표 9. 품질 특성별 주요 결함 ····· 47
표 10. 테스트 API 기능 리스트 ····· 55
표 11. 품질 검증 프레임워크 기능 리스트 ····· 57
표 12. 테스트슈트 목록 ····· 59
표 13. 빅데이터 분석용 DBMS 테스트케이스 ····· 99
표 14. 결과 폴더 설계 테이블 ····· 105
표 15. 테스트케이스 실행 결과 테이블 ····· 112
표 16. 백업대상 파일 ····· 112
표 17. 메모리 누수 유형 및 심볼 테이블 ····· 122
표 18. 시각화 도구 데이터 스키마 ····· 131
표 19. 품질 검증 자동화 프레임워크를 이용한 DBMS 시험항목 ····· 171
표 20. 세부사양 ····· 172
표 21. 품질 검증 자동화 프레임워크를 이용한 시스템 테스트 주요 결함 ····· 177

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

그 림 목 차

그림 1. ㈜웨어밸리의 해외 신시장 진출 상황 ····· 22
그림 2. 연구개발 위탁의 필요성 ····· 23
그림 3. 전세계 테스트 자동화 시장 매출 규모 ····· 27
그림 4. 가장 많이 사용되는 테스트 자동화 도구 ····· 28
그림 5. 데이터 솔루션 시장 규모 ····· 28
그림 6. 빅데이터 솔루션 기업의 국내 기술 수준 ····· 30
그림 7. 국내 데이터 솔루션 시장 점유율 ····· 30
그림 8. DBMS 모듈별 결함 내역 ····· 43
그림 9. 품질 특성별 결함 내역 ····· 45
그림 10. 마인드맵을 통한 ㈜웨어밸리의 요구사항 도출 ····· 48
그림 11. 테스트용 API와 실행 엔진과의 관계 ····· 58
그림 12. 테스트슈트 디렉토리 ····· 59
그림 13. 메모리 누수 테스트 결과 디렉토리 ····· 100
그림 14. 메모리 누수 로그 파일 ····· 101
그림 15. Root 및 Function 테스트 슈트 ····· 102
그림 16. 테스트 슈트 레파지토리 구조 ····· 103
그림 17. 테스트 슈트 레파지토리와 테스트 자동화 도구와의 데이터 흐름 ····· 103
그림 18. 테스트슈트 레파지토리와 테스트 자동화 도구간의 관계 ····· 104
그림 19. 결과 폴더 구조 ····· 104
그림 20. 품질검증 자동화 프레임워크 클래스 다이어그램 ····· 106
그림 21. 테스트슈트를 이용한 의존 관계 설정 ····· 106
그림 22. 의존성을 고려한 테스트케이스 실행 ····· 107
그림 23. 순차적 테스트케이스 실행 ····· 107
그림 24. 예외상황 감지 메커니즘 ····· 108
그림 25. 서버 비정상 종료 시, 중단 없이 다음 테스트로 진행하는 화면 ····· 108
그림 26. 네트워크 예외 발생기 개념 ····· 109
그림 27. 네트워크 단절 상황을 고려한 테스트케이스 ····· 110
그림 28. 테스트케이스 실행 모니터링 ····· 111
그림 29. 데이터베이스를 초기화하는 테스트케이스 ····· 113
그림 30. 유틸리티를 제어하는 테스트케이스 ····· 113
그림 31. 원격 서버 제어 개념도 ····· 114
그림 32. DB Link 초기화 테스트케이스 ····· 115
그림 33. 스토리지가 부족한 상황을 재현한 테스트케이스 ····· 116

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

그림 34. 스토리지가 부족한 상황을 재현한 테스트케이스 실행 결과 ····· 117
그림 35. 100개의 클라이언트를 생성하여 동시에 INSERT 쿼리문을 실행하는 테스트케이스 ····· 118
그림 36. TPC-H 환경을 설정하는 테스트케이스 ····· 119
그림 37. TPC-H 1번 쿼리를 실행하는 테스트케이스 ····· 119
그림 38. db.conf ····· 120
그림 39. 메모리 누수 결과 모니터링 ····· 120
그림 40. 테스트케이스별 메모리 누수 결과 ····· 121
그림 41. valgrind_filter.conf ····· 123
그림 42. 타 DBMS(PostgreSQL) 테스트케이스 ····· 124
그림 43. 타 DBMS(PostgreSQL) 테스트케이스 실행 결과 ····· 125
그림 44. 테스트 자동화 실행 결과 검증 엔진 구성도 ····· 126
그림 45. 테스트케이스(createTable_basic.py) ····· 126
그림 46. 정답 파일 (createTable_basic.exout) ····· 127
그림 47. Test Summary 화면 ····· 127
그림 48. dbtest.log ····· 128
그림 49. 시각화 도구와 테스트 폴더간의 인터페이스 ····· 129
그림 50. Web Report 안내 화면 ····· 132
그림 51. 시각화 도구(웹) 메인 페이지 화면 ····· 132
그림 52. 테스트 실행 결과 통계 화면 ····· 133
그림 53. 실패한 테스트케이스 목록 화면 ····· 133
그림 54. 테스트 결과 필터링 화면 ····· 134
그림 55. 실패한 테스트케이스에 대한 상세 정보 조회 ····· 134
그림 56. 메모리 옵션을 사용한 테스트 실행 결과 조회 화면 ····· 135
그림 57. 메모리 누수 검출 로그 화면 ····· 135
그림 58. 테스트 정보 조회 화면 ····· 135
그림 59. 비교 대상 테스트 선택 화면 ····· 136
그림 60. 테스트 결과 비교 화면 ····· 136
그림 61. 시험환경구성도 ····· 172
그림 62. 시험 유형별 결함 내역 ····· 173
그림 63. 장애 유형별 결함 내역 ····· 173
그림 64. 연구개발 추진 체계 ····· 183
그림 65. 참여기관(한국정보통신기술협회) 연구개발 추진 내역 ····· 184
그림 66. 국내 빅데이터 시장 규모 ····· 185
그림 67. 국내 빅데이터 시장의 기업 유형 ····· 186
그림 68. 국내 빅데이터 기술 수준 ····· 186
그림 69. SWOT 분석 ····· 187

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

그림 70. ㈜웨어밸리(주관기업) 보안 솔루션 ····· 188
그림 71. 국내외 매출 현황 및 예상 매출 ····· 188
그림 72. Business Intelligence용 PetaSQL ····· 189
그림 73. 국내 클라우드 시장 현황 ····· 189
그림 74. ㈜웨어밸리의 주요 총판·협력사 ····· 190
그림 75. ㈜웨어밸리의 해외 진출 계획 ····· 190

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 1 장 서론

제 1 절 개발기술의 중요성 및 필요성

1. 빅데이터 솔루션 안정성 및 신뢰성 확보 어려움

가. 품질 검증에 대한 기술적 · 인력적 한계

(1) (품질 검증에 대한 기술적, 인력적 한계) DBMS 제품은 고 신뢰도를 요구하는 시스템 소프트웨어로 ㈜웨어밸리는 국내 시장 뿐만 아니라 해외 시장 진출을 위해 체계적인 품질 검증을 시도하였으나, 여전히 다양한 품질문제가 발생하고 있고 전문인력 부족으로 기술적, 인력적 한계 발생

① 제품의 상용화 준비를 위해 자사 제품의 데이터 관리를 위한 DBMS를 MySQL에서 PetaSQL 2.0으로 시범 적용한 결과 다양한 장애와 버그 발생
- (주요 발생 장애) 기능 오류, 동기화 오류, 데이터 유실/불일치, 프로그램 비정상 종료 등

② 공공기관 등에 PetaSQL를 납품하여 필드 검증을 시도하고 있지만, 표면적인 버그만 발견되고 있어 근본적인 품질 검증은 어려움

(2) (전문지식 및 역량을 갖춘 검증기관 지원 필요) 중소기업에서 DBMS 검증 기술을 보유한 전문 인력 확보가 현실적으로 불가능하고, 관련 전문 테스팅 기업도 없는 상황

① ㈜웨어밸리는 품질 전담 조직 및 인력은 보유하고 있지만, DBMS에 대한 검증 기술을 보유한 전문 인력이 없고, 중소기업에서 해당 분야 기술력을 보유한 전문 인력을 채용하는 것은 현실적으로 불가능

② 국내 다수의 SW 테스팅 기업들이 존재하지만 대부분 단순 테스팅을 수행하는 아웃소싱 기업으로 DBMS의 전문지식 및 검증 역량을 갖춘 기업은 전무한 상황

나. 주관기관의 신시장 창출 의지와 연구개발 위탁 필요성

(1) (신시장 창출 · 탐색 경험과 노력) 오픈소스 기반의 빅데이터 분석용 DBMS가 일부 존재하지만 제품 안정성, 유지보수 등의 이유로 구매자가 도입을 꺼려하고 있고, 외산 솔루션의 경우 고가의 도입비용 및 유지보수 비용으로 인해 신뢰도와 안정성을 갖춘 국내 자체 기술로 개발된 상용 제품에 대한 수요가 발생하고 있음

① ㈜웨어밸리는 일본, 중국, 대만, 미국 등 해외시장으로 DB 관련 솔루션을 수출하고 있는 기업으로, 중국 합작 법인을 2017년에 설립하여 빅데이터 분석용 솔루션에 대해 중국 시장 진출 시도하고 있음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 1] ㈜웨어밸리의 해외 신시장 진출 상황

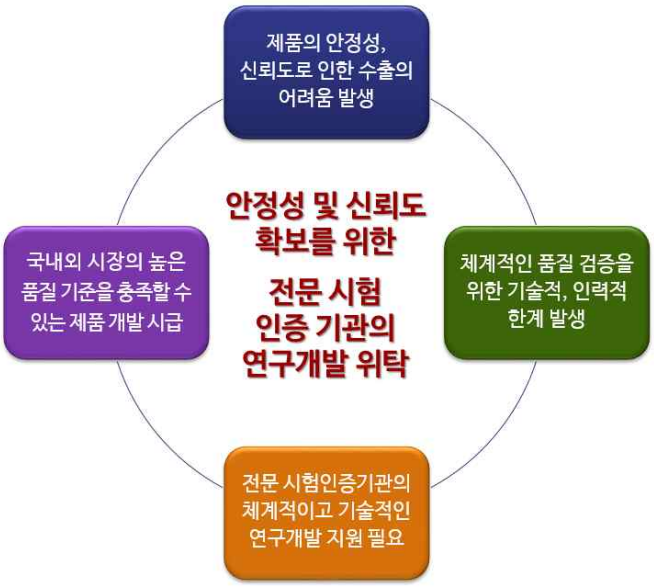
(2) (제품 아이디어의 혁신성 및 시장 진출 시급성) 주관기업인 ㈜웨어밸리는 칼럼 방식의 데이터 저장 (Column-Store) 기술을 적용하여 국내 기업 최초로 OLAP용 DBMS(PetaSQL) 개발하고 시장 진출을 시도하고 있음

① 국내 시장의 다양한 요구사항을 분석하여 빅데이터 분석에 필요한 핵심 기술(Column-Stored, Massive Parallel Processing, Real-Time Data Replication, 데이터 마스킹 및 컬럼 암호화 등)을 연구개발

- 빅데이터 솔루션에 대한 핵심 기술인 이중화, 파티션 테이블, 데이터베이스 링크, 백업 및 복구, Auto Sharding 등을 지속적으로 연구 개발 중

② ㈜웨어밸리는 국내뿐만 아니라 해외 시장에서 DB 보안솔루션 판매에 결실을 보고 있는 수출 성공 기업으로, 외산 의존도가 높아지고 있는 빅데이터 시장에 대한 국내 시장 진출을 시도하는 상황

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 2] 연구개발 위탁의 필요성

(3) (고품질 제품 개발을 위한 위탁연구개발 필요성) ㈜웨어밸리는 국내 시장 뿐만 아니라 해외 시장 진출을 시도하고 있으나, 체계적인 품질 검증에는 기술적, 인력적인 한계가 발생하고 있고, 글로벌 기업 수준의 품질 투자가 현실적으로 어려운 상황으로 전문 시험인증기관의 체계적이고 기술적인 연구개발 절실한 상황

① 공공기관 등에 PetaSQL를 납품하여 필드 검증을 시도하고 있지만, 표면적인 버그 수정으로만 국한되어 근본적인 품질 검증은 어려움

- 시스템 분야 글로벌 기업에서는 高수준의 안정성 및 신뢰도를 확보하기 위해 일반 응용 SW 대비 약 2~10배 이상의 품질 검증 비용을 투자하고 있지만, 국내 중소기업에게는 글로벌 수준의 투자가 어려운 상황
- 주관기업인 ㈜웨어밸리는 중국, 일본, 대만 등의 아시아 시장을 진출을 시도하고 있는 중소기업으로 해외 구매자의 신뢰도 확보를 위해 체계적인 품질 검증이 시급한 상황

☞ ㈜웨어밸리는 국내 시장 뿐만 아니라 해외 시장 진출을 시도하고 있는 기업으로, 고신뢰도 및 안정성을 요구하는 빅데이터 분석용 DBMS에 대한 **검증 전문지식과 다양한 검증 경험을 보유한 TTA 소프트웨어시험인증연구소의 연구개발이 절실**

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 2 절 국내·외 관련 기술 및 시장의 현황

1. 국내·외 기술 현황

가. 품질 검증 자동화 기술

(1) (응용 어플리케이션 검증 자동화 기술) 공통 플랫폼 환경에서 수행되는 웹 어플리케이션, 모바일 등을 대상으로 개발되고 있음

순번	제품명	라이선스	제품개요
1	Canoo WebTest	Apache License v2	웹 애플리케이션 테스트 자동화 도구
2	Jameleon	LGPL v2	데이터기반 자동화 테스팅 프레임워크
3	Maveryx	GPL v2	오픈소스 테스트 자동화 프레임워크. 자바 및 안드로이드의 기능테스트, 회귀 테스트 (Regression test), GUI 테스트 기능 지원
4	NTAF	GPL v2	NHN 테스트 자동화 프레임워크이며 FitNesse의 장점인 협업기능과 STAF의 장점인 재사용과 확장성을 통합
5	Robot Framework	Apache 2.0	파이썬으로 제작된 ATDD(Acceptance Test-Driven Development) 용 테스트 자동화 프레임워크
6	Sahi	Apache 2.0	오픈소스 웹 자동화 테스팅 툴
7	Twister	Apache 2.0	Luxoft의 테스트 자동화 프레임워크
8	Watir	BSD	테스트 자동화 도구
9	Selendroid	Apache License	안드로이드 및 iOS 모바일 환경을 대상으로 어플리케이션 (native and hybrid application)의 UI와 모바일 웹을 테스트 할 수 있도록 시스템 구성 요소를 제공하는 테스트 자동화 프레임워크

[표 1] 주요 오픈소스 기반 테스트 자동화 프레임워크 (출처: NIPA)

(2) (빅데이터 분야 검증 자동화 기술) DBMS 기능 실행 및 결과 확인 방법이 DBMS마다 상이하여, 상용 또는 오픈소스 테스트 자동화 프레임워크가 활성화 되지 않음

① **(기능 검증 도구)** 테스트 자동화 프레임워크에 대한 오픈소스가 존재하지만 빅데이터 솔루션 특성상 각 솔루션 마다 기능 및 실행 방법 등이 상이하여 해당 솔루션에 종속적으로 활용

- 대표적인 오픈소스 DBMS인 MySQL과 PostgreSQL의 경우 각 DBMS에 특화된 테스트 자동화 프레임워크가 오픈소스로 제공
- 오라클, 마이크로소프트, IBM 등 주요 빅데이터 분석용 DBMS 기업의 경우 테스트 자동화 프레임워크를 자체 개발 운영

② **(성능 시험 도구)** DBMS 성능 측정 및 분석을 위한 Apache Jmeter, BenchmarkSQL 등의 오픈소스 존재

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

나. 국내·외 지식재산권 현황

(1) 관련 기술/제품의 국내 지식재산권(특허 등) 현황

연번	기관/기업	특허명	특허번호	특허내용
1	한국전자통신연구원	소프트웨어 결함 유발 데이터를 삽입한 파일을 이용하여 소프트웨어의 결함을 검출하는 자동화 시스템 및 그 방법	10-0897412	파일에 결함 유발 데이터 삽입, 결함 유발 파일의 실행 그리고 파일 실행 중 발생하는 이상 현상 감시 및 분석의 전과정을 자동화함으로써 소프트웨어 테스팅의 효율성을 향상
2	(재)한국데이터진흥원	데이터 품질 진단 방법과 시스템	10-1125417	정형 데이터 및 비정형 데이터에 대한 품질 진단 기준을 제시하는 데이터 품질 진단 시스템 서비스 제공
3	㈜국민은행	품질테스트자동화시스템	10-0969877	테스트 데이터를 일괄 입력, 수행 및 결과를 자동으로 보고하고 테스트 데이터를 재사용
4	마이크로소프트 코포레이션	공통 데이터 플랫폼에 대한 스토리지 ＡＰＩ	10-1201061	서로 다른 프레임워크와 유기적으로 연동되어 액세스할 수 있는 데이터 스토어 API 제공

[표 2] 국내 지식재산권 현황

(2) 관련 기술/제품의 국외 지식재산권(특허 등) 현황

연번	기관/기업	특허명	특허번호	특허내용
1	CARDINAL HEALTH 414, LLC	METHOD AND SYSTEM FOR AUTOMATED QUALITY CONTROL PLATFORM	US 20130018618 A1	자동화된 품질 관리 플랫폼을 위한 방법과 시스템
2	International Business Machines Corporation	Software testing automation framework	US 8914679 B2	소프트웨어 시험 자동화 프레임워크
3	Microsoft Corporation	Data warehouse test automation framework	US 8019795 B2	데이터 웨어하우스 테스트 자동화 프레임워크
4	International Business Machines Corporation	Automating modular manual tests including framework for test automation	US 8239826 B2	테스트 자동화를 위한 프레임워크를 포함하는 모듈화된 매뉴얼 시험을 자동화하는 것
5	Accenture Global Services GmbH	Software testing capability assessment framework	EP 1939749 A3	소프트웨어 시험 성능 평가 프레임워크
6	AVERY DENNISON RETAIL INFORMATION SERVICES, LLC	System and method for automated RFID quality control	US 9569714 B2	자동화된 RFID 품질 관리를 위한 시스템과 방법

[표 3] 국외 지식재산권 현황

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

다. 국내·외 표준화 현황

(1) 본 기술/제품과 직접적으로 관련 있는 국내 표준화 현황

표준(과제) 번호	표준 제목	제/개정일
TTAK.KO-10.0378	데이터 품질관리 프레임워크	표준 제정 (2009-12-22)
TTAK.KO-10.0801	빅데이터 기반 데이터 웨어하우스 참조모델	표준 제정 (2015-06-23)
TTAK.KO-11.0218	소프트웨어 제품 품질 평가 항목	표준 제정 (2016-12-27)

[표 4] 국내 표준화 현황

(2) 본 기술/제품과 직접적으로 관련 있는 국외 표준화 현황

분과	WG6 SQuaRE 표준 문서		현 황
품질 관리	ISO/IEC 25000	Guide to SQuaRE	IS(2014)
	ISO/IEC 25001	Planning and management	IS(2014)
품질 모델	ISO/IEC 25010	System and software quality models	IS(2011)
	ISO/IEC 25012	Data quality model	IS(2008)
품질 측정	ISO/IEC 25020	Measurement reference model	IS(2007)
	ISO/IEC 25021	Quality measure elements	IS(2012)
	ISO/IEC 25022	Measurement of Quality in Use	IS(2016)
	ISO/IEC 25023	Measurement of System & SW Product Quality	IS(2016)
	ISO/IEC 25024	Measurement of Data Quality	IS(2015)
품질 요구사항	ISO/IEC 25030	Quality requirements	IS(2007)
품질 평가	ISO/IEC 25040	Quality Evaluation Process	IS(2011)
	ISO/IEC 25041	Evaluation guide for developers, acquirers and independent evaluators	IS(2012)
	ISO/IEC 25045	Evaluation module for recoverability	IS(2010)
SQuaRE 확장	ISO/IEC 25051	Requirements for quality of Ready to Use Software Product(RUSP) and instructions for testing	IS(2014)

[표 5] 국제 표준화 현황

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 국내·외 시장 현황

가. 품질 검증 자동화 시장 현황

(1) Zion Market Research 보고서에 따르면 2015년~2021년 동안 전세계 테스트 자동화 시장이 연평균 23.01% 성장할 것으로 예측함

① 전 산업분야가 유기적으로 융합하는 4차 산업혁명 시대에서 SW품질의 중요성이 높아지는 상황으로 SW 품질을 보증하기 위해서는 테스트 자동화가 필수적인 상황으로 인식



[그림 3] 전세계 테스트 자동화 시장 매출 규모

② 인도의 나스콤(NASCOM)에 따르면 세계 SW개발 시장에서 테스팅이 차지하는 비중은 약 50%로 분석

③ 웹(Web) 또는 모바일 앱(Mobile App)에 대한 수요가 증가 하면서, 웹 및 모바일 테스트 자동화 도구가 전체 테스트 자동화 시장의 성장을 견인하고 있음

- (Selenium) 다양한 브라우저의 웹 어플리케이션 테스트를 자동화 해주는 대표적인 웹 브라우저 기반 테스팅 프레임워크
- (Katalon Studio) 웹 및 모바일 SW를 대상으로 테스트케이스를 생성, 실행 관리하는 테스팅 프레임워크
- (Unified Functional Testing) 웹, 모바일 및 API(SOAP, REST) 테스트를 자동화 해주는 테스팅 프레임워크
- (TestComplete) 웹, 모바일 및 응용 SW 테스트를 자동화 해주는 테스팅 프레임워크
- (watir) 루비(ruby) 기반의 웹 브라우저 기반 테스팅 프레임워크

④ 국내의 경우 SW 테스팅 시장은 정확히 집계된 적이 없을 정도로 소외된 영역이며, 시장 규모가 약 3,000억 ~ 5,000억 규모로 추산하고 있음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



Product	Selenium	Katalon Studio	Unified Functional Testing	TestComplete	watir
Available since	2004	2015	1998	1999	2006
Application Under Test	Web apps	Web (UI & API), Mobile apps	Web (UI &API), Mobile, Desktop, Packaged apps	Web (UI &API), Mobile, Desktop apps	Web apps
Pricing	Free	Free	$$$$	$$	Free
Supported Platforms	Windows Linux OS X	Windows Linux OS X	Windows	Windows	Windows Linux OS X
Scripting languages	Java, C#, Perl, Python, JavaScript, Ruby, PHP	Java/Groovy	VBScript	JavaScript, Python, VBScript, JScript, Delphi, C++ and C#	Ruby
Programming skills	Advanced skills needed to integrate various tools	Not required. Recommended for advanced test scripts	Not required. Recommended for advanced test scripts	Not required. Recommended for advanced test scripts	Advanced skills needed to integrate various tools
Ease of installation and Use	Require advanced skills to install and use	Easy to setup and use	Complex in installation. Need training to properly use the tool	Easy to setup. Need training to properly use the tool	Advanced skills needed to integrate various tools

[그림 4] 가장 많이 사용되는 테스트 자동화 도구

※출처: https://medium.com

나. 빅데이터 솔루션 시장 현황

(1) **(빅데이터 활용 증가)** 기업의 빅데이터 활용은 고객의 행동을 사전에 예측하고 대처방안을 마련해 기업경쟁력을 강화시키고 생산성 향상과 비즈니스 혁신을 가능하게 하고 있고, 정부 및 공공기관의 경우 빅데이터를 분석 및 활용하여 품질 높은 공공 서비스 제공함으로써 사회적 비용 감소와 공공 서비스 품질 향상에 기여하고 있음

(2) **(빅데이터 솔루션 시장 규모)** SNS 데이터, 공공데이터 등에 대한 빅데이터 활용이 중요해짐에 따라 빅데이터를 분석 및 가시화하고, 처리 및 저장을 위한 빅데이터 솔루션 규모가 매년 커지고 있는 상황



[그림 5] 데이터 솔루션 시장 규모 (출처: 한국데이터진흥원)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(3) (OLAP용 DBMS 필요성 증가) 알티베이스, 티맥스데이터, 선재소프트 등의 국내 기업에서 OLTP(On-Line Transaction Processing)용 DBMS를 개발하여 국내 DBMS 시장을 일부 점유하고 있는 상황이지만, 대용량 데이터 분석에는 적합하지 않아 OLTP DBMS를 보유한 기업에서도 OLAP(On-Line Analytical Processing)용 DBMS 도입이 필요한 상황

① OLAP용 DBMS는 대용량 데이터의 추출 및 분석을 위한 제품으로, 다양한 비즈니스 관점에서 쉽고 빠르게 다차원적인 데이터를 분석할 수 있는 기술이 적용된 DBMS임

② 반면 OLTP용 DBMS는 온라인 트랜잭션 처리 목적으로 개발된 제품으로, 네트워크상의 여러 이용자가 실시간으로 데이터베이스의 데이터를 갱신하거나 조회하는 등의 실시간성의 단위 작업 처리에 강점을 가지지만, 방대한 데이터 분석에는 적합하지 않아 OLAP용 DBMS 필요성 증가

다. 국내 빅데이터 산업 현실

(1) (외산에 의존한 국내 빅데이터 솔루션 시장) 방대한 데이터를 분석해 새로운 가치를 발견할 수 있도록 도와주는 빅데이터 분석용 DBMS 수요가 증가하고 있지만, **외산 솔루션이 국내 시장을 장악하고 있는 상황**

① 빅데이터 분석을 위한 OLAP(On-Line Analytical Processing) 기술은 사용자가 대용량 데이터를 쉽고 다양한 관점에서 추출 및 분석할 수 있도록 지원하는 비즈니스 인텔리전스(Business Intelligence) 기술로, 빅데이터 시장에서 다차원의 복잡한 질의를 고속으로 처리하는 데이터 분석 기술로 주목받고 있음

② 국내 OLAP용 DBMS 시장은 외산제품이 장악하고 있는 상황으로 외산 제품의 높은 구입 및 유지보수 비용이 기업·정부의 재정 부담으로 작용하고 있고, 4차 산업혁명의 핵심 기술을 외산 기술에 의존하고 있는 실정

구분	도입 비용	유지보수 비용
Oracle Exadata	12억원 / 1Machine	2억6천만원/년
Sybase IQ	5억원 / 1CPU	1억2천만원/년

[표 6] OLAP용 외산 DBMS 구입 및 유지보수 예상 비용

(2) (낮은 안정성 및 신뢰성이 기술 격차의 원인) 빅데이터 솔루션 기업의 국내 기술 수준은 2016년 기준 선진국 대비 약 65.7점, 기술수준 격차는 약 3.1년, 선진기술 도달시간은 약 3.4년

① 국내 제품의 낮은 안정성 및 신뢰성 등이 외산제품 대비 경쟁력 저하의 주요 원인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

구분	국내 기술수준 (선진기술 100 기준)	수준 격차(년)	선진기술 도달 시간(년)
2015년	62.6	3.3	3.6
2016년	65.7	3.1	3.4
증감	+3.1	-0.2	-0.2

※ '15년 100개, '16년 114개 응답 기준

[그림 6] 빅데이터 솔루션 기업의 국내 기술 수준 (출처: 한국데이터진흥원)

② 국내 제품의 낮은 안정성과 신뢰성 등으로 인해 국내 데이터 솔루션 시장의 약 69.2%이상을 외산 솔루션이 차지하고, DBMS의 경우 약 90%에 육박



[그림 7] 국내 데이터 솔루션 시장 점유율 (출처: 한국데이터진흥원)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 3 절 기술개발 시 예상되는 기술적·경제적 파급효과

1. 기술적 파급 효과

가. 국내 최고의 품질 검증 기관인 TTA 소프트웨어시험인증연구소의 억대 가치의 품질검증 자동화 프레임워크 및 지적 자산을 전수 받음으로서 **빅데이터 분석용 DBMS의 안정성 및 신뢰도 향상에 크게 기여할 것으로 예상**

(1) DBMS 전문 지식을 보유하고, 다양한 검증 경험을 보유한 TTA의 지적 자산 및 노하우를 ㈜웨어밸리의 빅데이터 분석용 DBMS에 반영

(2) ㈜웨어밸리의 DBMS 검증인력 부족으로 인한 기술적 한계를 극복

(3) TTA 소프트웨어시험인증연구소가 개발한 품질 검증 자동화 프레임워크를 도입함으로서 수작업으로 진행해 왔던 DBMS 검증을 자동화함에 따라 품질 관리 비용뿐만 아니라 장애 발생에 대한 대응 비용 대폭 절감

나. DBMS의 고유 특성으로 범용적인 검증 도구 사용이 불가능했지만 ㈜웨어밸리의 빅데이터 분석용 DBMS의 맞춤형 프레임워크를 개발 및 적용함으로서 제품 안정성 및 신뢰도 향상에 크게 기여

다. 확장성 및 변경성을 지원하는 품질 검증 자동화 프레임워크를 통해 **중소 협력사 및 고객사 요구사항 추가 및 변경**에 대한 **체계적인 품질 검증 대응 가능**

(1) DBMS의 특성상 요구사항 추가, 변경 과정에서 심각한 장애가 발생하는 경우가 다수 발생하였기 때문에 기능 추가 및 변경 후 전체 기능에 대한 품질검증을 자동화할 수 있는 본 연구개발 결과물을 통해 체계적이고 완벽한 대응 가능

2 경제적 파급 효과

가. 낮은 품질로 인해 발생했던 결함 수정 비용 및 유지보수 비용을 획기적으로 감소 시킬 수 있고, 수작업 테스트를 자동화함에 따라 **품질 관리 비용을 절감**함으로서 **기업의 연구개발 투자 증가 및 기업 경쟁력 향상**

(1) 국내 중소기업의 독자적인 기술로 개발된 고신뢰도 제품을 개발함으로서 외산 기술에 의존적인 시장 체제에서 국내 기술로 경쟁하는 체제로 전환

(2) 안정성 및 신뢰도를 확보한 DBMS를 상용화함에 따라 국내뿐 아니라 해외시장의 구매자 신뢰도를 확보할 수 있고, 공공기관 및 중소기업 등은 저렴한 비용의 솔루션 도입으로 구입 및 유지보수 비용 절감

나. ㈜웨어밸리는 보안 솔루션을 중국, 대만, 일본, 미국 등에 수출하는 수출 성공 기업으로 DBMS에 대한 **국내 시장 레퍼런스 확보를 통해 해외 신시장 진출에 기여**

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 2 장 기술개발 내용 및 방법

제 1 절 최종 목표 및 평가 방법

1. 최종 목표

4차 산업혁명의 핵심 분야인 빅데이터 솔루션에 대한 사업화 및 신시장 진출 지원을 목표로 “빅데이터 분석용 DBMS의 안정성 및 신뢰도를 확보하기 위한 품질 검증 자동화 프레임워크 개발”

구분	내용
최종목표	○ 4차 산업혁명의 핵심 분야인 빅데이터 솔루션에 대한 사업화 및 신시장 진출 지원을 목표로 “빅데이터 분석용 DBMS의 안정성 및 신뢰도를 확보하기 위한 품질 검증 자동화 프레임워크 개발” ○ End Product – 빅데이터 분석용 DBMS 맞춤형 품질 검증 자동화 프레임워크 (SW) – 빅데이터 분석용 DBMS 맞춤형 테스트케이스 설계 및 실행 스크립트 – 빅데이터 분석용 DBMS 맞춤형 테스트 슈트 레파지토리(SW) – 빅데이터 분석용 DBMS 맞춤형 테스트 자동화 관리 엔진(SW) – 빅데이터 분석용 DBMS 맞춤형 테스트 자동화 실행 엔진(SW) – 빅데이터 분석용 DBMS 맞춤형 테스트 자동화 실행 결과 검증 엔진(SW) – 빅데이터 분석용 DBMS 맞춤형 테스트 결과 시각화 도구(SW) – 자동화 프레임워크 설치 매뉴얼 1종 – 레파지토리 운영 및 테스트케이스 작성 가이드 1종 – 테스트 자동화 도구 사용자 매뉴얼 1종 – 트라블슈팅(Troubleshooting) 가이드 1종 – 품질 검증 자동화 프레임워크를 활용한 품질 컨설팅 1건 – 테스트 결과 보고서 1건

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 평가 방법

<table>
<tr><th>정량적 평가 항목</th><th>평가방법</th></tr>
<tr><td>1. 테스트케이스 및 실행 스크립트 개발</td><td>공인시험기관을 통해 테스트케이스 및 실행 스크립트 개발 여부 확인
- 빅데이터 분석용 DBMS의 각 요소기술을 테스트하기 위한 테스트케이스 및 실행 스크립트 1,000건 이상 개발</td></tr>
<tr><td>2. 품질검증 자동화 프레임워크 적합성</td><td>공인시험기관을 통해 유사한 기능을 가지는 타 DBMS 품질검증 자동화 프레임워크와 본 연구개발물의 아래 기능을 비교하여 기능 적합성 및 우수성 확인
<table>
<tr><th>순번</th><th>기능명</th><th>순번</th><th>기능명</th></tr>
<tr><td>1</td><td>SQL 실행 기능</td><td>14</td><td>쿼리문 수행 시간 검증 기능</td></tr>
<tr><td>2</td><td>동시 세션 생성 기능</td><td>15</td><td>이중화 제어 기능</td></tr>
<tr><td>3</td><td>동시성 제어 기능</td><td>16</td><td>DB Link 제어 기능</td></tr>
<tr><td>4</td><td>테이블 간 정합성 비교 기능</td><td>17</td><td>백업 및 복구 기능</td></tr>
<tr><td>5</td><td>환경 변수 제어 기능</td><td>18</td><td>트랜잭션 설정 기능</td></tr>
<tr><td>6</td><td>파일 제어 기능</td><td>19</td><td>유틸리티 제어 기능</td></tr>
<tr><td>7</td><td>프로세스 제어 기능</td><td>20</td><td>인터페이스(JDBC, ODBC) 제어 기능</td></tr>
<tr><td>8</td><td>시스템 리소스 제어 기능</td><td>21</td><td>벤치마크 테스트 실행 기능</td></tr>
<tr><td>9</td><td>자원 모니터링 기능</td><td>22</td><td>입력 데이터 생성 기능</td></tr>
<tr><td>10</td><td>네트워크 제어 기능</td><td>23</td><td>DBMS 예외 상황 관리 기능</td></tr>
<tr><td>11</td><td>DBMS 설치 기능</td><td>24</td><td>출력 제어 기능</td></tr>
<tr><td>12</td><td>데이터베이스 관리 기능</td><td>25</td><td>로그 제어 기능</td></tr>
<tr><td>13</td><td>시스템 명령어 실행 기능</td><td></td><td></td></tr>
</table></td></tr>
<tr><td>3. 품질검증 자동화 프레임워크 구성 엔진 및 모듈 수</td><td>공인시험기관을 통해 자동화 프레임워크 구성 엔진 및 엔진별 모듈 수 확인
<table>
<tr><th>엔진명</th><th>모듈명</th></tr>
<tr><td>테스트 슈트 레파지토리 관리</td><td>테스트 자동화 스크립트</td></tr>
<tr><td>테스트 자동화 관리</td><td>테스트 슈트 분석, 테스트 실행 관리,
예외 상황 처리</td></tr>
<tr><td>테스트 자동화 실행</td><td>테스트 케이스 실행, 입력 데이터 생성,
프로세스 제어, 메모리 누수 검출 도구 제어</td></tr>
<tr><td>테스트 실행 결과 검증</td><td>테스트 케이스 결과 비교, 오류 정보 저장</td></tr>
<tr><td>테스트 결과 시각화 도구</td><td>시각화 모듈</td></tr>
</table></td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4. 품질검증 자동화 프레임 워크 실행 결과 정확성	공인시험기관을 통해 아래 사항에 대한 정확성 확인 - 자동화 검증 프레임워크의 엔진(5종) 정상 동작유무 · 테스트 슈트 레파지토리 관리 · 테스트 자동화 관리 엔진 · 테스트 자동화 실행 엔진 · 테스트 실행 결과 검증 엔진 · 테스트 결과 시각화 도구 - 테스트 실행 결과(성공/실패) 정확도 유무
5. 품질검증 자동화 프레임워크 사용자 매뉴얼	4종 사용자 매뉴얼에 대해 ㈜웨어밸리가 학습성 및 이해도 측면으로 평가 수행 - 자동화 프레임워크 설치 매뉴얼 - 레파지토리 운영 및 테스트케이스 작성 가이드 - 테스트 자동화 도구 사용자 매뉴얼 - 트라블슈팅(Troubleshooting) 가이드
6. 품질검증 자동화 프레임워크를 활용한 품질 컨설팅 보고서	㈜웨어밸리의 컨설팅만족도 평가

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 2 절 연차별 개발 내용 및 개발범위

1. 1차년도(2018년~2019년)

최종결과물 (제품명)	세부내용			기술개발 내용	개발 기관
	기술명	구분	비중		
빅데이터 분석용 DBMS 맞춤형 품질 검증 자동화 프레임워크	테스트 케이스 설계 기술	기술개발	20%	빅데이터 분석용 DBMS 맞춤형 테스트케이스 설계 기술	TTA 소프트웨어 시험인증 연구소
	레파지토리 구축 기술	기술개발	10%	테스트 슈트 구조 설계 및 레파지토리 관리 기술	
	테스트 자동화 관리 기술	기술개발	5%	테스트 슈트를 분석하여 순서대로 테스트 실행을 요청하는 기술	
	자동화 예외상황 처리 기술	기술개발	10%	검증 대상 SW 모니터링 하여 예외 상황 발생을 탐지하는 기술	
	테스트 자동화 실행 기술	기술개발	20%	빅데이터 솔루션 핵심 기술에 대한 테스트 자동화 실행 지원 기술	
	데이터 생성 기술	기술개발	5%	다양한 데이터 생성을 위한 데이터 생성 지원 기술	
	연동 도구 제어 기술	기술개발	5%	연동 메모리 누수 탐지 도구 제어 기술	
	테스트 자동화 결과 검증 기술	기술개발	10%	자동화 실행 결과에 대한 검증 기술	
	결과 확인 시각화 기술	기술개발	10%	실행 결과 확인 및 실패 원인 분석을 지원하는 시각화 기술	
	메모리 검증 결과 필터링 기술	기술개발	5%	메모리 누수 검증 결과를 필터링 하는 기술	
			100%		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 3 장 결과 및 향후 계획

제 1 절 연구개발 결과

1. 연차 연구개발 추진 일정 및 실적

1차년도(2018~2019년)														
일련번호	개발내용	추진 일정												책임자(소속기관)
		8월	9월	10월	11월	12월	1월	2월	3월	4월	5월	6월	7월	
빅데이터 분석용 DBMS 분석 및 요구사항 도출														
1	빅데이터 분석용 DBMS의 기능 및 핵심기술 분석													김현정 (TTA 소프트웨어시험인증연구소)
2	빅데이터 분석용 DBMS 장애 발생 사례 조사													김현정 (TTA 소프트웨어시험인증연구소)
3	빅데이터 분석용 DBMS에 대한 주관기업의 품질 검증 자동화 요구사항 도출													김현정 (TTA 소프트웨어시험인증연구소)
품질 검증 자동화 프레임워크 요소기술 연구														
1	효율적인 테스트케이스 관리를 위한 레파지토리 구축 방법 연구													김현정 (TTA 소프트웨어시험인증연구소)
2	장애유형별, SQL-92 표준 기반, 품질특성별, 핵심요소기술별 검증 방법 연구													김현정 (TTA 소프트웨어시험인증연구소)
3	주요 치명 결함(메모리 누수, 무결성 오류 등)의 원인을 찾을 수 있는 동적 테스트 방법 연구													김현정 (TTA 소프트웨어시험인증연구소)
품질 검증 자동화 프레임워크 개발을 위한 테스트 자동화 엔진 설계/개발														
1	테스트 자동화 관리 엔진 설계 및 개발													김현정 (TTA 소프트웨어시험인증연구소)
2	테스트 자동화 실행 엔진 설계 및 개발													김현정 (TTA 소프트웨어시험인증연구소)
3	테스트 실행 결과 검증 엔진 설계 및 개발													김현정 (TTA 소프트웨어시험인증연구소)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1차년도(2018~2019년)														
일련번호	개발내용	추진 일정												책임자(소속기관)
		8월	9월	10월	11월	12월	1월	2월	3월	4월	5월	6월	7월	
품질 검증 자동화 프레임워크 개발을 위한 테스트 슈트 레파지토리 설계/개발														
1	테스트케이스 설계 및 실행 스크립트 개발													김현정 (TTA 소프트웨어시험인증연구소)
2	테스트 슈트 레파지토리 개발													김현정 (TTA 소프트웨어시험인증연구소)
품질 검증 자동화 프레임워크 개발을 위한 테스트 자동화 결과 시각화 도구 개발														
1	테스트 실행 결과(성공 및 실패)에 대한 요약 정보 제공 도구 개발													김현정 (TTA 소프트웨어시험인증연구소)
2	실패 테스트 케이스에 대한 분석 도구 개발													김현정 (TTA 소프트웨어시험인증연구소)
3	메모리 관리 검증 결과 리포팅 도구 개발													김현정 (TTA 소프트웨어시험인증연구소)
품질 검증 자동화 프레임워크 매뉴얼 개발														
1	자동화 프레임워크 설치 매뉴얼 개발													김현정 (TTA 소프트웨어시험인증연구소)
2	레파지토리 운영 및 테스트 케이스 작성 가이드 개발													김현정 (TTA 소프트웨어시험인증연구소)
3	테스트 자동화 도구 사용자 매뉴얼 개발													김현정 (TTA 소프트웨어시험인증연구소)
4	트라블슈팅 (Troubleshooting) 가이드 개발													김현정 (TTA 소프트웨어시험인증연구소)
품질 검증 자동화 프레임워크 검증														
1	품질 검증 자동화 프레임워크 시범 적용 및 개선													김현정 (TTA 소프트웨어시험인증연구소)
2	품질 검증 자동화 프레임워크를 활용한 시스템 테스트													김현정 (TTA 소프트웨어시험인증연구소)
3	품질 개선을 위한 컨설팅 보고서													김현정 (TTA 소프트웨어시험인증연구소)

예상일정 추진실적

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 연차 연구개발 추진 실적

1차년도(2018~2019년)			
일련번호	개발내용	추진 실적	
		수행 결과	산출물명
빅데이터 분석용 DBMS 분석 및 요구사항 도출			
1	빅데이터 분석용 DBMS의 기능 및 핵심기술 분석	완료	- 빅데이터 분석용 DBMS의 기능 및 핵심 기술 분석서 - 요구사항 명세서
2	빅데이터 분석용 DBMS 장애 발생 사례 조사	완료	
3	빅데이터 분석용 DBMS에 대한 주관기업의 품질 검증 자동화 요구사항 도출	완료	
품질 검증 자동화 프레임워크 요소기술 연구			
1	효율적인 테스트케이스 관리를 위한 레파지토리 구축 방법 연구	완료	- 요소기술 연구 결과서
2	장애유형별, SQL-92 표준 기반, 품질특성별, 핵심요소기술별 검증 방법 연구	완료	
3	주요 치명 결함(메모리 누수, 무결성 오류 등)의 원인을 찾을 수 있는 동적 테스트 방법 연구	완료	
품질 검증 자동화 프레임워크 개발을 위한 테스트 자동화 엔진 설계/개발			
1	테스트 자동화 관리 엔진 설계 및 개발	완료	테스트 자동화 엔진
2	테스트 자동화 실행 엔진 설계 및 개발	완료	
3	테스트 실행 결과 검증 엔진 설계 및 개발	완료	
품질 검증 자동화 프레임워크 개발을 위한 테스트 슈트 레파지토리 설계/개발			
1	테스트케이스 설계 및 실행 스크립트 개발	완료	- 테스트케이스 설계 및 실행 스크립트 - 테스트 슈트 레파지토리
2	테스트 슈트 레파지토리 개발	완료	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1차년도(2018~2019년)			
일련번호	개발내용	추진 실적	
		수행 결과	산출물명
품질 검증 자동화 프레임워크 개발을 위한 테스트 자동화 결과 시각화 도구 개발			
1	테스트 실행 결과(성공 및 실패)에 대한 요약 정보 제공 도구 개발	완료	시각화 도구
2	실패 테스트 케이스에 대한 분석 도구 개발	완료	
3	메모리 관리 검증 결과 리포팅 도구 개발	완료	
품질 검증 자동화 프레임워크 매뉴얼 개발			
1	자동화 프레임워크 설치 매뉴얼 개발	완료	자동화 프레임워크 설치 매뉴얼
2	레파지토리 운영 및 테스트케이스 작성 가이드 개발	완료	레파지토리 운영 및 테스트케이스 작성 가이드
3	테스트 자동화 도구 사용자 매뉴얼 개발	완료	테스트 자동화 도구 사용자 매뉴얼
4	트러블슈팅 (Troubleshooting) 가이드 개발	완료	트러블슈팅 (Troubleshooting) 가이드
품질 검증 자동화 프레임워크 검증			
1	품질 검증 자동화 프레임워크 시범 적용 및 개선	완료	시범 적용 결과
2	품질 검증 자동화 프레임워크를 활용한 시스템 테스트	완료	시스템 테스트 결과서
3	품질 개선을 위한 컨설팅 보고서	완료	컨설팅 보고서

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 기술개발 결과의 유형 및 무형 성과 전체를 기재

1. 정량적 목표 달성 결과

가. 기술 개발 성과

평가 항목	개발 목표치	달성 결과	평가 방법
1. 테스트케이스 및 실행 스크립트 개발	1,000건	1,200건	공인 시험성적서 제출(V&V)
2. 품질 검증 자동화 프레임워크 적합성	110%	140%	공인 시험성적서 제출(V&V)
3. 품질검증 자동화 프레임워크 구성 엔진 개발	5종	5종	공인 시험성적서 제출(V&V)
4. 품질검증 자동화 프레임워크 실행 결과 정확도	95%	100%	공인 시험성적서 제출(V&V)
5. 품질검증 자동화 프레임워크 사용자 매뉴얼	4종	4종	자체 평가 (주관기관 평가)
6. 품질검증 자동화 프레임워크를 활용한 품질컨설팅 보고서	1건	1건	자체 평가 (주관기관 평가)

나. 정량적 성과

평가 항목	개발 목표치	달성 결과	비고 (학회명)
1. 국내 특허 출원	1건	1건	-
2. 국내 논문 제출 및 발표	1건	1건	2019 한국소프트웨어공학학술대회(KCSE)
3. 국제 논문 제출 및 발표	1건	1건	14th IEEE/ACM International Workshop on Automation of Software Test
4. 프로그램 등록	1건	1건	-

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 기술개발 결과

가. 빅데이터 분석용 DBMS 분석 및 요구사항 도출 및 데이터 분석용 DBMS 맞춤형 품질 검증 자동화 프레임워크 요소기술 연구

(1) 핵심 기능 및 기술 분석

- ㈜웨어밸리의 빅데이터 분석용 DBMS(PetaSQL)를 분석하여 핵심 기능 및 기술에 대한 리스트를 도출함

대분류	중분류	소분류	기능설명
클라이언트	-	-	데이터베이스에 접속하여 SQL문을 실행하는 기능
데이터베이스 관리	데이터베이스	create/start/ stop/destroy	데이터베이스 생성/구동/중지/삭제하는 기능
데이터 정의 언어(DDL)	사용자	create/alter/drop	사용자/테이블/인덱스(btree, hash)/뷰/제약조건 에 대한 DDL 문을 생성/변경/삭제하는 기능 ※ DDL : Data Definition Language
	테이블		
	인덱스		
	뷰		
	제약조건		
	권한	grant/revoke	select, insert, update 등에 대한 SQL 실행 권한을 사용자에게 부여/철회하는 기능
데이터 조작 언어(DML)	테이블	insert/select/ update/delete	데이터를 삽입/조회/변경/삭제하는 기능 ※ DML : Data Manipulation Language
데이터 제어 언어(DCL)	트랜잭션	read commited/ serializable	트랜잭션 격리수준을 설정하는 기능
		commit/rollback	DML 실행을 통해 조작된 데이터를 데이터베이스에 반영하거나, DML 실행 이전 상태로 되돌리는 기능 ※ DCL : Data Control Language
저장 프로시저	프로시저	create/call/drop	프로시저 및 함수를 생성/실행/삭제하는 기능
	함수		
내장 함수	통계 함수	avg, count, max, min, sum 등 10개	데이터를 집계(avg, count, max, min, sum 등)하는 함수
	수치 연산 함수	sign, abs, log, cos, sqrt 등 24개	데이터를 수치 연산(sign, abs, log, cos, sqrt 등) 하는 함수
	분석 함수	rank over, median, dense_rank over 등 4개	데이터를 분석(rank_over, median, dense_rank over 등)하는 함수
	문자열 함수	upper, lower, length, lpad, rpad 등 29개	문자열을 처리(upper, lower, length, lpad, rpad 등)하는 함수

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

대분류	중분류	소분류	기능설명
	URL 주소 처리 함수	getbasename, getcontent 등 12개	URL 주소를 처리(getbasename, getcontent 등)하는 함수
	날짜 및 시간 함수	sysdate, week 등 16개	날짜 및 시간을 처리(sysdate, week, local_timezone 등)하는 함수
내장 함수	변환 함수	date_to_str, str_to_date 등 5개	데이터 타입을 변환(date_to_str, str_to_date 등)하는 함수
	system 함수	storage, clientid, clients, sessions 등 8개	데이터베이스의 정보(storage, clientid, clients, sessions 등)를 출력하는 함수
백업 및 복구 관리	백업	-	데이터를 백업하는 기능
	복구	-	데이터를 복구하는 기능
감사 기록	-	-	데이터베이스 구동시각, 내부 오류 등의 정보를 로그 파일로 기록하는 기능
이중화 (Replication)	복제	-	테이블을 동종 제품의 데이터베이스에 복제하는 기능
	생성/구동/중지/삭제		복제 서비스를 생성/구동/중지/삭제하는 기능
DB Link	조회	-	분산된 데이터를 조회하기 위해 동종 데이터베이스를 연결하는 기능
	생성/구동/삭제	-	DB Link를 생성/구동/삭제하는 기능
인터페이스	JDBC	-	자바(JAVA)에서 시험대상제품에 접속할 수 있도록 지원하는 API ※ JDBC: Java Database Connectivity
	ODBC	-	ODBC 드라이버를 통해 시험대상제품에 접속할 수 있도록 지원하는 API ※ ODBC: Open Database Connectivity

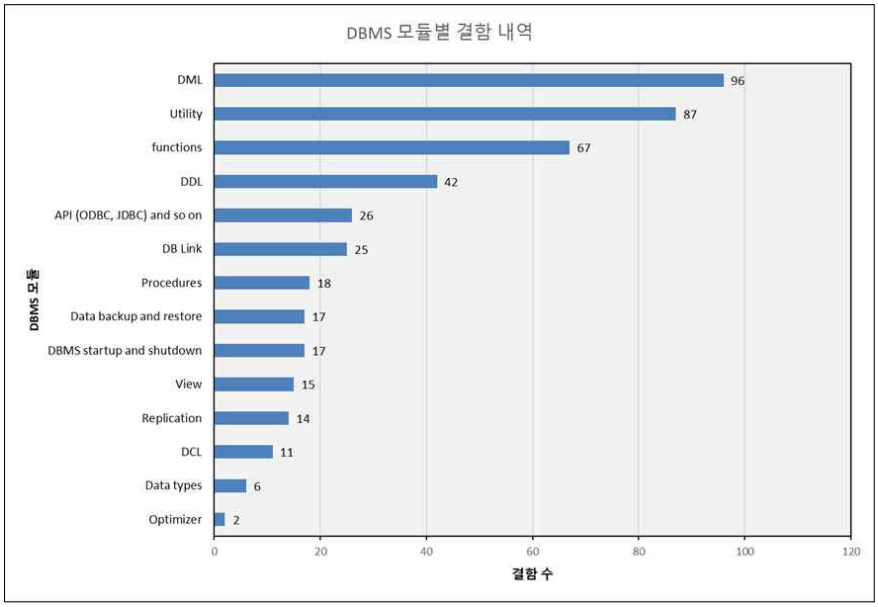
[표 7] 빅데이터 분석용 DBMS의 핵심 기능 리스트

(2) 장애 발생 사례 조사

① 통신 오류, 프로세스 비정상 종료, 데이터 무결성 오류 등 장애유형별 사례 수집

- TTA 소프트웨어시험인증연구소에서 시험한 다양한 종류의 DBMS의 시험 결과를 분석하여 DBMS의 주요 장애 사례를 수집하고 분석함
- 주요 장애 유형을 분석한 결과 DML(데이터 조작 언어) 관련 기능에서 가장 많은 결함이 발생한 것으로 확인되었으며 주요 결함 내용은 [표 8]과 같음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 8] DBMS 모듈별 결함 내역

순번	결함 요약	결함 설명
1	select 결과 출력 오류	특정 상황에서 select 결과가 화면에 출력되지 않음 <시나리오> 1. create table foo (id integer, bar1 integer, bar2 integer); 2. create table bar (barid integer, value char(10)); 3. insert into bar values (1, 'aaa'); 4. insert into bar values (2, 'bbb'); 5. insert into bar values (3, 'ccc'); 6. insert into foo values (100, 1, 2); 7. insert into foo values (101, 2, 3); 8. select b.value, f.id from bar b left join foo f on (f.bar1 = b.barid or f.bar2 = b.barid); → 결과가 화면에 출력되지 않음
2	rank 함수 기능오류	rank() 함수 실행 시, 원인을 알 수 없는 에러 메시지가 발생하고, 기능이 동작하지 않음 <시나리오> 1. create table t3 (f1 int, f2 int); 2. insert into t3 values (1,1), (2,2), (3,3), (4,4), (5,5); 3. select f1,f2, rank() over (order by f2 range between 2 PRECEDING AND CURRENT ROW) from t3;

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	결함 요약	결함 설명
		(에러 메시지) TypeException:user.s_1[18]:'algebra.subsort' undefined in: (_34:any, r1_34:any, r2_34:any) := algebra.subsort(_29:int, r1_30:bat[:oid,:oid], r2_30:bat[:oid,:oid], _37:bit, _33:bit) program contains errors
3	insert 기능 오류	특정 데이터 타입(clob, string, text)으로 테이블을 생성하고 숫자값 데이터 입력 시, 해당 테이블에 숫자값이 저장되어야 하지만 빈(empty) 값으로 잘못 저장됨 <시나리오> 1. create table t1(c1 string, c2 clob, c3 text); 2. insert into t1 values(1,1,1); 3. select * from t1;
4	서버 비정상 종료	특정 업데이트 쿼리문 실행 시, 서버가 비정상 종료함 <시나리오> 1. create table foo (col integer); 2. update foo set col = row_number() over (order by col);

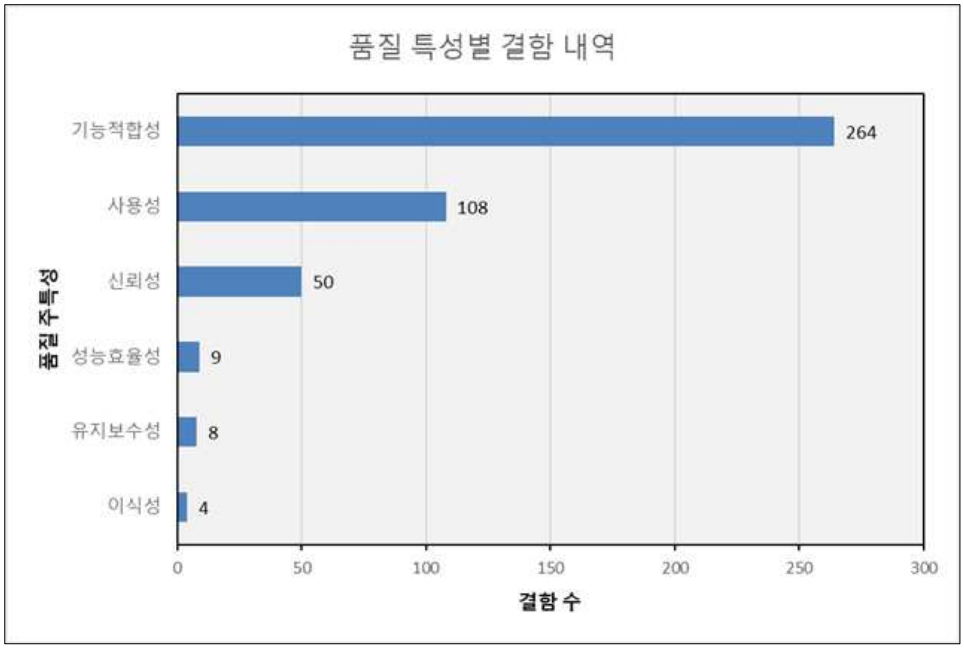
5	이중화
	기능 오류
		이중화 실행 후, 프로시저를 사용하여 데이터 삽입 시, slave 서버에 해당 데이터가 저장되지 않음
		
		<시나리오>
		1. create table t1(c1 int);
		2. create procedure proc1(n int)
		begin
		declare x int;
		set x=1;
		while x <= n do
		insert into t1 values(x);
		set x = x + 1;
		end while;
		end;
		3. call proc1(100);

[표 8] DBMS 모듈별 주요 결함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

② 기능적합성, 신뢰성, 성능효율성, 호환성 등 품질특성별 주요 오류 사례 수집

- TTA에서 시험한 DBMS 시험 결과를 품질특성별로 분석한 결과 기능적합성 품질에 대한 결함이 가장 많이 발생한 것을 확인할 수 있으며, 품질특성별 주요 결함은 [표 9]와 같음



[그림 9] 품질 특성별 결함 내역

순번	품질특성	결함 설명
1	기능적합성	"temporary table' 생성 후, 특정 상황에서 알수 없는 오류 메시지가 발생하고 기능이 동작하지 않음 <시나리오> 1. drop table test; 2. create local temporary table test(c1 int) on commit delete rows; 3. insert into test values(1); 4. select * from test; (에러 메시지) GDK reported error. BATproject: does not match always

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	품질특성	결함 설명
2	사용성	특정 max() 함수가 포함된 조회 쿼리문 실행 시, 원인을 알 수 없는 에러 메시지가 발생함 <쿼리문> select count(*) from t1 where c1=max(c1); (에러 메시지) Subquery result missing

3	신뢰성	특정 상황에서 서버가 비정상 종료함
		
		<시나리오>
		1. create table t1(c1 int, c2 int, c3 int, c4 text);
		2. create procedure proc2(n int)
		begin
		declare x int;
		set x=1;
		while x <= n do
		insert into t1 values(rand(), rand(rand()),rand() % 10, rpad('tta',1000,'tta'));
		set x = x + 1;
		end while;
		end;
		3. 4개의 클라이언트가 'proc2' 를 100회 반복수행
		- call proc2(1000000);
		4. 1개의 클라이언트가 select count(c1) from t1 where c1 <> 1; 를 100회 반복수행
		-> "Connection terminated" 에러 메시지가 발생하고, 서버가 비정상 종료함

4	성능효율성	TPC-H 데이터(300GB) 삽입 후, 아래와 같은 인덱스 생성 시, 48시간이 지나도 인덱스가 생성되지 않음 --- 아래 --- create index idx_lineitem_part_supp using btree on lineitem (l_partkey,l_suppkey);

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

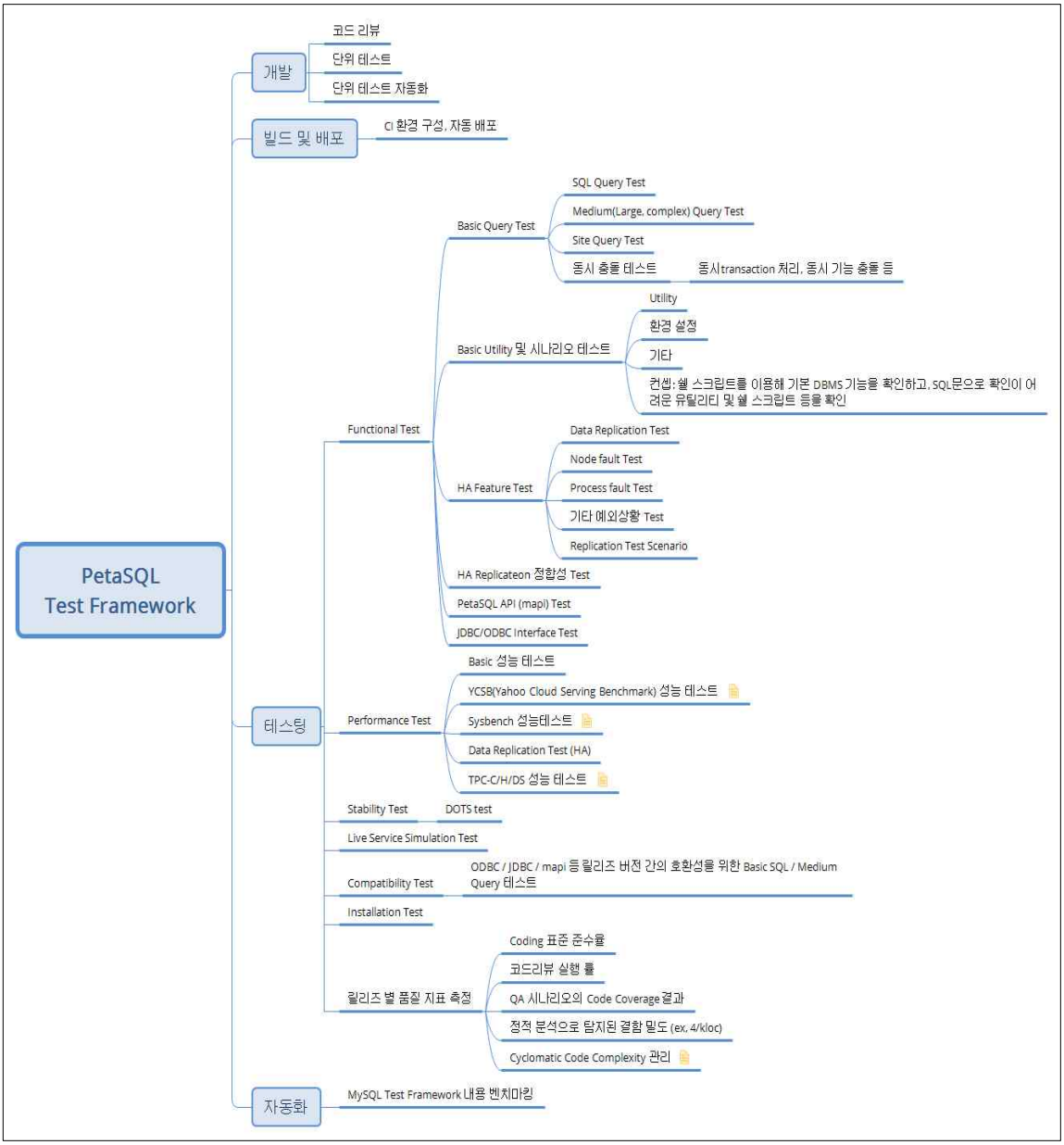
순번	품질특성	결함 설명
5	유지보수성	특정 환경변수 변경 시, 데이터 베이스 구동에 실패함 <시나리오> 1. export LANG="en_us.UTF-8" 2. petactl create db1 3. petactl release db1 4. petactl start db1
6	이식성	Install-Version 설치 수행시, 에레메시지가 발생하고 설치에 실패함 <시나리오> 1. tar xvf Petasql_rpms.tar.gz 2. cd Petasql_rpms 3. ./install.sh (에러 메시지) file /usr/lib64/libsnappy.so.1 from install of PetaSQL-server-2.0.4-CentOS6.4.x86_64 conflicts with file from package snappy-1.1.0-1.el6.x86_64 >> PetaSQL: Install Failed.

[표 9] 품질 특성별 주요 결함

(3) 품질 검증 자동화 요구사항 도출

① ㈜웨어밸리의 요구사항 수집 및 도출

- 주기적인 회의를 통해 주관기관인 ㈜웨어밸리의 품질 검증 자동화 프레임워크의 요구사항을 수집 및 도출함
- 마인드맵을 통해 요구사항을 시각화함으로써 요구사항 누락 및 오류를 최소화함

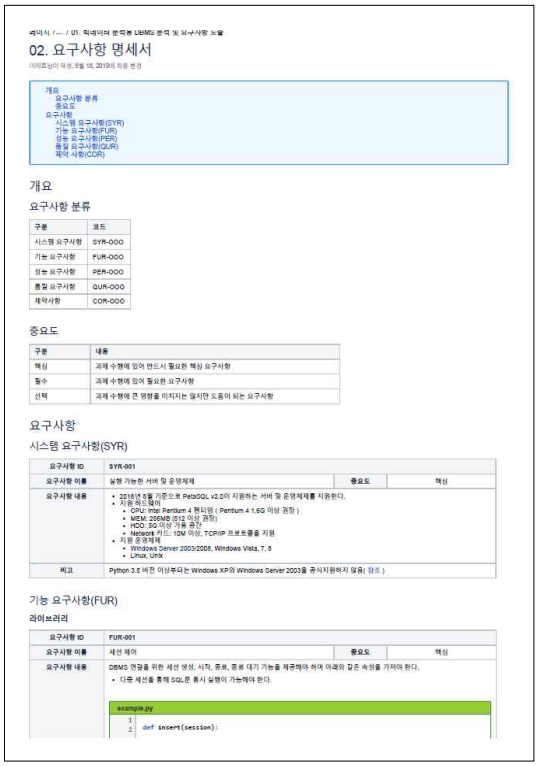


[그림 10] 마인드맵을 통한 ㈜웨어밸리의 요구사항 도출

② 요구사항 명세서

- 빅데이터 분석용 DBMS의 핵심 기능, 장애 유형, 품질 특성별 오류 및 웨어밸리의 요구사항을 분석하여 품질 검증 자동화 프레임워크의 요구사항 명세서를 작성함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



메뉴얼 / ... / 01. 학생버전 분석용 DBMS 분석 및 요구사항 노출

02. 요구사항 명세서

이재호님이 작성, 8월 16, 2019에 최종 변경

개요
요구사항 분류
중요도
요구사항
시스템 요구사항(SYR)
기능 요구사항(FUR)
성능 요구사항(PER)
품질 요구사항(QUR)
제약 사항(COR)

개요

요구사항 분류

구분	코드
시스템 요구사항	SYR-OOO
기능 요구사항	FUR-OOO
성능 요구사항	PER-OOO
품질 요구사항	QUR-OOO
제약사항	COR-OOO

중요도

구분	내용
핵심	과제 수행에 있어 반드시 필요한 핵심 요구사항
필수	과제 수행에 있어 필요한 요구사항
선택	과제 수행에 큰 영향을 미치지는 않지만 도움이 되는 요구사항

요구사항

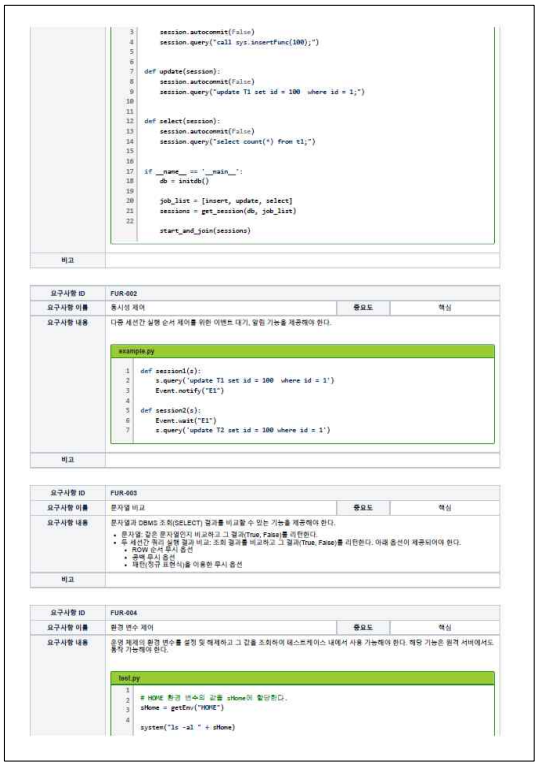
시스템 요구사항(SYR)

요구사항 ID	SYR-001		
요구사항 이름	실행 가능한 서버 및 운영체제	중요도	핵심
요구사항 내용	• 2018년 8월 기준으로 PetaSQL v2.0이 지원하는 서버 및 운영체제를 지원한다. • 지원 하드웨어 • CPU: Intel Pentium 4 현식명 (Pentium 4 1.6G 이상 권장) • MEM: 256MB (512 이상 권장) • HDD : 5G 이상 가용 공간 • Network 카드: 10M 이상, TCP/IP 프로토콜을 지원 • 지원 운영체제 • Windows Server 2003/2008, Windows Vista, 7, 8 • Linux, Unix		
비고	Python 3.5 버전 이상부터는 Windows XP와 Windows Server 2003을 공식지원하지 않음(참조)		

기능 요구사항(FUR)

라이브러리

요구사항 ID	FUR-001		
요구사항 이름	세션 제어	중요도	핵심
요구사항 내용	DBMS 연결을 위한 세션 생성, 시작, 종료, 종료 대기 기능을 제공해야 하며 아래와 같은 속성을 가져야 한다. • 다중 세션을 통해 SQL문 동시 실행이 가능해야 한다. example.py 1 def insert(session):



```
3       session.autocommit(False)
4       session.query("call sys.insertFunc(100);")
5
6
7   def update(session):
8       session.autocommit(False)
9       session.query("update T1 set id = 100  where id = 1;")
10
11
12  def select(session):
13      session.autocommit(False)
14      session.query("select count(*) from t1;")
15
16
17  if __name__ == '__main__':
18      db = initdb()
19
20      job_list = [insert, update, select]
21      sessions = get_session(db, job_list)
22
        start_and_join(sessions)
```

비고	

요구사항 ID	FUR-002		
요구사항 이름	동시성 제어	중요도	핵심
요구사항 내용	다중 세션간 실행 순서 제어를 위한 이벤트 대기, 알림 기능을 제공해야 한다. example.py 1 def session1(s): 2 s.query('update T1 set id = 100 where id = 1') 3 Event.notify("E1") 4 5 def session2(s): 6 Event.wait("E1") 7 s.query('update T2 set id = 100 where id = 1')
비고	

요구사항 ID	FUR-003		
요구사항 이름	문자열 비교	중요도	핵심
요구사항 내용	문자열과 DBMS 조회(SELECT) 결과를 비교할 수 있는 기능을 제공해야 한다. • 문자열 값은 문자열인지 비교하고 그 결과(True, False)를 리턴한다. • 두 세션간 쿼리 실행 결과 비교: 조회 결과를 비교하고 그 결과(True, False)를 리턴한다. 아래 옵션이 제공되어야 한다. • ROW 순서 무시 옵션 • 공백 무시 옵션 • 패턴(정규 표현식)을 이용한 무시 옵션
비고	

요구사항 ID	FUR-004		
요구사항 이름	환경 변수 제어	중요도	핵심
요구사항 내용	운영 체제의 환경 변수를 설정 및 제제하고 그 값을 조회하여 테스트케이스 내에서 사용 가능해야 한다. 해당 기능은 원격 서버에서도 동작 가능해야 한다. test.py 1 2 # HOME 환경 변수의 값을 sHome에 할당한다. 3 sHome = getEnv("HOME") 4 system("ls -al " + sHome)



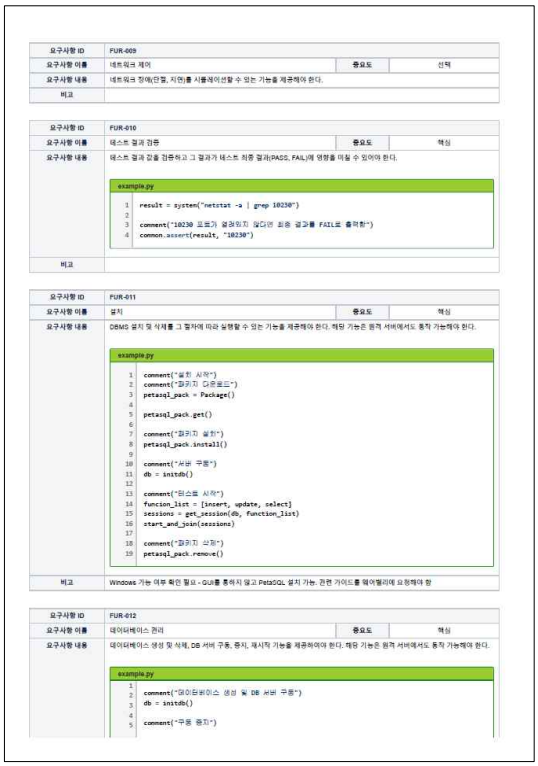
비고	

요구사항 ID	FUR-005		
요구사항 이름	주석	중요도	핵심
요구사항 내용	결과 파일에 주석을 기록할 수 있는 기능을 제공해야 한다. test.py 1 # comment 함수를 통한 주석을 결과 파일에 기록함 2 comment("db 구동")
비고	

요구사항 ID	FUR-006		
요구사항 이름	파일 제어	중요도	핵심
요구사항 내용	아래와 같이 DBMS에서 제공하는 파일(로그 파일)을 조회할 수 있는 기능을 제공해야 한다. 해당 기능은 원격 서버에서도 동작 가능해야 한다. • 로그 파일 읽기 • 로그 파일의 일부분 읽기(리눅스 head와 동일) • 로그 파일의 뒷부분 읽기(리눅스 tail과 동일) test.py 1 log_file = File(db, type="log") 2 log = log_file.read() 3 4 head = log_file.head() 5 tail_line = log_file.tail(100)
비고	

요구사항 ID	FUR-007		
요구사항 이름	프로세스 제어	중요도	핵심
요구사항 내용	DBMS의 프로세스를 제어(시그널 전송, PID 조회)할 수 있는 기능을 제공해야 한다. 해당 기능은 원격 서버에서도 동작 가능해야 한다. test.py 1 # petasqld(데몬프로세스) 강제 종료 2 petasqld = get_pid(db, name="petasqld") 3 4 petasqld.signal(SIGINT)
비고	

요구사항 ID	FUR-008		
요구사항 이름	자원(리소스) 제어	중요도	필수
요구사항 내용	CPU, 메모리, 스토리지 등 자원을 제어할 수 있도록 아래와 같은 기능을 제공해야 한다. 해당 기능은 원격 서버에서도 동작 가능해야 한다. • 메모리 소진 • 스토리지 소진 • CPU 소진 • 메모리 cache/buff 정리 • 자원 모니터링
비고	



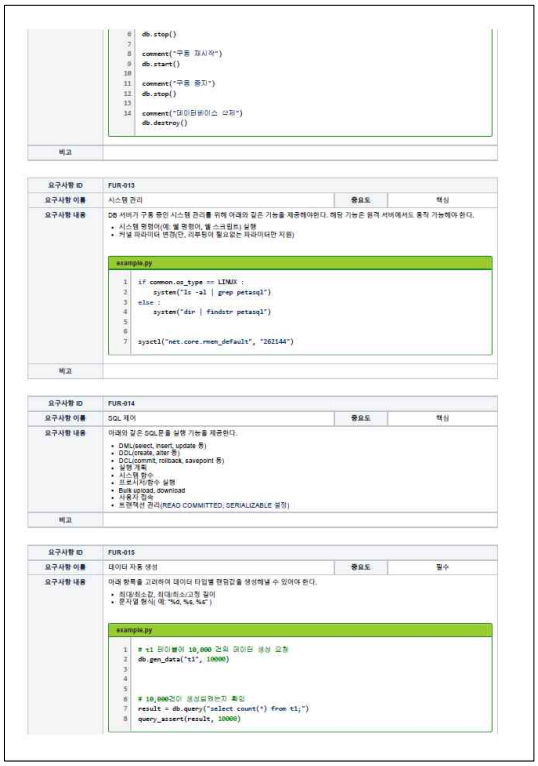
요구사항 ID	FUR-009		
요구사항 이름	네트워크 제어	중요도	선택
요구사항 내용	네트워크 장애(단절, 지연)를 시뮬레이션할 수 있는 기능을 제공해야 한다.		
비고			

요구사항 ID	FUR-010		
요구사항 이름	테스트 결과 검증	중요도	핵심
요구사항 내용	테스트 결과 값을 검증하고 그 결과가 테스트 최종 결과(PASS, FAIL)에 영향을 미칠 수 있어야 한다. example.py 1 result = system("netstat -a | grep 10230") 2 3 comment("10230 포트가 열려있지 않다면 최종 결과를 FAIL로 출력한") 4 common.assert(result, "10230")
비고			

요구사항 ID	FUR-011		
요구사항 이름	설치	중요도	핵심
요구사항 내용	DBMS 설치 및 삭제를 그 절차에 따라 실행할 수 있는 기능을 제공해야 한다. 해당 기능은 원격 서버에서도 동작 가능해야 한다. example.py 1 comment("설치 시작") 2 comment("패키지 다운로드") 3 petasql_pack = Package() 4 5 petasql_pack.get() 6 7 comment("패키지 설치") 8 petasql_pack.install() 9 10 comment("서버 구동") 11 db = initdb() 12 13 comment("테스트 시작") 14 funcion_list = [insert, update, select] 15 sessions = get_session(db, function_list) 16 start_and_join(sessions) 17 18 comment("패키지 삭제") 19 petasql_pack.remove()
비고	Windows 가능 여부 확인 필요 - GUI를 통하지 않고 PetaSQL 설치 가능. 관련 가이드를 메뉴얼에 요청해야 함		

요구사항 ID	FUR-012		
요구사항 이름	데이터베이스 관리	중요도	핵심
요구사항 내용	데이터베이스 생성 및 삭제, DB 서버 구동, 중지, 재시작 기능을 제공하여야 한다. 해당 기능은 원격 서버에서도 동작 가능해야 한다. example.py 1 2 comment("데이터베이스 생성 및 DB 서버 구동") 3 db = initdb() 4 5 comment("구동 중지")

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



db.stop()

comment("구동 재시작")
db.start()

comment("구동 종지")
db.stop()

comment("데이터베이스 삭제")
db.destroy()

비고

요구사항 ID	FUR-013		
요구사항 이름	시스템 관리	중요도	핵심
요구사항 내용	DB 서버가 구동 중인 시스템 관리를 위해 아래와 같은 기능을 제공해야한다. 해당 기능은 원격 서버에서도 동작 가능해야 한다. • 시스템 정보(메모리, 엘 정보)의, 쉘 스크립트 실행 • 커널 파라미터 변경(단, 리부팅이 필요없는 파라미터만 지원)		

example.py

```
if common.os_type == LINUX :
    system("ls -al | grep petasql")
else :
    system("dir | findstr petasql")


sysctl("net.core.rmem_default", "262144")
```

비고

요구사항 ID	FUR-014		
요구사항 이름	SQL 제어	중요도	핵심
요구사항 내용	아래와 같은 SQL문을 실행 기능을 제공한다. • DML(select, insert, update 등) • DDL(create, alter 등) • DCL(commit, rollback, savepoint 등) • 실행 계획 • 시스템 함수 • 프로시저/함수 실행 • Bulk upload, download • 사용자 접속 • 트랜잭션 관리(READ COMMITTED, SERIALIZABLE 설정)		

비고

요구사항 ID	FUR-015		
요구사항 이름	데이터 자동 생성	중요도	필수
요구사항 내용	아래 항목을 고려하여 데이터 타입별 랜덤값을 생성해낼 수 있어야 한다. • 최대/최소값, 최대/최소길이 정의 • 문자열 형식 예: "%d_%s_%s")		

example.py

```
# t1 테이블에 10,000 건의 데이터 생성 요청
db.gen_data("t1", 10000)


# 10,000건이 생성되었는지 확인
result = db.query("select count(*) from t1;")
query_assert(result, 10000)
```

비고

요구사항 ID	FUR-016		
요구사항 이름	쿼리 실행 결과에 대한 검증 또는 ASSERT 기능 제공	중요도	필수
요구사항 내용	쿼리 실행 결과에 측정 데이터가 포함되어 있는지 여부를 확인하여 테스트 결과(PASS 또는 FAIL)에 영향을 미치는 기능을 제공한다.		

example.py

```
# 조회 결과가 26이 아닐 경우 test fail
query_test("select count(*) from tables where id > high;", 26);

# 비교를 달리 할 수도 있다.
# 20~26 사이의 값이 하나라도 있으면 test fail
mylist = [20, 21, 22, 23, 24, 25, 26]
query_test("select count(*) from tables where id > high;", mylist);

# mylist에 포함되는 값이 하나라도 있으면 test fail
mylist = ["fail","abort","no"]
query_assert("select count(*) from tables where id > high;", mylist);

# mylist에 포함되는 값이 모두(AND 조건) 존재할 경우 test fail (default 는 OR)
mylist = ["fail","abort","no"]
query_assert("select count(*) from tables where id > high;", mylist, "AND");
```

비고

요구사항 ID	FUR-017		
요구사항 이름	SQL문 실행 결과 무시	중요도	핵심
요구사항 내용	검증과 관련없는 SQL문은 실행 결과를 무시할 수 있는 기능을 제공해야 한다.		

test.py

```
if __name__ == '__main__':
    # 테스트 실행이 앞서 t1 테이블을 초기화(삭제)함
    # 앞 테스트 케이스에서 t1 테이블을 생성했는지 여부에 관계없이 drop table은 결과가 달라질 수 있지
    session.query("drop table t1;", ignore=True)

    session.query("CREATE TABLE T1 ( C1 INT );")
```

비고

요구사항 ID	FUR-018		
요구사항 이름	이중화 제어	중요도	핵심
요구사항 내용	아래와 같은 이중화 제어 기능을 제공해야 한다. • 생성 • 구동 • 중지 • 마스터 및 슬레이브 DB들을 대상으로 쿼리 실행 • 각 노드 간 데이터 정합성 검증		

비고

요구사항 ID	FUR-019		
요구사항 이름	DB Link 제어	중요도	핵심
요구사항 내용	아래와 같은 DB Link 제어 기능을 제공해야 한다. • 생성 • 조회 • 사용 • 로컬 및 원격 DB들을 대상으로 쿼리 실행 • DB Link를 통해 조회된 데이터의 정합성 검증		



비고

요구사항 ID	FUR-020		
요구사항 이름	백업 및 복구	중요도	핵심
요구사항 내용	DBMS 백업 및 복구 기능을 제공해야 한다.		

example.py

```
db = initdb()


db.backup()


(중략)

db.restore()


restore_data = db.query("select * from " + table_name + ";")

select_diff(org_data, restore_data)
```

비고

요구사항 ID	FUR-021		
요구사항 이름	유틸리티 제어	중요도	핵심
요구사항 내용	PetaSQL에서 제공하는 아래 유틸리티에 대한 제어(실행, 종료, 실행 결과 획득) 기능을 제공해야 한다. • petasql • petadump : DB Migration • bindump : 데이터 백업/복구 • petaloader : 파일 데이터 DB 업로드		

example.py

```
if __name__ == '__main__':
    db = initdb()

    querylist = ["\>out.file",
                 "insert into t1 values(\'한글입력테스트\');",
                 "select * from t1;"]
    psql(encoding="euc-kr", query=querylist)

    cat("out_file")
```

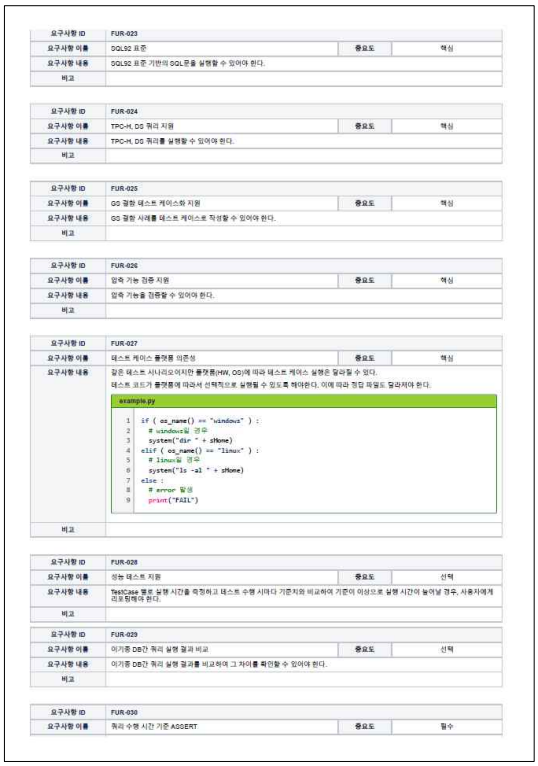
비고

요구사항 ID	FUR-022		
요구사항 이름	인터페이스 제어	중요도	핵심
요구사항 내용	JDBC, ODBC, mapi/MAL을 테스트하기 위한 컴파일, 실행 기능을 제공한다.		

example.py

```
db.compile(target="app1")
db.make()
```

비고



요구사항 ID	FUR-023		
요구사항 이름	SQL92 표준	중요도	핵심
요구사항 내용	SQL92 표준 기반의 SQL문을 실행할 수 있어야 한다.		

비고

요구사항 ID	FUR-024		
요구사항 이름	TPC-H, DS 쿼리 지원	중요도	핵심
요구사항 내용	TPC-H, DS 쿼리를 실행할 수 있어야 한다.		

비고

요구사항 ID	FUR-025		
요구사항 이름	GS 결함 테스트 케이스화 지원	중요도	핵심
요구사항 내용	GS 결함 사례를 테스트 케이스로 작성할 수 있어야 한다.		

비고

요구사항 ID	FUR-026		
요구사항 이름	압축 기능 검증 지원	중요도	핵심
요구사항 내용	압축 기능을 검증할 수 있어야 한다.		

비고

요구사항 ID	FUR-027		
요구사항 이름	테스트 케이스 플랫폼 의존성	중요도	핵심
요구사항 내용	같은 테스트 시나리오이지만 플랫폼(HW, OS)에 따라 테스트 케이스 실행은 달라질 수 있다. 테스트 코드가 플랫폼에 따라서 선택적으로 실행될 수 있도록 해야한다. 이에 따라 정답 파일도 달라져야 한다.		

example.py

```
if ( os_name() == "windows" ) :
  # windows일 경우
  system("dir " + sHome)
elif ( os_name() == "linux" ) :
  # linux일 경우
  system("ls -al " + sHome)
else :
  # error 일경
  print("FAIL")
```

비고

요구사항 ID	FUR-028		
요구사항 이름	성능 테스트 지원	중요도	선택
요구사항 내용	TestCase 별로 실행 시간을 측정하고 테스트 수행 시마다 기준치와 비교하여 기준이 이상으로 실행 시간이 늘어날 경우, 사용자에게 리포팅해야 한다.		

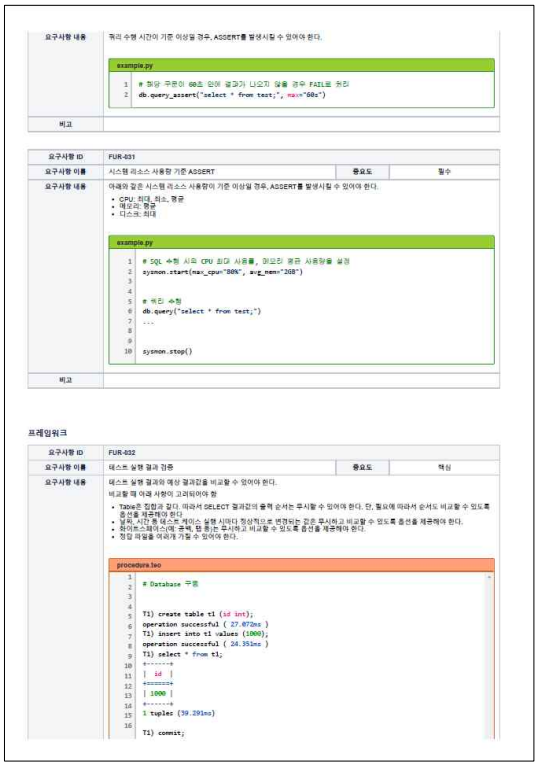
비고

요구사항 ID	FUR-029		
요구사항 이름	이기종 DB간 쿼리 실행 결과 비교	중요도	선택
요구사항 내용	이기종 DB간 쿼리 실행 결과를 비교하여 그 차이를 확인할 수 있어야 한다.		

비고

요구사항 ID	FUR-030		
요구사항 이름	쿼리 수행 시간 기준 ASSERT	중요도	필수

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



요구사항 내용	쿼리 수행 시간이 기준 이상일 경우, ASSERT를 발생시킬 수 있어야 한다.

example.py

```
# 최상 쿼문이 60초 안에 결과가 나오지 않을 경우 FAIL로 처리
db.query_assert("select * from test;", max="60s")
```

비고	

요구사항 ID	FUR-031		
요구사항 이름	시스템 리소스 사용량 기준 ASSERT	중요도	필수
요구사항 내용	아래와 같은 시스템 리소스 사용량이 기준 이상일 경우, ASSERT를 발생시킬 수 있어야 한다. • CPU: 최대, 최소, 평균 • 메모리: 평균 • 디스크: 최대		

example.py

```
# SQL 수행 시의 CPU 최대 사용률, 메모리 평균 사용량을 설정
sysmon.start(max_cpu="80%", avg_mem="2GB")


# 쿼리 수행
db.query("select * from test;")
...


sysmon.stop()
```

비고	

프레임워크

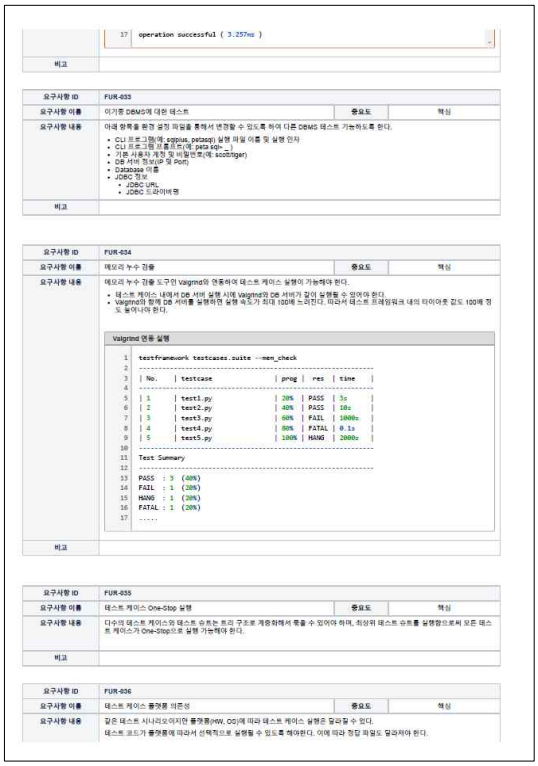
요구사항 ID	FUR-032		
요구사항 이름	테스트 실행 결과와 검증	중요도	핵심
요구사항 내용	테스트 실행 결과와 예상 결과값을 비교할 수 있어야 한다. 비교할 때 아래 사항이 고려되어야 함 • Table의 칼럼과 같이, 따라서 SELECT 결과값의 출력 순서는 무시할 수 있어야 한다. 단, 필요에 따라서 순서도 비교할 수 있도록 옵션을 제공해야 한다. • 날짜, 시간 등 테스트 케이스 실행 시마다 정상적으로 변경되는 값은 무시하고 비교할 수 있도록 옵션을 제공해야 한다. • 하이프스페이스(예: 줄바꿈, 탭 등)는 무시하고 비교할 수 있도록 옵션을 제공해야 한다. • 정답 파일을 여러개 가질 수 있어야 한다.		

procedure.tso

```
# Database 구동


T1) create table t1 (id int);
operation successful ( 27.072ms )
T1) insert into t1 values (1000);
operation successful ( 28.353ms )
T1) select * from t1;
+------+
| id   |
+======+
| 1000 |
+------+
1 tuples (39.291ms)

T1) commit;
```



```
operation successful ( 3.257ms )
```

비고	

요구사항 ID	FUR-033		
요구사항 이름	이기종 DBMS에 대한 테스트	중요도	핵심
요구사항 내용	아래 항목을 환경 설정 파일을 통해서 변경할 수 있도록 하여 다른 DBMS 테스트 가능하도록 한다. • CLI 프로그램(예: sqlplus, psql) 실행 파일 이름 및 실행 인자 • CLI 프로그램 프롬프트(예: psql 와 =) • 기본 사용자 계정 및 비밀번호(예: scott/tiger) • DB 서버 정보(IP 및 Port) • Database 이름 • JDBC 정보 • JDBC URL • JDBC 드라이버명		

비고	

요구사항 ID	FUR-034		
요구사항 이름	메모리 누수 검출	중요도	핵심
요구사항 내용	메모리 누수 검출 도구인 Valgrind와 연동하여 테스트 케이스 실행이 가능해야 한다. • 테스트 케이스 내에서 DB 서버 실행 시에 Valgrind와 DB 서버가 같이 실행될 수 있어야 한다. • Valgrind와 함께 DB 서버를 실행하면 실행 속도가 최대 100배 느려진다. 따라서 테스트 프레임워크 내의 타이아웃 값도 100배 정도 늘어나야 한다.		

Valgrind 연동 실행

```
testframework testcases.suite --mem_check
-------------------------------------------------------
| No.  | testcase          | prog | res  | time    |
-------------------------------------------------------
| 1    | test1.py          | 20%  | PASS | 1s      |
| 2    | test2.py          | 40%  | PASS | 10s     |
| 3    | test3.py          | 60%  | FAIL | 1000s   |
| 4    | test4.py          | 80%  | FATAL | 0.1s   |
| 5    | test5.py          | 100% | HANG | 2000s   |
-------------------------------------------------------

Test Summary

PASS : 3 (60%)
FAIL : 1 (20%)
HANG : 1 (20%)
FATAL : 1 (20%)
-----
```

비고	

요구사항 ID	FUR-035		
요구사항 이름	테스트 케이스 One-Stop 실행	중요도	핵심
요구사항 내용	다수의 테스트 케이스와 테스트 슈트는 트리 구조로 계층화해서 묶을 수 있어야 하며, 최상위 테스트 슈트를 실행함으로써 모든 테스트 케이스가 One-Stop으로 실행 가능해야 한다.		

비고	

요구사항 ID	FUR-036		
요구사항 이름	테스트 케이스 플랫폼 의존성	중요도	핵심
요구사항 내용	같은 테스트 시나리오이지만 플랫폼(HW, OS)에 따라 테스트 케이스 실행은 달라질 수 있다. 테스트 코드가 플랫폼에 따라서 선택적으로 실행될 수 있도록 해야한다. 이에 따라 정답 파일도 달라져야 한다.		



example.py

```
if ( os_name() == "windows" ) :
  # windows일 경우
  system("dir " + sHome)
elif ( os_name() == "linux" ) :
  # linux일 경우
  system("ls -al " + sHome)
else :
  # error 발생
  print("FAIL")
```

비고	

요구사항 ID	FUR-037		
요구사항 이름	DB 서버 상태 모니터링	중요도	핵심
요구사항 내용	테스트 케이스 실행 전 DB 서버의 상태를 추가적으로 모니터링하고 아래와 같은 상황 발생 시 사용자에게 리포팅한 후 다음 테스트 케이스를 실행하지 한다. • FATAL: DB 서버가 비정상 종료된 상태 • HANG: DB 서버에서 비정상 응답(에러 발생 시간동안 쿼리 결과가 오지 않는 상태 • 논의를 통해 추가		
비고	모니터링 방법에 대해서는 추가 논의 필요		

요구사항 ID	FUR-038		
요구사항 이름	심각한 결함 발생 시 분석 정보 백업	중요도	핵심
요구사항 내용	심각한 결함(FATAL, HANG) 발생 시, 분석에 필요한 정보(예: stack, db error log 등)을 웹도의 공간에 저장해야 한다.		
비고			

요구사항 ID	FUR-039		
요구사항 이름	심각한 결함 발생 시 DB 초기화 기능	중요도	핵심
요구사항 내용	심각한 결함(FATAL, HANG) 발생 시, 이를 감지하고 DB를 초기화해주어야 한다. 그렇지 않으면 이후 테스트 케이스는 정상적인 실행이 불가능하다.		
비고			

요구사항 ID	FUR-040		
요구사항 이름	실패 테스트 케이스에 대한 정보 저장	중요도	핵심
요구사항 내용	실패(FAIL)된 테스트 케이스의 경우, 원인 분석을 위해 실행 결과를 별도의 파일로 저장하도록 한다.		
비고			

요구사항 ID	FUR-041		
요구사항 이름	테스트 진행 상황 모니터링	중요도	핵심
요구사항 내용	현재 어떤 테스트 코드가 실행 중인지 모니터링 가능해야 한다.		

테스트 모니터링

```
testmon


T1) CREATE PROCEDURE sys.myfunc() EXTERNAL NAME sql.myfunc;
operation successful ( 92.658ms )
T1) drop table if exists t1;
operation successful ( 2.124ms )
T1) create table t1 (id int);
.........
```



비고	

요구사항 ID	FUR-042		
요구사항 이름	다국어 지원	중요도	핵심
요구사항 내용	아래와 같은 언어를 테스트 케이스에서 사용할 수 있어야 한다. • 한국어 • 중국어 • 일본어		
비고	UTF-8로 테스트 케이스를 작성하도록 지원할 예정		

요구사항 ID	FUR-043		
요구사항 이름	테스트 슈트 및 케이스 반복 수행 지원	중요도	핵심
요구사항 내용	테스트 슈트 또는 케이스를 사용자가 지정한 회수 만큼 반복 수행할 수 있어야 하며, 아래와 같은 속성을 가져야 한다. • 반복 수행 시 이전 결과/결과 및 결과 파일을 유지해야 한다.(즉, 덮어쓰면 안됨) • 총 반복 횟수, 성공 및 실패 횟수 정보를 남겨야 한다.		
비고			

요구사항 ID	FUR-044		
요구사항 이름	기본 정보 시각화	중요도	핵심
요구사항 내용	테스트의 기본 정보(Project, 시작시간, 종료시간, IP 주소, 등등)를 시각화 하는 기능을 제공해야 한다.		
비고			

요구사항 ID	FUR-045		
요구사항 이름	테스트 결과 필터링	중요도	핵심
요구사항 내용	테스트의 결함(1. 실행 결과, 2. Diff, 3. 메모리 결함, 4. 원인)을 필터링하여 필요한 정보만을 제공해야 한다.		
비고			

요구사항 ID	FUR-046		
요구사항 이름	테스트 결과 차트로 시각화	중요도	핵심
요구사항 내용	필터링 한 결함 정보(실행 결과, Diff, 메모리 결함, 원인)를 각각의 통계 차트로 시각화하는 기능을 제공해야 한다.		
비고			

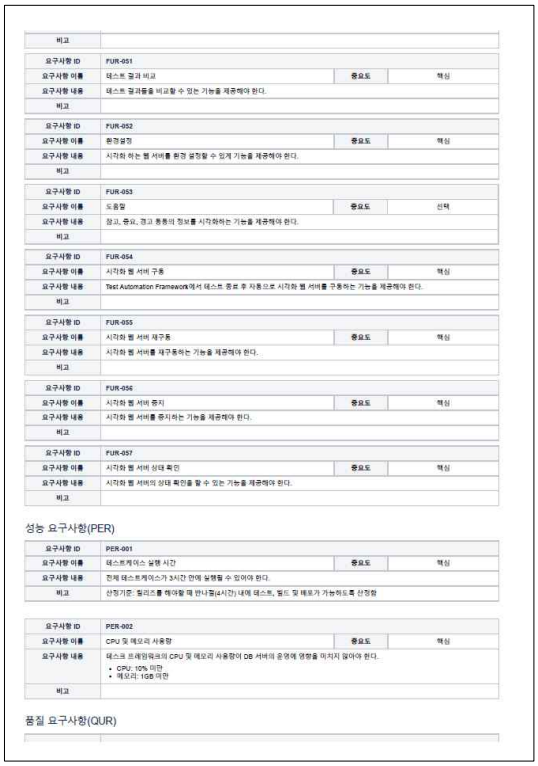
요구사항 ID	FUR-047		
요구사항 이름	상세 정보 검색 및 조회	중요도	핵심
요구사항 내용	테스트의 상세 정보(테스트케이스명, 실행 결과, Diff(아이콘), TC(아이콘), Diff분류, 메모리 결함, 원인, 의견)를 시각화하는 기능을 제공해야 한다.		
비고			

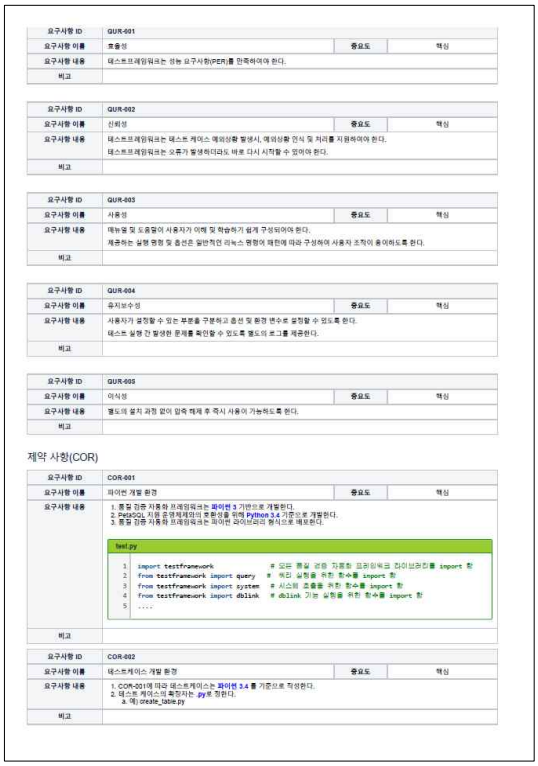
요구사항 ID	FUR-048		
요구사항 이름	분석 정보를 변경 및 초기화	중요도	핵심
요구사항 내용	테스트의 분석 정보(Diff분류, 메모리 결함, 원인, 의견)를 변경 및 초기화하는 기능을 제공해야 한다.		
비고	하나씩 변경 가능, 일괄적으로 변경 가능		

요구사항 ID	FUR-049		
요구사항 이름	저장 및 불러오기	중요도	핵심
요구사항 내용	저장 및 불러오기 기능을 제공해야 한다.		
비고	Zip확장자		

요구사항 ID	FUR-050		
요구사항 이름	결과 리포트 저장	중요도	핵심
요구사항 내용	테스트 결과를 리포트 파일로 저장하는 기능을 제공해야 한다.		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다





- 요구사항 분석을 통해 자동화 실행에 필요한 테스트 API 목록을 도출하였으며, 도출 결과는 [표 10]과 같음

순번	중분류	소분류	package	API
1	세션 제어	세션 생성	session	Session()
2		동기 및 비동기 세션 실행	session	execute()
3		비동기 세션 종료 대기	session	wait()
4		세션 DB 변경	query	set_session()
5			query	unset_session()
6		쓰레드 생성	session	Worker()
7		쓰레드 실행	session	start()
8		쓰레드 종료 대기	session	join()
9	동시성 제어	이벤트 발생 대기	event	wait()
		이벤트 발생 알림	event	notify()
		동기화 배리어(장벽)	event	sync_barrier()
10	문자열 비교	텍스트 비교	utils	diff()
		SELECT 조회 결과 비교	utils	select_diff()
11	환경변수	환경 변수 설정	environment	set_env()

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

		환경 변수 가져오기		get_env()
		환경 변수 해제하기		rm_env()
12	주석	-	utils	comment()
13		블록 구분자	utils	section()
14	결과 파일 기록	결과 파일에 문자열 기록	utils	write_result()
15	파일 제어	로그 파일 조회	utils	read_logfile()
16	프로세스 제어	petasql 프로세스 강제 종료	process	kill()
		petasql 프로세스 시그널 전송		send_signal()
		petasql 프로세스 ID 조회		get_pid()
17	자원(resource) 제어	자원(CPU, 메모리, 디스크) 소진하기	resource	start_stress()
				stop_stress()
		메모리 cache/buff 정리하기	resource	drop_caches()
		자원 모니터링	resource	start_monitoring()
				stop_monitoring()
				get_memory_stat()
				get_cpu_stat()
				get_storage_stat()
18	네트워크 제어	네트워크 절체하기	resource.network	disconnect_network()
19				connect_network()
20	테스트 결과 제어	결과 검증	utils	taf_test()
				taf_assert()
21	패키지 제어	패키지 다운로드	package	get()
22		패키지 설치		install()
23		패키지 삭제		remove()
24	DBMS 제어	데이터베이스 삭제 및 생성	database	clear()
25		데이터베이스 삭제/생성/구동		initdb()
26		데이터베이스 생성		create()
27		데이터베이스 구동		start()
		데이터베이스 재시작		restart()

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

28		데이터베이스 중지		stop()
29		데이터베이스 삭제		destroy()
30		DB_HOME 삭제		remove_dbms_home()
31	시스템 명령어 실행	로컬(원격) 실행	system	system()
32	커널 파라미터 제어	-	system	sysctl()
33	SQL 실행	SQL 실행	query	query()
		SQL 실행 및 결과 검증	query	query_test()
			query	query_assert()
		DB User 변경	query	connect()
		bulk input/output	query	copy()
		DCL	query	transaction()
		DCL	query	autocommit()
			query	commit()
			query	rollback()
			query	savepoint()
		프로시저/함수 실행	query	call()
		데이터 생성	query	gen_data()
34	실행계획	-	query	explain()
		-	query	plan()
35	시스템 함수	BBP	query	BBP()
		STORAGE	query	STORAGE()
		CLIENTID	query	CLIENTID()
		CLIENTS	query	CLIENTS()
		SESSIONS	query	SESSIONS()
		ENVIRONMENT	query	ENVIRONMENT()
		VAR	query	VAR()
		storagemodelinit()	query	storagemodelinit()
		sys_kill	query	sys_kill()
36	쿼리 수행 시간	-	query	query_assert(query, max_latency="60s")
37	이중화 제어	이중화 생성	replication	create_repl()

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

38		이중화 구동	replication	start()
39		이중화 중지	replication	pause()
40		이중화 재개	replication	resume()
41		이중화 정합성 검증	replication	table_diff()
42	DB Link 제어	dblink 생성	dblink	create_dblink()
43		dblink 구동	dblink	start()
44		dblink 조회	dblink	select()
45		dblink 삭제	dblink	drop()
46		dblink 정합성 검증	dblink	dblink_diff()
47		dblink 이름 조회	dblink	get_dblink_name()
48	백업	-	recovery	backup()
49	복구	-		restore()
50	유틸리티 제어	petasql 실행	utility	psql() / client()
		petasql명령 수행	utility	send_wait()
		petasql종료	utility	quit()
		petasql강제 종료	utility	kill()
		prepare() 실행	utility	prepare()
		exec() 실행	utility	exec()
51		petadump	utility	petadump()
52		bindump	utility	bindump()
53		petaloader	utility	petaloader()
54	JDBC	컴파일	interface	javac()
		실행	interface	java()
55		컴파일 초기화	interface	java_clean()
56	ODBC	컴파일	interface	make()
		실행	interface	odbc()
57	TPC-H	TPC-H 스키마, 데이터, 인덱스 생성	benchmark	tpch_create()
		TPC-H 쿼리 수행 1~22번	benchmark	tpch_query()
58	TPC-DS	TPC-DS 스키마, 데이터, 인덱스 생성	benchmark	tpcds_create()
		TPC-DS 쿼리 수행 1~99번	benchmark	tpcds_query()

[표 10] 테스트 API 기능 리스트

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 자동화 도구 및 시각화 도구에 대한 요구사항을 분석하여 아래와 같이 프레임워크 개발에 필요한 기능 리스트를 도출하였음

대분류	중분류	소분류	설 명
자동화 도구	다국어 지원	–	한국어, 중국어, 일본어를 테스트케이스에서 사용할 수 있도록 지원하는 기능
	테스트 반복 수행 지원	–	테스트 슈트 및 케이스를 지정한 횟수만큼 반복 수행하는 기능
	테스트 케이스 실행	–	최상위 슈트를 실행함으로써 모든 하위 테스트 케이스를 One-Stop으로 실행하는 기능
	테스트 실행 결과 검증	–	SQL 실행 결과를 정답 파일(테스트 오라클)과 비교하여 검증하는 기능
	메모리 누수 검출 도구 연동	–	PetaSQL이 메모리 누수 검출 도구(valgrind)와 연동된 상태에서 테스트 케이스가 실행될 수 있도록 지원 하는 기능
	치명적 에러 처리	탐지	DB 서버의 상태를 모니터링 하여 비정상 종료, 중지를 탐지하는 기능
		분석 정보 저장	치명적 결함 발생시, 분석에 필요한 정보를 별도의 공간에 저장하는 기능
		DB 초기화 기능	탐지 및 분석 정보 저장 완료 후 다음 테스트 케이스 실행을 위해 DB를 초기화하여 다시 구동하는 기능
	테스트 실행 결과 분석 정보 저장	–	검증에 실패한 테스트 케이스(FAIL) 발생 시 원인 분석이 가능하도록 분석 정보를 저장하는 기능
	진행 상황 모니터링	–	현재 실행 중인 테스트코드를 모니터링 하는 기능
시각화 도구 (웹)	테스트케이스 관리	테스트 실행 결과 요약	테스트 실행 결과 요약 정보(총 수행 건수, 성공 및 실패 건수, 수행 시간 등) 를 제공하는 기능
		실패된 테스트케이스 목록 조회	실패된 테스트케이스 목록을 조회하는 기능
		실패된 테스트케이스 검색	실패된 테스트케이스명을 검색하는 기능

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

대분류	중분류	소분류	설 명
		실패된 테스트케이스 분류	실패된 테스트케이스를 분류(미확인/결함 등)하는 기능
		실패된 테스트케이스 분석	실패된 테스트케이스 실행 결과 와 테스트오라클간의 차이점 비교 분석하는 기능
		반복 수행 결과 분석	실행 차수별로 테스트 결과를 비교할 수 있는 기능
	메모리 결함 관리	메모리 결함 목록 조회	메모리 결함 목록을 조회하는 기능
		메모리 결함 검색	메모리 결함을 조회하는 기능
		메모리 결함 분류	메모리 결함을 분류하는 기능
		메모리 결함 필터링	메모리 결함을 필터링 하는 기능
	리포트 파일 다운로드	-	리포트 파일을 다운로드 하는 기능
	저장 / 불러오기	-	-

[표 11] 품질 검증 프레임워크 기능 리스트

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

나. 빅데이터 분석용 DBMS 품질 검증 자동화 프레임워크 설계 및 개발

(1) 테스트케이스 설계 및 실행 스크립트 설계 및 개발

- 테스트케이스 설계를 용이하게 할 수 있도록 파이썬 기반의 테스트 API를 제공함으로써 테스트케이스를 쉽게 설계 하고 유지보수 할 수 있도록 지원함



[그림 11] 테스트용 API와 실행 엔진과의 관계

① 장애유형별, SQL-92 표준 기반, 품질특성별, 핵심요소기술별 테스트케이스

- 장애유형별, SQL-92 표준, 품질특성별 및 핵심요소기술별로 약 1,200여개의 테스트케이스 및 실행 스크립트를 설계함

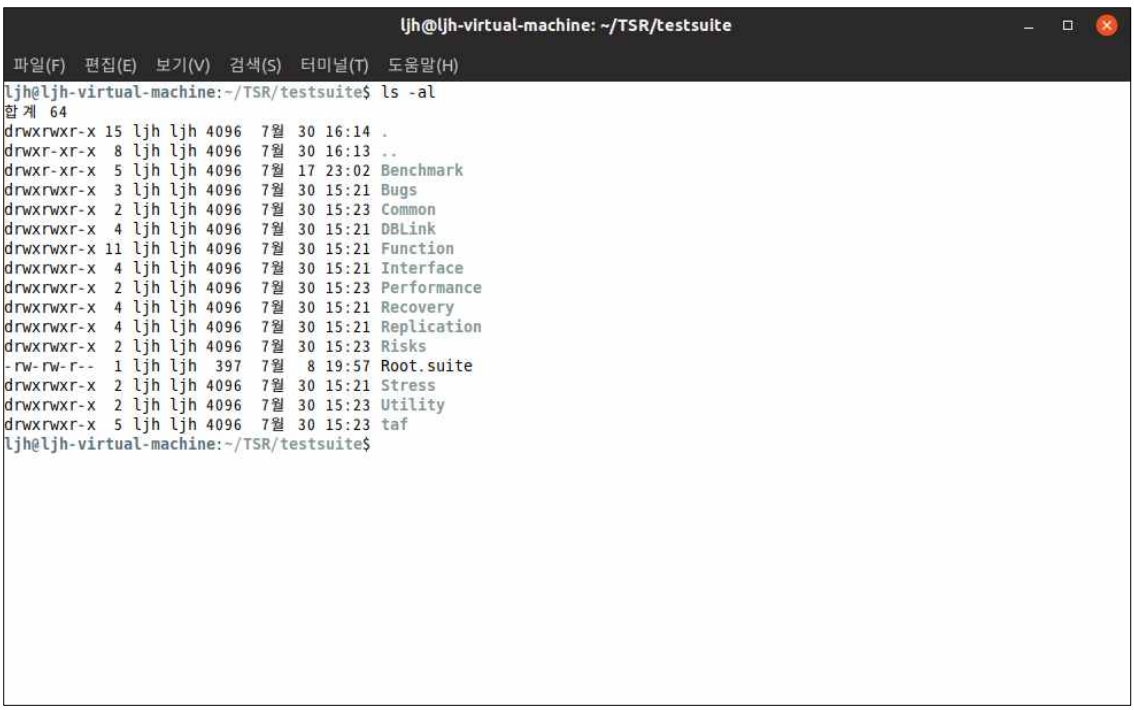
순번	테스트 슈트	설명
1	Function	SQL, 트랜잭션 등 핵심 요소 기술 검증 슈트
2	Replication	이중화 기능 검증 슈트
3	DBLink	DBLink 기능 검증 슈트
4	Interface	JDBC 및 ODBC API 기능 검증 슈트

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	테스트 슈트	설명
5	Utility	petaloader, petasql 등 유틸리티 기능 검증 슈트
6	Performance	성능 검증 슈트
7	Stress	스트레스 검증 슈트
8	Bugs	수정된 결함에 대한 회귀 슈트
9	Risks	장애 유형별 검증 슈트
10	Benchmark	SQL92, TPC-H, TPC-DS 벤치마크 검증 슈트

[표 12] 테스트슈트 목록

- 각 테스트케이스 및 실행 스크립트는 테스트슈트 레파지토리 구조화 하였으며, 테스트슈트 레파지토리는 Subversion으로 형상관리 하도록 설계함



[그림 12] 테스트슈트 디렉토리

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 약 1,200개 이상의 장애유형별, SQL92 표준, 품질특성별, 핵심요소기술별 테스트케이스를 설계하였으며, 테스트케이스는 아래 [표13]과 같음

번호	테스트케이스 스크립트	테스트 시나리오
1	createTable_basic.py	테이블이 생성되는지 확인
2	createTable_manyColumn.py	2000개 컬럼을 포함하는 테이블이 생성되는지 확인
3	createTable_manyTypes.py	다수 타입(25개)을 포함하는 테이블이 생성되는지 확인
4	insertTable_basic.py	테이블에 1만건의 데이터가 삽입되는지 확인
5	selectTable_basic.py	테이블의 데이터가 조회되는지 확인
6	projection_manyProjection.py	50개 이상의 projection 결과가 조회되는지 확인
7	projection_BigDataProjection.py	10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
8	projection_manyRows.py	1000개 이상의 결과가 조회되는지 확인
9	projection_inlineView.py	인라인 뷰를 이용하여 결과가 조회되는지 확인
10	projection_alias.py	alias를 이용하여 결과가 조회되는지 확인
11	projection_nested_function.py	nested(예: sum(avg(c1)) 되는 내장 함수를 이용하여 결과가 조회되는지 확인
12	predicate_null.py	조건절에 IS NULL, IS NOT NULL을 이용하여 결과가 조회되는지 확인
13	predicate_like.py	조건절에 Like를 이용하여 결과가 조회되는지 확인
14	predicate_comparision.py	조건절에 비교 연산자를 이용하여 결과가 조회되는지 확인
15	pridicate_bultiin_function.py	조건절에 내장 함수를 이용하여 결과가 조회되는지 확인
16	subquery_in.py	IN 구문을 이용하여 결과가 조회되는지 확인
17	subquery_exists.py	EXIST 구문을 이용하여 결과가 조회되는지 확인
18	subquery_any_some.py	ANY, SOME 구문을 이용하여 결과가 조회되는지 확인
19	subquery_all.py	ALL 구문을 이용하여 결과가 조회되는지 확인
20	subquery_many.py	2개 이상의 서브쿼리를 이용하여 결과가 조회되는지 확인
21	basicJoin.py	JOIN을 이용하여 결과가 조회되는지 확인
22	crossJoin.py	CROSS JOIN을 이용하여 결과가 조회되는지 확인
23	naturalJoin.py	NATURAL JOIN을 이용하여 결과가 조회되는지 확인
24	unionJoin.py	UNION JOIN을 이용하여 결과가 조회되는지 확인
25	outerJoin.py	OUTER JOIN을 이용하여 결과가 조회되는지 확인
26	innerJoin.py	INNER JOIN을 이용하여 결과가 조회되는지 확인
27	selectTable_groupby.py	GROUP BY 구문을 이용하여 결과가 조회되는지 확인
28	selectTable_having.py	HAVING 구문을 이용하여 결과가 조회되는지 확인
29	selectTable_orderby.py	ORDER BY 구문을 이용하여 결과가 조회되는지 확인
30	manyClinets_oneTable_lowSelectivity.py	50개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)
31	manyClinets_oneTable_highSelectivity.py	10개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
32	manyClients_manyTable_lowSelectivity.py	30개의 클라이언트를 이용하여 30개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)
33	manyClients_manyTable_highSelectivity.py	10개의 클라이언트를 이용하여 10개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
34	update_basic.py	테이블의 데이터가 갱신되는지 확인
35	update_manyColumn.py	13개의 컬럼에 대한 데이터가 갱신되는지 확인
36	update_manyRows.py	1000개의 데이터가 갱신되는지 확인
37	update_manyColumn_Rows.py	13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
38	update_character.py	문자열 최대 크기로 확장하여 갱신되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
39	manyClinets_oneTable_notConflict.py	50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
40	manyClinets_oneTable_Conflict.py	50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
41	manyClients_manyTable_notConflict.py	50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
42	manyClients_manyTable_Conflict.py	50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
43	delete_basic.py	테이블의 데이터가 삭제되는지 확인
44	delete_predicate.py	조건에 따른 데이터가 삭제되는지 확인
45	delete_manyRows.py	1000건 이상의 데이터가 삭제되는지 확인
46	delete_iteration.py	1000건의 데이터가 반복하여 삽입, 삭제되는지 확인
47	manyClinets_oneTable_notConflict.py	50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인
48	manyClinets_oneTable_Conflict.py	50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
49	manyClients_manyTable_notConflict.py	50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
50	manyClients_manyTable_Conflict.py	50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
51	constraint_null.py	NULL 제약조건이 적용되는지 확인
52	constraint_unique.py	UNIQUE 제약조건이 적용되는지 확인
53	constraint_primarykey.py	PRIMARY KEY 제약조건이 적용되는지 확인
54	constraint_default.py	DEFAULT 제약조건이 적용되는지 확인
55	reference_on_update_no_action.py	원본 테이블 레코드 업데이트 시, 아무 작업도 하지 않는지 확인
56	reference_on_update_cascade.py	원본 테이블 레코드 업데이트 시, 원본 테이블과 같은 값으로 업데이트 되는지 확인
57	reference_on_update_restrict.py	원본 테이블 레코드 업데이트 시, 해당 키를 현재 테이블에서 참조하고 있을 경우 업데이트가 실패하는지 확인
58	reference_on_update_set_null.py	원본 테이블 레코드 업데이트 시, 해당 키를 참조하는 레코드의 값이 NULL로 업데이트 되는지 확인
59	reference_on_update_set_default.py	원본 테이블 레코드 업데이트 시, 현재 레코드의 값이 디폴트값으로 업데이트 되는지 확인
60	reference_on_delete_no_action.py	원본 테이블 레코드 삭제 시, 아무 작업도 하지 않는지 확인
61	reference_on_delete_cascade.py	원본 테이블 레코드 삭제 시, 원본 테이블과 같은 값으로 업데이트되는지 확인
62	reference_on_delete_restrict.py	원본 테이블 레코드 삭제 시, 해당 키를 현재 테이블에서 참조하고 있을 경우 삭제가 실패하는지 확인
63	reference_on_delete_set_null.py	원본 테이블 레코드 삭제 시, 해당 키를 참조하는 레코드의 값이 NULL로 업데이트 되는지 확인
64	reference_on_delete_set_default.py	원본 테이블 레코드 삭제 시, 현재 레코드의 값이 디폴트값으로 업데이트 되는지 확인
65	alter_table_add_column.py	테이블에 컬럼이 추가되는지 확인
66	alter_table_add_column_after.py	테이블의 특정 칼럼 뒤에 컬럼이 추가되는지 확인
67	alter_table_add_column_before.py	테이블의 특정 칼럼 앞에 컬럼이 추가되는지 확인
68	alter_table_add_foreign_key.py	칼럼에 외래키가 추가되는지 확인
69	alter_table_add_primary_key.py	칼럼에 기본키가 추가되는지 확인
70	alter_table_default_value.py	칼럼에 기본값이 추가되는지 확인
71	alter_table_drop_column.py	테이블의 특정 컬럼이 제거되는지 확인
72	alter_table_read_only.py	읽기 전용 테이블로 변경되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
73	alter_table_read_write.py	삽입 가능 테이블로 변경되는지 확인
74	alter_table_rename.py	테이블명이 변경되는지 확인
75	alter_table_set_null.py	테이블의 특정 칼럼의 값이 Null로 설정되는지 확인
76	rangePartition_create.py	레인지 파티션 테이블을 대상으로 테이블이 생성되는지 확인
77	rangePartition_create_manyPartition.py	레인지 파티션 테이블을 대상으로 100개의 파티션을 포함한 테이블이 생성되는지 확인
78	rangePartition_insert.py	레인지 파티션 테이블을 대상으로 테이블에 1만건의 데이터가 삽입되는지 확인
79	rangePartition_insert_data_skew.py	레인지 파티션 테이블을 대상으로 데이터 쏠림 현상이 발생하는지 확인
80	rangePartition_projection_manyProjection.py	레인지 파티션 테이블을 대상으로 50개 이상의 projection 결과가 조회되는지 확인
81	rangePartition_projection_longByteProjection.py	레인지 파티션 테이블을 대상으로 10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
82	rangePartition_projection_manyRows.py	레인지 파티션 테이블을 대상으로 1000개 이상의 결과가 조회되는지 확인
83	rangePartition_predicate_null.py	레인지 파티션 테이블을 대상으로 조건절에 IS NULL, IS NOT NULL을 이용하여 결과가 조회되는지 확인
84	rangePartition_predicate_like.py	레인지 파티션 테이블을 대상으로 조건절에 Like를 이용하여 결과가 조회되는지 확인
85	rangePartition_predicate_comparision.py	레인지 파티션 테이블을 대상으로 조건절에 비교 연산자를 이용하여 결과가 조회되는지 확인
86	rangePartition_pridicate_bultiin_function.py	레인지 파티션 테이블을 대상으로 조건절에 내장 함수를 이용하여 결과가 조회되는지 확인
87	rangePartition_subquery_in.py	레인지 파티션 테이블을 대상으로 IN 구문을 이용하여 결과가 조회되는지 확인
88	rangePartition_subquery_exists.py	레인지 파티션 테이블을 대상으로 EXIST 구문을 이용하여 결과가 조회되는지 확인
89	rangePartition_subquery_any_some.py	레인지 파티션 테이블을 대상으로 ANY, SOME 구문을 이용하여 결과가 조회되는지 확인
90	rangePartition_subquery_all.py	레인지 파티션 테이블을 대상으로 ALL 구문을 이용하여 결과가 조회되는지 확인
91	rangePartition_subquery_many.py	레인지 파티션 테이블을 대상으로 2개 이상의 서브쿼리를 이용하여 결과가 조회되는지 확인
92	rangePartition_basicJoin.py	레인지 파티션 테이블을 대상으로 JOIN을 이용하여 결과가 조회되는지 확인
93	rangePartition_crossJoin.py	레인지 파티션 테이블을 대상으로 CROSS JOIN을 이용하여 결과가 조회되는지 확인
94	rangePartition_naturalJoin.py	레인지 파티션 테이블을 대상으로 NATURAL JOIN을 이용하여 결과가 조회되는지 확인
95	rangePartition_unionJoin.py	레인지 파티션 테이블을 대상으로 UNION JOIN을 이용하여 결과가 조회되는지 확인
96	rangePartition_outerJoin.py	레인지 파티션 테이블을 대상으로 OUTER JOIN을 이용하여 결과가 조회되는지 확인
97	rangePartition_innerJoin.py	레인지 파티션 테이블을 대상으로 INNER JOIN을 이용하여 결과가 조회되는지 확인
98	rangePartition_selectTable_groupby.py	레인지 파티션 테이블을 대상으로 GROUP BY 구문을 이용하여 결과가 조회되는지 확인
99	rangePartition_selectTable_having.py	레인지 파티션 테이블을 대상으로 HAVING 구문을 이용하여 결과가 조회되는지 확인
100	rangePartition_selectTable_orderby.py	레인지 파티션 테이블을 대상으로 ORDER BY 구문을 이용하여 결과가 조회되는지 확인
101	rangePartition_manyClinets_oneTable_lowSelectivity.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
102	rangePartition_manyClinets_oneTable_HighSelectivity.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
103	rangePartition_manyClients_manyTable_lowSelectivity.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)
104	rangePartition_manyClients_manyTable_HighSelectivity.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
105	rangePartition_update_basic.py	레인지 파티션 테이블을 대상으로 테이블의 데이터가 갱신되는지 확인
106	rangePartition_update_manyColumn.py	레인지 파티션 테이블을 대상으로 13개의 컬럼에 대한 데이터가 갱신되는지 확인
107	rangePartition_update_manyRows.py	레인지 파티션 테이블을 대상으로 1000개의 데이터가 갱신되는지 확인
108	rangePartition_update_manyColumn_Rows.py	레인지 파티션 테이블을 대상으로 13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
109	rangePartition_update_character.py	레인지 파티션 테이블을 대상으로 문자열 최대 크기로 확장하여 갱신되는지 확인
110	rangePartition_manyClinets_oneTable_notConflict.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
111	rangePartition_manyClinets_oneTable_Conflict.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
112	rangePartition_manyClients_manyTable_notConflict.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
113	rangePartition_manyClients_manyTable_Conflict.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
114	rangePartition_delete_basic.py	레인지 파티션 테이블을 대상으로 테이블의 데이터가 삭제되는지 확인
115	rangePartition_delete_predicate.py	레인지 파티션 테이블을 대상으로 조건에 따른 데이터가 삭제되는지 확인
116	rangePartition_delete_manyRows.py	레인지 파티션 테이블을 대상으로 1000건 이상의 데이터가 삭제되는지 확인
117	rangePartition_delete_iteration.py	레인지 파티션 테이블을 대상으로 1000건의 데이터가 반복하여 삽입, 삭제되는지 확인
118	rangePartition_manyClinets_oneTable_notConflict.py	레인지 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인
119	rangePartition_manyClinets_oneTable_Conflict.py	레인지 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
120	rangePartition_manyClients_manyTable_notConflict.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
121	rangePartition_manyClients_manyTable_Conflict.py	레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
122	range_alter_table_add_column.py	레인지 파티션 테이블을 대상으로 테이블에 컬럼이 추가되는지 확인
123	range_alter_table_add_column_after.py	레인지 파티션 테이블을 대상으로 테이블의 특정 칼럼 뒤에 컬럼이 추가되는지 확인
124	range_alter_table_add_column_before.py	레인지 파티션 테이블을 대상으로 테이블의 특정 칼럼 앞에 컬럼이 추가되는지 확인
125	range_alter_table_add_foreign_key.py	레인지 파티션 테이블을 대상으로 칼럼에 외래키가 추가되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
126	range_alter_table_add_primary_key.py	레인지 파티션 테이블을 대상으로 칼럼에 기본키가 추가되는지 확인
127	range_alter_table_default_value.py	레인지 파티션 테이블을 대상으로 칼럼에 기본값이 추가되는지 확인
128	range_alter_table_drop_column.py	레인지 파티션 테이블을 대상으로 테이블의 특정 컬럼이 제거되는지 확인
129	range_alter_table_read_only.py	레인지 파티션 테이블을 대상으로 읽기 전용 테이블로 변경되는지 확인
130	range_alter_table_read_write.py	레인지 파티션 테이블을 대상으로 삽입 가능 테이블로 변경되는지 확인
131	range_alter_table_rename.py	레인지 파티션 테이블을 대상으로 테이블명이 변경되는지 확인
132	range_alter_table_set_null.py	레인지 파티션 테이블을 대상으로 테이블의 특정 칼럼의 값이 Null로 설정되는지 확인
133	listPartition_rangePartition_create.py	리스트 파티션 테이블을 대상으로 테이블이 생성되는지 확인
134	listPartition_create_manyPartition.py	리스트 파티션 테이블을 대상으로 100개의 파티션을 포함한 테이블이 생성되는지 확인
135	listPartition_insert.py	리스트 파티션 테이블을 대상으로 테이블에 1만건의 데이터가 삽입되는지 확인
136	listPartition_insert_data_skew.py	리스트 파티션 테이블을 대상으로 데이터 쏠림 현상이 발생하는지 확인
137	listPartition_projection_manyProjection.py	리스트 파티션 테이블을 대상으로 50개 이상의 projection 결과가 조회되는지 확인
138	listPartition_projection_longByteProjection.py	리스트 파티션 테이블을 대상으로 10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
139	listPartition_projection_manyRows.py	리스트 파티션 테이블을 대상으로 1000개 이상의 결과가 조회되는지 확인
140	listPartition_predicate_null.py	리스트 파티션 테이블을 대상으로 조건절에 IS NULL, IS NOT NULL을 이용하여 결과가 조회되는지 확인
141	listPartition_predicate_like.py	리스트 파티션 테이블을 대상으로 조건절에 Like를 이용하여 결과가 조회되는지 확인
142	listPartition_predicate_comparision.py	리스트 파티션 테이블을 대상으로 조건절에 비교 연산자를 이용하여 결과가 조회되는지 확인
143	listPartition_pridicate_bultiin_function.py	리스트 파티션 테이블을 대상으로 조건절에 내장 함수를 이용하여 결과가 조회되는지 확인
144	listPartition_subquery_in.py	리스트 파티션 테이블을 대상으로 IN 구문을 이용하여 결과가 조회되는지 확인
145	listPartition_subquery_exists.py	리스트 파티션 테이블을 대상으로 EXIST 구문을 이용하여 결과가 조회되는지 확인
146	listPartition_subquery_any_some.py	리스트 파티션 테이블을 대상으로 ANY, SOME 구문을 이용하여 결과가 조회되는지 확인
147	listPartition_subquery_all.py	리스트 파티션 테이블을 대상으로 ALL 구문을 이용하여 결과가 조회되는지 확인
148	listPartition_subquery_many.py	리스트 파티션 테이블을 대상으로 2개 이상의 서브쿼리를 이용하여 결과가 조회되는지 확인
149	listPartition_basicJoin.py	리스트 파티션 테이블을 대상으로 JOIN을 이용하여 결과가 조회되는지 확인
150	listPartition_crossJoin.py	리스트 파티션 테이블을 대상으로 CROSS JOIN을 이용하여 결과가 조회되는지 확인
151	listPartition_naturalJoin.py	리스트 파티션 테이블을 대상으로 NATURAL JOIN을 이용하여 결과가 조회되는지 확인
152	listPartition_unionJoin.py	리스트 파티션 테이블을 대상으로 UNION JOIN을 이용하여 결과가 조회되는지 확인
153	listPartition_outerJoin.py	리스트 파티션 테이블을 대상으로 OUTER JOIN을 이용하여 결과가 조회되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
154	listPartition_innerJoin.py	리스트 파티션 테이블을 대상으로 INNER JOIN을 이용하여 결과가 조회되는지 확인
155	listPartition_selectTable_groupby.py	리스트 파티션 테이블을 대상으로 GROUP BY 구문을 이용하여 결과가 조회되는지 확인
156	listPartition_selectTable_having.py	리스트 파티션 테이블을 대상으로 HAVING 구문을 이용하여 결과가 조회되는지 확인
157	listPartition_selectTable_orderby.py	리스트 파티션 테이블을 대상으로 ORDER BY 구문을 이용하여 결과가 조회되는지 확인
158	listPartition_manyClinets_oneTable_lowSelectivity.py	리스트 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)
159	listPartition_manyClinets_oneTable_HighSelectivity.py	리스트 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
160	listPartition_manyClients_manyTable_lowSelectivity.py	리스트 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)
161	listPartition_manyClients_manyTable_HighSelectivity.py	리스트 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
162	listPartition_update_basic.py	리스트 파티션 테이블을 대상으로 테이블의 데이터가 갱신되는지 확인
163	listPartition_update_manyColumn.py	리스트 파티션 테이블을 대상으로 13개의 컬럼에 대한 데이터가 갱신되는지 확인
164	listPartition_update_manyRows.py	리스트 파티션 테이블을 대상으로 1000개의 데이터가 갱신되는지 확인
165	listPartition_update_manyColumn_Rows.py	리스트 파티션 테이블을 대상으로 13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
166	listPartition_update_character.py	리스트 파티션 테이블을 대상으로 문자열 최대 크기로 확장하여 갱신되는지 확인
167	listPartition_manyClinets_oneTable_notConflict.py	리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
168	listPartition_manyClinets_oneTable_Conflict.py	리스트 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
169	listPartition_manyClients_manyTable_notConflict.py	리스트 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 30개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
170	listPartition_manyClients_manyTable_Conflict.py	리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
171	listPartition_delete_basic.py	리스트 파티션 테이블을 대상으로 테이블의 데이터가 삭제되는지 확인
172	listPartition_delete_predicate.py	리스트 파티션 테이블을 대상으로 조건에 따른 데이터가 삭제되는지 확인
173	listPartition_delete_manyRows.py	리스트 파티션 테이블을 대상으로 1000건 이상의 데이터가 삭제되는지 확인
174	listPartition_delete_iteration.py	리스트 파티션 테이블을 대상으로 1000건의 데이터가 반복하여 삽입, 삭제되는지 확인
175	listPartition_manyClinets_oneTable_notConflict.py	리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인
176	listPartition_manyClinets_oneTable_Conflict.py	리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
177	listPartition_manyClients_manyTable_Conflict.py	리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
178	listPartition_manyClients_manyTable_Conflict.py	리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
179	list_alter_table_add_column.py	리스트 파티션 테이블을 대상으로 테이블에 컬럼이 추가되는지 확인
180	list_alter_table_add_column_after.py	리스트 파티션 테이블을 대상으로 테이블의 특정 칼럼 뒤에 컬럼이 추가되는지 확인
181	list_alter_table_add_column_before.py	리스트 파티션 테이블을 대상으로 테이블의 특정 칼럼 앞에 컬럼이 추가되는지 확인
182	list_alter_table_add_foreign_key.py	리스트 파티션 테이블을 대상으로 칼럼에 외래키가 추가되는지 확인
183	list_alter_table_add_primary_key.py	리스트 파티션 테이블을 대상으로 칼럼에 기본키가 추가되는지 확인
184	list_alter_table_default_value.py	리스트 파티션 테이블을 대상으로 칼럼에 기본값이 추가되는지 확인
185	list_alter_table_drop_column.py	리스트 파티션 테이블을 대상으로 테이블의 특정 컬럼이 제거되는지 확인
186	list_alter_table_read_only.py	리스트 파티션 테이블을 대상으로 읽기 전용 테이블로 변경되는지 확인
187	list_alter_table_read_write.py	리스트 파티션 테이블을 대상으로 삽입 가능 테이블로 변경되는지 확인
188	list_alter_table_rename.py	리스트 파티션 테이블을 대상으로 테이블명이 변경되는지 확인
189	list_alter_table_set_null.py	리스트 파티션 테이블을 대상으로 테이블의 특정 칼럼의 값이 NULL로 설정되는지 확인
190	hashPartition_create.py	해시 파티션 테이블을 대상으로 테이블이 생성되는지 확인
191	hashPartition_create_manyPartition.py	해시 파티션 테이블을 대상으로 100개의 파티션을 포함한 테이블이 생성되는지 확인
192	hashPartition_insert.py	해시 파티션 테이블을 대상으로 테이블에 1만건의 데이터가 삽입되는지 확인
193	hashPartition_insert_data_skew.py	해시 파티션 테이블을 대상으로 데이터 쏠림 현상이 발생하는지 확인
194	hashPartition_projection_manyProjection.py	해시 파티션 테이블을 대상으로 50개 이상의 projection 결과가 조회되는지 확인
195	hashPartition_selectTable_LongByteProjection.py	해시 파티션 테이블을 대상으로 10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
196	hashPartition_projection_manyRows.py	해시 파티션 테이블을 대상으로 1000개 이상의 결과가 조회되는지 확인
197	hashPartition_predicate_null.py	해시 파티션 테이블을 대상으로 조건절에 IS NULL, IS NOT NULL을 이용하여 결과가 조회되는지 확인
198	hashPartition_predicate_like.py	해시 파티션 테이블을 대상으로 조건절에 Like를 이용하여 결과가 조회되는지 확인
199	hashPartition_predicate_comparision.py	해시 파티션 테이블을 대상으로 조건절에 비교 연산자를 이용하여 결과가 조회되는지 확인
200	hashPartition_pridicate_bultiin_function.py	해시 파티션 테이블을 대상으로 조건절에 내장 함수를 이용하여 결과가 조회되는지 확인
201	hashPartition_subquery_in.py	해시 파티션 테이블을 대상으로 IN 구문을 이용하여 결과가 조회되는지 확인
202	hashPartition_subquery_exists.py	해시 파티션 테이블을 대상으로 EXIST 구문을 이용하여 결과가 조회되는지 확인
203	hashPartition_subquery_any_some.py	해시 파티션 테이블을 대상으로 ANY, SOME 구문을 이용하여 결과가 조회되는지 확인
204	hashPartition_subquery_all.py	해시 파티션 테이블을 대상으로 ALL 구문을 이용하여 결과가 조회되는지 확인
205	hashPartition_subquery_many.py	해시 파티션 테이블을 대상으로 2개 이상의 서브쿼리를 이용하여 결과가 조회되는지 확인
206	hashPartition_basicJoin.py	해시 파티션 테이블을 대상으로 JOIN을 이용하여 결과가 조회되는지 확인
207	hashPartition_crossJoin.py	해시 파티션 테이블을 대상으로 CROSS JOIN을 이용하여 결과가 조회되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
208	hashPartition_naturalJoin.py	해시 파티션 테이블을 대상으로 NATURAL JOIN을 이용하여 결과가 조회되는지 확인
209	hashPartition_unionJoin.py	해시 파티션 테이블을 대상으로 UNION JOIN을 이용하여 결과가 조회되는지 확인
210	hashPartition_outerJoin.py	해시 파티션 테이블을 대상으로 OUTER JOIN을 이용하여 결과가 조회되는지 확인
211	hashPartition_innerJoin.py	해시 파티션 테이블을 대상으로 INNER JOIN을 이용하여 결과가 조회되는지 확인
212	hashPartition_selectTable_groupby.py	해시 파티션 테이블을 대상으로 GROUP BY 구문을 이용하여 결과가 조회되는지 확인
213	hashPartition_selectTable_having.py	해시 파티션 테이블을 대상으로 HAVING 구문을 이용하여 결과가 조회되는지 확인
214	hashPartition_selectTable_orderby.py	해시 파티션 테이블을 대상으로 ORDER BY 구문을 이용하여 결과가 조회되는지 확인
215	hashPartition_manyClinets_oneTable_lowSelectivity.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)
216	hashPartition_manyClinets_oneTable_HighSelectivity.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
217	hashPartition_manyClients_manyTable_lowSelectivity.py	해시 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 낮음)
218	hashPartition_manyClients_manyTable_HighSelectivity.py	해시 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에서 동시에 결과가 조회되는지 확인(선택도 높음)
219	hashPartition_update_basic.py	해시 파티션 테이블을 대상으로 테이블의 데이터가 갱신되는지 확인
220	hashPartition_update_manyColumn.py	해시 파티션 테이블을 대상으로 13개의 컬럼에 대한 데이터가 갱신되는지 확인
221	hashPartition_update_manyRows.py	해시 파티션 테이블을 대상으로 1000개의 데이터가 갱신되는지 확인
222	hashPartition_update_manyColumn_Rows.py	해시 파티션 테이블을 대상으로 13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
223	hashPartition_update_character.py	해시 파티션 테이블을 대상으로 문자열 최대 크기로 확장하여 갱신되는지 확인
224	hashPartition_manyClinets_oneTable_notConflict.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
225	hashPartition_manyClinets_oneTable_Conflict.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
226	hashPartition_manyClients_manyTable_notConflict.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
227	hashPartition_manyClients_manyTable_Conflict.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
228	hashPartition_delete_basic.py	해시 파티션 테이블을 대상으로 테이블의 데이터가 삭제되는지 확인
229	hashPartition_delete_predicate.py	해시 파티션 테이블을 대상으로 조건에 따른 데이터가 삭제되는지 확인
230	hashPartition_delete_manyRows.py	해시 파티션 테이블을 대상으로 1000건 이상의 데이터가 삭제되는지 확인
231	hashPartition_delete_iteration.py	해시 파티션 테이블을 대상으로 1000건의 데이터가 반복하여 삽입, 삭제되는지 확인
232	hashPartition_manyClinets_oneTable_notConflict.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인
233	hashPartition_manyClinets_oneTable_Conflict.py	해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
234	hashPartition_manyClients_manyTable_notConflict.py	해시 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 30개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
235	hashPartition_manyClients_manyTable_Conflict.py	해시 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 30개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
236	hash_alter_table_add_column.py	해시 파티션 테이블을 대상으로 테이블에 컬럼이 추가되는지 확인
237	hash_alter_table_add_column_after.py	해시 파티션 테이블을 대상으로 테이블의 특정 칼럼 뒤에 컬럼이 추가되는지 확인
238	hash_alter_table_add_column_before.py	해시 파티션 테이블을 대상으로 테이블의 특정 칼럼 앞에 컬럼이 추가되는지 확인
239	hash_alter_table_add_foreign_key.py	해시 파티션 테이블을 대상으로 칼럼에 외래키가 추가되는지 확인
240	hash_alter_table_add_primary_key.py	해시 파티션 테이블을 대상으로 칼럼에 기본키가 추가되는지 확인
241	hash_alter_table_default_value.py	해시 파티션 테이블을 대상으로 칼럼에 기본값이 추가되는지 확인
242	hash_alter_table_drop_column.py	해시 파티션 테이블을 대상으로 테이블의 특정 컬럼이 제거되는지 확인
243	hash_alter_table_read_only.py	해시 파티션 테이블을 대상으로 읽기 전용 테이블로 변경되는지 확인
244	hash_alter_table_read_write.py	해시 파티션 테이블을 대상으로 삽입 가능 테이블로 변경되는지 확인
245	hash_alter_table_rename.py	해시 파티션 테이블을 대상으로 테이블명이 변경되는지 확인
246	hash_alter_table_set_null.py	해시 파티션 테이블을 대상으로 테이블의 특정 칼럼의 값이 Null로 설정되는지 확인
247	temp_createTable.py	Temporary 테이블이 생성되는지 확인
248	temp_insertTable.py	Temporary 테이블에 데이터가 삽입되는지 확인
249	temp_selectTable.py	Temporary 테이블의 데이터가 조회되는지 확인
250	temp_updateTable.py	Temporary 테이블의 데이터가 변경되는지 확인
251	temp_deleteTable.py	Temporary 테이블의 데이터가 삭제되는지 확인
252	temp_tableConstraint.py	Temporary 테이블에 제약사항이 적용되는지 확인
253	basic_hashPartitionTableJoin.py	Basic Table 과 HashPartition Table이 조인되는지 확인
254	basic_listPartitionTableJoin.py	Basic Table 과 ListPartition Table이 조인되는지 확인
255	basic_rangePartitionTableJoin.py	Basic Table 과 RangePartition Table이 조인되는지 확인
256	basic_tempTableJoin.py	Basic Table 과 Temp Table이 조인되는지 확인
257	temp_hashpartitionTableJoin.py	Temp Table 과 HashPartition Table이 조인되는지 확인
258	temp_listPartitionTableJoin.py	Temp Table 과 ListPartition Table이 조인되는지 확인
259	temp_rangePartitionTableJoin.py	Temp Table 과 RangePartition Table이 조인되는지 확인
260	hash_listPartitionTableJoin.py	HashPartition Table 과 ListPartition Table이 조인되는지 확인
261	hash_rangePartitionTableJoin.py	HashPartition Table 과 RangePartition Table이 조인되는지 확인
262	range_listPartitionTableJoin.py	RangePartition Table 과 ListPartition Table이 조인되는지 확인
263	basicTable_index_basic.py	테이블에 인덱스가 생성되는지 확인
264	basicTable_index_many_column.py	다수의 인덱스 컬럼이 생성되는지 확인
265	basicTable_index_many_for_one_table.py	하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
266	basicTable_index_many_split.py	인덱스 split이 발생하는지 확인
267	basicTable_index_data_skew.py	인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
268	basicTable_index_many_dml.py	인덱스 생성시, 반복하여 DML이 수행되는지 확인
269	basicTable_index_ddl.py	반복하여 인덱스가 생성, 제거되는지 확인
270	basicTable_index_bigdata.py	빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
271	basicTable_index_manyClients.py	다수 클라이언트 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
272	basicTable_index_bad_clusteringFactor.py	클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
273	rangePartition_index_basic.py	레인지 파티션 테이블을 대상으로 테이블에 인덱스가 생성되는지 확인
274	rangePartition_index_many_column.py	레인지 파티션 테이블을 대상으로 다수의 인덱스 컬럼이 생성되는지 확인
275	rangePartition_index_many_for_one_table.py	레인지 파티션 테이블을 대상으로 하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
276	rangePartition_RangePartition_index_many_split.py	레인지 파티션 테이블을 대상으로 인덱스 split이 발생하는지 확인
277	rangePartition_index_data_skew.py	레인지 파티션 테이블을 대상으로 인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
278	rangePartition_index_many_dml.py	레인지 파티션 테이블을 대상으로 인덱스 생성시, 반복하여 DML이 수행되는지 확인
279	rangePartition_index_ddl.py	레인지 파티션 테이블을 대상으로 반복하여 인덱스가 생성, 제거되는지 확인
280	rangePartition_index_bigdata.py	레인지 파티션 테이블을 대상으로 빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
281	rangePartition_index_bad_clusteringFactor.py	레인지 파티션 테이블을 대상으로 클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
282	listPartition_index_basic.py	리스트 파티션 테이블을 대상으로 테이블에 인덱스가 생성되는지 확인
283	listPartition_index_many_column.py	리스트 파티션 테이블을 대상으로 다수의 인덱스 컬럼이 생성되는지 확인
284	listPartition_index_many_for_one_table.py	리스트 파티션 테이블을 대상으로 하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
285	listPartition_index_many_split.py	리스트 파티션 테이블을 대상으로 인덱스 split이 발생하는지 확인
286	listPartition_index_data_skew.py	리스트 파티션 테이블을 대상으로 인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
287	listPartition_index_many_dml.py	리스트 파티션 테이블을 대상으로 인덱스 생성시, 반복하여 DML이 수행되는지 확인
288	listPartition_index_ddl.py	리스트 파티션 테이블을 대상으로 반복하여 인덱스가 생성, 제거되는지 확인
289	listPartition_index_bigdata.py	리스트 파티션 테이블을 대상으로 빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
290	listPartition_index_bad_clusteringFactor.py	리스트 파티션 테이블을 대상으로 클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
291	hashPartition_index_basic.py	해시 파티션 테이블을 대상으로 테이블에 인덱스가 생성되는지 확인
292	hashPartition_index_many_column.py	해시 파티션 테이블을 대상으로 다수의 인덱스 컬럼이 생성되는지 확인
293	hashPartition_index_many_for_one_table.py	해시 파티션 테이블을 대상으로 하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
294	hashPartition_index_many_split.py	해시 파티션 테이블을 대상으로 인덱스 split이 발생하는지 확인
295	hashPartition_index_data_skew.py	해시 파티션 테이블을 대상으로 인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
296	hashPartition_index_many_dml.py	해시 파티션 테이블을 대상으로 인덱스 생성시, 반복하여 DML이 수행되는지 확인
297	hashPartition_index_ddl.py	해시 파티션 테이블을 대상으로 반복하여 인덱스가 생성, 제거되는지 확인
298	hashPartition_index_bigdata.py	해시 파티션 테이블을 대상으로 빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
299	hashPartition_index_bad_clusteringFactor.py	해시 파티션 테이블을 대상으로 클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
300	basicTable_index_basic.py	테이블에 인덱스가 생성되는지 확인
301	basicTable_index_many_column.py	다수의 인덱스 컬럼이 생성되는지 확인
302	basicTable_index_many_for_one_table.py	하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
303	basicTable_index_many_split.py	인덱스 split이 발생하는지 확인
304	basicTable_index_data_skew.py	인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
305	basicTable_index_many_dml.py	인덱스 생성시, 반복하여 DML이 수행되는지 확인
306	basicTable_index_ddl.py	반복하여 인덱스가 생성, 제거되는지 확인
307	basicTable_index_bigdata.py	빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
308	basicTable_index_manyClients.py	다수 클라이언트 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
309	basicTable_index_bad_clusteringFactor.py	클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
310	rangePartition_index_basic.py	레인지 파티션 테이블을 대상으로 테이블에 인덱스가 생성되는지 확인
311	rangePartition_index_many_column.py	레인지 파티션 테이블을 대상으로 다수의 인덱스 컬럼이 생성되는지 확인
312	rangePartition_index_many_split.py	레인지 파티션 테이블을 대상으로 인덱스 split이 발생하는지 확인
313	rangePartition_index_many_for_one_table.py	레인지 파티션 테이블을 대상으로 하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
314	rangePartition_index_data_skew.py	레인지 파티션 테이블을 대상으로 인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
315	rangePartition_index_many_dml.py	레인지 파티션 테이블을 대상으로 인덱스 생성시, 반복하여 DML이 수행되는지 확인
316	rangePartition_index_ddl.py	레인지 파티션 테이블을 대상으로 반복하여 인덱스가 생성, 제거되는지 확인
317	rangePartition_index_bigdata.py	레인지 파티션 테이블을 대상으로 빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
318	rangePartition_index_bad_clusteringFactor.py	레인지 파티션 테이블을 대상으로 클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
319	listPartition_index_basic.py	리스트 파티션 테이블을 대상으로 테이블에 인덱스가 생성되는지 확인
320	listPartition_index_many_column.py	리스트 파티션 테이블을 대상으로 다수의 인덱스 컬럼이 생성되는지 확인
321	listPartition_index_many_split.py	리스트 파티션 테이블을 대상으로 인덱스 split이 발생하는지 확인
322	listPartition_index_many_for_one_table.py	리스트 파티션 테이블을 대상으로 하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
323	listPartition_index_data_skew.py	리스트 파티션 테이블을 대상으로 인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
324	listPartition_index_many_dml.py	리스트 파티션 테이블을 대상으로 인덱스 생성시, 반복하여 DML이 수행되는지 확인
325	listPartition_index_ddl.py	리스트 파티션 테이블을 대상으로 반복하여 인덱스가 생성, 제거되는지 확인
326	listPartition_index_bigdata.py	리스트 파티션 테이블을 대상으로 빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
327	listPartition_index_bad_clusteringFactor.py	리스트 파티션 테이블을 대상으로 클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
328	hashPartition_index_basic.py	해시 파티션 테이블을 대상으로 테이블에 인덱스가 생성되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
329	hashPartition_index_many_column.py	해시 파티션 테이블을 대상으로 다수의 인덱스 컬럼이 생성되는지 확인
330	hashPartition_index_many_split.py	해시 파티션 테이블을 대상으로 인덱스 split이 발생하는지 확인
331	hashPartition_index_many_for_one_table.py	해시 파티션 테이블을 대상으로 하나의 테이블에 다수(5개 이상)의 인덱스 생성시, 인덱스가 적용되어 결과가 조회되는지 확인
332	hashPartition_index_data_skew.py	해시 파티션 테이블을 대상으로 인덱스 생성시, 데이터 쏠림 현상이 발생하는지 확인
333	hashPartition_index_many_dml.py	해시 파티션 테이블을 대상으로 인덱스 생성시, 반복하여 DML이 수행되는지 확인
334	hashPartition_index_ddl.py	해시 파티션 테이블을 대상으로 반복하여 인덱스가 생성, 제거되는지 확인
335	hashPartition_index_bigdata.py	해시 파티션 테이블을 대상으로 빅데이터 환경에서 인덱스가 적용되어 결과가 조회되는지 확인
336	hashPartition_index_bad_clusteringFactor.py	해시 파티션 테이블을 대상으로 클러스터링 팩터가 좋지 않은 상황에서, 인덱스가 적용되어 결과가 조회되는지 확인
337	commit_basic.py	DDL 쿼리문에 대한 commit이 적용되는지 확인
338	rollback_basic.py	DDL 쿼리문에 대한 rollback이 적용되는지 확인
339	rollback_bigdata.py	빅데이터 환경에서 rollback이 적용되는지 확인
340	savepoint_basic.py	savepoint로 rollback 되는지 확인
341	savepoint_many.py	20개 이상의 savepoint 환경에서 rollback 되는지 확인
342	readCommitted.py	commit이 완료된 데이터만 조회되는지 확인
343	serializable.py	모든 명령이 순서대로 처리되는지 확인
344	character_char_varchar.py	char, varchar 데이터 타입이 제공되는지 확인
345	character_text_string.py	text, string 데이터 타입이 제공되는지 확인
346	numeric.py	numeric 데이터 타입이 제공되는지 확인
347	binary_clob.py	clob 데이터 타입이 제공되는지 확인
348	binary_blob.py	blob 데이터 타입이 제공되는지 확인
349	compress_lz4.py	lz4 데이터 타입이 제공되는지 확인
350	compress_snappy.py	snappy 데이터 타입이 제공되는지 확인
351	compress_lz4_snappy.py	lz4, snappy 데이터 타입이 제공되는지 확인
352	temporal.py	temporal 데이터 타입이 제공되는지 확인
353	sequence_basic.py	sequence가 생성되는지 확인
354	sequence_boundary.py	데이터 타입에 따른 squence 값의 범위를 확인
355	user_basic.py	계정이 생성되는지 확인
356	user_many.py	100명 이상의 계정이 생성되는지 확인
357	grant.py	권한 부여 기능이 적용되는지 확인
358	grant_with_option.py	부여 받은 권한을 다른 유저에게 부여 가능한지 확인
359	revoke.py	권한 해제 기능이 적용되는지 확인
360	createProcedure_basic.py	프로시저가 생성되는지 확인
361	createProcedure_invalid_syntax.py	유효하지 않은 문법 사용 시, 프로시저가 생성되는지 확인
362	createProcedure_valid_syntax.py	유효한 문법 사용 시, 프로시저가 생성되는지 확인
363	createProcedure_value_boundary.py	254 byte의 프로시저명으로 생성되는지 확인
364	createProcedure_object_boundary.py	245개의 객체를 포함하는 프로시저가 생성되는지 확인
365	executeProcedure_basic.py	프로시저가 실행되는지 확인
366	executeProcedure_variable.py	프로시저에 변수가 사용 가능한지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
367	executeProcedure_control.py	프로시저에 제어 구문이 적용되는지 확인
368	executeProcedure_insert.py	프로시저를 이용하여 삽입되는지 확인
369	executeProcedure_update.py	프로시저를 이용하여 갱신되는지 확인
370	createFunction_basic.py	함수가 생성되는지 확인
371	createFunction_invalid_syntax.py	유효하지 않은 문법 사용 시, 함수가 생성되는지 확인
372	createFunction_valid_syntax.py	유효한 문법 사용 시, 함수가 생성되는지 확인
373	createFunction_value_boundary.py	긴 길이(254byte)의 함수명으로 함수가 생성되는지 확인
374	createFunction_object_boundary.py	245개의 객체를 포함하는 함수가 생성되는지 확인
375	executeFunction_basic.py	함수가 실행되는지 확인
376	executeFunction_dataType.py	함수에 데이터 타입이 사용 가능한지 확인
377	executeFunction_statement.py	함수 내에 사용 가능한 쿼리문이 실행되는지 확인
378	executeFunction_control.py	함수에 제어 구문이 적용되는지 확인
379	ifthenelse.py	일반 테이블을 대상으로 ifthenelse 함수가 실행되는지 확인
380	avg.py	일반 테이블을 대상으로 avg 함수가 실행되는지 확인
381	count.py	일반 테이블을 대상으로 count 함수가 실행되는지 확인
382	max.py	일반 테이블을 대상으로 max 함수가 실행되는지 확인
383	min.py	일반 테이블을 대상으로 min 함수가 실행되는지 확인
384	prod.py	일반 테이블을 대상으로 prod 함수가 실행되는지 확인
385	quantile.py	일반 테이블을 대상으로 quantile 함수가 실행되는지 확인
386	stddev_samp.py	일반 테이블을 대상으로 stddev_samp 함수가 실행되는지 확인
387	sum.py	일반 테이블을 대상으로 sum 함수가 실행되는지 확인
388	var_pop.py	일반 테이블을 대상으로 var_pop 함수가 실행되는지 확인
389	var_samp.py	일반 테이블을 대상으로 var_samp 함수가 실행되는지 확인
390	sign.py	일반 테이블을 대상으로 sign 함수가 실행되는지 확인
391	mod.py	일반 테이블을 대상으로 mod 함수가 실행되는지 확인
392	abs.py	일반 테이블을 대상으로 abs 함수가 실행되는지 확인
393	sin.py	일반 테이블을 대상으로 sin 함수가 실행되는지 확인
394	cos.py	일반 테이블을 대상으로 cos 함수가 실행되는지 확인
395	tan.py	일반 테이블을 대상으로 tan 함수가 실행되는지 확인
396	acos.py	일반 테이블을 대상으로 acos 함수가 실행되는지 확인
397	asin.py	일반 테이블을 대상으로 asin 함수가 실행되는지 확인
398	atan.py	일반 테이블을 대상으로 atan 함수가 실행되는지 확인
399	sinh.py	일반 테이블을 대상으로 sinh 함수가 실행되는지 확인
400	cosh.py	일반 테이블을 대상으로 cosh 함수가 실행되는지 확인
401	tanh.py	일반 테이블을 대상으로 tanh 함수가 실행되는지 확인
402	cot.py	일반 테이블을 대상으로 cot 함수가 실행되는지 확인
403	ceiling.py	일반 테이블을 대상으로 ceiling 함수가 실행되는지 확인
404	rand.py	일반 테이블을 대상으로 rand 함수가 실행되는지 확인
405	pi.py	일반 테이블을 대상으로 pi 함수가 실행되는지 확인
406	log10.py	일반 테이블을 대상으로 log10 함수가 실행되는지 확인
407	exp.py	일반 테이블을 대상으로 exp 함수가 실행되는지 확인
408	floor.py	일반 테이블을 대상으로 floor 함수가 실행되는지 확인
409	sqrt.py	일반 테이블을 대상으로 sqrt 함수가 실행되는지 확인
410	power.py	일반 테이블을 대상으로 power 함수가 실행되는지 확인
411	log.py	일반 테이블을 대상으로 log 함수가 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
412	radians.py	일반 테이블을 대상으로 radians 함수가 실행되는지 확인
413	degrees.py	일반 테이블을 대상으로 degrees 함수가 실행되는지 확인
414	rank_over.py	일반 테이블을 대상으로 rank_over 함수가 실행되는지 확인
415	dense_rank_over.py	일반 테이블을 대상으로 dense_rank_over 함수가 실행되는지 확인
416	row_number_over.py	일반 테이블을 대상으로 row_number_over 함수가 실행되는지 확인
417	median.py	일반 테이블을 대상으로 median 함수가 실행되는지 확인
418	locate.py	일반 테이블을 대상으로 locate 함수가 실행되는지 확인
419	upper.py	일반 테이블을 대상으로 upper 함수가 실행되는지 확인
420	lower.py	일반 테이블을 대상으로 lower 함수가 실행되는지 확인
421	replace.py	일반 테이블을 대상으로 replace 함수가 실행되는지 확인
422	translate.py	일반 테이블을 대상으로 translate 함수가 실행되는지 확인
423	repeat.py	일반 테이블을 대상으로 repeat 함수가 실행되는지 확인
424	space.py	일반 테이블을 대상으로 space 함수가 실행되는지 확인
425	length.py	일반 테이블을 대상으로 length 함수가 실행되는지 확인
426	octet_length.py	일반 테이블을 대상으로 octet_length 함수가 실행되는지 확인
427	right.py	일반 테이블을 대상으로 right 함수가 실행되는지 확인
428	left.py	일반 테이블을 대상으로 left 함수가 실행되는지 확인
429	concat.py	일반 테이블을 대상으로 concat 함수가 실행되는지 확인
430	substring.py	일반 테이블을 대상으로 substring 함수가 실행되는지 확인
431	code.py	일반 테이블을 대상으로 code 함수가 실행되는지 확인
432	ascii.py	일반 테이블을 대상으로 ascii 함수가 실행되는지 확인
433	truncate.py	일반 테이블을 대상으로 truncate 함수가 실행되는지 확인
434	lpad.py	일반 테이블을 대상으로 lpad 함수가 실행되는지 확인
435	rpad.py	일반 테이블을 대상으로 rpad 함수가 실행되는지 확인
436	insert.py	일반 테이블을 대상으로 insert 함수가 실행되는지 확인
437	trim.py	일반 테이블을 대상으로 trim 함수가 실행되는지 확인
438	ltrim.py	일반 테이블을 대상으로 ltrim 함수가 실행되는지 확인
439	rtrim.py	일반 테이블을 대상으로 rtrim 함수가 실행되는지 확인
440	isnull.py	일반 테이블을 대상으로 isnull 함수가 실행되는지 확인
441	levenshtein.py	일반 테이블을 대상으로 levenshtein 함수가 실행되는지 확인
442	soundex.py	일반 테이블을 대상으로 soundex 함수가 실행되는지 확인
443	getbasename.py	일반 테이블을 대상으로 getbasename 함수가 실행되는지 확인
444	getextension.py	일반 테이블을 대상으로 getextension 함수가 실행되는지 확인
445	getdomain.py	일반 테이블을 대상으로 getdomain 함수가 실행되는지 확인
446	getfile.py	일반 테이블을 대상으로 getfile 함수가 실행되는지 확인
447	gethost.py	일반 테이블을 대상으로 gethost 함수가 실행되는지 확인
448	getport.py	일반 테이블을 대상으로 getport 함수가 실행되는지 확인
449	getprotocol.py	일반 테이블을 대상으로 getprotocol 함수가 실행되는지 확인
450	getquery.py	일반 테이블을 대상으로 getquery 함수가 실행되는지 확인
451	getroboturl.py	일반 테이블을 대상으로 getroboturl 함수가 실행되는지 확인
452	getanchor.py	일반 테이블을 대상으로 getanchor 함수가 실행되는지 확인
453	getcontext.py	일반 테이블을 대상으로 getcontext 함수가 실행되는지 확인
454	getcontent.py	일반 테이블을 대상으로 getcontent 함수가 실행되는지 확인
455	current_date.py	일반 테이블을 대상으로 current_date 함수가 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
456	current_time.py	일반 테이블을 대상으로 current_time 함수가 실행되는지 확인
457	current_timestamp.py	일반 테이블을 대상으로 current_timestamp 함수가 실행되는지 확인
458	dayofmonth.py	일반 테이블을 대상으로 dayofmonth 함수가 실행되는지 확인
459	dayofweek.py	일반 테이블을 대상으로 dayofweek 함수가 실행되는지 확인
460	dayofyear.py	일반 테이블을 대상으로 dayofyear 함수가 실행되는지 확인
461	weekofyear.py	일반 테이블을 대상으로 weekofyear 함수가 실행되는지 확인
462	from_unixtime.py	일반 테이블을 대상으로 from_unixtime 함수가 실행되는지 확인
463	unix_timestamp.py	일반 테이블을 대상으로 unix_timestamp 함수가 실행되는지 확인
464	local_timezone.py	일반 테이블을 대상으로 local_timezone 함수가 실행되는지 확인
465	quarter.py	일반 테이블을 대상으로 quarter 함수가 실행되는지 확인
466	date_format.py	일반 테이블을 대상으로 date_format 함수가 실행되는지 확인
467	date_to_str.py	일반 테이블을 대상으로 date_to_str 함수가 실행되는지 확인
468	str_to_date.py	일반 테이블을 대상으로 str_to_date 함수가 실행되는지 확인
469	str_to_timestamp.py	일반 테이블을 대상으로 str_to_timestamp 함수가 실행되는지 확인
470	timestamp_to_str.py	일반 테이블을 대상으로 timestamp_to_str 함수가 실행되는지 확인
471	bbp.py	일반 테이블을 대상으로 bbp 함수가 실행되는지 확인
472	storage.py	일반 테이블을 대상으로 storage 함수가 실행되는지 확인
473	clientid.py	일반 테이블을 대상으로 clientid 함수가 실행되는지 확인
474	clients.py	일반 테이블을 대상으로 clients 함수가 실행되는지 확인
475	sessions.py	일반 테이블을 대상으로 sessions 함수가 실행되는지 확인
476	environment.py	일반 테이블을 대상으로 environment 함수가 실행되는지 확인
477	var.py	일반 테이블을 대상으로 var 함수가 실행되는지 확인
478	hash_ifthenelse.py	해시 파티션 테이블을 대상으로 ifthenelse 함수가 실행되는지 확인
479	hash_avg.py	해시 파티션 테이블을 대상으로 avg 함수가 실행되는지 확인
480	hash_count.py	해시 파티션 테이블을 대상으로 count 함수가 실행되는지 확인
481	hash_max.py	해시 파티션 테이블을 대상으로 max 함수가 실행되는지 확인
482	hash_min.py	해시 파티션 테이블을 대상으로 min 함수가 실행되는지 확인
483	hash_prod.py	해시 파티션 테이블을 대상으로 prod 함수가 실행되는지 확인
484	hash_quantile.py	해시 파티션 테이블을 대상으로 quantile 함수가 실행되는지 확인
485	hash_stddev_samp.py	해시 파티션 테이블을 대상으로 stddev_samp 함수가 실행되는지 확인
486	hash_sum.py	해시 파티션 테이블을 대상으로 sum 함수가 실행되는지 확인
487	hash_var_pop.py	해시 파티션 테이블을 대상으로 var_pop 함수가 실행되는지 확인
488	hash_var_samp.py	해시 파티션 테이블을 대상으로 var_samp 함수가 실행되는지 확인
489	hash_sign.py	해시 파티션 테이블을 대상으로 sign 함수가 실행되는지 확인
490	hash_mod.py	해시 파티션 테이블을 대상으로 mod 함수가 실행되는지 확인
491	hash_abs.py	해시 파티션 테이블을 대상으로 abs 함수가 실행되는지 확인
492	hash_sin.py	해시 파티션 테이블을 대상으로 sin 함수가 실행되는지 확인
493	hash_cos.py	해시 파티션 테이블을 대상으로 cos 함수가 실행되는지 확인
494	hash_tan.py	해시 파티션 테이블을 대상으로 tan 함수가 실행되는지 확인
495	hash_acos.py	해시 파티션 테이블을 대상으로 acos 함수가 실행되는지 확인
496	hash_asin.py	해시 파티션 테이블을 대상으로 asin 함수가 실행되는지 확인
497	hash_atan.py	해시 파티션 테이블을 대상으로 atan 함수가 실행되는지 확인
498	hash_sinh.py	해시 파티션 테이블을 대상으로 sinh 함수가 실행되는지 확인
499	hash_cosh.py	해시 파티션 테이블을 대상으로 cosh 함수가 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
500	hash_tanh.py	해시 파티션 테이블을 대상으로 tanh 함수가 실행되는지 확인
501	hash_cot.py	해시 파티션 테이블을 대상으로 cot 함수가 실행되는지 확인
502	hash_ceiling.py	해시 파티션 테이블을 대상으로 ceiling 함수가 실행되는지 확인
503	hash_rand.py	해시 파티션 테이블을 대상으로 rand 함수가 실행되는지 확인
504	hash_log10.py	해시 파티션 테이블을 대상으로 log10 함수가 실행되는지 확인
505	hash_exp.py	해시 파티션 테이블을 대상으로 exp 함수가 실행되는지 확인
506	hash_floor.py	해시 파티션 테이블을 대상으로 floor 함수가 실행되는지 확인
507	hash_sqrt.py	해시 파티션 테이블을 대상으로 sqrt 함수가 실행되는지 확인
508	hash_power.py	해시 파티션 테이블을 대상으로 power 함수가 실행되는지 확인
509	hash_log.py	해시 파티션 테이블을 대상으로 log 함수가 실행되는지 확인
510	hash_radians.py	해시 파티션 테이블을 대상으로 radians 함수가 실행되는지 확인
511	hash_degrees.py	해시 파티션 테이블을 대상으로 degrees 함수가 실행되는지 확인
512	hash_rank_over.py	해시 파티션 테이블을 대상으로 rank_over 함수가 실행되는지 확인
513	hash_dense_rank_over.py	해시 파티션 테이블을 대상으로 dense_rank_over 함수가 실행되는지 확인
514	hash_row_number_over.py	해시 파티션 테이블을 대상으로 row_number_over 함수가 실행되는지 확인
515	hash_median.py	해시 파티션 테이블을 대상으로 median 함수가 실행되는지 확인
516	hash_locate.py	해시 파티션 테이블을 대상으로 locate 함수가 실행되는지 확인
517	hash_upper.py	해시 파티션 테이블을 대상으로 upper 함수가 실행되는지 확인
518	hash_lower.py	해시 파티션 테이블을 대상으로 lower 함수가 실행되는지 확인
519	hash_replace.py	해시 파티션 테이블을 대상으로 replace 함수가 실행되는지 확인
520	hash_translate.py	해시 파티션 테이블을 대상으로 translate 함수가 실행되는지 확인
521	hash_repeat.py	해시 파티션 테이블을 대상으로 repeat 함수가 실행되는지 확인
522	hash_space.py	해시 파티션 테이블을 대상으로 space 함수가 실행되는지 확인
523	hash_length.py	해시 파티션 테이블을 대상으로 length 함수가 실행되는지 확인
524	hash_octet_length.py	해시 파티션 테이블을 대상으로 octet_length 함수가 실행되는지 확인
525	hash_right.py	해시 파티션 테이블을 대상으로 right 함수가 실행되는지 확인
526	hash_left.py	해시 파티션 테이블을 대상으로 left 함수가 실행되는지 확인
527	hash_concat.py	해시 파티션 테이블을 대상으로 concat 함수가 실행되는지 확인
528	hash_substring.py	해시 파티션 테이블을 대상으로 substring 함수가 실행되는지 확인
529	hash_code.py	해시 파티션 테이블을 대상으로 code 함수가 실행되는지 확인
530	hash_ascii.py	해시 파티션 테이블을 대상으로 ascii 함수가 실행되는지 확인
531	hash_truncate.py	해시 파티션 테이블을 대상으로 truncate 함수가 실행되는지 확인
532	hash_lpad.py	해시 파티션 테이블을 대상으로 lpad 함수가 실행되는지 확인
533	hash_rpad.py	해시 파티션 테이블을 대상으로 rpad 함수가 실행되는지 확인
534	hash_insert.py	해시 파티션 테이블을 대상으로 insert 함수가 실행되는지 확인
535	hash_trim.py	해시 파티션 테이블을 대상으로 trim 함수가 실행되는지 확인
536	hash_ltrim.py	해시 파티션 테이블을 대상으로 ltrim 함수가 실행되는지 확인
537	hash_rtrim.py	해시 파티션 테이블을 대상으로 rtrim 함수가 실행되는지 확인
538	hash_isnull.py	해시 파티션 테이블을 대상으로 isnull 함수가 실행되는지 확인
539	hash_levenshtein.py	해시 파티션 테이블을 대상으로 levenshtein 함수가 실행되는지 확인
540	hash_soundex.py	해시 파티션 테이블을 대상으로 soundex 함수가 실행되는지 확인
541	hash_getbasename.py	해시 파티션 테이블을 대상으로 getbasename 함수가 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
542	hash_getextension.py	해시 파티션 테이블을 대상으로 getextension 함수가 실행되는지 확인
543	hash_getdomain.py	해시 파티션 테이블을 대상으로 getdomain 함수가 실행되는지 확인
544	hash_getfile.py	해시 파티션 테이블을 대상으로 getfile 함수가 실행되는지 확인
545	hash_gethost.py	해시 파티션 테이블을 대상으로 gethost 함수가 실행되는지 확인
546	hash_getport.py	해시 파티션 테이블을 대상으로 getport 함수가 실행되는지 확인
547	hash_getprotocol.py	해시 파티션 테이블을 대상으로 getprotocol 함수가 실행되는지 확인
548	hash_getquery.py	해시 파티션 테이블을 대상으로 getquery 함수가 실행되는지 확인
549	hash_getroboturl.py	해시 파티션 테이블을 대상으로 getroboturl 함수가 실행되는지 확인
550	hash_getanchor.py	해시 파티션 테이블을 대상으로 getanchor 함수가 실행되는지 확인
551	hash_getcontext.py	해시 파티션 테이블을 대상으로 getcontext 함수가 실행되는지 확인
552	hash_getcontent.py	해시 파티션 테이블을 대상으로 getcontent 함수가 실행되는지 확인
553	hash_dayofmonth.py	해시 파티션 테이블을 대상으로 dayofmonth 함수가 실행되는지 확인
554	hash_dayofweek.py	해시 파티션 테이블을 대상으로 dayofweek 함수가 실행되는지 확인
555	hash_dayofyear.py	해시 파티션 테이블을 대상으로 dayofyear 함수가 실행되는지 확인
556	hash_weekofyear.py	해시 파티션 테이블을 대상으로 weekofyear 함수가 실행되는지 확인
557	hash_from_unixtime.py	해시 파티션 테이블을 대상으로 from_unixtime 함수가 실행되는지 확인
558	hash_unix_timestamp.py	해시 파티션 테이블을 대상으로 unix_timestamp 함수가 실행되는지 확인
559	hash_quarter.py	해시 파티션 테이블을 대상으로 quarter 함수가 실행되는지 확인
560	hash_date_format.py	해시 파티션 테이블을 대상으로 date_format 함수가 실행되는지 확인
561	hash_date_to_str.py	해시 파티션 테이블을 대상으로 date_to_str 함수가 실행되는지 확인
562	hash_str_to_date.py	해시 파티션 테이블을 대상으로 str_to_date 함수가 실행되는지 확인
563	hash_str_to_timestamp.py	해시 파티션 테이블을 대상으로 str_to_timestamp 함수가 실행되는지 확인
564	hash_timestamp_to_str.py	해시 파티션 테이블을 대상으로 timestamp_to_str 함수가 실행되는지 확인
565	explain.py	explain 기능 실행 시, 쿼리 실행 계획이 mal 코드로 출력되는지 확인
566	plan.py	plan 기능 실행 시, 쿼리 실행 계획이 출력되는지 확인
567	prepare.py	prepare 기능 실행 후, exec() 함수를 통해 쿼리문 실행이 가능한지 확인
568	create_view.py	테이블의 View가 생성되는지 확인
569	select_view.py	테이블의 View 데이터가 조회되는지 확인
570	drop_view.py	테이블의 View가 삭제되는지 확인
571	hash_create_view.py	HashPartition 테이블의 View가 생성되는지 확인
572	hash_select_view.py	HashPartition 테이블의 View 데이터가 조회되는지 확인
573	hash_drop_view.py	HashPartition 테이블의 View가 삭제되는지 확인
574	list_create_view.py	ListPartition 테이블의 View가 생성되는지 확인
575	list_select_view.py	ListPartition 테이블의 View 데이터가 조회되는지 확인
576	list_drop_view.py	ListPartition 테이블의 View가 삭제되는지 확인
577	range_create_view.py	RangePartition 테이블의 View가 생성되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
578	range_select_view.py	RangePartition 테이블의 View 데이터가 조회되는지 확인
579	range_drop_view.py	RangePartition 테이블의 View가 삭제되는지 확인
580	replication_initialization.py	이중화 설정 기능이 동작하는지 확인
581	replication_finalization.py	이중화 종료 기능이 오류 없이 동작하는지 확인
582	createTable_basic.py	이중화를 설정한 상태에서 테이블이 생성되는지 확인
583	createTable_manyColumn.py	이중화를 설정한 상태에서 2000개 컬럼을 포함하는 테이블이 생성되는지 확인
584	createTable_manyTypes.py	이중화를 설정한 상태에서 다수 타입(25개)을 포함하는 테이블이 생성되는지 확인
585	insertTable_basic.py	이중화를 설정한 상태에서 테이블에 1만건의 데이터가 삽입되는지 확인
586	update_basic.py	이중화를 설정한 상태에서 테이블의 데이터가 갱신되는지 확인
587	update_manyColumn.py	이중화를 설정한 상태에서 13개의 컬럼에 대한 데이터가 갱신되는지 확인
588	update_manyRows.py	이중화를 설정한 상태에서 1000개의 데이터가 갱신되는지 확인
589	update_manyColumn_Rows.py	이중화를 설정한 상태에서 13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
590	update_character.py	이중화를 설정한 상태에서 문자열 최대 크기로 확장하여 갱신되는지 확인
591	manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
592	manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
593	manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
594	manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
595	delete_basic.py	이중화를 설정한 상태에서 테이블의 데이터가 삭제되는지 확인
596	delete_predicate.py	이중화를 설정한 상태에서 조건에 따른 데이터가 삭제되는지 확인
597	delete_manyRows.py	이중화를 설정한 상태에서 1000건 이상의 데이터가 삭제되는지 확인
598	delete_iteration.py	이중화를 설정한 상태에서 1000건의 데이터가 반복하여 삽입, 삭제되는지 확인
599	manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인
600	manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
601	manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
602	manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
603	constraint_null.py	이중화를 설정한 상태에서 NULL 제약조건이 적용되는지 확인
604	constraint_unique.py	이중화를 설정한 상태에서 UNIQUE 제약조건이 적용되는지 확인
605	constraint_primarykey.py	이중화를 설정한 상태에서 PRIMARY KEY 제약조건이 적용되는지 확인
606	constraint_default.py	이중화를 설정한 상태에서 DEFAULT 제약조건이 적용되는지 확인
607	reference_on_update_no_action.py	이중화를 설정한 상태에서 원본 테이블 레코드 업데이트 시, 아무 작업도 하지 않는지 확인
608	reference_on_update_cascade.py	이중화를 설정한 상태에서 원본 테이블 레코드 업데이트 시, 원본 테이블과 같은 값으로 업데이트 되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
609	reference_on_update_restrict.py	이중화를 설정한 상태에서 원본 테이블 레코드 업데이트 시, 해당 키를 현재 테이블에서 참조하고 있을 경우 업데이트가 실패하는지 확인
610	reference_on_update_set_null.py	이중화를 설정한 상태에서 원본 테이블 레코드 업데이트 시, 해당 키를 참조하는 레코드의 값이 NULL로 업데이트 되는지 확인
611	reference_on_update_set_default.py	이중화를 설정한 상태에서 원본 테이블 레코드 업데이트 시, 현재 레코드의 값이 디폴트값으로 업데이트 되는지 확인
612	reference_on_delete_no_action.py	이중화를 설정한 상태에서 원본 테이블 레코드 삭제 시, 아무 작업도 하지 않는지 확인
613	reference_on_delete_cascade.py	이중화를 설정한 상태에서 원본 테이블 레코드 삭제 시, 원본 테이블과 같은 값으로 업데이트되는지 확인
614	reference_on_delete_restrict.py	이중화를 설정한 상태에서 원본 테이블 레코드 삭제 시, 해당 키를 현재 테이블에서 참조하고 있을 경우 삭제가 실패하는지 확인
615	reference_on_delete_set_null.py	이중화를 설정한 상태에서 원본 테이블 레코드 삭제 시, 해당 키를 참조하는 레코드의 값이 NULL로 업데이트 되는지 확인
616	reference_on_delete_set_default.py	이중화를 설정한 상태에서 원본 테이블 레코드 삭제 시, 현재 레코드의 값이 디폴트값으로 업데이트 되는지 확인
617	rangePartition_create.py	이중화를 설정한 상태에서 레인지 파티션 테이블이 생성되는지 확인
618	rangePartition_create_manyPartition.py	이중화를 설정한 상태에서 다수의 파티션을 포함한 레인지 파티션 테이블이 생성되는지 확인
619	rangePartition_insert.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 테이블에 1만건의 데이터가 삽입되는지 확인
620	rangePartition_insert_data_skew.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 데이터 쏠림 현상이 발생하는지 확인
621	rangePartition_update_basic.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 테이블의 데이터가 갱신되는지 확인
622	rangePartition_update_manyColumn.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 13개의 컬럼에 대한 데이터가 갱신되는지 확인
623	rangePartition_update_manyRows.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 1000개의 데이터가 갱신되는지 확인
624	rangePartition_update_manyColumn_Rows.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
625	rangePartition_update_character.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 문자열 최대 크기로 확장하여 갱신되는지 확인
626	rangePartition_manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
627	rangePartition_manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
628	rangePartition_manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
629	rangePartition_manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
630	rangePartition_delete_basic.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 테이블의 데이터가 삭제되는지 확인
631	rangePartition_delete_predicate.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 조건에 따른 데이터가 삭제되는지 확인
632	rangePartition_delete_manyRows.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 1000건 이상의 데이터가 삭제되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
633	rangePartition_delete_iteration.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 1000건의 데이터가 반복하여 삽입, 삭제되는지 확인
634	rangePartition_manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인
635	rangePartition_manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
636	rangePartition_manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
637	rangePartition_manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 레인지 파티션 테이블을 대상으로 10개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
638	listPartition_rangePartition_create.py	이중화를 설정한 상태에서 이중화를 설정한 상태에서 리스트 파티션 테이블이 생성되는지 확인
639	listPartition_create_manyPartition.py	이중화를 설정한 상태에서 이중화를 설정한 상태에서 다수의 파티션을 포함한 리스트 파티션 테이블이 생성되는지 확인
640	listPartition_insert.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 테이블에 1만건의 데이터가 삽입되는지 확인
641	listPartition_insert_data_skew.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 데이터 쏠림 현상이 발생하는지 확인
642	listPartition_update_basic.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 테이블의 데이터가 갱신되는지 확인
643	listPartition_update_manyColumn.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 13개의 컬럼에 대한 데이터가 갱신되는지 확인
644	listPartition_update_manyRows.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 1000개의 데이터가 갱신되는지 확인
645	listPartition_update_manyColumn_Rows.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
646	listPartition_update_character.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 문자열 최대 크기로 확장하여 갱신되는지 확인
647	listPartition_manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
648	listPartition_manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
649	listPartition_manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 30개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
650	listPartition_manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
651	listPartition_delete_basic.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 테이블의 데이터가 삭제되는지 확인
652	listPartition_delete_predicate.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 조건에 따른 데이터가 삭제되는지 확인
653	listPartition_delete_manyRows.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 1000건 이상의 데이터가 삭제되는지 확인
654	listPartition_delete_iteration.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 1000건의 데이터가 반복하여 삽입, 삭제되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
655	listPartition_manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인
656	listPartition_manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
657	listPartition_manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
658	listPartition_manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 리스트 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
659	hashPartition_create.py	이중화를 설정한 상태에서 이중화를 설정한 상태에서 해시 파티션 테이블이 생성되는지 확인
660	hashPartition_create_manyPartition.py	이중화를 설정한 상태에서 이중화를 설정한 상태에서 다수의 파티션을 포함한 해시 파티션 테이블이 생성되는지 확인
661	hashPartition_insert.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 테이블에 1만건의 데이터가 삽입되는지 확인
662	hashPartition_insert_data_skew.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 데이터 쏠림 현상이 발생하는지 확인
663	hashPartition_update_basic.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 테이블의 데이터가 갱신되는지 확인
664	hashPartition_update_manyColumn.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 13개의 컬럼에 대한 데이터가 갱신되는지 확인
665	hashPartition_update_manyRows.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 1000개의 데이터가 갱신되는지 확인
666	hashPartition_update_manyColumn_Rows.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 13개의 컬럼을 포함하는 100개의 데이터가 갱신되는지 확인
667	hashPartition_update_character.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 문자열 최대 크기로 확장하여 갱신되는지 확인
668	hashPartition_manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인
669	hashPartition_manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
670	hashPartition_manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 10개의 테이블에 대한 데이터가 동시에 갱신되는지 확인
671	hashPartition_manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 50개의 테이블에 대한 데이터가 동시에 갱신되는지 확인(데이터 갱신 충돌)
672	hashPartition_delete_basic.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 테이블의 데이터가 삭제되는지 확인
673	hashPartition_delete_predicate.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 조건에 따른 데이터가 삭제되는지 확인
674	hashPartition_delete_manyRows.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 1000건 이상의 데이터가 삭제되는지 확인
675	hashPartition_delete_iteration.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 1000건의 데이터가 반복하여 삽입, 삭제되는지 확인
676	hashPartition_manyClients_oneTable_notConflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
677	hashPartition_manyClients_oneTable_Conflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 50개의 클라이언트를 이용하여 하나의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
678	hashPartition_manyClients_manyTable_notConflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 30개의 테이블에 대한 데이터가 동시에 삭제되는지 확인
679	hashPartition_manyClients_manyTable_Conflict.py	이중화를 설정한 상태에서 해시 파티션 테이블을 대상으로 30개의 클라이언트를 이용하여 30개의 테이블에 대한 데이터가 동시에 삭제되는지 확인(데이터 충돌)
680	dblink_initialization.py	DB 초기화 및 DB Link가 설정되는지 확인
681	dblink_finalization.py	DB Link 해제 및 DB 초기화가 설정되는지 확인
682	selectTable_basic.py	DB Link 설정 후, 데이터가 조회되는지 확인
683	projection_manyProjection.py	DB Link 설정 후, 50개 이상의 projection 결과가 조회되는지 확인
684	projection_BigDataProjection.py	DB Link 설정 후, 10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
685	projection_manyRows.py	DB Link 설정 후, 인라인 뷰를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
686	projection_inlineView.py	DB Link 설정 후, alias를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
687	projection_alias.py	DB Link 설정 후, nested(예: sum(avg(c1)) 되는 내장 함수를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
688	projection_nested_function.py	DB Link 설정 후, 조건절에 IS NULL, IS NOT NULL을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
689	predicate_null.py	DB Link 설정 후, 조건절에 NULL을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
690	predicate_like.py	DB Link 설정 후, 조건절에 Like를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
691	predicate_comparision.py	DB Link 설정 후, 조건절에 비교 연산자를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
692	pridicate_bultiin_function.py	DB Link 설정 후, 조건절에 내장 함수를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
693	subquery_in.py	DB Link 설정 후, IN 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
694	subquery_exists.py	DB Link 설정 후, EXIST 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
695	subquery_any_some.py	DB Link 설정 후, ANY, SOME 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
696	subquery_all.py	DB Link 설정 후, ALL 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
697	subquery_many.py	DB Link 설정 후, 2개 이상의 서브쿼리를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
698	basicJoin.py	DB Link 설정 후, JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
699	crossJoin.py	DB Link 설정 후, CROSS JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
700	naturalJoin.py	DB Link 설정 후, NATURAL JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
701	unionJoin.py	DB Link 설정 후, UNION JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
702	outerJoin.py	DB Link 설정 후, OUTER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
703	innerJoin.py	DB Link 설정 후, INNER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
704	selectTable_groupby.py	DB Link 설정 후, GROUP BY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
705	selectTable_having.py	DB Link 설정 후, HAVING 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
706	selectTable_orderby.py	DB Link 설정 후, ORDER BY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
707	rangePartition_projection_manyProjection.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 50개 이상의 projection 결과가 조회되는지 확인
708	rangePartition_projection_longByteProjection.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
709	rangePartition_projection_manyRows.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 인라인 뷰를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
710	rangePartition_predicate_null.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 조건절에 NULL을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
711	rangePartition_predicate_like.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 조건절에 Like를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
712	rangePartition_predicate_comparision.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 조건절에 비교 연산자를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
713	rangePartition_pridicate_bultiin_function.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 조건절에 내장 함수를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
714	rangePartition_subquery_in.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 IN 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
715	rangePartition_subquery_exists.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 EXIST 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
716	rangePartition_subquery_any_some.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 ANY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
717	rangePartition_subquery_all.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 ALL 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
718	rangePartition_subquery_many.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 2개 이상의 서브쿼리를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
719	rangePartition_basicJoin.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
720	rangePartition_crossJoin.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 CROSS JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
721	rangePartition_naturalJoin.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 NATURAL JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
722	rangePartition_unionJoin.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 UNION JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
723	rangePartition_outerJoin.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 OUTER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
724	rangePartition_innerJoin.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 INNER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
725	rangePartition_selectTable_groupby.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 GROUP BY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
726	rangePartition_selectTable_having.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 HAVING 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
727	rangePartition_selectTable_orderby.py	DB Link 설정 후 레인지 파티션 테이블을 대상으로 ORDER BY

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
		구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
728	listPartition_projection_manyProjection.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 50개 이상의 projection 결과가 조회되는지 확인
729	listPartition_projection_longByteProjection.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
730	listPartition_projection_manyRows.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 인라인 뷰를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
731	listPartition_predicate_null.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 조건절에 NULL을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
732	listPartition_predicate_like.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 조건절에 Like를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
733	listPartition_predicate_comparision.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 조건절에 비교 연산자를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
734	listPartition_pridicate_bultiin_function.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 조건절에 내장 함수를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
735	listPartition_subquery_in.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 IN 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
736	listPartition_subquery_exists.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 EXIST 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
737	listPartition_subquery_any_some.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 ANY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
738	listPartition_subquery_all.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 ALL 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
739	listPartition_subquery_many.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 2개 이상의 서브쿼리를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
740	listPartition_basicJoin.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
741	listPartition_crossJoin.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 CROSS JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
742	listPartition_naturalJoin.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 NATURAL JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
743	listPartition_unionJoin.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 UNION JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
744	listPartition_outerJoin.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 OUTER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
745	listPartition_innerJoin.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 INNER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
746	listPartition_selectTable_groupby.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 GROUP BY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
747	listPartition_selectTable_having.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 HAVING 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
748	listPartition_selectTable_orderby.py	DB Link 설정 후 리스트 파티션 테이블을 대상으로 ORDER BY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
749	hashPartition_projection_manyProjection.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 50개 이상의 projection 결과가 조회되는지 확인
750	hashPartition_selectTable_LongByteProjection.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 10000바이트 이상의 긴 길이를 가지는 결과가 조회되는지 확인
751	hashPartition_projection_manyRows.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 인라인 뷰를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
752	hashPartition_predicate_null.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 조건절에 NULL을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
753	hashPartition_predicate_like.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 조건절에 Like를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
754	hashPartition_predicate_comparision.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 조건절에 비교 연산자를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
755	hashPartition_pridicate_bultiin_function.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 조건절에 내장 함수를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
756	hashPartition_subquery_in.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 IN 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
757	hashPartition_subquery_exists.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 EXIST 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
758	hashPartition_subquery_any_some.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 ANY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
759	hashPartition_subquery_all.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 ALL 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
760	hashPartition_subquery_many.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 2개 이상의 서브쿼리를 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
761	hashPartition_basicJoin.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
762	hashPartition_crossJoin.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 CROSS JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
763	hashPartition_naturalJoin.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 NATURAL JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
764	hashPartition_unionJoin.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 UNION JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
765	hashPartition_outerJoin.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 OUTER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
766	hashPartition_innerJoin.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 INNER JOIN을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
767	hashPartition_selectTable_groupby.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 GROUP BY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
768	hashPartition_selectTable_having.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 HAVING 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
769	hashPartition_selectTable_orderby.py	DB Link 설정 후 해시 파티션 테이블을 대상으로 ORDER BY 구문을 이용하여 원격지 DB로 부터 결과가 조회되는지 확인
770	connect_invalidApi.py	유효하지 않은 데이터를 입력하여 Connect 함수 실행 시, 오류 메시지가 출력되는지 확인
771	connect_manyConnections.py	1000개 이상의 Connection 객체 생성이 가능한지 확인
772	connect_unitTest.py	Connection 함수의 각 파라미터의 값 변경이 가능한지 확인
773	creaeteStatement_manyStatement.py	100개 이상의 createStatement 생성이 가능한지 확인
774	execute_basic_1.py	INSERT 쿼리문이 실행되는지 확인
775	execute_basic_2.py	RAND 함수를 포함한 INSERT 쿼리문이 실행되는지 확인
776	execute_basic_3.py	한글이 포함된 문자를 대상으로 INSERT 쿼리문이 실행되는지 확인
777	execute_manyInsert.py	AutoCommit 상태에서 1000건의 INSERT 쿼리문이 실행되는지 확인
778	execute_sqlError.py	DML, DDL, DCL 쿼리문을 대상으로 오류 발생 시, JDBC에서 해당 오류를 출력해 주는지 확인
779	execute_transaction.py	foreign key on update 기능이 실행되는지 확인
780	executeQuery_basic.py	SELECT 쿼리문이 실행되는지 확인
781	executeQuery_executeTime.py	시간이 오래 걸리는 쿼리문이 실행되는지 확인
782	executeQuery_longQuerylength_1.py	4000자 이상의 길이를 가지는 쿼리문이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
783	executeQuery_longQuerylength_2.py	유효하지 않은 크기(1MB 이상)의 SELECT 쿼리문 실행 시, JDBC에서 해당 오류를 출력해 주는지 확인
784	executeQuery_sqlError.py	SELECT 쿼리문 실행 오류 발생 시, JDBC에서 해당오류를 출력하는지 확인
785	getMethod_basic_1.py	CAST('0000-00-00' AS TIMESTAMP) 함수를 갖는 select 쿼리문 결과 값을 getTimestamp() 함수로 반환 시, 값이 출력되는지 확인
786	getMethod_basic_2.py	소수점 10자리의 수를 가진 결과 값을 getfloat() 함수로 반환 시, 값이 일치하는지 확인
787	getMethod_null.py	null 값을 갖는 select 쿼리문 결과 값을 getObject 함수로 반환 시, null 값이 출력되는지 확인
788	preparedStatement_concurrency.py	4개의 클라이언트가 동시에 INSERT, SELECT, UPDATE, DELETE 쿼리문 실행 시, 해당 쿼리문이 오류 없이 실행되는지 확인
789	preparedStatement_basic_1.py	파라미터 값을 가지는 INSERT 쿼리문 실행이 가능한지 확인
790	preparedStatement_basic_2.py	bigInteger 데이터 타입의 값을 setObject() 함수로 바인딩이 가능한지 확인
791	preparedStatement_null.py	NULL 값을 바인딩 하여 preparedStatement 실행이 가능한지 확인
792	preparedStatement_manyParameters.py	30개의 파라미터를 이용하여 INSERT 쿼리문이 실행되는지 확인
793	preparedStatement_invalidPrecision.py	파라미터의 데이터 타입 보다 큰 값으로 바인딩 시, 오류 메시지가 출력되는지 확인
794	preparedStatement_invalidType.py	유효하지 않은 데이터 타입으로 값을 바인딩 시, 오류 메시지가 출력되는지 확인
795	SQLAllocConnect.py	SQLAllocConnect 함수가 실행되는지 확인
796	SQLAllocEnv.py	SQLAllocEnv API 함수가 실행되는지 확인
797	SQLAllocHandle.py	SQLAllocHandle API 함수가 실행되는지 확인
798	SQLAllocStmt.py	SQLAllocStmt 함수가 실행되는지 확인
799	SQLBindCol.py	SQLBindCol 함수가 실행되는지 확인
800	SQLBindParameter.py	SQLBindParameter 함수가 실행되는지 확인
801	SQLCloseCursor.py	SQLCloseCursor 함수가 실행되는지 확인
802	SQLColAttribute.py	SQLColAttribute 함수가 실행되는지 확인
803	SQLColumns.py	SQLColumns 함수가 실행되는지 확인
804	SQLConnect.py	SQLConnect 함수가 실행되는지 확인
805	SQLDescribeCol.py	SQLDescribeCol 함수가 실행되는지 확인
806	SQLDescribeParam.py	SQLDescribeParam 함수가 실행되는지 확인
807	SQLDisconnect.py	SQLDisconnect 함수가 실행되는지 확인
808	SQLDriverConnect.py	SQLDriverConnect 함수가 실행되는지 확인
809	SQLEndTran.py	SQLEndTran 함수가 실행되는지 확인
810	SQLError.py	SQLError 함수가 실행되는지 확인
811	SQLExecDirect.py	SQLExecDirect 함수가 실행되는지 확인
812	SQLExecute.py	SQLExecute 함수가 실행되는지 확인
813	SQLFetch.py	SQLFetch 함수가 실행되는지 확인
814	SQLFetchScroll.py	SQLFetchScroll 함수가 실행되는지 확인
815	SQLForeignKeys.py	SQLForeignKeys 함수가 실행되는지 확인
816	SQLFreeConnect.py	SQLFreeConnect 함수가 실행되는지 확인
817	SQLFreeEnv.py	SQLFreeEnv 함수가 실행되는지 확인
818	SQLFreeHandle.py	SQLFreeHandle 함수가 실행되는지 확인
819	SQLFreeStmt.py	SQLFreeStmt 함수가 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
820	SQLGetConnectAttr.py	SQLGetConnectAttr 함수가 실행되는지 확인
821	SQLGetData.py	SQLGetData 함수가 실행되는지 확인
822	SQlGetDiagField.py	SQlGetDiagField 함수가 실행되는지 확인
823	SQlGetDiagRec.py	SQlGetDiagRec 함수가 실행되는지 확인
824	SQlGetEnvAttr.py	SQlGetEnvAttr 함수가 실행되는지 확인
825	SQlGetFunctions.py	SQlGetFunctions 함수가 실행되는지 확인
826	SQLGetInfo.py	SQLGetInfo 함수가 실행되는지 확인
827	SQLGetStmtAttr.py	SQLGetStmtAttr 함수가 실행되는지 확인
828	SQLGetTypeInfo.py	SQLGetTypeInfo 함수가 실행되는지 확인
829	SQLMoreResults.py	SQLMoreResults 함수가 실행되는지 확인
830	SQLNativeSql.py	SQLNativeSql 함수가 실행되는지 확인
831	SQLNumParams.py	SQLNumParams 함수가 실행되는지 확인
832	SQLNumResultCols.py	SQLNumResultCols 함수가 실행되는지 확인
833	SQLParamData.py	SQLParamData 함수가 실행되는지 확인
834	SQLPrepare.py	SQLPrepare 함수가 실행되는지 확인
835	SQLPrimaryKeys.py	SQLPrimaryKeys 함수가 실행되는지 확인
836	SQLProcedureColumns.py	SQLProcedureColumns 함수가 실행되는지 확인
837	SQLProcedures.py	SQLProcedures 함수가 실행되는지 확인
838	SQLRowCount.py	SQLRowCount 함수가 실행되는지 확인
839	SQLSetConnectrAttr.py	SQLSetConnectrAttr 함수가 실행되는지 확인
840	SQLSetEnvAttr.py	SQLSetEnvAttr 함수가 실행되는지 확인
841	SQLSetStmtAttr.py	SQLSetStmtAttr 함수가 실행되는지 확인
842	SQLSpecialColums.py	SQLSpecialColums 함수가 실행되는지 확인
843	SQLStatistics.py	SQLStatistics 함수가 실행되는지 확인
844	SQLTablePrivileges.py	SQLTablePrivileges 함수가 실행되는지 확인
845	SQLTables.py	SQLTables 함수가 실행되는지 확인
846	SQLTransact.py	SQLTransact 함수가 실행되는지 확인
847	petasql.py	petasql 클라이언트의 주요 기능이 동작하는지 확인
848	petaloader.py	petaloader 유틸리티의 주요 기능이 동작하는지 확인
849	recovery.py	일반 테이블을 대상으로 데이터 백업 및 복구 기능이 동작하는지 확인
850	dump.py	일반 테이블을 대상으로 petadump 및 bindump의 주요 기능이 동작하는지 확인
851	restore.py	일반 테이블을 대상으로 데이터 10만건 삽입 후 서버 강제 종료 시 데이터 복구 기능이 동작하는지 확인
852	hash_recovery.py	해시 파티션 테이블을 대상으로 데이터 백업 및 복구 기능이 동작하는지 확인
853	hash_dump.py	해시 파티션 테이블을 대상으로 petadump 및 bindump의 주요 기능이 동작하는지 확인
854	hash_restore.py	해시 파티션 테이블 대상으로 데이터 10만건 삽입 후 서버 강제 종료 시 데이터 복구 기능이 동작하는지 확인
855	list_recovery.py	리스트 파티션 테이블을 대상으로 데이터 백업 및 복구 기능이 동작하는지 확인
856	list_dump.py	리스트 파티션 테이블을 대상으로 petadump 및 bindump의 주요 기능이 동작하는지 확인
857	list_restore.py	리스트 파티션 테이블을 대상으로 데이터 10만건 삽입 후 서버 강제 종료 시 데이터 복구 기능이 동작하는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
858	range_recovery.py	레인지 파티션 테이블을 대상으로 데이터 백업 및 복구 기능이 동작하는지 확인
859	range_dump.py	레인지 파티션 테이블을 대상으로 petadump 및 bindump의 주요 기능이 동작하는지 확인
860	range_restore.py	레인지 파티션 테이블을 대상으로 데이터 10만건 삽입 후 서버 강제 종료 시 데이터 복구 기능이 동작하는지 확인
861	insert_stress.py	100개의 클라이언트가 동시에 INSERT 쿼리문 실행이 가능한지 확인
862	update_stress.py	100개의 클라이언트가 동시에 UPDATE 쿼리문 실행이 가능한지 확인
863	delete_stress.py	100개의 클라이언트가 동시에 DELETE 쿼리문 실행이 가능한지 확인
864	create_drop_stress.py	100개의 클라이언트가 동시에 CREATE 및 DROP 쿼리문 실행이 가능한지 확인
865	alter_stress.py	100개의 클라이언트가 동시에 동일한 테이블을 대상으로 ALTER 쿼리문 실행이 가능한지 확인
866	dml_ddl_stress.py	100개의 클라이언트가 동시에 동일한 테이블을 대상으로 DML 및 DDL 쿼리문 실행이 가능한지 확인
867	insert_risk.py	스토리지가 부족한 상황에서 INSERT 쿼리문 실행 시, 서버가 비정상 종료하지 않고 관련 오류 메시지가 출력되는지 확인
868	update_risk.py	스토리지가 부족한 상황에서 UPDATE 쿼리문 실행 시, 서버가 비정상 종료하지 않고 관련 오류 메시지가 출력되는지 확인
869	index_risk.py	스토리지가 부족한 상황에서 INDEX 쿼리문 실행 시, 서버가 비정상 종료하지 않고 관련 오류 메시지가 출력되는지 확인
870	tpch_initialization.py	TPC-H 데이터가 생성(dat 파일 압축해제, 스키마 생성, 데이터 삽입, 제약조건, 인덱스 생성)되는지 확인
871	tpch_query1.py	TPC-H 1번 쿼리문이 실행되는지 확인
872	tpch_query2.py	TPC-H 2번 쿼리문이 실행되는지 확인
873	tpch_query3.py	TPC-H 3번 쿼리문이 실행되는지 확인
874	tpch_query4.py	TPC-H 4번 쿼리문이 실행되는지 확인
875	tpch_query5.py	TPC-H 5번 쿼리문이 실행되는지 확인
876	tpch_query6.py	TPC-H 6번 쿼리문이 실행되는지 확인
877	tpch_query7.py	TPC-H 7번 쿼리문이 실행되는지 확인
878	tpch_query8.py	TPC-H 8번 쿼리문이 실행되는지 확인
879	tpch_query9.py	TPC-H 9번 쿼리문이 실행되는지 확인
880	tpch_query10.py	TPC-H 10번 쿼리문이 실행되는지 확인
881	tpch_query11.py	TPC-H 11번 쿼리문이 실행되는지 확인
882	tpch_query12.py	TPC-H 12번 쿼리문이 실행되는지 확인
883	tpch_query13.py	TPC-H 13번 쿼리문이 실행되는지 확인
884	tpch_query14.py	TPC-H 14번 쿼리문이 실행되는지 확인
885	tpch_query15.py	TPC-H 15번 쿼리문이 실행되는지 확인
886	tpch_query16.py	TPC-H 16번 쿼리문이 실행되는지 확인
887	tpch_query17.py	TPC-H 17번 쿼리문이 실행되는지 확인
888	tpch_query18.py	TPC-H 18번 쿼리문이 실행되는지 확인
889	tpch_query19.py	TPC-H 19번 쿼리문이 실행되는지 확인
890	tpch_query20.py	TPC-H 20번 쿼리문이 실행되는지 확인
891	tpch_query21.py	TPC-H 21번 쿼리문이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
892	tpch_query22.py	TPC-H 22번 쿼리문이 실행되는지 확인
893	tpch_initialization.py	TPC-H 데이터가 생성(dat 파일 압축해제, 스키마 생성(파티션 테이블), 데이터 삽입, 제약조건, 인덱스 생성)되는지 확인
894	tpch_query1.py	파티션 테이블을 대상으로 TPC-H 1번 쿼리문이 실행되는지 확인
895	tpch_query2.py	파티션 테이블을 대상으로 TPC-H 2번 쿼리문이 실행되는지 확인
896	tpch_query3.py	파티션 테이블을 대상으로 TPC-H 3번 쿼리문이 실행되는지 확인
897	tpch_query4.py	파티션 테이블을 대상으로 TPC-H 4번 쿼리문이 실행되는지 확인
898	tpch_query5.py	파티션 테이블을 대상으로 TPC-H 5번 쿼리문이 실행되는지 확인
899	tpch_query6.py	파티션 테이블을 대상으로 TPC-H 6번 쿼리문이 실행되는지 확인
900	tpch_query7.py	파티션 테이블을 대상으로 TPC-H 7번 쿼리문이 실행되는지 확인
901	tpch_query8.py	파티션 테이블을 대상으로 TPC-H 8번 쿼리문이 실행되는지 확인
902	tpch_query9.py	파티션 테이블을 대상으로 TPC-H 9번 쿼리문이 실행되는지 확인
903	tpch_query10.py	파티션 테이블을 대상으로 TPC-H 10번 쿼리문이 실행되는지 확인
904	tpch_query11.py	파티션 테이블을 대상으로 TPC-H 11번 쿼리문이 실행되는지 확인
905	tpch_query12.py	파티션 테이블을 대상으로 TPC-H 12번 쿼리문이 실행되는지 확인
906	tpch_query13.py	파티션 테이블을 대상으로 TPC-H 13번 쿼리문이 실행되는지 확인
907	tpch_query14.py	파티션 테이블을 대상으로 TPC-H 14번 쿼리문이 실행되는지 확인
908	tpch_query15.py	파티션 테이블을 대상으로 TPC-H 15번 쿼리문이 실행되는지 확인
909	tpch_query16.py	파티션 테이블을 대상으로 TPC-H 16번 쿼리문이 실행되는지 확인
910	tpch_query17.py	파티션 테이블을 대상으로 TPC-H 17번 쿼리문이 실행되는지 확인
911	tpch_query18.py	파티션 테이블을 대상으로 TPC-H 18번 쿼리문이 실행되는지 확인
912	tpch_query19.py	파티션 테이블을 대상으로 TPC-H 19번 쿼리문이 실행되는지 확인
913	tpch_query20.py	파티션 테이블을 대상으로 TPC-H 20번 쿼리문이 실행되는지 확인
914	tpch_query21.py	파티션 테이블을 대상으로 TPC-H 21번 쿼리문이 실행되는지 확인
915	tpch_query22.py	파티션 테이블을 대상으로 TPC-H 22번 쿼리문이 실행되는지 확인
916	tpch_initialization.py	TPC-H 데이터가 생성(dat 파일 압축해제, 스키마 생성(파티션 테이블), 데이터 삽입, 제약조건, 인덱스 생성)되는지 확인
917	tpch_query1.py	TPC-H 1번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
918	tpch_query2.py	TPC-H 2번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
919	tpch_query3.py	TPC-H 3번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
920	tpch_query4.py	TPC-H 4번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
921	tpch_query5.py	TPC-H 5번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
922	tpch_query6.py	TPC-H 6번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
923	tpch_query7.py	TPC-H 7번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
924	tpch_query8.py	TPC-H 8번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
925	tpch_query9.py	TPC-H 9번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
926	tpch_query10.py	TPC-H 10번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
927	tpch_query11.py	TPC-H 11번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
928	tpch_query12.py	TPC-H 12번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
929	tpch_query13.py	TPC-H 13번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
930	tpch_query14.py	TPC-H 14번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
931	tpch_query15.py	TPC-H 15번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
932	tpch_query16.py	TPC-H 16번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
933	tpch_query17.py	TPC-H 17번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
934	tpch_query18.py	TPC-H 18번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
935	tpch_query19.py	TPC-H 19번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
936	tpch_query20.py	TPC-H 20번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
937	tpch_query21.py	TPC-H 21번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
938	tpch_query22.py	TPC-H 22번 쿼리문이 DBLink를 통해 원격으로 실행되는지 확인
939	tpch_throughtput.py	다수 클라이언트가 동시에 TPC-H 쿼리문이 오류 없이 실행되는지 확인
940	tpcds_initialization.py	TPC-DS 데이터가 생성(dat 파일 압축해제, 스키마 생성, 데이터 삽입, 제약조건, 인덱스 생성)되는지 확인
941	tpcds_query1.py	TPC-DS 1번 쿼리문이 실행되는지 확인
942	tpcds_query2.py	TPC-DS 2번 쿼리문이 실행되는지 확인
943	tpcds_query3.py	TPC-DS 3번 쿼리문이 실행되는지 확인
944	tpcds_query4.py	TPC-DS 4번 쿼리문이 실행되는지 확인
945	tpcds_query5.py	TPC-DS 5번 쿼리문이 실행되는지 확인
946	tpcds_query6.py	TPC-DS 6번 쿼리문이 실행되는지 확인
947	tpcds_query7.py	TPC-DS 7번 쿼리문이 실행되는지 확인
948	tpcds_query8.py	TPC-DS 8번 쿼리문이 실행되는지 확인
949	tpcds_query9.py	TPC-DS 9번 쿼리문이 실행되는지 확인
950	tpcds_query10.py	TPC-DS 10번 쿼리문이 실행되는지 확인
951	tpcds_query11.py	TPC-DS 11번 쿼리문이 실행되는지 확인
952	tpcds_query12.py	TPC-DS 12번 쿼리문이 실행되는지 확인
953	tpcds_query13.py	TPC-DS 13번 쿼리문이 실행되는지 확인
954	tpcds_query14.py	TPC-DS 14번 쿼리문이 실행되는지 확인
955	tpcds_query15.py	TPC-DS 15번 쿼리문이 실행되는지 확인
956	tpcds_query16.py	TPC-DS 16번 쿼리문이 실행되는지 확인
957	tpcds_query17.py	TPC-DS 17번 쿼리문이 실행되는지 확인
958	tpcds_query18.py	TPC-DS 18번 쿼리문이 실행되는지 확인
959	tpcds_query19.py	TPC-DS 19번 쿼리문이 실행되는지 확인
960	tpcds_query20.py	TPC-DS 20번 쿼리문이 실행되는지 확인
961	tpcds_query21.py	TPC-DS 21번 쿼리문이 실행되는지 확인
962	tpcds_query22.py	TPC-DS 22번 쿼리문이 실행되는지 확인
963	tpcds_query23.py	TPC-DS 23번 쿼리문이 실행되는지 확인
964	tpcds_query24.py	TPC-DS 24번 쿼리문이 실행되는지 확인
965	tpcds_query25.py	TPC-DS 25번 쿼리문이 실행되는지 확인
966	tpcds_query26.py	TPC-DS 26번 쿼리문이 실행되는지 확인
967	tpcds_query27.py	TPC-DS 27번 쿼리문이 실행되는지 확인
968	tpcds_query28.py	TPC-DS 28번 쿼리문이 실행되는지 확인
969	tpcds_query29.py	TPC-DS 29번 쿼리문이 실행되는지 확인
970	tpcds_query30.py	TPC-DS 30번 쿼리문이 실행되는지 확인
971	tpcds_query31.py	TPC-DS 31번 쿼리문이 실행되는지 확인
972	tpcds_query32.py	TPC-DS 32번 쿼리문이 실행되는지 확인
973	tpcds_query33.py	TPC-DS 33번 쿼리문이 실행되는지 확인
974	tpcds_query34.py	TPC-DS 34번 쿼리문이 실행되는지 확인
975	tpcds_query35.py	TPC-DS 35번 쿼리문이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
976	tpcds_query36.py	TPC-DS 36번 쿼리문이 실행되는지 확인
977	tpcds_query37.py	TPC-DS 37번 쿼리문이 실행되는지 확인
978	tpcds_query38.py	TPC-DS 38번 쿼리문이 실행되는지 확인
979	tpcds_query39.py	TPC-DS 39번 쿼리문이 실행되는지 확인
980	tpcds_query40.py	TPC-DS 40번 쿼리문이 실행되는지 확인
981	tpcds_query41.py	TPC-DS 41번 쿼리문이 실행되는지 확인
982	tpcds_query42.py	TPC-DS 42번 쿼리문이 실행되는지 확인
983	tpcds_query43.py	TPC-DS 43번 쿼리문이 실행되는지 확인
984	tpcds_query44.py	TPC-DS 44번 쿼리문이 실행되는지 확인
985	tpcds_query45.py	TPC-DS 45번 쿼리문이 실행되는지 확인
986	tpcds_query46.py	TPC-DS 46번 쿼리문이 실행되는지 확인
987	tpcds_query47.py	TPC-DS 47번 쿼리문이 실행되는지 확인
988	tpcds_query48.py	TPC-DS 48번 쿼리문이 실행되는지 확인
989	tpcds_query49.py	TPC-DS 49번 쿼리문이 실행되는지 확인
990	tpcds_query50.py	TPC-DS 50번 쿼리문이 실행되는지 확인
991	tpcds_query51.py	TPC-DS 51번 쿼리문이 실행되는지 확인
992	tpcds_query52.py	TPC-DS 52번 쿼리문이 실행되는지 확인
993	tpcds_query53.py	TPC-DS 53번 쿼리문이 실행되는지 확인
994	tpcds_query54.py	TPC-DS 54번 쿼리문이 실행되는지 확인
995	tpcds_query55.py	TPC-DS 55번 쿼리문이 실행되는지 확인
996	tpcds_query56.py	TPC-DS 56번 쿼리문이 실행되는지 확인
997	tpcds_query57.py	TPC-DS 57번 쿼리문이 실행되는지 확인
998	tpcds_query58.py	TPC-DS 58번 쿼리문이 실행되는지 확인
999	tpcds_query59.py	TPC-DS 59번 쿼리문이 실행되는지 확인
1000	tpcds_query60.py	TPC-DS 60번 쿼리문이 실행되는지 확인
1001	tpcds_query61.py	TPC-DS 61번 쿼리문이 실행되는지 확인
1002	tpcds_query62.py	TPC-DS 62번 쿼리문이 실행되는지 확인
1003	tpcds_query63.py	TPC-DS 63번 쿼리문이 실행되는지 확인
1004	tpcds_query64.py	TPC-DS 64번 쿼리문이 실행되는지 확인
1005	tpcds_query65.py	TPC-DS 65번 쿼리문이 실행되는지 확인
1006	tpcds_query66.py	TPC-DS 66번 쿼리문이 실행되는지 확인
1007	tpcds_query67.py	TPC-DS 67번 쿼리문이 실행되는지 확인
1008	tpcds_query68.py	TPC-DS 68번 쿼리문이 실행되는지 확인
1009	tpcds_query69.py	TPC-DS 69번 쿼리문이 실행되는지 확인
1010	tpcds_query70.py	TPC-DS 70번 쿼리문이 실행되는지 확인
1011	tpcds_query71.py	TPC-DS 71번 쿼리문이 실행되는지 확인
1012	tpcds_query72.py	TPC-DS 72번 쿼리문이 실행되는지 확인
1013	tpcds_query73.py	TPC-DS 73번 쿼리문이 실행되는지 확인
1014	tpcds_query74.py	TPC-DS 74번 쿼리문이 실행되는지 확인
1015	tpcds_query75.py	TPC-DS 75번 쿼리문이 실행되는지 확인
1016	tpcds_query76.py	TPC-DS 76번 쿼리문이 실행되는지 확인
1017	tpcds_query77.py	TPC-DS 77번 쿼리문이 실행되는지 확인
1018	tpcds_query78.py	TPC-DS 78번 쿼리문이 실행되는지 확인
1019	tpcds_query79.py	TPC-DS 79번 쿼리문이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1020	tpcds_query80.py	TPC-DS 80번 쿼리문이 실행되는지 확인
1021	tpcds_query81.py	TPC-DS 81번 쿼리문이 실행되는지 확인
1022	tpcds_query82.py	TPC-DS 82번 쿼리문이 실행되는지 확인
1023	tpcds_query83.py	TPC-DS 83번 쿼리문이 실행되는지 확인
1024	tpcds_query84.py	TPC-DS 84번 쿼리문이 실행되는지 확인
1025	tpcds_query85.py	TPC-DS 85번 쿼리문이 실행되는지 확인
1026	tpcds_query86.py	TPC-DS 86번 쿼리문이 실행되는지 확인
1027	tpcds_query87.py	TPC-DS 87번 쿼리문이 실행되는지 확인
1028	tpcds_query88.py	TPC-DS 88번 쿼리문이 실행되는지 확인
1029	tpcds_query89.py	TPC-DS 89번 쿼리문이 실행되는지 확인
1030	tpcds_query90.py	TPC-DS 90번 쿼리문이 실행되는지 확인
1031	tpcds_query91.py	TPC-DS 91번 쿼리문이 실행되는지 확인
1032	tpcds_query92.py	TPC-DS 92번 쿼리문이 실행되는지 확인
1033	tpcds_query93.py	TPC-DS 93번 쿼리문이 실행되는지 확인
1034	tpcds_query94.py	TPC-DS 94번 쿼리문이 실행되는지 확인
1035	tpcds_query95.py	TPC-DS 95번 쿼리문이 실행되는지 확인
1036	tpcds_query96.py	TPC-DS 96번 쿼리문이 실행되는지 확인
1037	tpcds_query97.py	TPC-DS 97번 쿼리문이 실행되는지 확인
1038	tpcds_query98.py	TPC-DS 98번 쿼리문이 실행되는지 확인
1039	tpcds_query99.py	TPC-DS 99번 쿼리문이 실행되는지 확인
1040	create_user.py	SQL92 유저가 생성되는지 확인
1041	create_schema_hu.py	HU 사용자 계정의 스키마가 생성되는지 확인
1042	create_schema_sun.py	SUN 사용자 계정의 스키마가 생성되는지 확인
1043	create_schema_cts1.py	CTS1 사용자 계정의 스키마가 생성되는지 확인
1044	create_schema_cts2.py	CTS2 사용자 계정의 스키마가 생성되는지 확인
1045	create_schema_sullivan1.py	SULLIVAN1 사용자 계정의 스키마가 생성되는지 확인
1046	create_schema_sullivan.py	SULLIVAN 사용자 계정의 스키마가 생성되는지 확인
1047	create_schema_cugini.py	CUGINI 사용자 계정의 스키마가 생성되는지 확인
1048	create_schema_schanzle.py	SCHANZLE 사용자 계정의 스키마가 생성되는지 확인
1049	create_schema_flater.py	FLATER 사용자 계정의 스키마가 생성되는지 확인
1050	create_schema_shirley_hurwitz.py	SHIRLEY 사용자 계정의 스키마가 생성되는지 확인
1051	create_schema_len_gallagher.py	LEN_GALLAGHER 사용자 계정의 스키마가 생성되는지 확인
1052	create_schema_hydrography.py	HYDROGRAPHY 사용자 계정의 스키마가 생성되는지 확인
1053	create_schema_tides.py	TIDES 사용자 계정의 스키마가 생성되는지 확인
1054	create_schema_mcginn.py	MCGINN 사용자 계정의 스키마가 생성되는지 확인
1055	data_load_hu.py	HU 사용자 계정의 데이터가 삽입되는지 확인
1056	data_load_sun_0.py	SUN 사용자 계정의 데이터가 삽입되는지 확인
1057	data_load_sun_1.py	SUN 사용자 계정의 데이터가 삽입되는지 확인
1058	data_load_sun_2.py	SUN 사용자 계정의 데이터가 삽입되는지 확인
1059	data_load_sun_3.py	SUN 사용자 계정의 데이터가 삽입되는지 확인
1060	data_load_cts1.py	CTS1 사용자 계정의 데이터가 삽입되는지 확인
1061	data_load_cts2.py	CTS2 사용자 계정의 데이터가 삽입되는지 확인
1062	data_load_flater.py	FLATER 사용자 계정의 데이터가 삽입되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1063	data_load_tides.py	TIDES 사용자 계정의 데이터가 삽입되는지 확인
1064	data_load_sullivan.py	SULLIVAN 사용자 계정의 데이터가 삽입되는지 확인
1065	cdr001.py	SQL92의 cdr001.sql 파일이 실행되는지 확인
1066	cdr002.py	SQL92의 cdr002.sql 파일이 실행되는지 확인
1067	cdr003.py	SQL92의 cdr003.sql 파일이 실행되는지 확인
1068	cdr004.py	SQL92의 cdr004.sql 파일이 실행되는지 확인
1069	cdr005.py	SQL92의 cdr005.sql 파일이 실행되는지 확인
1070	cdr006.py	SQL92의 cdr006.sql 파일이 실행되는지 확인
1071	cdr007.py	SQL92의 cdr007.sql 파일이 실행되는지 확인
1072	cdr008.py	SQL92의 cdr008.sql 파일이 실행되는지 확인
1073	cdr009.py	SQL92의 cdr009.sql 파일이 실행되는지 확인
1074	cdr010.py	SQL92의 cdr010.sql 파일이 실행되는지 확인
1075	cdr011.py	SQL92의 cdr011.sql 파일이 실행되는지 확인
1076	cdr012.py	SQL92의 cdr012.sql 파일이 실행되는지 확인
1077	cdr013.py	SQL92의 cdr013.sql 파일이 실행되는지 확인
1078	cdr014.py	SQL92의 cdr014.sql 파일이 실행되는지 확인
1079	cdr015.py	SQL92의 cdr015.sql 파일이 실행되는지 확인
1080	cdr016.py	SQL92의 cdr016.sql 파일이 실행되는지 확인
1081	cdr017.py	SQL92의 cdr017.sql 파일이 실행되는지 확인
1082	cdr018.py	SQL92의 cdr018.sql 파일이 실행되는지 확인
1083	cdr019.py	SQL92의 cdr019.sql 파일이 실행되는지 확인
1084	cdr020.py	SQL92의 cdr020.sql 파일이 실행되는지 확인
1085	cdr021.py	SQL92의 cdr021.sql 파일이 실행되는지 확인
1086	cdr022.py	SQL92의 cdr022.sql 파일이 실행되는지 확인
1087	cdr023.py	SQL92의 cdr023.sql 파일이 실행되는지 확인
1088	cdr024.py	SQL92의 cdr024.sql 파일이 실행되는지 확인
1089	cdr025.py	SQL92의 cdr025.sql 파일이 실행되는지 확인
1090	cdr026.py	SQL92의 cdr026.sql 파일이 실행되는지 확인
1091	cdr027.py	SQL92의 cdr027.sql 파일이 실행되는지 확인
1092	cdr028.py	SQL92의 cdr028.sql 파일이 실행되는지 확인
1093	cdr029.py	SQL92의 cdr029.sql 파일이 실행되는지 확인
1094	cdr030.py	SQL92의 cdr030.sql 파일이 실행되는지 확인
1095	cdr031.py	SQL92의 cdr031.sql 파일이 실행되는지 확인
1096	dml001.py	SQL92의 dml001.sql 파일이 실행되는지 확인
1097	dml004.py	SQL92의 dml004.sql 파일이 실행되는지 확인
1098	dml005.py	SQL92의 dml005.sql 파일이 실행되는지 확인
1099	dml008.py	SQL92의 dml008.sql 파일이 실행되는지 확인
1100	dml009.py	SQL92의 dml009.sql 파일이 실행되는지 확인
1101	dml010.py	SQL92의 dml010.sql 파일이 실행되는지 확인
1102	dml011.py	SQL92의 dml011.sql 파일이 실행되는지 확인
1103	dml012.py	SQL92의 dml012.sql 파일이 실행되는지 확인
1104	dml013.py	SQL92의 dml013.sql 파일이 실행되는지 확인
1105	dml014.py	SQL92의 dml014.sql 파일이 실행되는지 확인
1106	dml015.py	SQL92의 dml015.sql 파일이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1107	dml016.py	SQL92의 dml016.sql 파일이 실행되는지 확인
1108	dml018.py	SQL92의 dml018.sql 파일이 실행되는지 확인
1109	dml019.py	SQL92의 dml019.sql 파일이 실행되는지 확인
1110	dml020.py	SQL92의 dml020.sql 파일이 실행되는지 확인
1111	dml021.py	SQL92의 dml021.sql 파일이 실행되는지 확인
1112	dml022.py	SQL92의 dml022.sql 파일이 실행되는지 확인
1113	dml023.py	SQL92의 dml023.sql 파일이 실행되는지 확인
1114	dml024.py	SQL92의 dml024.sql 파일이 실행되는지 확인
1115	dml025.py	SQL92의 dml025.sql 파일이 실행되는지 확인
1116	dml026.py	SQL92의 dml026.sql 파일이 실행되는지 확인
1117	dml027.py	SQL92의 dml027.sql 파일이 실행되는지 확인
1118	dml029.py	SQL92의 dml029.sql 파일이 실행되는지 확인
1119	dml033.py	SQL92의 dml033.sql 파일이 실행되는지 확인
1120	dml034.py	SQL92의 dml034.sql 파일이 실행되는지 확인
1121	dml035.py	SQL92의 dml035.sql 파일이 실행되는지 확인
1122	dml037.py	SQL92의 dml037.sql 파일이 실행되는지 확인
1123	dml038.py	SQL92의 dml038.sql 파일이 실행되는지 확인
1124	dml039.py	SQL92의 dml039.sql 파일이 실행되는지 확인
1125	dml040.py	SQL92의 dml040.sql 파일이 실행되는지 확인
1126	dml041.py	SQL92의 dml041.sql 파일이 실행되는지 확인
1127	dml042.py	SQL92의 dml042.sql 파일이 실행되는지 확인
1128	dml043.py	SQL92의 dml043.sql 파일이 실행되는지 확인
1129	dml044.py	SQL92의 dml044.sql 파일이 실행되는지 확인
1130	dml045.py	SQL92의 dml045.sql 파일이 실행되는지 확인
1131	dml046.py	SQL92의 dml046.sql 파일이 실행되는지 확인
1132	dml047.py	SQL92의 dml047.sql 파일이 실행되는지 확인
1133	dml049.py	SQL92의 dml049.sql 파일이 실행되는지 확인
1134	dml050.py	SQL92의 dml050.sql 파일이 실행되는지 확인
1135	dml051.py	SQL92의 dml051.sql 파일이 실행되는지 확인
1136	dml052.py	SQL92의 dml052.sql 파일이 실행되는지 확인
1137	dml053.py	SQL92의 dml053.sql 파일이 실행되는지 확인
1138	dml055.py	SQL92의 dml055.sql 파일이 실행되는지 확인
1139	dml056.py	SQL92의 dml056.sql 파일이 실행되는지 확인
1140	dml057.py	SQL92의 dml057.sql 파일이 실행되는지 확인
1141	dml058.py	SQL92의 dml058.sql 파일이 실행되는지 확인
1142	dml059.py	SQL92의 dml059.sql 파일이 실행되는지 확인
1143	dml060.py	SQL92의 dml060.sql 파일이 실행되는지 확인
1144	dml061.py	SQL92의 dml061.sql 파일이 실행되는지 확인
1145	dml062.py	SQL92의 dml062.sql 파일이 실행되는지 확인
1146	dml064.py	SQL92의 dml064.sql 파일이 실행되는지 확인
1147	dml065.py	SQL92의 dml065.sql 파일이 실행되는지 확인
1148	dml068.py	SQL92의 dml068.sql 파일이 실행되는지 확인
1149	dml069.py	SQL92의 dml069.sql 파일이 실행되는지 확인
1150	dml070.py	SQL92의 dml070.sql 파일이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1151	dml073.py	SQL92의 dml073.sql 파일이 실행되는지 확인
1152	dml075.py	SQL92의 dml075.sql 파일이 실행되는지 확인
1153	dml076.py	SQL92의 dml076.sql 파일이 실행되는지 확인
1154	dml077.py	SQL92의 dml077.sql 파일이 실행되는지 확인
1155	dml079.py	SQL92의 dml079.sql 파일이 실행되는지 확인
1156	dml080.py	SQL92의 dml080.sql 파일이 실행되는지 확인
1157	dml081.py	SQL92의 dml081.sql 파일이 실행되는지 확인
1158	dml082.py	SQL92의 dml082.sql 파일이 실행되는지 확인
1159	dml083.py	SQL92의 dml083.sql 파일이 실행되는지 확인
1160	dml084.py	SQL92의 dml084.sql 파일이 실행되는지 확인
1161	dml085.py	SQL92의 dml085.sql 파일이 실행되는지 확인
1162	dml086.py	SQL92의 dml086.sql 파일이 실행되는지 확인
1163	dml087.py	SQL92의 dml087.sql 파일이 실행되는지 확인
1164	dml090.py	SQL92의 dml090.sql 파일이 실행되는지 확인
1165	dml091.py	SQL92의 dml091.sql 파일이 실행되는지 확인
1166	dml099.py	SQL92의 dml099.sql 파일이 실행되는지 확인
1167	dml100.py	SQL92의 dml100.sql 파일이 실행되는지 확인
1168	dml104.py	SQL92의 dml104.sql 파일이 실행되는지 확인
1169	dml106.py	SQL92의 dml106.sql 파일이 실행되는지 확인
1170	dml108.py	SQL92의 dml108.sql 파일이 실행되는지 확인
1171	dml112.py	SQL92의 dml112.sql 파일이 실행되는지 확인
1172	dml114.py	SQL92의 dml114.sql 파일이 실행되는지 확인
1173	dml117.py	SQL92의 dml117.sql 파일이 실행되는지 확인
1174	dml119.py	SQL92의 dml119.sql 파일이 실행되는지 확인
1175	dml121.py	SQL92의 dml121.sql 파일이 실행되는지 확인
1176	dml130.py	SQL92의 dml130.sql 파일이 실행되는지 확인
1177	dml131.py	SQL92의 dml131.sql 파일이 실행되는지 확인
1178	dml132.py	SQL92의 dml132.sql 파일이 실행되는지 확인
1179	dml133.py	SQL92의 dml133.sql 파일이 실행되는지 확인
1180	dml134.py	SQL92의 dml134.sql 파일이 실행되는지 확인
1181	dml135.py	SQL92의 dml135.sql 파일이 실행되는지 확인
1182	dml136.py	SQL92의 dml136.sql 파일이 실행되는지 확인
1183	dml137.py	SQL92의 dml137.sql 파일이 실행되는지 확인
1184	dml139.py	SQL92의 dml139.sql 파일이 실행되는지 확인
1185	dml141.py	SQL92의 dml141.sql 파일이 실행되는지 확인
1186	dml142.py	SQL92의 dml142.sql 파일이 실행되는지 확인
1187	dml143.py	SQL92의 dml143.sql 파일이 실행되는지 확인
1188	dml144.py	SQL92의 dml144.sql 파일이 실행되는지 확인
1189	dml147.py	SQL92의 dml147.sql 파일이 실행되는지 확인
1190	dml148.py	SQL92의 dml148.sql 파일이 실행되는지 확인
1191	dml149.py	SQL92의 dml149.sql 파일이 실행되는지 확인
1192	dml153.py	SQL92의 dml153.sql 파일이 실행되는지 확인
1193	dml154.py	SQL92의 dml154.sql 파일이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1194	dml155.py	SQL92의 dml155.sql 파일이 실행되는지 확인
1195	dml157.py	SQL92의 dml157.sql 파일이 실행되는지 확인
1196	dml158.py	SQL92의 dml158.sql 파일이 실행되는지 확인
1197	dml160.py	SQL92의 dml160.sql 파일이 실행되는지 확인
1198	dml161.py	SQL92의 dml161.sql 파일이 실행되는지 확인
1199	dml162.py	SQL92의 dml162.sql 파일이 실행되는지 확인
1200	dml163.py	SQL92의 dml163.sql 파일이 실행되는지 확인
1201	dml164.py	SQL92의 dml164.sql 파일이 실행되는지 확인
1202	dml165.py	SQL92의 dml165.sql 파일이 실행되는지 확인
1203	dml167.py	SQL92의 dml167.sql 파일이 실행되는지 확인
1204	dml168.py	SQL92의 dml168.sql 파일이 실행되는지 확인
1205	dml169.py	SQL92의 dml169.sql 파일이 실행되는지 확인
1206	dml170.py	SQL92의 dml170.sql 파일이 실행되는지 확인
1207	dml171.py	SQL92의 dml171.sql 파일이 실행되는지 확인
1208	dml172.py	SQL92의 dml172.sql 파일이 실행되는지 확인
1209	dml173.py	SQL92의 dml173.sql 파일이 실행되는지 확인
1210	dml174.py	SQL92의 dml174.sql 파일이 실행되는지 확인
1211	dml175.py	SQL92의 dml175.sql 파일이 실행되는지 확인
1212	dml176.py	SQL92의 dml176.sql 파일이 실행되는지 확인
1213	dml177.py	SQL92의 dml177.sql 파일이 실행되는지 확인
1214	dml178.py	SQL92의 dml178.sql 파일이 실행되는지 확인
1215	dml179.py	SQL92의 dml179.sql 파일이 실행되는지 확인
1216	dml181.py	SQL92의 dml181.sql 파일이 실행되는지 확인
1217	dml182.py	SQL92의 dml182.sql 파일이 실행되는지 확인
1218	dml183.py	SQL92의 dml183.sql 파일이 실행되는지 확인
1219	dml184.py	SQL92의 dml184.sql 파일이 실행되는지 확인
1220	dml185.py	SQL92의 dml185.sql 파일이 실행되는지 확인
1221	dml186.py	SQL92의 dml186.sql 파일이 실행되는지 확인
1222	flg005.py	SQL92의 flg005.sql 파일이 실행되는지 확인
1223	flg006.py	SQL92의 flg006.sql 파일이 실행되는지 확인
1224	flg008.py	SQL92의 flg008.sql 파일이 실행되는지 확인
1225	flg009.py	SQL92의 flg009.sql 파일이 실행되는지 확인
1226	flg011.py	SQL92의 flg011.sql 파일이 실행되는지 확인
1227	flg012.py	SQL92의 flg012.sql 파일이 실행되는지 확인
1228	isi001.py	SQL92의 isi001.sql 파일이 실행되는지 확인
1229	isi002.py	SQL92의 isi002.sql 파일이 실행되는지 확인
1230	isi003.py	SQL92의 isi003.sql 파일이 실행되는지 확인
1231	isi004.py	SQL92의 isi004.sql 파일이 실행되는지 확인
1232	isi005.py	SQL92의 isi005.sql 파일이 실행되는지 확인
1233	isi006.py	SQL92의 isi006.sql 파일이 실행되는지 확인
1234	isi007.py	SQL92의 isi007.sql 파일이 실행되는지 확인
1235	ist001.py	SQL92의 ist001.sql 파일이 실행되는지 확인
1236	ist002.py	SQL92의 ist002.sql 파일이 실행되는지 확인
1237	ist003.py	SQL92의 ist003.sql 파일이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1238	ist004.py	SQL92의 ist004.sql 파일이 실행되는지 확인
1239	ist005.py	SQL92의 ist005.sql 파일이 실행되는지 확인
1240	ist006.py	SQL92의 ist006.sql 파일이 실행되는지 확인
1241	ist007.py	SQL92의 ist007.sql 파일이 실행되는지 확인
1242	sdl001.py	SQL92의 sdl001.sql 파일이 실행되는지 확인
1243	sdl002.py	SQL92의 sdl002.sql 파일이 실행되는지 확인
1244	sdl003.py	SQL92의 sdl003.sql 파일이 실행되는지 확인
1245	sdl004.py	SQL92의 sdl004.sql 파일이 실행되는지 확인
1246	sdl005.py	SQL92의 sdl005.sql 파일이 실행되는지 확인
1247	sdl006.py	SQL92의 sdl006.sql 파일이 실행되는지 확인
1248	sdl007.py	SQL92의 sdl007.sql 파일이 실행되는지 확인
1249	sdl008.py	SQL92의 sdl008.sql 파일이 실행되는지 확인
1250	sdl009.py	SQL92의 sdl009.sql 파일이 실행되는지 확인
1251	sdl010.py	SQL92의 sdl010.sql 파일이 실행되는지 확인
1252	sdl011.py	SQL92의 sdl011.sql 파일이 실행되는지 확인
1253	sdl012.py	SQL92의 sdl012.sql 파일이 실행되는지 확인
1254	sdl013.py	SQL92의 sdl013.sql 파일이 실행되는지 확인
1255	sdl014.py	SQL92의 sdl014.sql 파일이 실행되는지 확인
1256	sdl015.py	SQL92의 sdl015.sql 파일이 실행되는지 확인
1257	sdl016.py	SQL92의 sdl016.sql 파일이 실행되는지 확인
1258	sdl017.py	SQL92의 sdl017.sql 파일이 실행되는지 확인
1259	sdl018.py	SQL92의 sdl018.sql 파일이 실행되는지 확인
1260	sdl019.py	SQL92의 sdl019.sql 파일이 실행되는지 확인
1261	sdl020.py	SQL92의 sdl020.sql 파일이 실행되는지 확인
1262	sdl021.py	SQL92의 sdl021.sql 파일이 실행되는지 확인
1263	sdl022.py	SQL92의 sdl022.sql 파일이 실행되는지 확인
1264	sdl023.py	SQL92의 sdl023.sql 파일이 실행되는지 확인
1265	sdl024.py	SQL92의 sdl024.sql 파일이 실행되는지 확인
1266	sdl025.py	SQL92의 sdl025.sql 파일이 실행되는지 확인
1267	sdl026.py	SQL92의 sdl026.sql 파일이 실행되는지 확인
1268	sdl027.py	SQL92의 sdl027.sql 파일이 실행되는지 확인
1269	sdl028.py	SQL92의 sdl028.sql 파일이 실행되는지 확인
1270	sdl029.py	SQL92의 sdl029.sql 파일이 실행되는지 확인
1271	sdl030.py	SQL92의 sdl030.sql 파일이 실행되는지 확인
1272	sdl031.py	SQL92의 sdl031.sql 파일이 실행되는지 확인
1273	sdl032.py	SQL92의 sdl032.sql 파일이 실행되는지 확인
1274	sdl033.py	SQL92의 sdl033.sql 파일이 실행되는지 확인
1275	sdl034.py	SQL92의 sdl034.sql 파일이 실행되는지 확인
1276	sdl035.py	SQL92의 sdl035.sql 파일이 실행되는지 확인
1277	sdl036.py	SQL92의 sdl036.sql 파일이 실행되는지 확인
1278	sdl037.py	SQL92의 sdl037.sql 파일이 실행되는지 확인
1279	xts700.py	SQL92의 xts700.sql 파일이 실행되는지 확인
1280	xts701.py	SQL92의 xts701.sql 파일이 실행되는지 확인
1281	xts702.py	SQL92의 xts702.sql 파일이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1282	xts703.py	SQL92의 xts703.sql 파일이 실행되는지 확인
1283	xts713.py	SQL92의 xts713.sql 파일이 실행되는지 확인
1284	xts714.py	SQL92의 xts714.sql 파일이 실행되는지 확인
1285	xts715.py	SQL92의 xts715.sql 파일이 실행되는지 확인
1286	xts716.py	SQL92의 xts716.sql 파일이 실행되는지 확인
1287	xts717.py	SQL92의 xts717.sql 파일이 실행되는지 확인
1288	xts718.py	SQL92의 xts718.sql 파일이 실행되는지 확인
1289	xts719.py	SQL92의 xts719.sql 파일이 실행되는지 확인
1290	xts720.py	SQL92의 xts720.sql 파일이 실행되는지 확인
1291	xts721.py	SQL92의 xts721.sql 파일이 실행되는지 확인
1292	xts722.py	SQL92의 xts722.sql 파일이 실행되는지 확인
1293	xts723.py	SQL92의 xts723.sql 파일이 실행되는지 확인
1294	xts724.py	SQL92의 xts724.sql 파일이 실행되는지 확인
1295	xts727.py	SQL92의 xts727.sql 파일이 실행되는지 확인
1296	xts728.py	SQL92의 xts728.sql 파일이 실행되는지 확인
1297	xts729.py	SQL92의 xts729.sql 파일이 실행되는지 확인
1298	xts730.py	SQL92의 xts730.sql 파일이 실행되는지 확인
1299	xts731.py	SQL92의 xts731.sql 파일이 실행되는지 확인
1300	xts732.py	SQL92의 xts732.sql 파일이 실행되는지 확인
1301	xts733.py	SQL92의 xts733.sql 파일이 실행되는지 확인
1302	xts734.py	SQL92의 xts734.sql 파일이 실행되는지 확인
1303	xts735.py	SQL92의 xts735.sql 파일이 실행되는지 확인
1304	xts736.py	SQL92의 xts736.sql 파일이 실행되는지 확인
1305	xts740.py	SQL92의 xts740.sql 파일이 실행되는지 확인
1306	xts741.py	SQL92의 xts741.sql 파일이 실행되는지 확인
1307	xts742.py	SQL92의 xts742.sql 파일이 실행되는지 확인
1308	xts744.py	SQL92의 xts744.sql 파일이 실행되는지 확인
1309	xts745.py	SQL92의 xts745.sql 파일이 실행되는지 확인
1310	xts746.py	SQL92의 xts746.sql 파일이 실행되는지 확인
1311	xts747.py	SQL92의 xts747.sql 파일이 실행되는지 확인
1312	xts748.py	SQL92의 xts748.sql 파일이 실행되는지 확인
1313	xts749.py	SQL92의 xts749.sql 파일이 실행되는지 확인
1314	xts750.py	SQL92의 xts750.sql 파일이 실행되는지 확인
1315	xts752.py	SQL92의 xts752.sql 파일이 실행되는지 확인
1316	xts753.py	SQL92의 xts753.sql 파일이 실행되는지 확인
1317	xts754.py	SQL92의 xts754.sql 파일이 실행되는지 확인
1318	xts755.py	SQL92의 xts755.sql 파일이 실행되는지 확인
1319	xts756.py	SQL92의 xts756.sql 파일이 실행되는지 확인
1320	xts760.py	SQL92의 xts760.sql 파일이 실행되는지 확인
1321	xts761.py	SQL92의 xts761.sql 파일이 실행되는지 확인
1322	xts762.py	SQL92의 xts762.sql 파일이 실행되는지 확인
1323	xts763.py	SQL92의 xts763.sql 파일이 실행되는지 확인
1324	xts764.py	SQL92의 xts764.sql 파일이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1325	xts765.py	SQL92의 xts765.sql 파일이 실행되는지 확인
1326	xts766.py	SQL92의 xts766.sql 파일이 실행되는지 확인
1327	xts767.py	SQL92의 xts767.sql 파일이 실행되는지 확인
1328	xts768.py	SQL92의 xts768.sql 파일이 실행되는지 확인
1329	xts769.py	SQL92의 xts769.sql 파일이 실행되는지 확인
1330	xts770.py	SQL92의 xts770.sql 파일이 실행되는지 확인
1331	xts771.py	SQL92의 xts771.sql 파일이 실행되는지 확인
1332	xts798.py	SQL92의 xts798.sql 파일이 실행되는지 확인
1333	xts799.py	SQL92의 xts799.sql 파일이 실행되는지 확인
1334	yts750.py	SQL92의 yts750.sql 파일이 실행되는지 확인
1335	yts751.py	SQL92의 yts751.sql 파일이 실행되는지 확인
1336	yts752.py	SQL92의 yts752.sql 파일이 실행되는지 확인
1337	yts753.py	SQL92의 yts753.sql 파일이 실행되는지 확인
1338	yts754.py	SQL92의 yts754.sql 파일이 실행되는지 확인
1339	yts755.py	SQL92의 yts755.sql 파일이 실행되는지 확인
1340	yts756.py	SQL92의 yts756.sql 파일이 실행되는지 확인
1341	yts757.py	SQL92의 yts757.sql 파일이 실행되는지 확인
1342	yts759.py	SQL92의 yts759.sql 파일이 실행되는지 확인
1343	yts760.py	SQL92의 yts760.sql 파일이 실행되는지 확인
1344	yts761.py	SQL92의 yts761.sql 파일이 실행되는지 확인
1345	yts762.py	SQL92의 yts762.sql 파일이 실행되는지 확인
1346	yts763.py	SQL92의 yts763.sql 파일이 실행되는지 확인
1347	yts764.py	SQL92의 yts764.sql 파일이 실행되는지 확인
1348	yts765.py	SQL92의 yts765.sql 파일이 실행되는지 확인
1349	yts766.py	SQL92의 yts766.sql 파일이 실행되는지 확인
1350	yts767.py	SQL92의 yts767.sql 파일이 실행되는지 확인
1351	yts768.py	SQL92의 yts768.sql 파일이 실행되는지 확인
1352	yts769.py	SQL92의 yts769.sql 파일이 실행되는지 확인
1353	yts770.py	SQL92의 yts770.sql 파일이 실행되는지 확인
1354	yts771.py	SQL92의 yts771.sql 파일이 실행되는지 확인
1355	yts772.py	SQL92의 yts772.sql 파일이 실행되는지 확인
1356	yts773.py	SQL92의 yts773.sql 파일이 실행되는지 확인
1357	yts774.py	SQL92의 yts774.sql 파일이 실행되는지 확인
1358	yts776.py	SQL92의 yts776.sql 파일이 실행되는지 확인
1359	yts777.py	SQL92의 yts777.sql 파일이 실행되는지 확인
1360	yts778.py	SQL92의 yts778.sql 파일이 실행되는지 확인
1361	yts779.py	SQL92의 yts779.sql 파일이 실행되는지 확인
1362	yts780.py	SQL92의 yts780.sql 파일이 실행되는지 확인
1363	yts781.py	SQL92의 yts781.sql 파일이 실행되는지 확인
1364	yts782.py	SQL92의 yts782.sql 파일이 실행되는지 확인
1365	yts788.py	SQL92의 yts788.sql 파일이 실행되는지 확인
1366	yts789.py	SQL92의 yts789.sql 파일이 실행되는지 확인
1367	yts790.py	SQL92의 yts790.sql 파일이 실행되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

번호	테스트케이스 스크립트	테스트 시나리오
1368	yts791.py	SQL92의 yts791.sql 파일이 실행되는지 확인
1369	yts792.py	SQL92의 yts792.sql 파일이 실행되는지 확인
1370	yts793.py	SQL92의 yts793.sql 파일이 실행되는지 확인
1371	yts794.py	SQL92의 yts794.sql 파일이 실행되는지 확인
1372	yts795.py	SQL92의 yts795.sql 파일이 실행되는지 확인
1373	yts796.py	SQL92의 yts796.sql 파일이 실행되는지 확인
1374	yts797.py	SQL92의 yts797.sql 파일이 실행되는지 확인
1375	yts798.py	SQL92의 yts798.sql 파일이 실행되는지 확인
1376	yts799.py	SQL92의 yts799.sql 파일이 실행되는지 확인
1377	yts800.py	SQL92의 yts800.sql 파일이 실행되는지 확인
1378	yts802.py	SQL92의 yts802.sql 파일이 실행되는지 확인
1379	yts803.py	SQL92의 yts803.sql 파일이 실행되는지 확인
1380	yts805.py	SQL92의 yts805.sql 파일이 실행되는지 확인
1381	yts806.py	SQL92의 yts806.sql 파일이 실행되는지 확인
1382	yts807.py	SQL92의 yts807.sql 파일이 실행되는지 확인
1383	yts808.py	SQL92의 yts808.sql 파일이 실행되는지 확인
1384	yts809.py	SQL92의 yts809.sql 파일이 실행되는지 확인
1385	yts810.py	SQL92의 yts810.sql 파일이 실행되는지 확인
1386	yts811.py	SQL92의 yts811.sql 파일이 실행되는지 확인
1387	yts812.py	SQL92의 yts812.sql 파일이 실행되는지 확인

[표 13] 빅데이터 분석용 DBMS 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

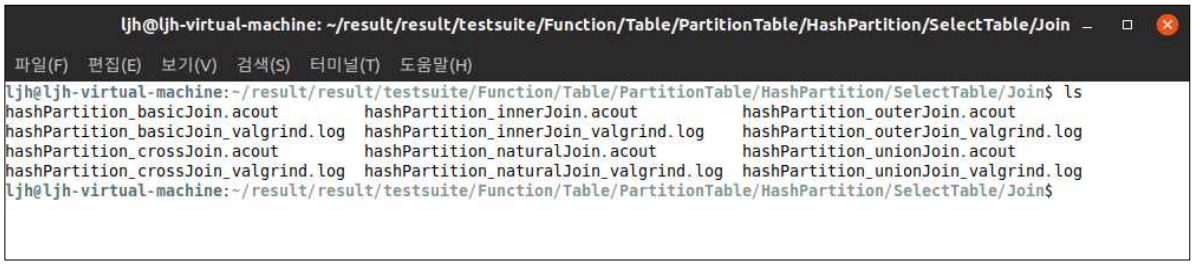
② 주요 치명 결함(메모리 누수, 무결성 오류 등)을 발견할 수 있는 테스트 케이스, 실행 스크립트

- 메모리 누수 탐지 연동을 위해 C언어 메모리 누수 검출 도구인 valgrind를 사용하였으며, 테스트 실행 시, 메모리 누수 탐지 옵션을 사용하도록 설계 및 개발함

```
$> taf Root.suite -m
```

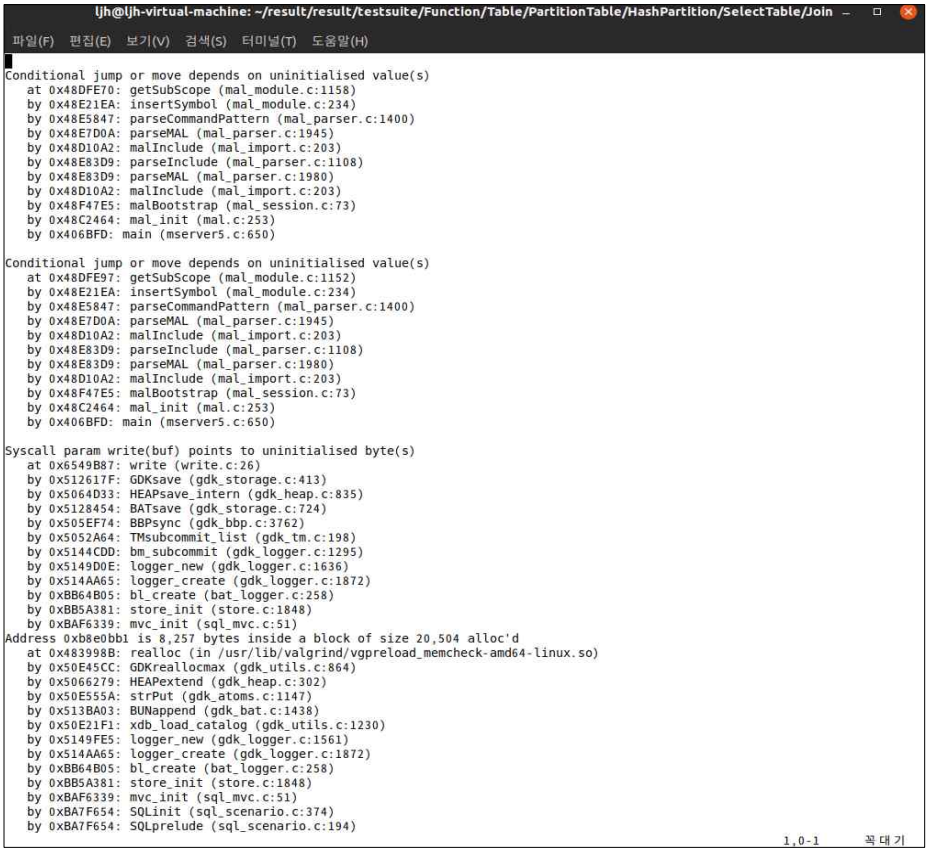
- 메모리 누수 옵션 사용 시, 자동화 도구에서 valgrind와 연동하도록 하여 DBMS 프로세스의 메모리 누수를 검증 하도록 함

- valgrind에서 발생되는 오류 로그는 하나의 로그파일로 저장되어 분석하기 매우 어려운데, 이를 개선하기 위해 각 테스트케이스 실행 마다 메모리 오류 로그를 별도로 저장하도록 설계 및 개발함



[그림 13] 메모리 누수 테스트 결과 디렉토리

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



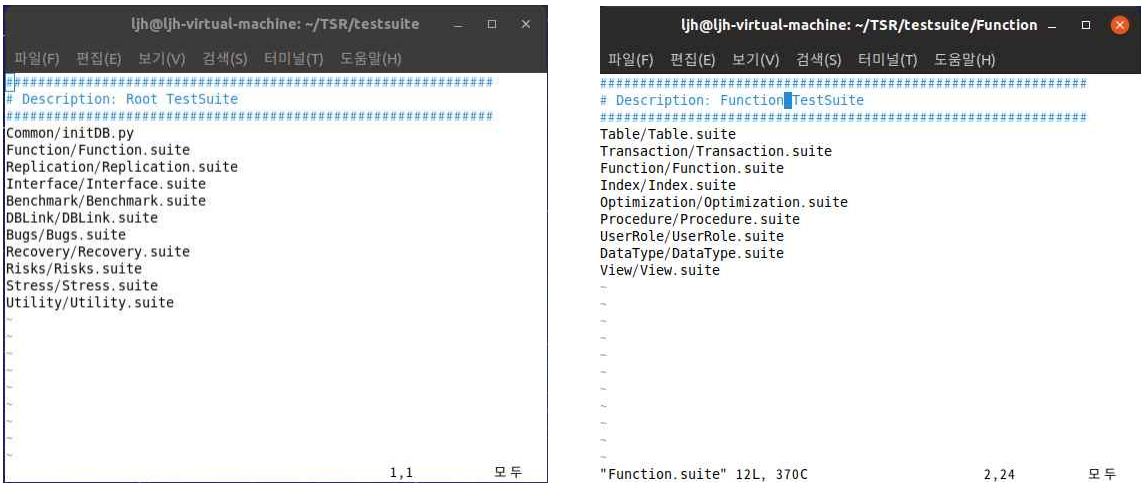
[그림 14] 메모리 누수 로그 파일

- 다중 트랜잭션 처리, 데이터 타입별 저장 방식, DB Link 등의 다양한 조건 및 환경을 고려하여 테스트 케이스를 설계함

③ 다수의 테스트케이스를 One-Stop으로 실행 할 수 있도록 테스트 자동화를 위한 스크립트

- 다수의 테스트케이스를 One-Stop으로 실행 할 수 있도록 다수 테스트케이스를 실행할 수 있도록 테스트 슈트(Test Suite)를 구성함
- 테스트 슈트는 다른 테스트 슈트를 포함 할 수 있도록 설계하여, Root 테스트 슈트(Root.suite) 실행 시, 해당 테스트 슈트에 포함되어 있는 모든 테스트 슈트 및 테스트케이스가 자동 실행되도록 설계 및 개발함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

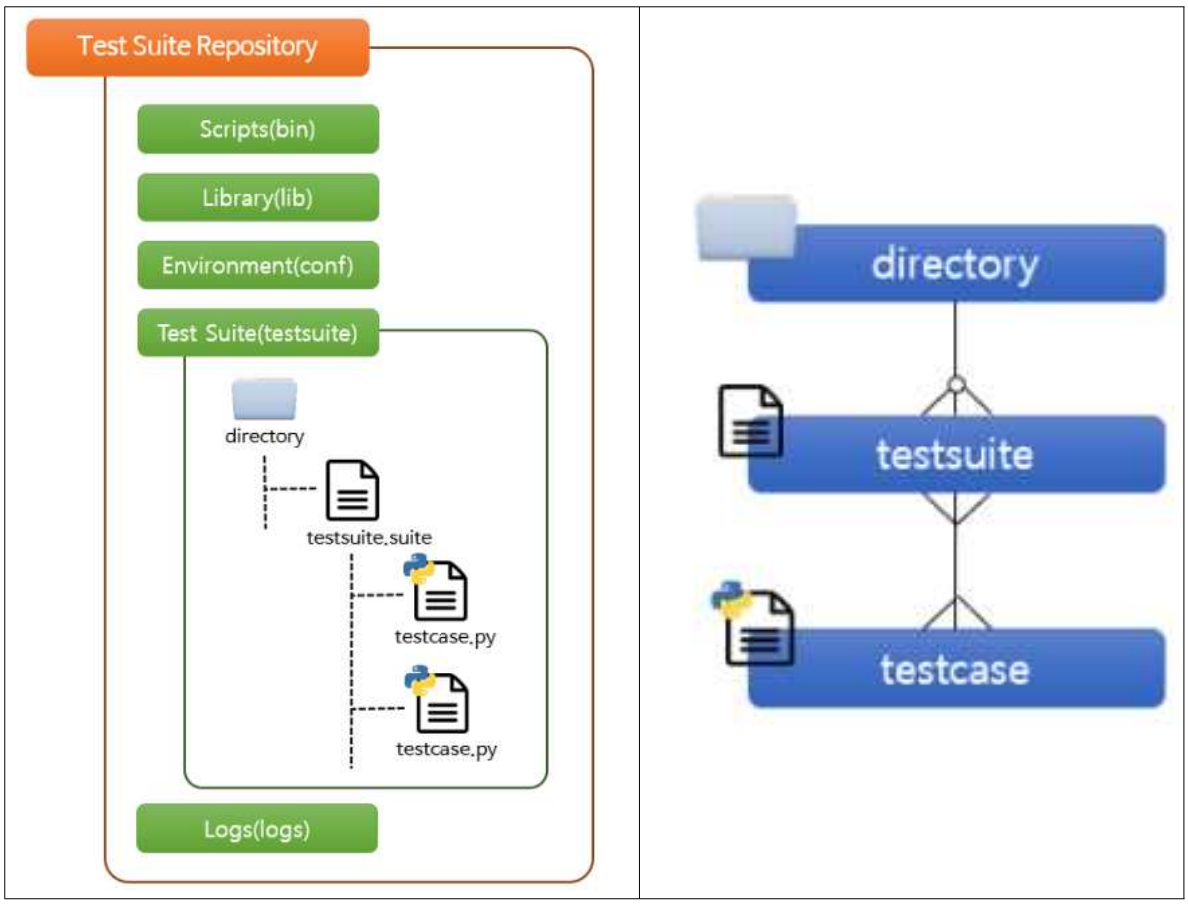


[그림 15] Root 및 Function 테스트 슈트

(2) 테스트 슈트 레파지토리(TSR) 설계 및 개발

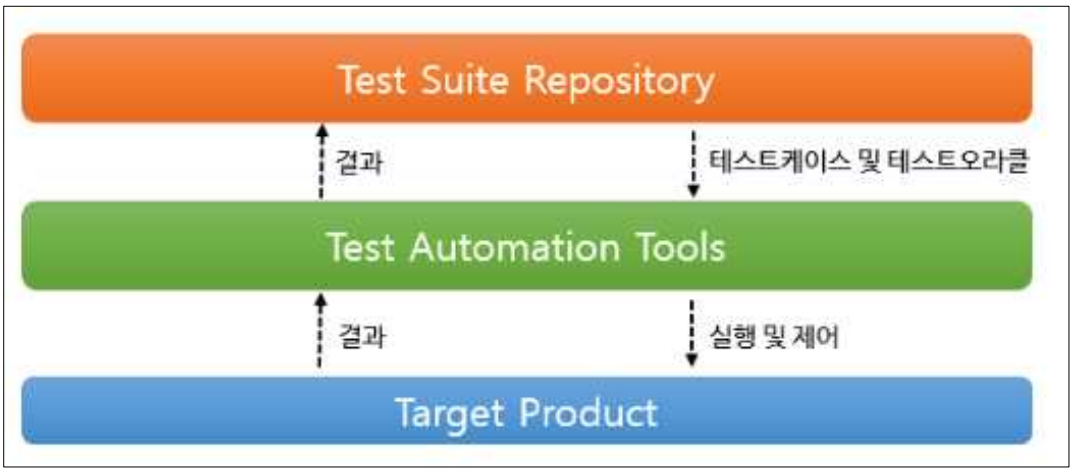
- 테스트케이스의 체계적인 관리를 위해 테스트 슈트 레파지토리를 구축하였으며, 형상관리 도구와의 연동을 위해 디렉토리 구조로 설계함
- 테스트의 의존도 및 조건 등을 고려하여 하나의 디렉토리는 다수의 테스트슈트를 포함하고, 하나의 테스트 슈트는 또 다른 테스트 슈트 또는 테스트케이스를 포함할 수 있는 구조로 설계하여 모든 테스트를 One-Stop으로 실행 하거나, 사용자가 원하는 테스트케이스를 조합하여 실행 할 수 있도록 테스트케이스 구조를 유연하게 설계함
- 테스트를 위한 라이브러리의 의존성을 해결하기 위해 별도의 테스트 라이브러리를 설치하지 않아도 테스트 할 수 있도록 Library(lib) 폴더를 생성함
- 다수 운영체제 또는 DBMS 프로세스가 실행되는 환경을 지원하기 위해 Environment(conf) 폴더를 생성하고, 설정파일을 통해 환경을 직접 설정할 수 있도록 설계함
- 테스트 실행에 필요한 별도의 유틸리티는 Scripts(bin)에 설치하도록 설계함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



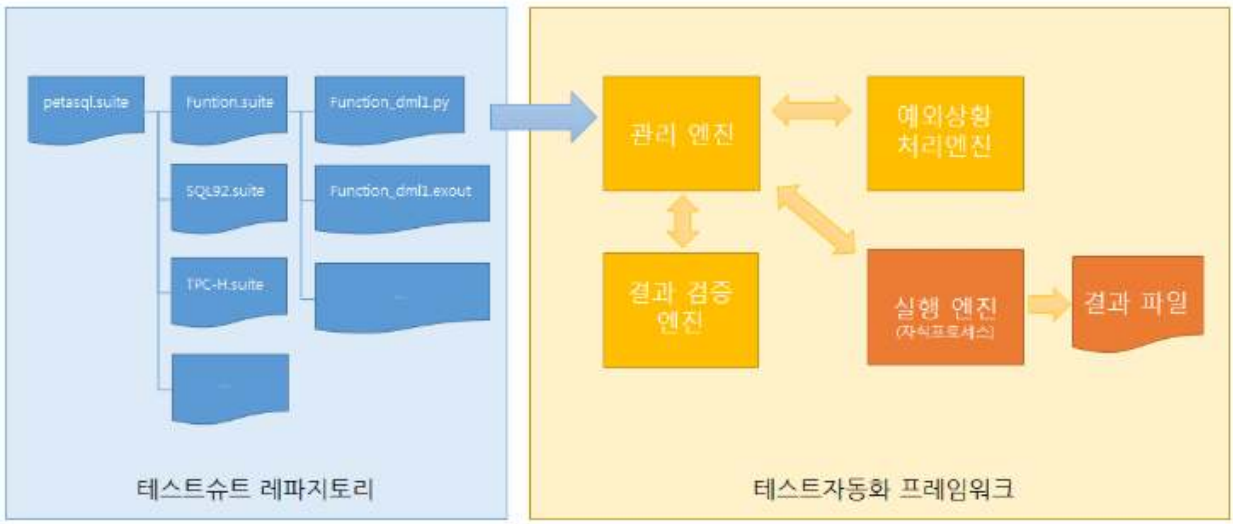
[그림 16] 테스트 슈트 레파지토리 구조

- 테스트 자동화 도구에서는 테스트 슈트 레파지토리의 테스트 슈트 또는 테스트케이스를 로딩하여 빅데이터 분석용 DBMS에 대한 테스트를 실행할 수 있도록 설계 하였고, 정답 파일인 테스트오라클을 테스트 슈트 레파지토리에 저장하도록 하여 실제 실행 결과와 예상 결과가 일치하는지 확인할 수 있도록 설계함



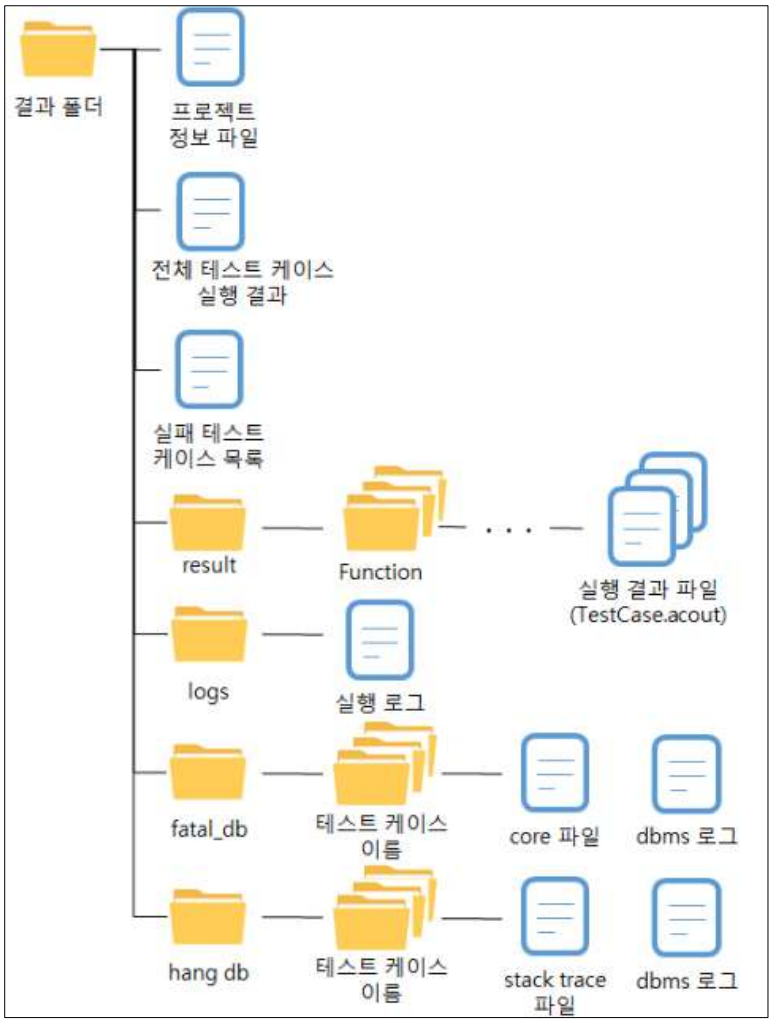
[그림 17] 테스트 슈트 레파지토리와 테스트 자동화 도구와의 데이터 흐름

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 18] 테스트슈트 레파지토리와 테스트 자동화 도구간의 관계

- 테스트 실행 결과 및 로그는 별도의 결과 폴더를 생성하여 기록 할 수 있도록 설계함



[그림 19] 결과 폴더 구조

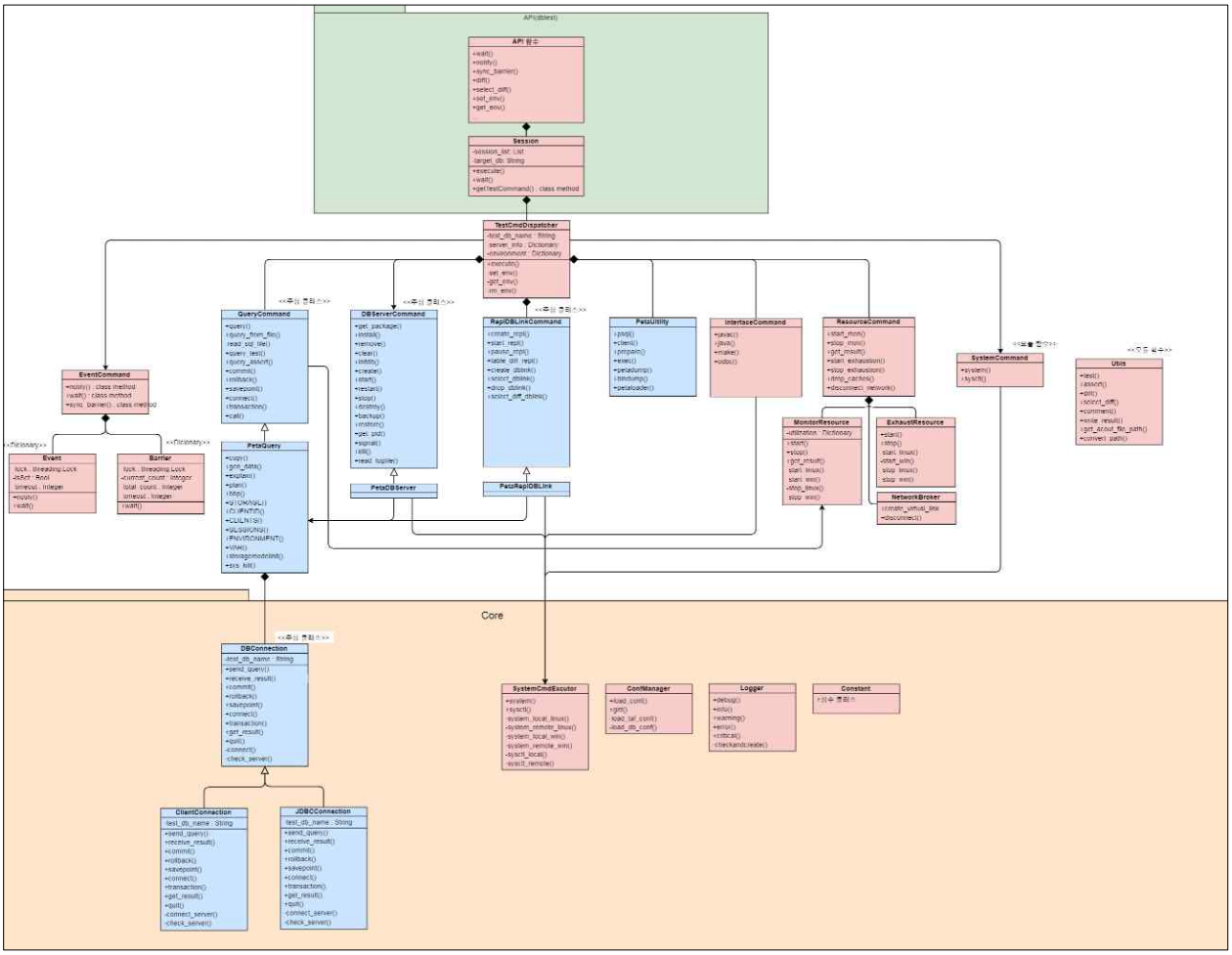
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

디렉토리	서브 디렉토리	결과물	파일명	데이터
결과 폴더	–	프로젝트 정보 파일	dbtest.info	• 수행 시작 시간, 수행 종료 시간, 테스트 수행 시간 • DBMS의 svn 정보(svn repository, revision) • 테스트슈트 레파지토리의 svn 정보(svn repository, revision) • 수행된 테스트 케이스 수, 성공/실패/FATAL/HANG 발생 수 (갯수만 요약해서 남김)
	–	실행 결과 목록	total.list	• 실행한 전체 테스트 케이스 목록 및 그 결과, 상위 suite 이름, 실행 옵션
	–	실패 테스트 케이스 목록	error.suite	• 실패, FATAL, HANG이 발생한 테스트 케이스 목록과 그 결과
	result	각 테스트 케이스별 디렉토리 및 실행 결과 파일		• result 밑의 디렉토리 구조는 테스트 슈트 레파지토리의 구조와 같게 생성됨 • 각 디렉토리에는 실패한 모든 Test Case의 실제 실행 결과 파일이 저장됨 • 메모리 결함 발생 시, 테스트 케이스별 메모리 로그가 별도의 파일로 출력됨
	logs	실행 로그	dbtest.log	• 테스트 프레임워크 실행 간 발생한 문제의 원인을 찾기 위한 로그 ○ 쿼리 실행, 그 결과 ○ 프로세스 실행, 그 결과 ○ 쉘 커멘드 실행, 그 결과 등
	fatal_db	core 파일	OS에서 생성한 파일명	프로그램 비정상 종료로 인해 core 파일이 OS에 의해서 생성되었다면 해당 core 파일을 백업함
		dbms 로그	DBMS 로그	dbms에서 남기는 실행 로그 또는 디버그 로그를 백업함
	hang_db	stack trace 파일	stack.trace	프로그램 비정상 중지 시, 리눅스의 pstack을 이용하여 DB 서버의 call-stack을 남김
		dbms 로그	DBMS 로그	dbms에서 남기는 실행 로그 또는 디버그 로그를 백업함

[표 14] 결과 폴더 설계 테이블

(3) 테스트 자동화 관리 엔진(TME) 설계 및 개발

- 품질검증 자동화 프레임워크에 대한 클래스 다이어그램을 작성하여 구조 설계를 진행함



[그림 20] 품질검증 자동화 프레임워크 클래스 다이어그램

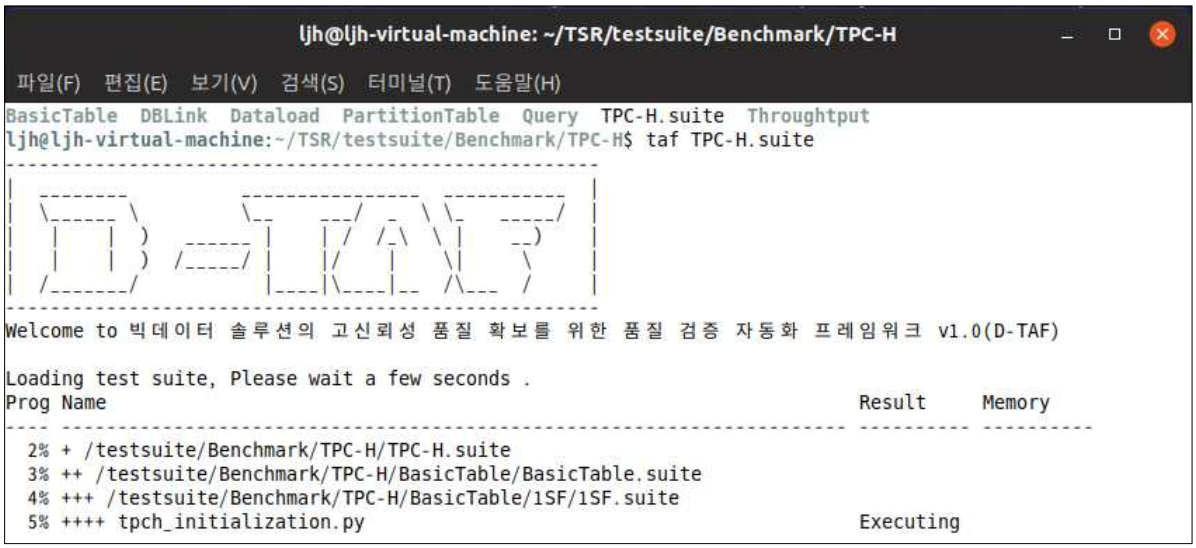
- 테스트케이스 실행 시, 의존 관계에 있는 테스트케이스를 먼저 실행할 수 있도록 의존 관계 분석기를 설계 및 개발함
- 의존 관계를 명시적으로 표현하는 경우를 고려하기 위해 테스트 슈트에 –init, --fini 옵션을 설정할 경우 해당 테스트케이스를 실행하기 전에 –init 옵션에 명시된 테스트케이스를 실행하고, 테스트케이스 실행 완료 후 –fini 옵션에 명시된 테스트케이스를 실행하도록 설계 및 개발함

```
ljh@ljh-virtual-machine: ~/TSR/testsuite/Benchmark/TPC-H/BasicTable
파일(F)  편집(E)  보기(V)  검색(S)  터미널(T)  도움말(H)
############################################################
# Description: TPC-H Basic Table TestSuite
############################################################
1SF/1SF.suite --init=1SF/tpch_initialization.py --fini=1SF/tpch_finalization.py
~
```

[그림 21] 테스트슈트를 이용한 의존 관계 설정

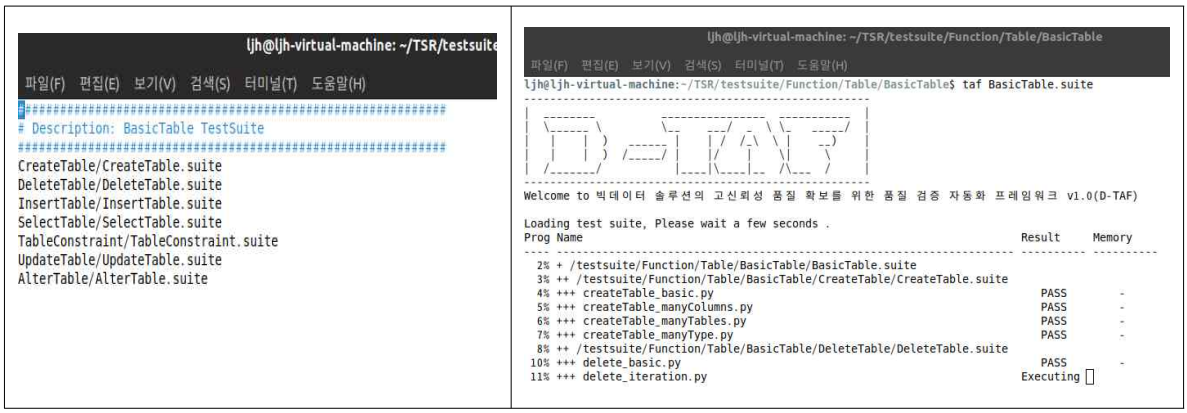
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 명시적으로 의존관계를 옵션으로 표현하지 않더라도, 패턴(init, fini)에 해당하는 테스트케이스가 테스트 디렉토리에 있을 경우 자동으로 테스트케이스간 의존관계를 분석하여 자동으로 의존관계에 있는 테스트케이스를 실행하도록 설계 및 개발함



[그림 22] 의존성을 고려한 테스트케이스 실행

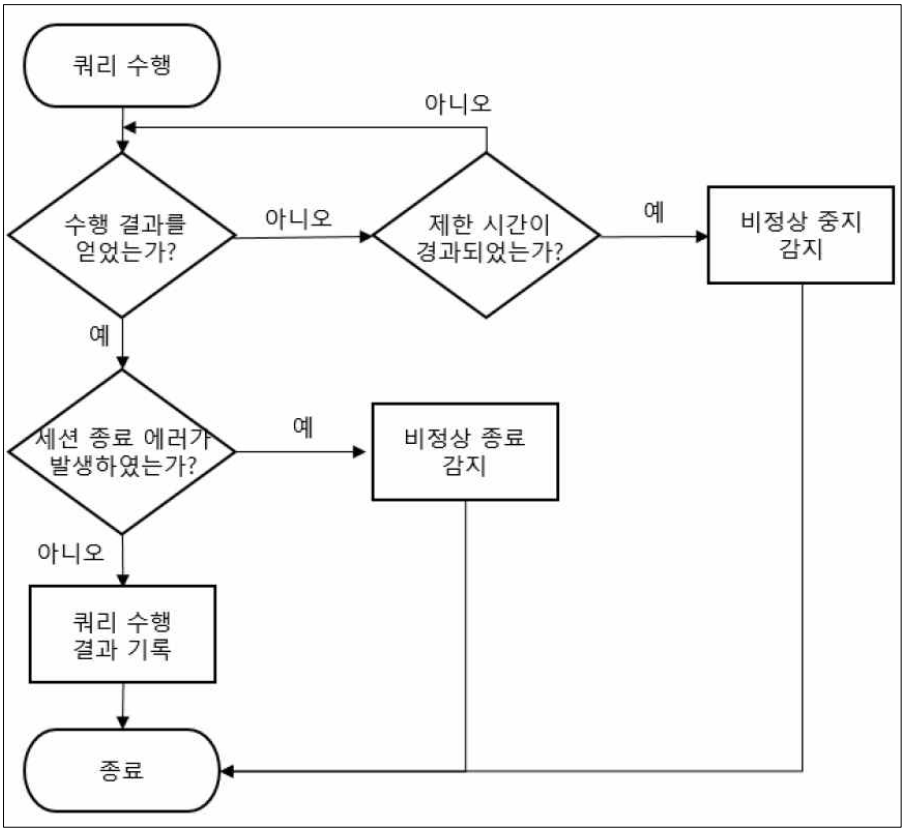
- 테스트 슈트를 통해 테스트케이스의 실행 순서를 제어하도록 설계 하여, 테스트 슈트에 기재된 테스트 슈트 및 테스트케이스의 순서에 따라 테스트케이스가 실행됨



[그림 23] 순차적 테스트케이스 실행

- 테스트 중 예외 상황 발생 시, 테스트가 지속될 수 있도록 예외 상황 감지 기능을 설계 및 개발 하였으며, 예외 상황 감지 메커니즘은 [그림 23] 과 같음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 24] 예외상황 감지 메커니즘

- DBMS 비정상 중지 및 종료 상황 발생 시, 자동으로 이를 감지하고, DBMS를 초기화 및 재구동하여 테스트 중단 없이 다음 테스트를 실행 할 수 있도록 설계 및 개발함

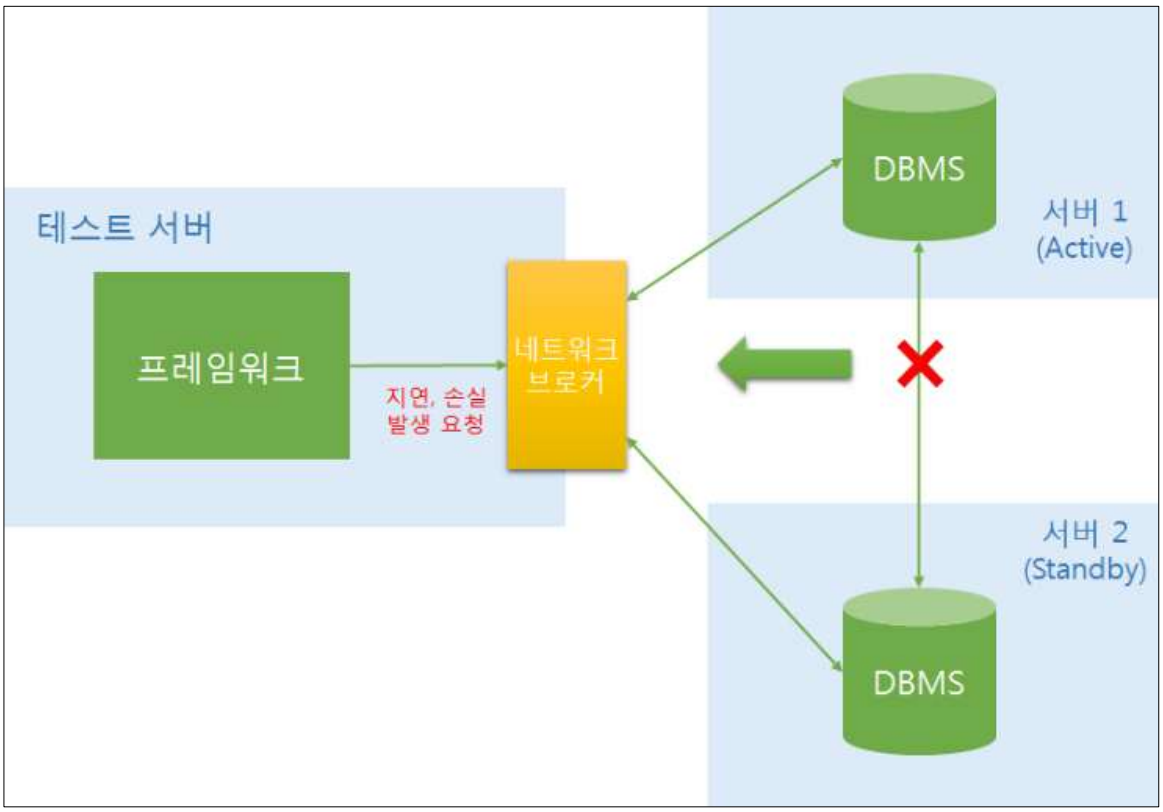
```
ljh@ljh-virtual-machine: ~/TSR/testsuite/Function/Index/HashIndex/PartitionTable/HashP
파일(F)  편집(E)  보기(V)  검색(S)  터미널(T)  도움말(H)
|  \______ \         \__    ___/  _  \ \_   _____/  |
|   |    |  )   ______ |    | /  /_\  \ |    __)    |
|   |    |  )  /_____/ |    |/    |    \|     \     |
|  /_______/           |____|\____|__  /\___  /     |
-------------------------------------------------------
Welcome to 빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 v1.0(D-TAF)

Loading test suite, Please wait a few seconds .
Prog Name                                                                    Result     Memory
---- ------------------------------------------------------------------------ ---------- ----------
 10% + /testsuite/Function/Index/HashIndex/PartitionTable/HashPartition/HashPartition.suite
 20% ++ hashPartition_index_basic.py                                          FATAL      -
 20% ----- init.py                                                           PASS       -
 30% ++ hashPartition_index_many_column.py                                    FATAL      -
 30% ----- init.py                                                           PASS       -
 40% ++ hashPartition_index_many_for_one_table.py                             FATAL      -
 40% ----- init.py                                                           PASS       -
 50% ++ hashPartition_index_many_split.py                                     PASS       -
 60% ++ hashPartition_index_data_skew.py                                      FAIL       -
 70% ++ hashPartition_index_many_dml.py                                       FATAL      -
 70% ----- init.py                                                           PASS       -
 80% ++ hashPartition_index_ddl.py                                            FATAL      -
 80% ----- init.py                                                           PASS       -
 90% ++ hashPartition_index_bigdata.py                                        PASS       -
100% ++ hashPartition_index_bad_clusteringFactor.py                           PASS       -
```

[그림 25] 서버 비정상 종료 시, 중단 없이 다음 테스트로 진행하는 화면

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 네트워크 단절, 지연 등의 예외 상황을 발생시키기 위해 네트워크 예외 발생기를 설계하여 네트워크 단절 API를 지원함



[그림 26] 네트워크 예외 발생기 개념

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

ljh@ljh-virtual-machine: ~/work/TAF/tests/basic/replication

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
# -*- coding: utf-8 -*-
import time

from dbtest import database
from dbtest.query import query, gen_data
from dbtest.replication import create_repl, start
from dbtest.resource.network import disconnect_network, remote_query
from dbtest.utils import comment, write_result

if __name__ == "__main__":
    comment("create database with broker")
    database.initdb()
    database.initdb(target_db="NODE1")

    comment("create replication")
    repl = create_repl("NODE1", master="DEFAULT", network_simul=True)

    start()

    query("create table t1 (c1 int);")

    # 네트워크 단절 API 실행
    disconnect_network("NODE1")

    # 100 건의 데이터 넣기
    row_count = 100
    gen_data("t1", row_count)

    # get count of "NODE1" server
    remote_result = remote_query("select count(*) from t1;", "NODE1")

    if remote_result.find("100") > -1:
        # 100개가 복제됐다면 네트워크 절체에 실패한 것임
        write_result("FAILURE")
    else:
        # 100개가 복제되지 않았다면 네트워크 절체에 설공
        write_result("SUCCESS")

    query("drop table t1;")
```

[그림 27] 네트워크 단절 상황을 고려한 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(4) 테스트 자동화 실행 엔진(TEE) 설계 및 개발

- 테스트케이스 실행 시, 실행 과정 및 결과를 CLI 화면에 출력하여 테스트 과정을 모니터링 할 수 있도록 설계 및 개발함

```
1% ++ initDB.py                                                                    PASS        -
1% ++ /testsuite/Function/Function.suite
1% +++ /testsuite/Function/Table/Table.suite
1% ++++ /testsuite/Function/Table/BasicTable/BasicTable.suite
1% +++++ /testsuite/Function/Table/BasicTable/CreateTable/CreateTable.suite
1% ++++++ createTable_basic.py                                                     FAIL        -
1% ++++++ createTable_manyColumns.py                                               PASS        -
1% ++++++ createTable_manyTables.py                                                PASS        -
1% ++++++ createTable_manyType.py                                                  PASS        -
1% +++++ /testsuite/Function/Table/BasicTable/DeleteTable/DeleteTable.suite
1% ++++++ delete_basic.py                                                          PASS        -
1% ++++++ delete_iteration.py                                                      PASS        -
1% ++++++ delete_manyRows.py                                                       PASS        -
1% ++++++ delete_predicate.py                                                      PASS        -
1% ++++++ /testsuite/Function/Table/BasicTable/DeleteTable/Concurrency/Concurrency.suite
1% +++++++ manyClients_oneTable_notConfilct.py                                     PASS        -
1% +++++++ manyClients_oneTable_Confilct.py                                        FAIL        -
2% +++++++ manyClients_manyTable_notConfilct.py                                    PASS        -
2% +++++++ manyClients_manyTable_Confilct.py                                       PASS        -
2% +++++ /testsuite/Function/Table/BasicTable/InsertTable/InsertTable.suite
2% ++++++ insertTable_basic.py                                                     PASS        -
2% +++++ /testsuite/Function/Table/BasicTable/SelectTable/SelectTable.suite
2% ++++++ selectTable_basic.py                                                     PASS        -
2% ++++++ /testsuite/Function/Table/BasicTable/SelectTable/Concurrency/Concurrency.suite
2% +++++++ manyClients_oneTable_lowSelectivity.py                                  PASS        -
2% +++++++ manyClients_oneTable_highSelectivity.py                                 PASS        -
2% +++++++ manyClients_manyTable_lowSelectivity.py                                 PASS        -
2% +++++++ manyClients_manyTable_highSelectivity.py                                PASS        -
2% ++++++ /testsuite/Function/Table/BasicTable/SelectTable/Groupby/Groupby.suite
2% +++++++ selectTable_groupby.py                                                  PASS        -
2% ++++++ /testsuite/Function/Table/BasicTable/SelectTable/Having/Having.suite
2% +++++++ selectTable_having.py                                                   PASS        -
2% ++++++ /testsuite/Function/Table/BasicTable/SelectTable/Join/Join.suite
2% +++++++ basicJoin.py                                                           PASS        -
2% +++++++ crossJoin.py                                                           PASS        -
3% +++++++ innerJoin.py                                                           PASS        -
3% +++++++ naturalJoin.py                                                         PASS        -
3% +++++++ outerJoin.py                                                           PASS        -
3% +++++++ unionJoin.py                                                           FAIL        -
3% ++++++ /testsuite/Function/Table/BasicTable/SelectTable/Orderby/Orderby.suite
3% +++++++ selectTable_orderby.py                                                 PASS        -
3% ++++++ /testsuite/Function/Table/BasicTable/SelectTable/Predicate/Predicate.suite
3% +++++++ predicate_builtin_function.py                                          PASS        -
3% +++++++ predicate_comparision.py                                               PASS        -
3% +++++++ predicate_like.py                                                      PASS        -
3% +++++++ predicate_null.py                                                      PASS        -
3% ++++++ /testsuite/Function/Table/BasicTable/SelectTable/Subquery/Subquery.suite
3% +++++++ subquery_all.py                                                        PASS        -
3% +++++++ subquery_any_some.py                                                   PASS        -
3% +++++++ subquery_exists.py                                                     PASS        -
3% +++++++ subquery_in.py                                                         PASS        -
3% +++++++ subquery_many.py                                                       PASS        -
3% +++++ /testsuite/Function/Table/BasicTable/TableConstraint/TableConstraint.suite
3% ++++++ constraint_default.py                                                   PASS        -
4% ++++++ constraint_null.py                                                      PASS        -
4% ++++++ constraint_primarykey.py                                                PASS        -
4% ++++++ constraint_uniquekey.py                                                 PASS        -
4% ++++++ /testsuite/Function/Table/BasicTable/TableConstraint/ReferenceOptions/ReferenceOptions.
4% +++++++ reference_on_delete_cascade.py                                         PASS        -
```

[그림 28] 테스트케이스 실행 모니터링

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 테스트 결과는 테스트가 실행 중인 상황을 포함하여 총 7개로 분류하였으며, 테스트 결과에 대한 분류는 [표 15] 와 같음

순번	테스트 결과	설명
1	Executing	테스트 실행 중
2	PASS	테스트 성공
3	FAIL	정답 파일과 다른 결과가 나옴
4	FATAL	테스트 중 DBMS가 비정상 종료됨
5	HANG	테스트 중 DBMS가 응답하지 않음
6	ERROR	테스트 중 테스트 자동화 프레임워크에서 에러가 발생함
7	COMMENTED	해당 테스트 케이스가 주석처리되어 실행되지 않음

[표 15] 테스트케이스 실행 결과 테이블

- DBMS 비정상 중지 및 종료 상황 발생 시, 테스트 정보를 백업하고 데이터베이스를 초기화한 후 서버를 재구동하도록 자동화 관리 모듈을 설계 및 개발하였으며, 백업 대상 파일은 [표 16]과 같음

순번	백업 대상	설명
1	Core 덤프 파일	메모리에 저장된 프로세스 상태 정보를 백업하여 비정상 종료 시, 메모리에 저장된 데이터를 분석하기 위해 활용함
2	Stack Trace	프로세스 실행과정을 추적할 수 있도록 콜스택(Call Stack) 정보를 백업하여 비정상 중지를 발생 시킨 함수, 쿼리 등을 분석하기 위해 활용함
3	DBMS 로그 파일	DBMS의 운영 로그, 에러 로그 등을 백업하여 예외 상황 발생 시, DBMS 오류의 원인을 분석하기 위해 활용함

[표 16] 백업대상 파일 설명

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 테스트케이스 작성을 위한 파이썬 테스트 라이브러리를 제공하여, DBMS 서버, 클라이언트 및 유틸리티를 제어 할 수 있도록 설계 및 개발함

ljh@ljh-virtual-machine: ~/TSR/testsuite/Common

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```python
#-*- coding: utf-8 -*-8

from dbtest.database import *
from dbtest.query import query

"""
 # Description : DBMS 초기화
 # Scenario :  N/A
"""

if __name__ == '__main__':

    #데이터베이스 초기화 API
    initdb()

    #쿼리문 실행 API
    query("create table t1(c1 int);");
    query("drop table t1;");
```

[그림 29] 데이터베이스를 초기화하는 테스트케이스

ljh@ljh-virtual-machine: ~/TSR/testsuite/Utility

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```python
# -*- coding: utf-8 -*-
# Copyright (C) 2019 TTA - All Rights Reserved
# Unauthorized copying of this file, via any medium is strictly prohibited
# Proprietary and confidential
# Modified by $Id: utililty_test.py 190 2019-05-09 06:00:45Z woosup.lim $
# $Date: 2019-05-08 23:00:45 -0700 (2019-05-08, 수) $

from dbtest.utility import psql, send_wait, quit, client
from dbtest.utils import write_result, comment
from dbtest.query import query
import re

if __name__ == "__main__":

    query("drop table if exists t1;", silent=True, query_silent=True)

    # Utility 제어에 대한 테스트
    comment("Start petasql")
    psql()

    query(sql_query="create table t1 ( id int );", silent=True, query_silent=True)

    for i in range(100):
        query("insert into t1 values( {} )".format(i), silent=True, query_silent=True)

    comment("Send query to petasql")
    result = send_wait("select count(*) from t1;")

    resultstr = re.sub('\(\S+\)','',result)

    write_result(resultstr)

    result = send_wait("start transaction;")

    write_result(result)

    result = send_wait("select count(*) from t1;")

    quit()

    client("-a")

    # -a가 적용되어 autocommit이 on이 되면 start transaction에서 에러가 발생함
    result = send_wait("start transaction;")

    write_result(result)

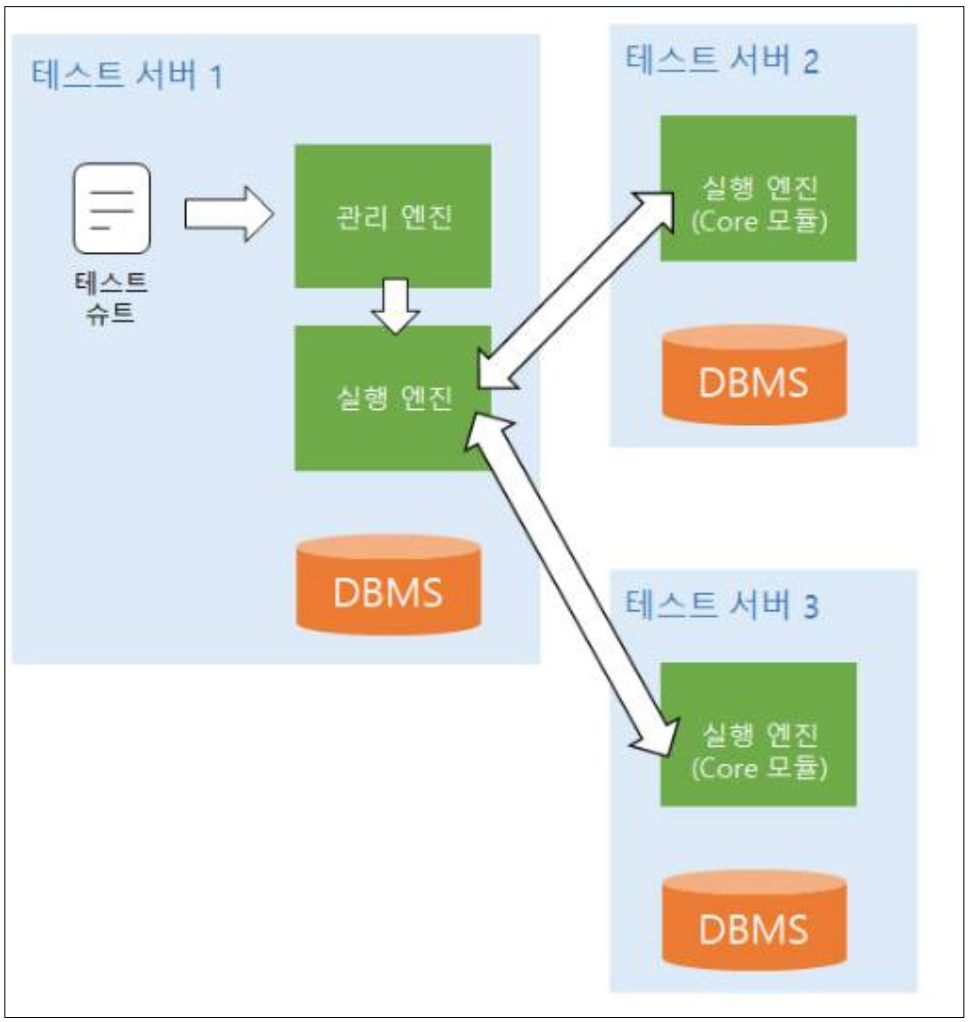
    query("drop table if exists t1;", silent=True, query_silent=True)

    quit()
```

[그림 30] 유틸리티를 제어하는 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 이중화, DBLink의 경우 2개 이상의 DBMS를 제어 및 관리해야하는데, 이를 지원하기 위해 각 서버(노드)에 설치된 실행 엔진을 통해 각 서버의 DBMS를 제어할 수 있도록 함



[그림 31] 원격 서버 제어 개념도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

ljh@ljh-virtual-machine: ~/TSR/testsuite/DBLink

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
# -*- coding: utf-8 -*-
from dbtest import database, dblink
from dbtest.query import *
from dbtest.session import Session
from dbtest.utils import comment, section


if __name__ == "__main__":

    section("DB Link Initialization")
    comment("DB Initialization")
    database.initdb()
    database.initdb(target_db="NODE1")

    comment("Create DB Link")
    newlink = dblink.create_dblink("NODE1")

    comment("Start DB Link")
    dblink.start()

    comment("Select db_link_info")
    dblink.select()

    comment("Smoke Test")

    set_session("NODE1")

    query("DROP TABLE SMOKE;", silent=True)

    query("CREATE TABLE SMOKE (IDX int);")

    for i in range(10):
        query("INSERT INTO SMOKE VALUES({})".format(i+1))

    unset_session()

    query("SELECT * FROM SYS.SMOKE@{};".format(newlink.getname()))
    query("SELECT * FROM SYS.SMOKE@{} AS S WHERE S.IDX > 5;".format(newlink.getname()))

    set_session("NODE1")

    query("DROP TABLE SMOKE;")

    unset_session()
```

[그림 32] DB Link 초기화 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 스토리지가 부족한 상황에서도 빅데이터 분석용 DBMS가 예상치 않은 오류(비정상 종료 등)가 발생하는지 확인하기 위해 스토리지의 남은 공간을 인위적으로 제어할 수 있는 테스트 API를 제공함

ljh@ljh-virtual-machine: ~/TSR/testsuite/Risks

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
# -*-coding:utf-8

from dbtest.query import *
from dbtest.utils import *
from dbtest.resource import *

"""
# Description :
# Scenario : .
"""

if __name__ == '__main__':
    comment("Initialization")

    query("DROP TABLE T1;", silent=True)

    # 30MB 공간만 남기고 스토리지 용량 소진하기
    start_stress(storage="30M")

    query("create table t1(c1 string);")

    query("insert into t1 values(rpad('x',1000000,'x'));")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")
    query("insert into t1 select * from t1;")

    comment("Finalization")

    #스토리지 소진 해제하기
    stop_stress()

    query("DROP TABLE T1;", silent=True)
```

[그림 33] 스토리지가 부족한 상황을 재현한 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 34] 스토리지가 부족한 상황을 재현한 테스트케이스 실행 결과

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 100개 이상의 다수 클라이언트가 동시에 SQL 쿼리문을 실행하는 상황을 지원하기 위해 동시에 세션을 생성하고 제어할 수 있는 테스트 API를 제공함

ljh@ljh-virtual-machine: ~/TSR/testsuite/Stress

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
# -*-coding:utf-8

from dbtest.query import *
from dbtest.utils import *
from dbtest.session import Session
from dbtest.event import sync_barrier

"""
 # Description : 다수 클라이언트가 하나의 테이블에 대한 데이터를 동시에 삽입
 # Scenario : 100개의 클라이언트가 하나의 테이블에 하나의 데이터를 동시에 삽입
"""

def client1() :

    section("Initialization")
    query("DROP TABLE BASIC;", silent=True)

    query("CREATE TABLE BASIC(  B_INT int NOT NULL AUTO_INCREMENT PRIMARY KEY, B_CHAR10 char(10) default 'TTA', "
            "B_CHAR100 char(100), B_CHAR1000 char(1000), B_VARCHAR10 varchar(10), B_VARCHAR100 varchar(100), "
            "B_VARCHAR1000 varchar(1000), B_TEXT text, B_STRING string, B_CLOB clob, B_TINYINT tinyint, "
            "B_SMALLINT smallint, B_BIGINT bigint, B_REAL real, B_DOUBLE double, B_BOOLEAN boolean NOT NULL, "
            "B_BLOB blob, B_DATE date, B_TIME time, B_TIME_WITH_TIME_ZONE time with time zone, B_TIMESTAMP timestamp, "
            "B_TIMESTAMP_WITH_TIME_ZONE timestamp with time zone, B_INTERVAL_HOUR interval hour, "
            "B_INTERVAL_MINUTE interval minute, B_INTERVAL_SECOND interval second);")
    section("TEST")

def client2() :
    for i in range(100) :
        query("INSERT INTO BASIC(B_CHAR100, B_CHAR1000, B_VARCHAR10, B_VARCHAR100, B_VARCHAR1000, B_TEXT, "
                "B_STRING, B_CLOB, B_TINYINT, B_SMALLINT, B_BIGINT, B_REAL, B_DOUBLE, B_BOOLEAN, B_BLOB, "
                "B_DATE, B_TIME, B_TIME_WITH_TIME_ZONE, B_TIMESTAMP, B_TIMESTAMP_WITH_TIME_ZONE, "
                "B_INTERVAL_HOUR, B_INTERVAL_MINUTE, B_INTERVAL_SECOND) values('ABCDE', RPAD('', 500, CODE("+ str(i) + "%5+65)), "
                "'ABCBCBDED', 'AAAAAAAAAABBBBBBBBBBCCCCCCCCCCDDDDDDDDDDEEEEEEEEEEFFFFFFFFFFGGGGGGGGGGHHHHHHHHHHIIIIIIIIIIJJJJJJJJJJ', "
                "'AAAAAAAAAABBBBBBBBBBCCCCCCCCCCDDDDDDDDDDEEEEEEEEEEFFFFFFFFFFGGGGGGGGGGHHHHHHHHHHIIIIIIIIIIJJJJJJJJJJ', "
                "RPAD('', "+str(i+1) + ", 'x'), 'AAAAAAAAAABBBBBBBBBBCCCCCCCCCCDDDDDDDDDDEEEEEEEEEEFFFFFFFFFFGGGGGGGGGGHHHHHHHHHHIIIIIIIIIIJ
JJJJJJJJJ', "
                "'AAAAAAAAAABBBBBBBBBBCCCCCCCCCCDDDDDDDDDDEEEEEEEEEEFFFFFFFFFFGGGGGGGGGGHHHHHHHHHHIIIIIIIIIIJJJJJJJJJJ', "
                "1, 500, 1234567890, 10000, 1234, "+ str(i) + "%2, CAST('12' AS BLOB), CAST('2018-11-28' AS DATE), "
                "CAST('12:30:55.123' AS TIME), CAST('21:45:34.890' AS TIME WITH TIME ZONE), "
                "CAST('2018-11-28' AS TIMESTAMP), CAST('2018-11-28' AS TIMESTAMP WITH TIME ZONE), 12, 30, 60);")

def client3() :
    query("SELECT COUNT(*) FROM BASIC WHERE B_CHAR10 = 'TTA';")

def client4() :
    section("Finalization")
    query("DROP TABLE t1;")

if __name__ == '__main__':

    s = Session("DEFAULT")

    s.execute(target_func=client1)

    s.wait()

    # 100개의 세션을 생성하고, 100개의 세션이 client2 함수를 동시 실행
    s.execute(target_func=client2, session_cnt=100)

    # 100개의 세션이 모두 끝날때 까지 기다림
    s.wait()

    s.execute(target_func=client3)

    s.wait()

    s.execute(target_func=client4)
```

[그림 35] 100개의 클라이언트를 생성하여 동시에 INSERT 쿼리문을 실행하는 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- Benchmark(TPC-H, TPC-DS) 테스트 검증을 위해 한 번의 함수 실행으로도 TPC 스키마 생성, 데이터 로딩, 인덱스 생성, 제약조건 설정 등을 할 수 있는 API를 개발 하였고, 복잡한 TPC 쿼리문을 쉽게 호출 할 수 있는 API를 제공함

ljh@ljh-virtual-machine: ~/TSR/testsuite/Benchmark/TPC-H/BasicTable/1SF

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
#-*- coding: utf-8 -*-8

from dbtest.benchmark import *
from dbtest.query import *

"""
 # Description : TPC-H 데이터 로딩
 # Scenario :
    1. dat 압축 해제
    2. 스키마 생성
    3. 데이터 로딩
    4. 제약조건 설정
    5. 인덱스 생성
"""

if __name__ == '__main__':

    # TPC-H 생성 함수
    # TPC-H 스키마 생성, 데이터 로딩, 인덱스 생성, 제약조건 설정을 한번에 진행
    tpch_create()

    # TPC-H 테이블 별 데이터 개수 확인
    query("SELECT COUNT(*) FROM REGION;")
    query("SELECT COUNT(*) FROM NATION;")
    query("SELECT COUNT(*) FROM CUSTOMER;")
    query("SELECT COUNT(*) FROM SUPPLIER;")
    query("SELECT COUNT(*) FROM PARTSUPP;")
    query("SELECT COUNT(*) FROM PART;")
    query("SELECT COUNT(*) FROM ORDERS;")
    query("SELECT COUNT(*) FROM LINEITEM;")
```

[그림 36] TPC-H 환경을 설정하는 테스트케이스

ljh@ljh-virtual-machine: ~/TSR/testsuite/Benchmark/TPC-H/BasicTable/1SF

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
# -*-coding:utf-8

from dbtest.benchmark import *

"""
 # Description : (Non-PartitionTable)TPC-H 1번 쿼리 수행
 # Scenario : N/A
"""

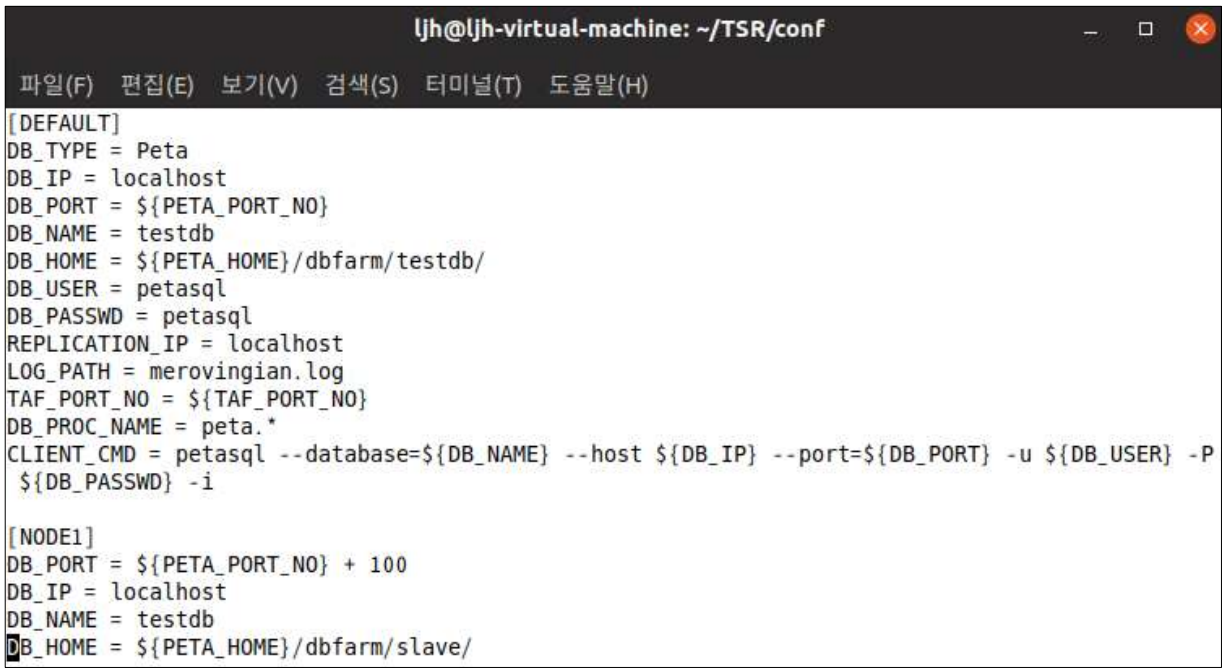
if __name__ == '__main__':

    tpch_query(1)
```

[그림 37] TPC-H 1번 쿼리를 실행하는 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

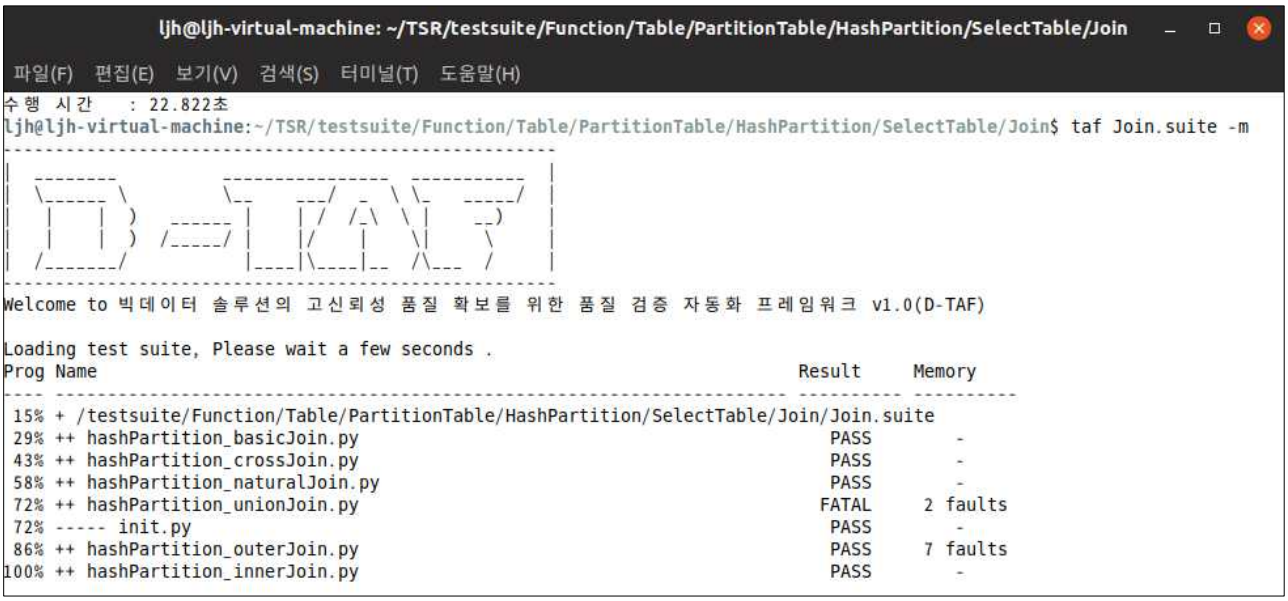
- 각 서버(노드)의 DBMS를 제어하기 위한 정보(IP주소, 포트번호, 데이터베이스명 등)를 설정하기 위한 설정 파일(db.conf)을 제공함



[그림 38] db.conf

- 메모리 누수 원인 분석 시, 작성된 테스트케이스를 재활용하기 위해 -m 옵션을 사용하여 테스트를 실행 할 경우 자동으로 valgrind와 연동하여 각 테스트케이스 실행 시, 메모리 누수 여부를 판단할 수 있도록 설계 및 개발함

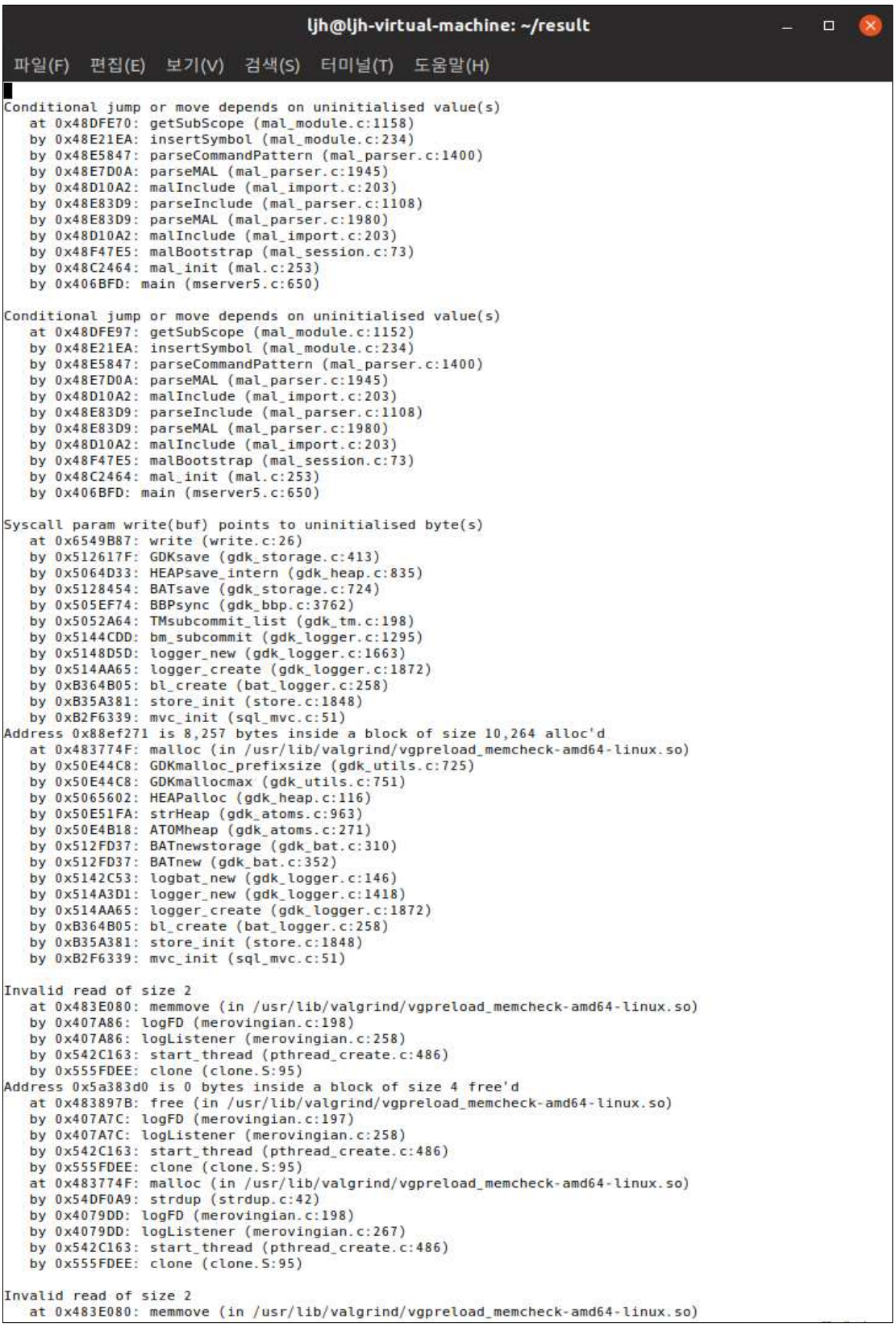
- 테스트케이스 실행 시, 메모리 누수가 발생될 경우 CLI 화면에 해당 테스트케이스의 메모리 누수 결과를 표시하도록 설계 및 개발함



[그림 39] 메모리 누수 결과 모니터링

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 결함 격리를 효과적으로 하기 위해 메모리 누수 결과를 테스트케이스 별로 저장되도록 설계함



[그림 40] 테스트케이스별 메모리 누수 결과

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 메모리 누수 결과의 일부가 오탐(False Positive)일 경우를 고려하기 위해 해당 메모리 누수 오류를 다시 나오지 않도록 메모리 누수 필터링 도구를 설계 및 개발함

순번	메모리 누수 유형	메모리 누수 심볼
1	Conditional jump or move depends on uninitialized value(s)	UNINIT_VALUE
2	Syscall param write(buf) points to uninitialised byte(s)	UNINIT_WRITE_PARAM_VALUE
3	Syscall param msync(error_code) contains uninitialised byte(s)	UNINIT_SYNC_PARAM_VALUE
4	Syscall param pwrite64(buf) points to uninitialised byte(s)	UNINIT_PWRITE_PARAM_VALUE
5	4 bytes in 1 blocks are definitely lost in loss record 1,783 of 2,936	DEFINITELY_LOST
6	4 bytes in 1 blocks are indirectly lost in loss record 1,783 of 2,936	INDIRECTLY_LOST
7	64 bytes in 2 blocks are possibly lost in loss record 1,782 of 2,936	POSSIBLY_LOST
8	1,176 bytes in 49 blocks are still reachable in loss record 2,456 of 2,936	STILL_REACHABLE_LOST
9	Invalid read of size 1	INVALID_MEM_READ_SIZE1
10	Invalid read of size 2	INVALID_MEM_READ_SIZE2
11	Invalid read of size 4	INVALID_MEM_READ_SIZE4
12	Invalid read of size 8	INVALID_MEM_READ_SIZE8
13	Invalid read of size 16	INVALID_MEM_READ_SIZE16
14	Invalid write of size 1	INVALID_MEM_WRITE_SIZE1
15	Invalid write of size 2	INVALID_MEM_WRITE_SIZE2
16	Invalid write of size 4	INVALID_MEM_WRITE_SIZE4
17	Invalid write of size 8	INVALID_MEM_WRITE_SIZE8
18	Invalid write of size 16	INVALID_MEM_WRITE_SIZE16
19	Jump to the invalid address stated on the next line	INVALID_ADDRESS
20	Invalid free	INVALID_FREE
21	Mismatched free	MISMATCHED_FREE
22	Argument 'size' of function malloc has a fishy (possibly negative) value: -3	FISHY_ARGUMENT_VALUE

[표 17] 메모리 누수 유형 및 심볼 테이블

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 41] valgrind_filter.conf

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- ㈜웨어밸리의 빅데이터 분석용 DBMS 뿐만 아니라, 다른 DBMS도 검증 가능하도록 확장성을 고려하여 설계 및 개발함

ljh@ljh-virtual-machine: ~

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
# -*-coding:utf-8

from dbtest.query import *
from dbtest.utils import *

"""
 # Description : PostgreSQL 연동 테스트
 # Scenario : PostgreSQL SQL문이 정상적으로 동작하는지 확인한다.
"""

if __name__ == '__main__':
    section("Initialization")

    query("DROP TABLE weather;", silent=True)

    section("TEST")

    comment("CREATE weather TABLE")

    query("CREATE TABLE weather ("
          "city            varchar(80),"
          "temp_lo         int,"
          "temp_hi         int,"
          "prcp            real,"
          "date            date"
          ");")

    comment("INSERT TABLE")

    query("INSERT INTO weather VALUES ('San Francisco', 46, 50, 0.25, '1994-11-27');")
    query("INSERT INTO weather VALUES ('San Francisco', 43, 55, 0, '1994-11-29');")
    query("INSERT INTO weather VALUES ('Hayward', 37, 54, 0.75, '1994-11-29');")

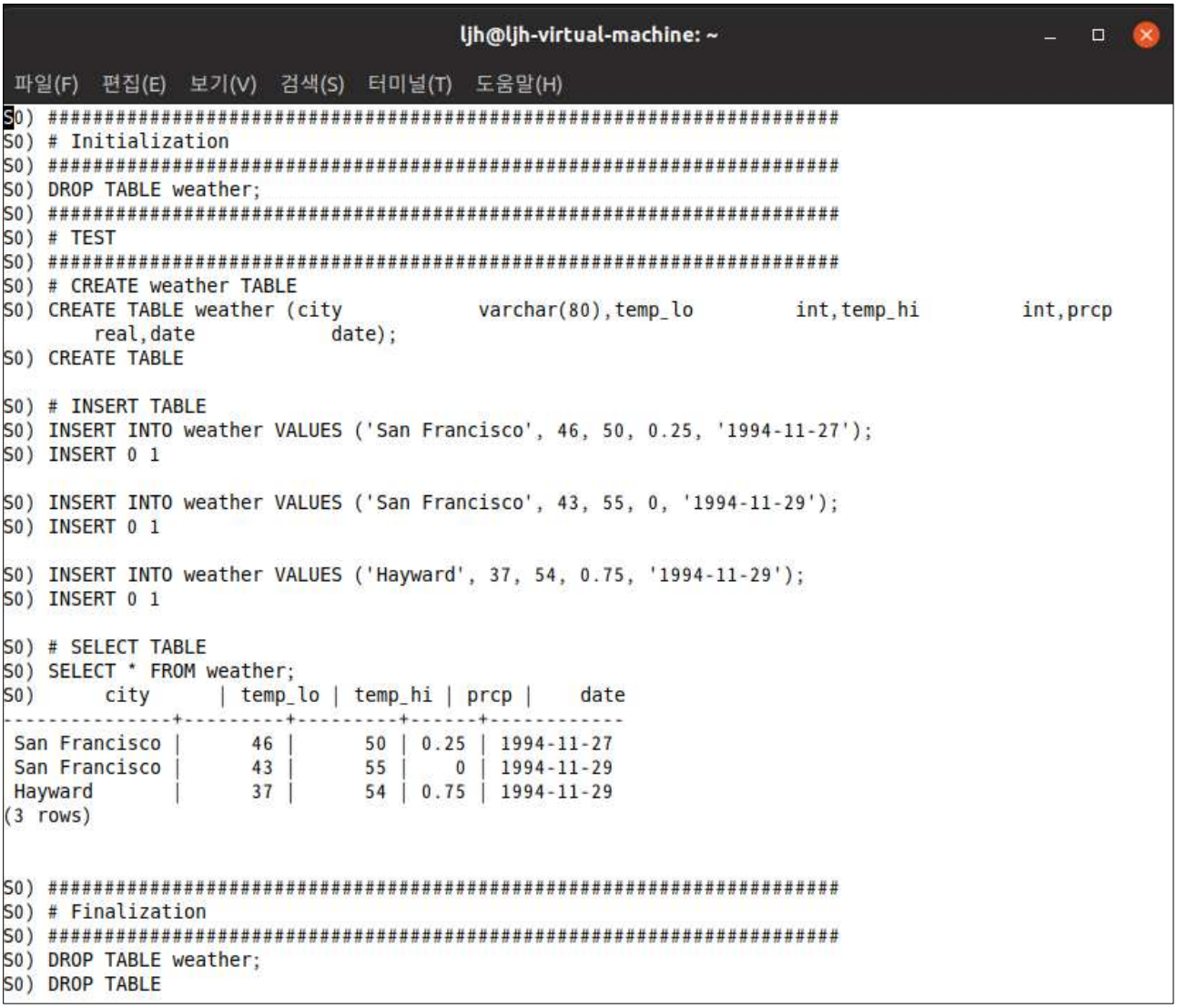
    comment("SELECT TABLE")
    query("SELECT * FROM weather;")

    section("Finalization")

    query("DROP TABLE weather;")
```

[그림 42] 타 DBMS(PostgreSQL) 테스트케이스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

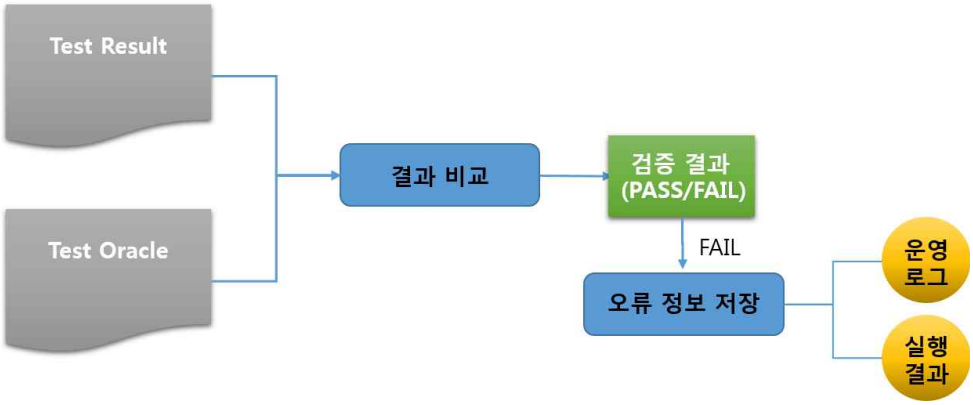


[그림 43] 타 DBMS(PostgreSQL) 테스트케이스 실행 결과

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(5) 테스트 실행 결과 검증 엔진 설계 및 개발

- 테스트 케이스별 기대 출력과 실제 출력을 비교하기 위해 테스트케이스 실행 결과 검증 엔진을 설계 및 개발하였으며, 구성도는 [그림 41]과 같음



[그림 44] 테스트 자동화 실행 결과 검증 엔진 구성도

- 테스트케이스(예: test.py) 실행 시, 실제 실행 결과는 test.acout 파일에 저장되며, 사전에 실행하여 검증된 정답파일(test.exout 파일)과 text 비교(diff)하여 정확히 일치할 경우, PASS 그렇지 않을 경우 FAIL로 결과를 판단함

ljh@ljh-virtual-machine: ~/TSR/testsuite/Function/Table/BasicTable/CreateTable

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
# -*-coding:utf-8

from dbtest.query import *
from dbtest.utils import *

"""
 # Description : 기본 테이블 생성 테스트
 # Scenario : N/A
"""

if __name__ == '__main__':
    section("Initialization")

    query("DROP TABLE T1;", silent=True)

    section("TEST")

    query("CREATE TABLE T1(C1 int, C2 CHAR(100));")

    query("INSERT INTO T1 VALUES(1, 'A');")
    query("INSERT INTO T1 VALUES(2, 'B');")
    query("INSERT INTO T1 VALUES(3, 'C');")
    query("INSERT INTO T1 VALUES(4, 'D');")
    query("INSERT INTO T1 VALUES(5, 'E');")

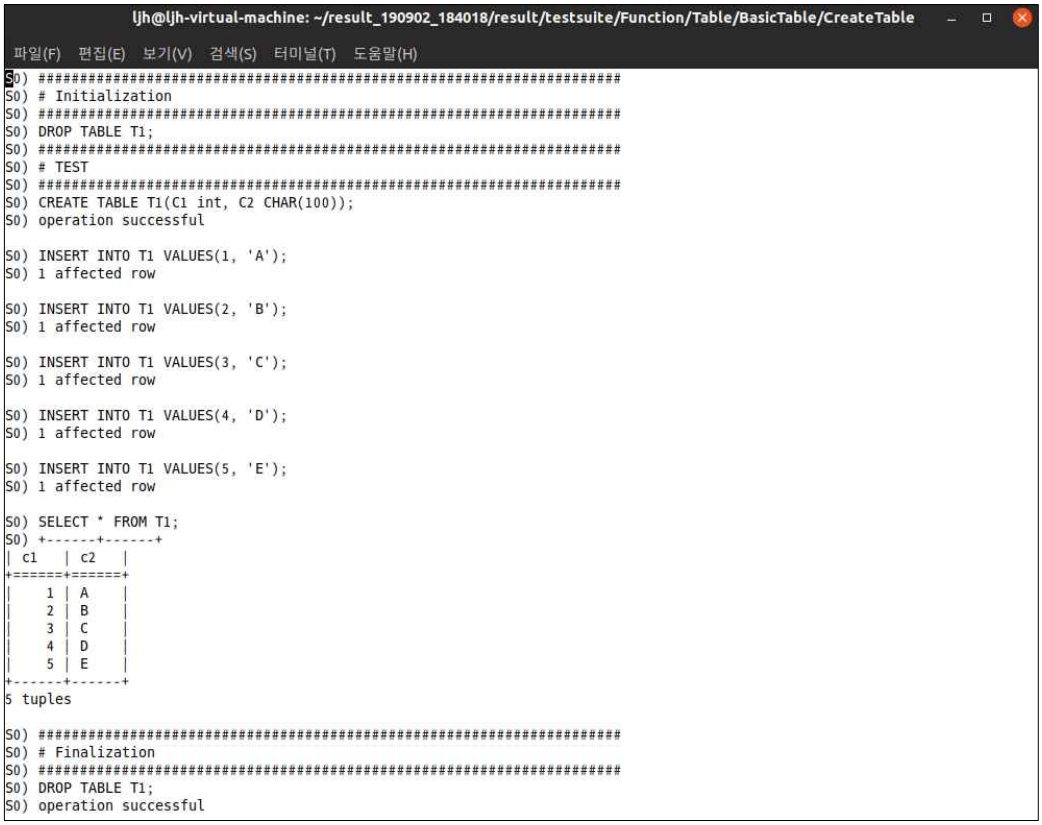
    query("SELECT * FROM T1;")

    section("Finalization")

    query("DROP TABLE T1;")
```

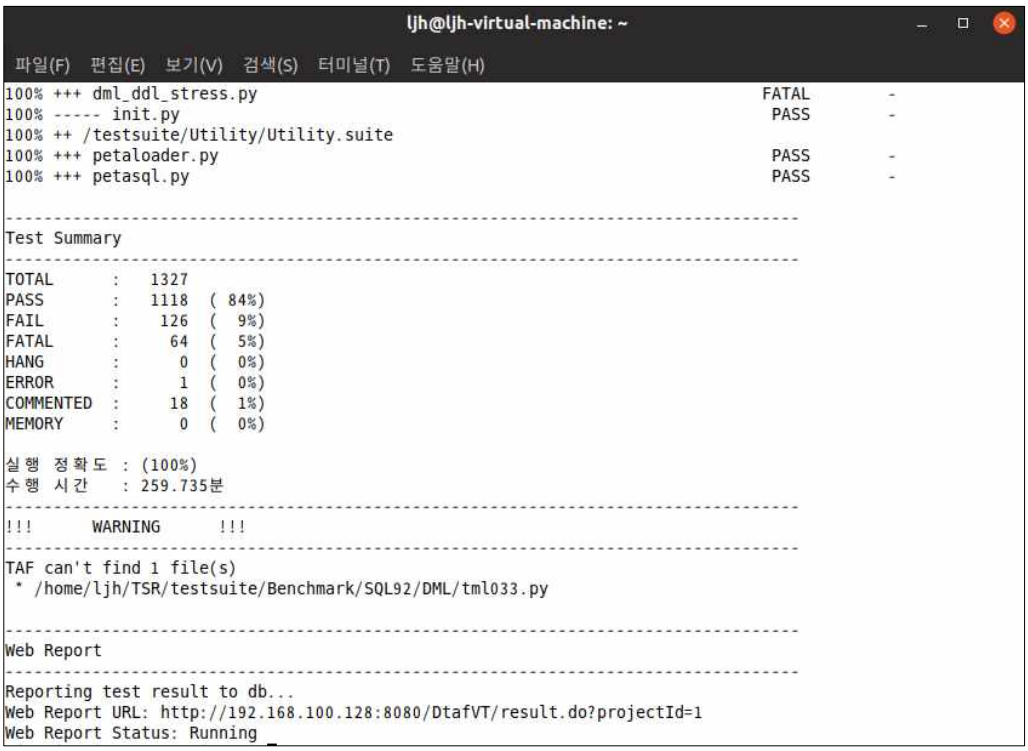
[그림 45] 테스트케이스(createTable_basic.py)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 46] 정답 파일 (createTable_basic.exout)

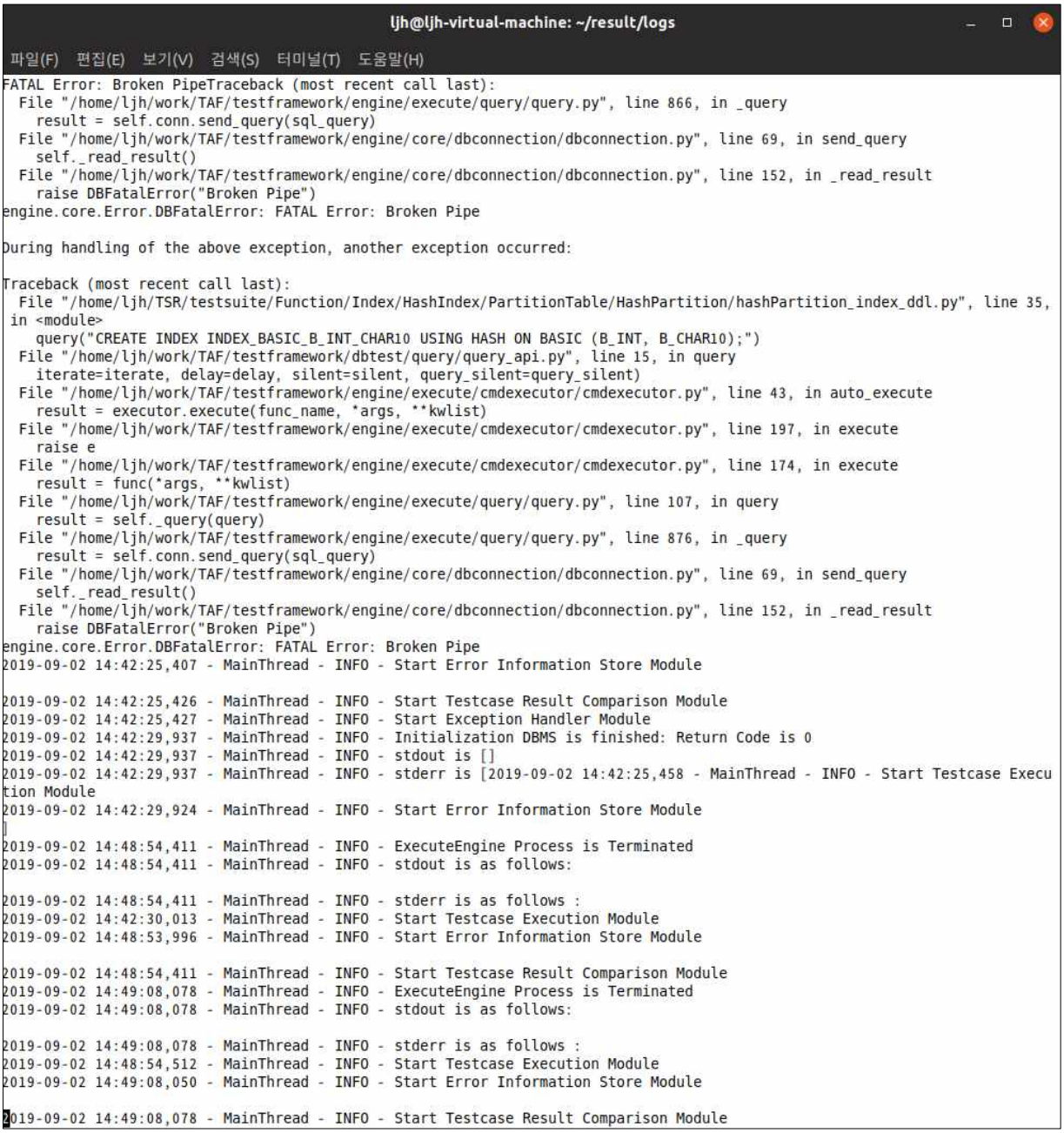
- 테스트 실행 완료 후, Test Summary를 제공하여 테스트 결과에 대한 통계 정보를 제공하도록 설계 및 개발함



[그림 47] Test Summary 화면

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 실패된 테스트케이스의 원인을 분석하기 위해 테스트케이스 실행 로그를 별도의 로그파일에 저장하도록 설계 및 개발함

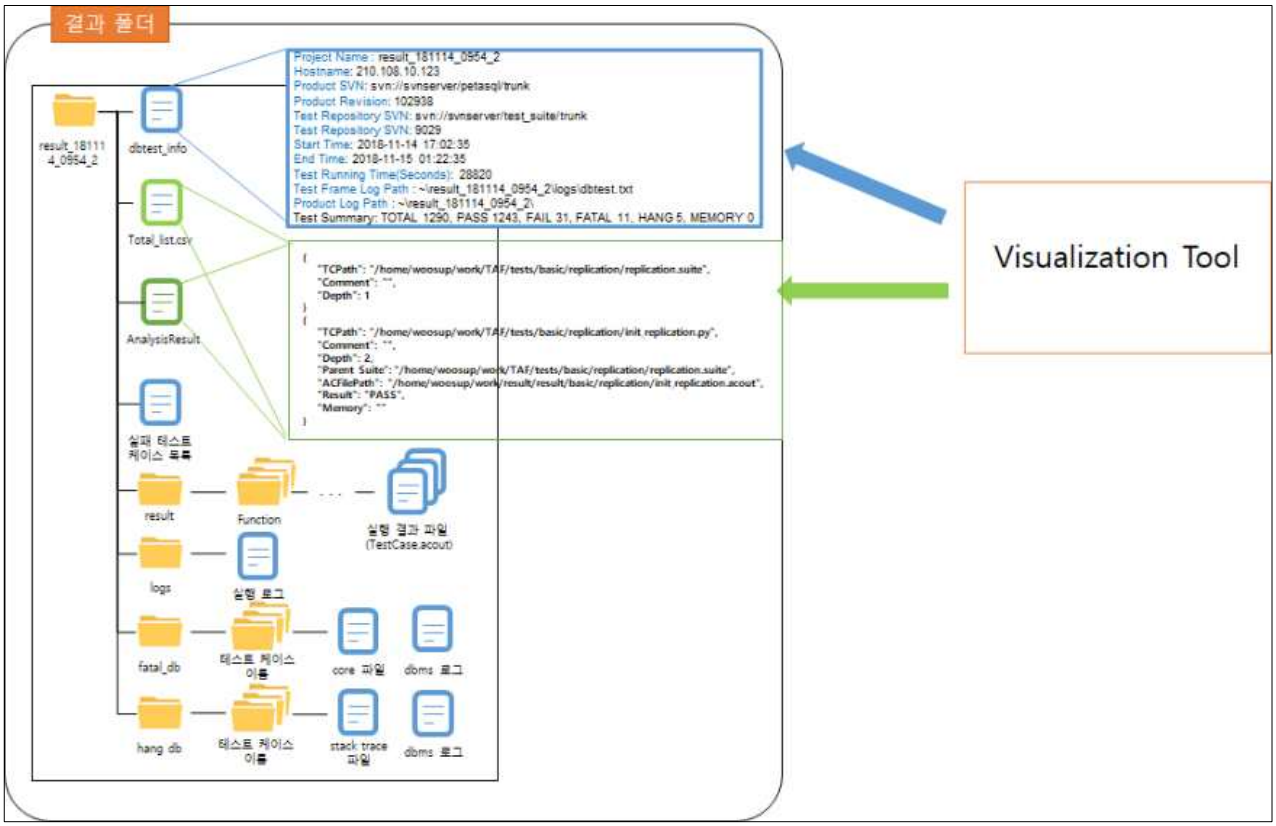


[그림 48] dbtest.log

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(6) 테스트 결과 시각화 도구 설계 및 개발

- 시각화 도구에서 테스트 결과를 보여주기 위해 테스트 결과 폴더에 시각화 도구 실행에 필요한 데이터를 분석하여 구조화 함



[그림 49] 시각화 도구와 테스트 폴더간의 인터페이스

- 시각화 도구에서 테스트 결과를 분석 및 관리하기 위해 데이터를 DBMS를 통해 관리하도록 설계하였음

```
CREATE TABLE USERS (
  NAME                          VARCHAR2(100),
  PASSWD                        VARCHAR2(100)
);
CREATE TABLE TEST_RESULT (
  RESULT_ID                     IDENTITY,
  RESULT_NAME                   VARCHAR2(20) UNIQUE
);
CREATE TABLE CATEGORY (
  CATEGORY_ID                   IDENTITY,
  CATEGORY_NAME                         VARCHAR2(20) UNIQUE
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
);
CREATE TABLE CAUSE (
  CAUSE_ID                    IDENTITY,
  CAUSE_NAME                  VARCHAR2(20) UNIQUE
);
CREATE TABLE PROJECTS (
  PROJECT_ID                  IDENTITY PRIMARY KEY,
  PROJECT_NAME                VARCHAR2(200),
  HOSTNAME                          VARCHAR2(100),
  PRODUCT_URL                 VARCHAR2(200),
  PRODUCT_REVISION    VARCHAR2(10),
  TSR_URL                           VARCHAR2(200),
  TSR_REVISION        VARCHAR2(10),
  START_TIME                  VARCHAR2(30),
  END_TIME                          VARCHAR2(30),
  DB_LOG_PATH                 VARCHAR2(200),
  DTAF_LOG_PATH               VARCHAR2(200),
  PASS_CNT                          VARCHAR2(10),
  FAIL_CNT                          VARCHAR2(10),
  FATAL_CNT                   VARCHAR2(10),
  HANG_CNT                          VARCHAR2(10),
  ERROR_CNT                   VARCHAR2(10),
  COMMENTED_CNT               VARCHAR2(10),
  MEMORY_CNT                  VARCHAR2(10),

  IS_MEMCHECK                 CHAR(1),
  TSR_HOME                    VARCHAR2(200),
  RESULT_ROOT                 VARCHAR2(200)
);
CREATE TABLE RESULTS
(  RESULT_ID                  IDENTITY PRIMARY KEY,
  PROJECT_ID          BIGINT,
  RESULT_NAME                 VARCHAR2(200),
  RESULT                            VARCHAR2(20),
  ORACLE_PATH                 VARCHAR2(200),
  RESULT_PATH                 VARCHAR2(200),
  TC_PATH                           VARCHAR2(200),
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
  MEMORY_FAULT                VARCHAR2(2000),
  MEMORY_LOG_PATH             VARCHAR2(200),
  CATEGORY                          VARCHAR2(30),
  CAUSE                       VARCHAR2(30),
  COMMENTS                          VARCHAR2(1000),
  CLASS                       VARCHAR2(100),
  RUNNING_TIME                DOUBLE,
  IS_EXOUT_UPDATED            VARCHAR2(100),
  FOREIGN KEY (PROJECT_ID) REFERENCES PROJECTS(PROJECT_ID),
  FOREIGN KEY (RESULT) REFERENCES TEST_RESULT(RESULT_NAME),
  FOREIGN KEY (CATEGORY) REFERENCES CATEGORY(CATEGORY_NAME),
  FOREIGN KEY (CAUSE) REFERENCES CAUSE(CAUSE_NAME)
);
INSERT INTO USERS (NAME, PASSWD) VALUES('admin',
'$2a$10$yt3kGaT4Yje/pHFrrOATY.oSWvgGTg0NuZZpjZRyP4OTCounzH5Tq');
INSERT INTO TEST_RESULT (RESULT_NAME) VALUES('PASS');
INSERT INTO TEST_RESULT (RESULT_NAME) VALUES('FAIL');
INSERT INTO TEST_RESULT (RESULT_NAME) VALUES('FATAL');
INSERT INTO TEST_RESULT (RESULT_NAME) VALUES('HANG');
INSERT INTO TEST_RESULT (RESULT_NAME) VALUES('ERROR');
INSERT INTO TEST_RESULT (RESULT_NAME) VALUES('MEMORY LEAK');
INSERT INTO TEST_RESULT (RESULT_NAME) VALUES('COMMENTED');
INSERT INTO CATEGORY (CATEGORY_NAME) VALUES('TIMING');
INSERT INTO CATEGORY (CATEGORY_NAME) VALUES('SPACE');
INSERT INTO CATEGORY (CATEGORY_NAME) VALUES('DATE');
INSERT INTO CATEGORY (CATEGORY_NAME) VALUES('ORDERING');
INSERT INTO CAUSE (CAUSE_NAME) VALUES('TRUE POSITIVE');
INSERT INTO CAUSE (CAUSE_NAME) VALUES('FALSE POSITIVE');
```

[표 18] 시각화 도구 데이터 스키마

- 테스트 자동화 도구를 사용하여 테스트를 실행 시, 시각화 도구에 접속 가능한 URL 주소가 출력되고, 사용자는 해당 URL 주소를 통해 테스트 결과 확인이 가능하도록 설계 및 개발함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

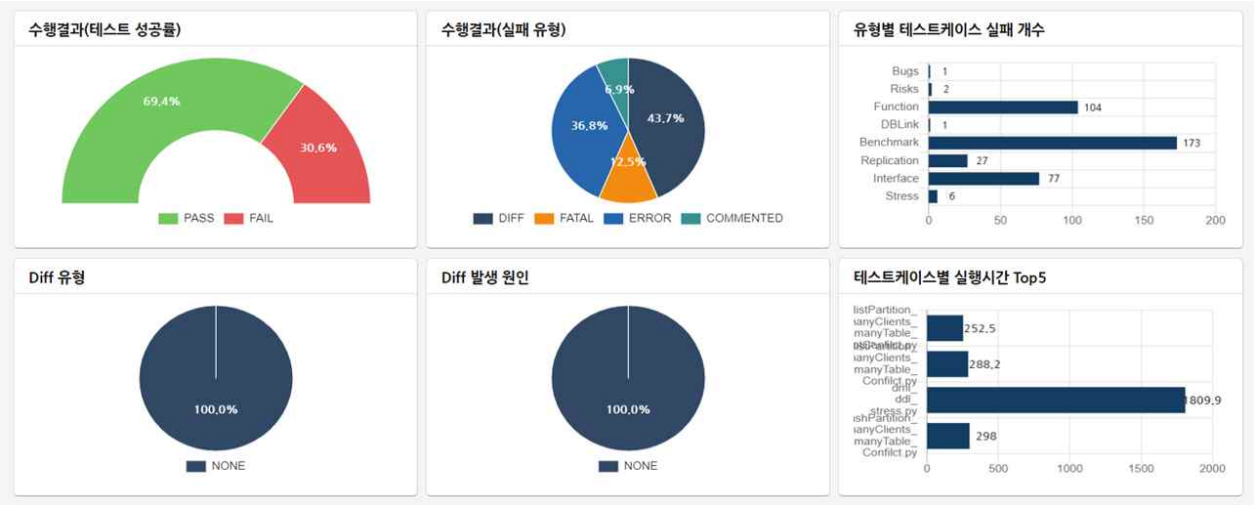
```
--------------------------------------------------------------------------------
Web Report
--------------------------------------------------------------------------------
Reporting test result to db...
Web Report URL: http://192.168.100.128:8080/DtafVT/result.do?projectId=1
Web Report Status: Running
```

[그림 50] Web Report 안내 화면



[그림 51] 시각화 도구(웹) 메인 페이지 화면

- 시각화 도구를 통해 실행된 테스트에 대한 테스트 성공률, 실패 유형 등 통계 정보를 그래프로 확인이 가능함



[그림 52] 테스트 실행 결과 통계 화면

- 실패한 각 테스트케이스의 정보(실행 결과, Diff, TC)가 출력되도록 하였고, 실패한 테스트케이스 확인 및 조치 사항 기록을 위해 Diff 분류, 분석 결과, Comments 항목을 추가함
- 조치 사항 기록을 통해 다수의 사용자가 시각화 도구에 접속하여 테스트케이스 실행 결과 및 조치 결과 공유 등 협업이 가능함



[그림 53] 실패한 테스트케이스 목록 화면

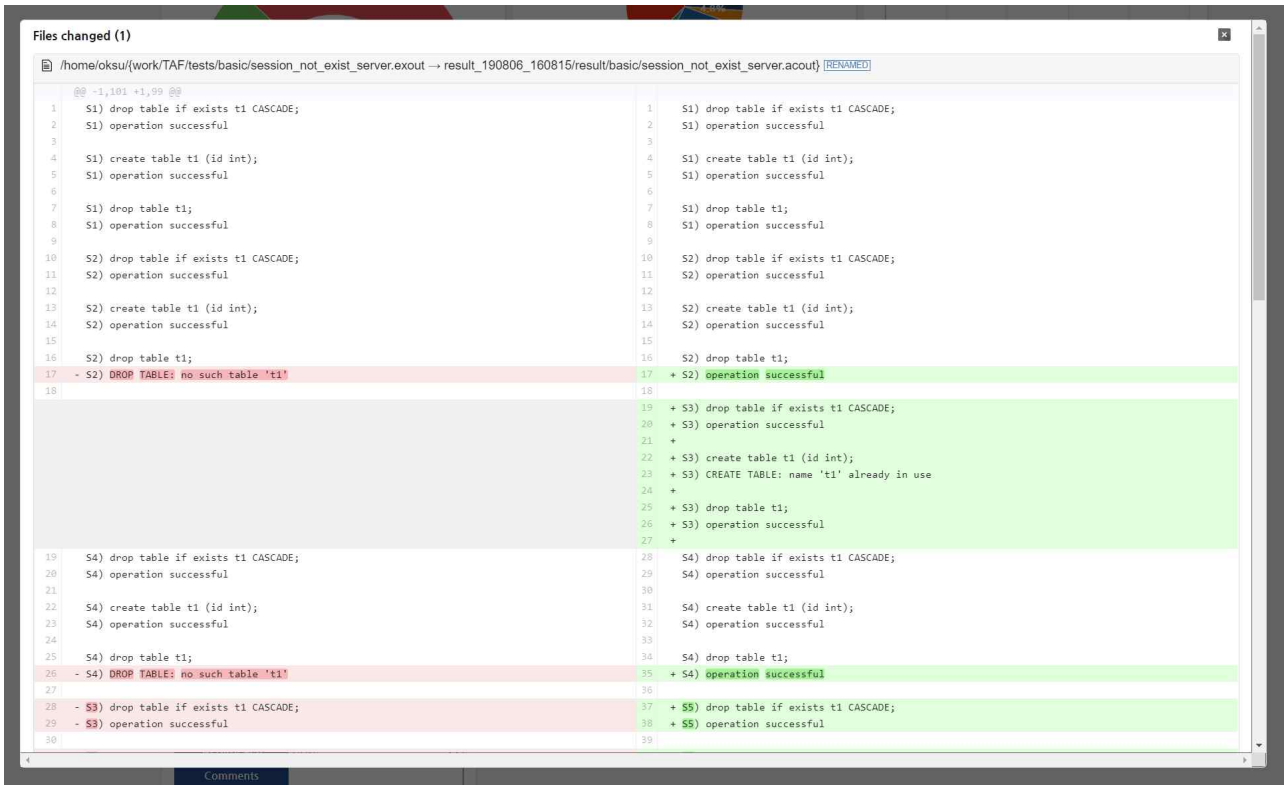
- 테스트케이스 및 DBMS 결함 수정 등으로 인해 정답 파일(exout)을 시각화 도구에서 업데이트할 수 있도록 정답파일 변경 기능을 제공함
- 특정 테스트케이스만 확인 및 분석 할 수 있도록 실패한 테스트케이스에 대한 필터링 기능을 제공함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 54] 테스트 결과 필터링 화면

- 실패한 테스트케이스 중 기대 결과와 실제 테스트 실행 결과가 다른 경우, 이를 비교할 수 있도록 'Diff' 모듈을 통해 확인이 가능함



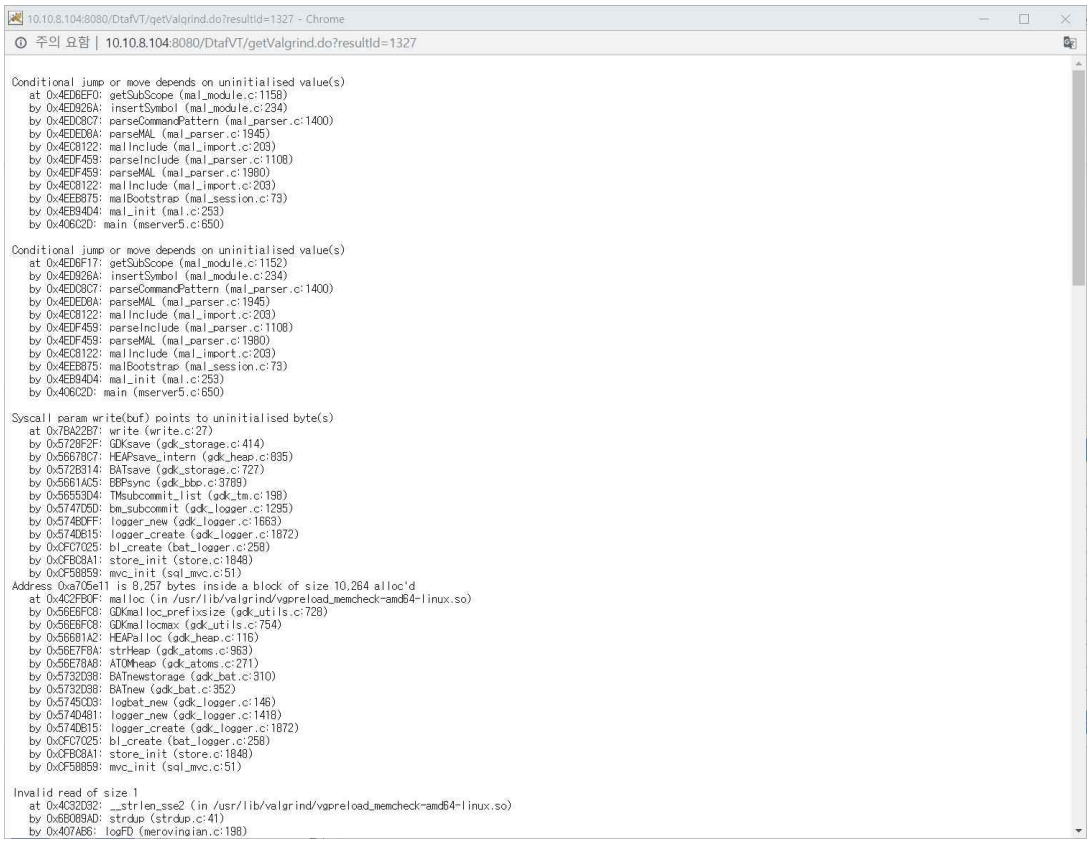
[그림 55] 실패한 테스트케이스에 대한 상세 정보 조회

- 메모리 누수 검출 도구(valgrind)와 연동하여 테스트 수행 시, 시각화 도구를 통해 각 테스트케이스에서 발견된 메모리 누수 오류를 출력하도록 설계 및 개발함
- 테스트케이스 실행 중 메모리 누수 발견 시, 해당 메모리 로그를 시각화 도구로 확인이 가능함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 56] 메모리 옵션을 사용한 테스트 실행 결과 조회 화면



[그림 57] 메모리 누수 검출 로그 화면

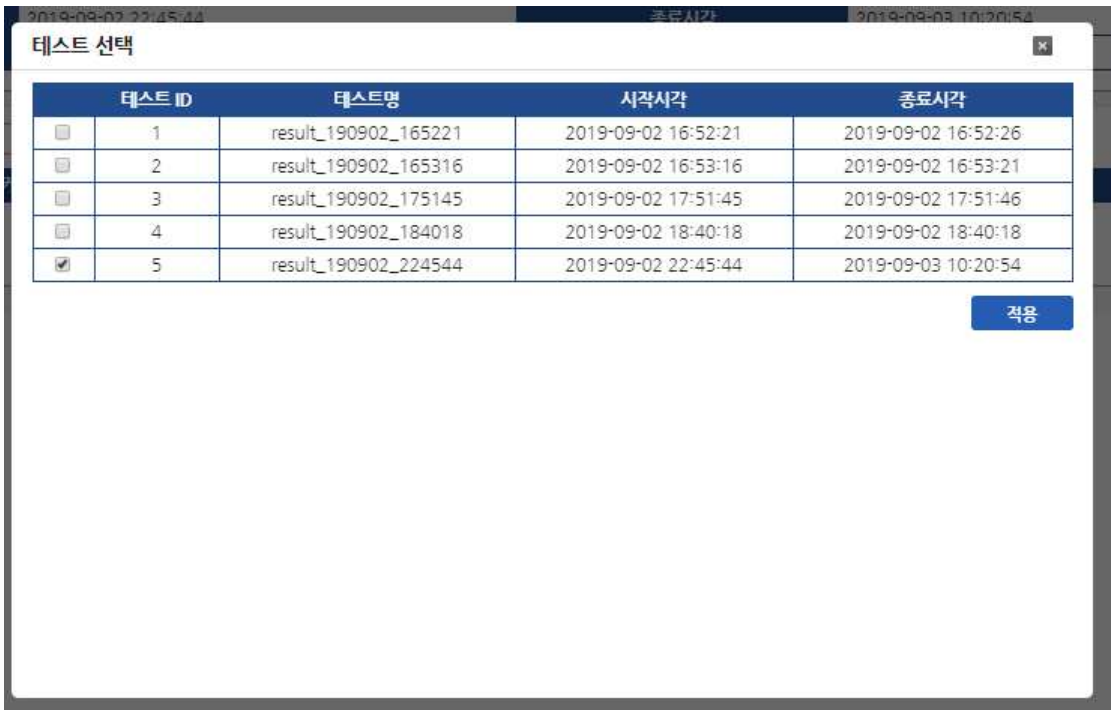
- 테스트 실행 정보(테스트 실행 결과 저장 폴더명, 테스트 대상 서버의 IP 주소, 테스트 대상 DBMS의 SVN 및 버전 정보, 테스트 시작 및 종료 시각 등)를 시각화 도구에서 확인이 가능함



[그림 58] 테스트 정보 조회 화면

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 현재 테스트 결과와 이전에 실행하였던 테스트 결과를 비교할 수 있도록 테스트 결과 비교 기능을 개발함



[그림 59] 비교 대상 테스트 선택 화면



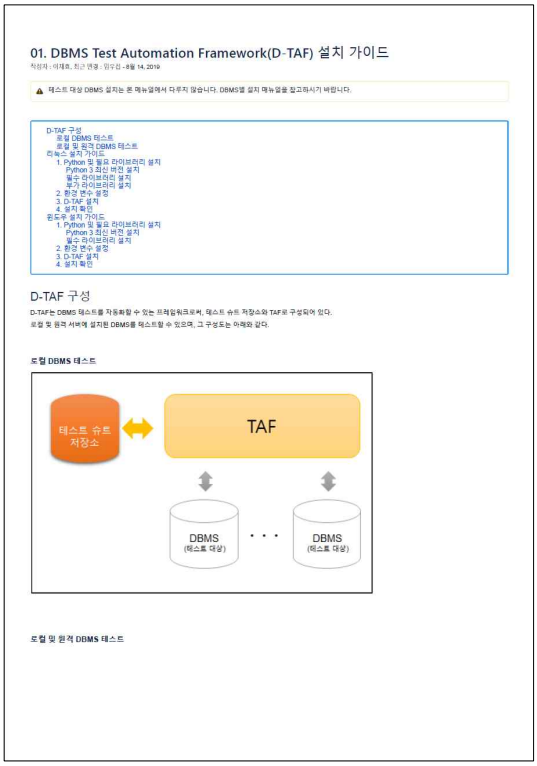
[그림 60] 테스트 결과 비교 화면

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

다. 품질 검증 자동화 프레임워크에 대한 사용자 매뉴얼 4종 개발

(1) 자동화 프레임워크 설치 매뉴얼

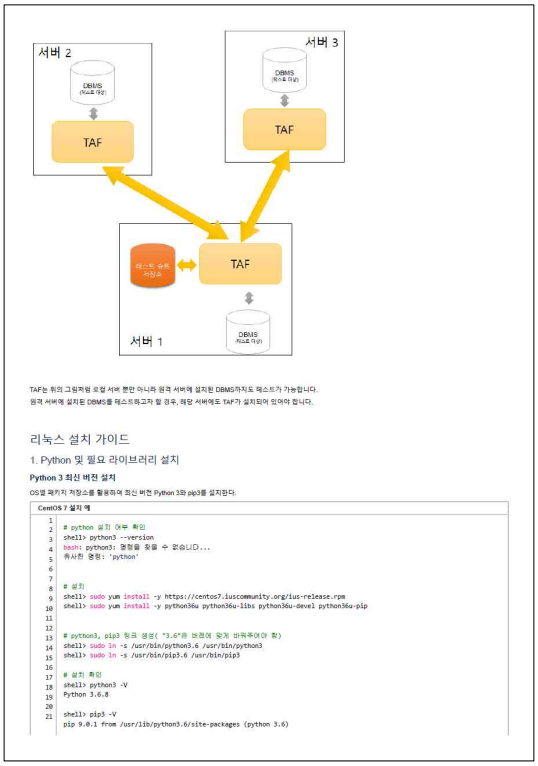
- 빅데이터 분석용 DBMS 품질 검증 자동화 프레임워크 실행을 위한 사전 설치 프로그램 및 환경설정 방법 등에 대한 가이드 문서를 작성함


01. DBMS Test Automation Framework(D-TAF) 설치 가이드

D-TAF 구성

로컬 DBMS 테스트

로컬 및 원격 DBMS 테스트



리눅스 설치 가이드

1. Python 및 필요 라이브러리 설치

Python 3 최신 버전 설치

CentOS 7 설치 예

```
# python 설치 여부 확인
shell> python3 --version
bash: python3: 명령을 찾을 수 없습니다...
유사한 명령: 'python'

# 설치
shell> sudo yum install -y https://centos7.iuscommunity.org/ius-release.rpm
shell> sudo yum install -y python36u python36u-libs python36u-devel python36u-pip

# python3, pip3 링크 생성( "3.6"은 버전에 맞게 바꿔주어야 함)
shell> sudo ln -s /usr/bin/python3.6 /usr/bin/python3
shell> sudo ln -s /usr/bin/pip3.6 /usr/bin/pip3

# 설치 확인
shell> python3 -V
Python 3.6.8

shell> pip3 -V
pip 9.0.1 from /usr/lib/python3.6/site-packages (python 3.6)
```


필수 라이브러리 설치

CentOS 7 설치 예

```
# 리눅스 빌드 도구 설치
shell > sudo yum groupinstall 'Development Tools'


# psutil 설치
shell> sudo pip3 install psutil


# java 설치 확인( 운용 경우 설치 필요, 시각화 도구 실행을 위해)
shell> java -version
openjdk version "1.8.0_181"
OpenJDK Runtime Environment (build 1.8.0_181-b13)
OpenJDK 64-Bit Server VM (build 25.181-b13, mixed mode)


# 메모리 누수 검출 도구 설치
$> sudo yum install valgrind
```

부가 라이브러리 설치

자원 제어 기능 사용 시

```
# stress 프로그램 설치 - 자원 제어 기능 사용 시 필요.
shell> sudo yum install stress
```

2. 환경 변수 설정

.bashrc

```
# DBMS 제어를 위해 필요한 환경 변수
export PETA_HOME=~/work/petasql
export PATH=${PETA_HOME}/bin:$PATH
export LD_LIBRARY_PATH=${PETA_HOME}/lib:$LD_LIBRARY_PATH

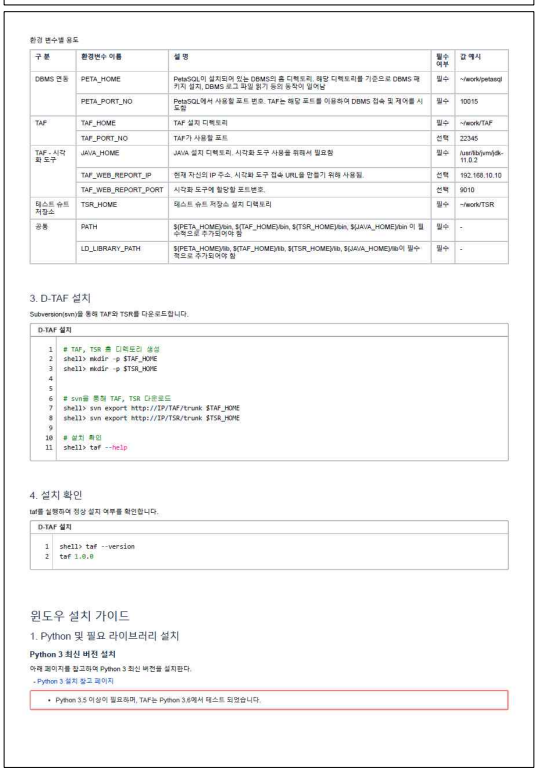
export PETA_PORT_NO=10115

# TAF 실행을 위한 환경 변수
export TAF_HOME=~/work/TAF
export TAF_PORT_NO=22345
export PATH=${TAF_HOME}/bin:$PATH
export LD_LIBRARY_PATH=${TAF_HOME}/lib:$LD_LIBRARY_PATH

# 원스톤 슈트 저장소를 위한 환경 변수
export TSR_HOME=~/work/TSR
export PATH=${TSR_HOME}/bin:$PATH
export LD_LIBRARY_PATH=${TSR_HOME}/lib:$LD_LIBRARY_PATH

export JAVA_HOME=/usr/lib/jvm/jdk-11.0.2
export PATH=$JAVA_HOME/bin:$PATH

export TAF_WEB_REPORT_IP=현재 자신의 IP 주소
export TAF_WEB_REPORT_PORT=9010
```


환경 변수별 용도

구분	환경변수 이름	설명	필수 여부	값 예시
DBMS 연동	PETA_HOME	PetaSQL이 설치되어 있는 DBMS의 홈 디렉토리. 해당 디렉토리를 기준으로 DBMS 패키지 설치, DBMS 로그 파일 참기 등의 동작이 일어남	필수	~/work/petasql
	PETA_PORT_NO	PetaSQL에서 사용할 포트 번호. TAF는 해당 포트를 이용하여 DBMS 접속 및 제어를 시도함	필수	10015
TAF	TAF_HOME	TAF 설치 디렉토리	필수	~/work/TAF
	TAF_PORT_NO	TAF가 사용할 포트	선택	22345
TAF - 시각화 도구	JAVA_HOME	JAVA 설치 디렉토리. 시각화 도구 사용을 위해서 필요함	필수	/usr/lib/jvm/jdk-11.0.2
	TAF_WEB_REPORT_IP	현재 자신의 IP 주소. 시각화 도구 접속 URL을 만들기 위해 사용됨.	선택	192.168.10.10
	TAF_WEB_REPORT_PORT	시각화 도구에 할당할 포트번호.	선택	9010
테스트 슈트 저장소	TSR_HOME	테스트 슈트 저장소 설치 디렉토리	필수	~/work/TSR
공통	PATH	${PETA_HOME}/bin, ${TAF_HOME}/bin, ${TSR_HOME}/bin, ${JAVA_HOME}/bin 이 필수적으로 추가되어야 함	필수	-
	LD_LIBRARY_PATH	${PETA_HOME}/lib, ${TAF_HOME}/lib, ${TSR_HOME}/lib, ${JAVA_HOME}/lib이 필수적으로 추가되어야 함	필수	-

3. D-TAF 설치

Subversion(svn)을 통해 TAF와 TSR를 다운로드합니다.

D-TAF 설치

```
# TAF, TSR 홈 디렉토리 생성
shell> mkdir -p $TAF_HOME
shell> mkdir -p $TSR_HOME


# svn을 통한 TAF, TSR 다운로드
shell> svn export http://IP/TAF/trunk $TAF_HOME
shell> svn export http://IP/TSR/trunk $TSR_HOME

# 설치 확인
shell> taf --help
```

4. 설치 확인

taf를 실행하여 정상 설치 여부를 확인합니다.

D-TAF 설치

```
shell> taf --version
taf 1.0.0
```

윈도우 설치 가이드

1. Python 및 필요 라이브러리 설치

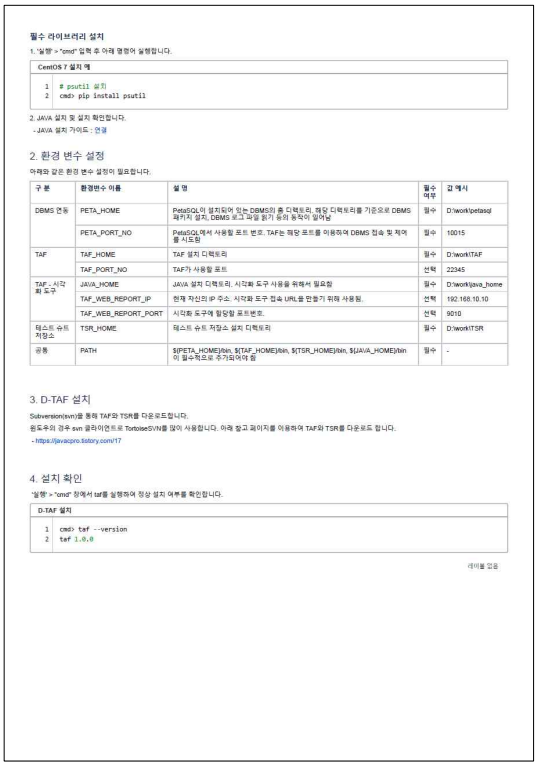
Python 3 최신 버전 설치

아래 페이지를 참고하여 Python 3 최신 버전을 설치합니다.

- Python 3 설치 참고 페이지

- Python 3.5 이상이 필요하며, TAF는 Python 3.6에서 테스트 되었습니다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



필수 라이브러리 설치

1. '실행' > "cmd" 입력 후 아래 명령어 실행합니다.

CentOS 7 설치 예

```
# psutil 설치
cmd> pip install psutil
```

2. JAVA 설치 및 설치 확인합니다.
- JAVA 설치 가이드 : 연결

2. 환경 변수 설정

아래와 같은 환경 변수 설정이 필요합니다.

구 분	환경변수 이름	설 명	필수 여부	값 예시
DBMS 연동	PETA_HOME	PetaSQL이 설치되어 있는 DBMS의 홈 디렉토리. 패당 디렉토리를 기준으로 DBMS 패키지 설치, DBMS 로그 파일 읽기 등의 동작이 일어남	필수	D:\work\petasql
	PETA_PORT_NO	PetaSQL에서 사용할 포트 번호. TAF는 해당 포트를 이용하여 DBMS 접속 및 제어를 시도함	필수	10015
TAF	TAF_HOME	TAF 설치 디렉토리	필수	D:\work\TAF
	TAF_PORT_NO	TAF가 사용할 포트	선택	22345
TAF_시각화 도구	JAVA_HOME	JAVA 설치 디렉토리. 시각화 도구 사용을 위해서 필요함	필수	D:\work\java_home
	TAF_WEB_REPORT_IP	현재 자신의 IP 주소. 시각화 도구 접속 URL을 만들기 위해 사용됨.	선택	192.168.10.10
	TAF_WEB_REPORT_PORT	시각화 도구에 할당할 포트번호.	선택	9010
테스트 슈트 저장소	TSR_HOME	테스트 슈트 저장소 설치 디렉토리	필수	D:\work\TSR
공통	PATH	${PETA_HOME}\bin, ${TAF_HOME}\bin, ${TSR_HOME}\bin, ${JAVA_HOME}\bin 이 필수적으로 추가되어야 함	필수	-

3. D-TAF 설치

Subversion(svn)을 통해 TAF와 TSR를 다운로드합니다.
윈도우의 경우 svn 클라이언트로 TortoiseSVN를 많이 사용합니다. 아래 참고 페이지를 이용하여 TAF와 TSR를 다운로드 합니다.
- https://javacpro.tistory.com/17

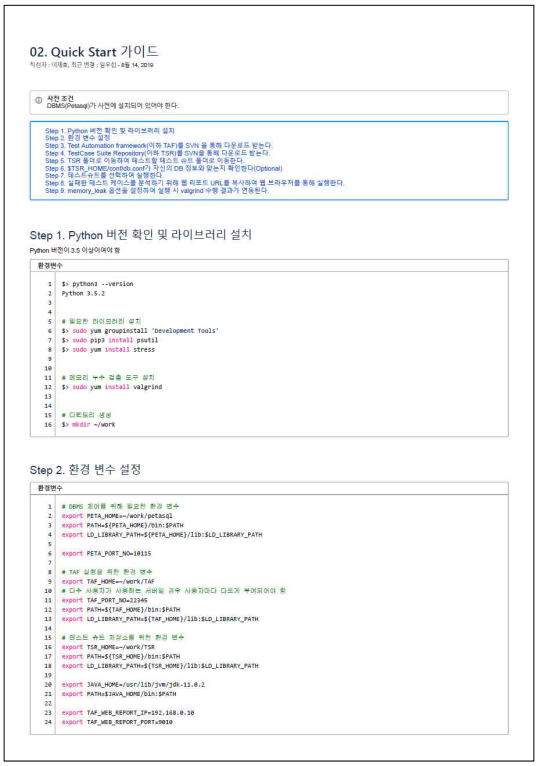
4. 설치 확인

"실행" > "cmd" 창에서 taf를 실행하여 정상 설치 여부를 확인합니다.

D-TAF 설치

```
cmd> taf --version
taf 1.0.0
```

데이블 없음



02. Quick Start 가이드

작성자 : 이태호, 최근 변경 : 임우진 - 8월 14, 2019

ⓘ 사전 조건
DBMS(Petasql)가 사전에 설치되어 있어야 한다.

Step 1. Python 버전 확인 및 라이브러리 설치
Step 2. 환경 변수 설정
Step 3. Test Automation framework(이하 TAF)를 SVN 을 통해 다운로드 받는다.
Step 4. TestCase Suite Repository(이하 TSR)를 SVN을 통해 다운로드 받는다.
Step 5. TSR 폴더로 이동하여 테스트할 테스트 슈트 폴더로 이동한다.
Step 6. $TSR_HOME/conf/db.conf가 자신의 DB 정보와 맞는지 확인한다(Optional)
Step 7. 테스트슈트를 선택하여 실행한다.
Step 8. 실패한 테스트 케이스를 분석하기 위해 웹 리포트 URL을 복사하여 웹 브라우저를 통해 실행한다.
Step 9. memory_leak 옵션을 설정하여 실행 시 valgrind 수행 결과가 연동된다.

Step 1. Python 버전 확인 및 라이브러리 설치

Python 버전이 3.5 이상이여야 함

환경변수

```
$> python3 --version
Python 3.5.2

# 필요한 라이브러리 설치
$> sudo yum groupinstall 'Development Tools'
$> sudo pip3 install psutil
$> sudo yum install stress

# 메모리 누수 검출 도구 설치
$> sudo yum install valgrind

# 디렉토리 생성
$> mkdir ~/work
```

Step 2. 환경 변수 설정

환경변수

```
# DBMS 접속을 위한 필요한 환경 변수
export PETA_HOME=~/work/petasql
export PATH=${PETA_HOME}/bin:$PATH
export LD_LIBRARY_PATH=${PETA_HOME}/lib:$LD_LIBRARY_PATH

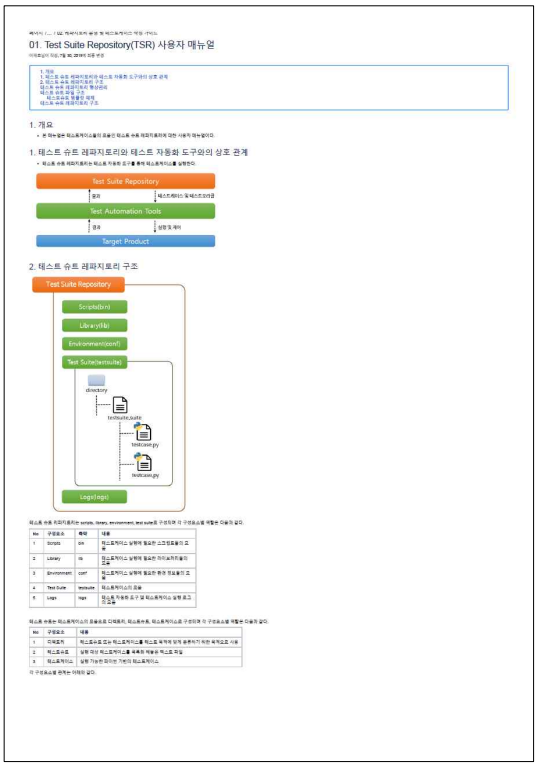
export PETA_PORT_NO=10115

# TAF 실행을 위한 환경 변수
export TAF_HOME=~/work/TAF
# 다수 사용자가 사용한는 서버일 경우 사용자마다 다르게 부여되어야 함
export TAF_PORT_NO=22345
export PATH=${TAF_HOME}/bin:$PATH
export LD_LIBRARY_PATH=${TAF_HOME}/lib:$LD_LIBRARY_PATH

# 테스트 슈트 저장소를 위한 환경 변수
export TSR_HOME=~/work/TSR
export PATH=${TSR_HOME}/bin:$PATH
export LD_LIBRARY_PATH=${TSR_HOME}/lib:$LD_LIBRARY_PATH

export JAVA_HOME=/usr/lib/jvm/jdk-11.0.2
export PATH=$JAVA_HOME/bin:$PATH

export TAF_WEB_REPORT_IP=192.168.0.10
export TAF_WEB_REPORT_PORT=9010
```



01. Test Suite Repository(TSR) 사용자 매뉴얼

1. 개요

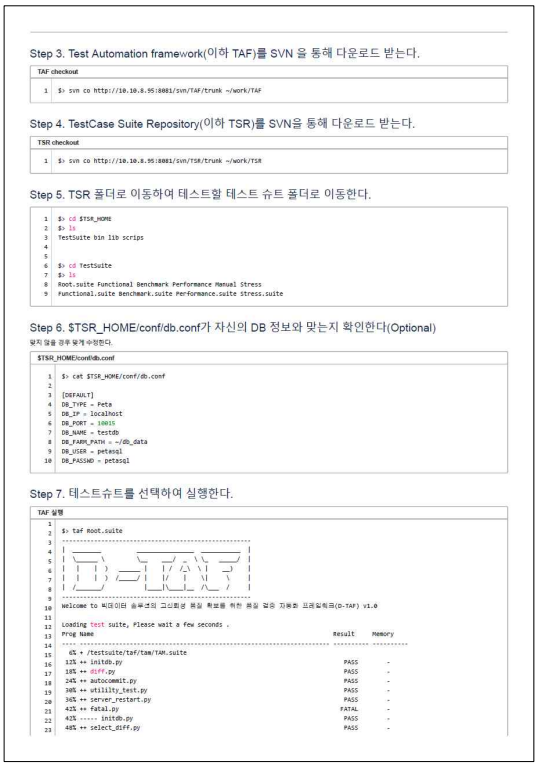
1. 테스트 슈트 레파지토리와 테스트 자동화 도구와의 상호 관계

2. 테스트 슈트 레파지토리 구조

No	구성요소	폴더	내용
1	Scripts	bin	테스트케이스 실행에 필요한 스크립트들이 모여
2	Library	lib	테스트케이스 실행에 필요한 라이브러리들의 모음
3	Environment	conf	테스트케이스 실행에 필요한 환경 정보들이 모여
4	Test Suite	testsuite	테스트케이스의 모음
5	Logs	logs	테스트 수행의 로그 및 테스트케이스 실행 로그 모음들

No	구성요소	내용
1	각 testcase	테스트케이스 단위 테스트케이스를 테스트 목적에 따라 실행하기 위한 목적으로 사용
2	테스트슈트	실행 대상 테스트케이스를 세부적 매뉴얼 텍스트 파일
3	테스트케이스	실행 가능한 파라미터 기반의 테스트케이스

각 구성요소별 역할은 아래와 같다.



Step 3. Test Automation framework(이하 TAF)를 SVN 을 통해 다운로드 받는다.

TAF checkout

```
$> svn co http://10.10.8.95:8081/svn/TAF/trunk ~/work/TAF
```

Step 4. TestCase Suite Repository(이하 TSR)를 SVN을 통해 다운로드 받는다.

TSR checkout

```
$> svn co http://10.10.8.95:8081/svn/TSR/trunk ~/work/TSR
```

Step 5. TSR 폴더로 이동하여 테스트할 테스트 슈트 폴더로 이동한다.

```
$> cd $TSR_HOME
$> ls
TestSuite bin lib scrips

$> cd TestSuite
$> ls
Root.suite Functional Benchmark Performance Manual Stress
Functional.suite Benchmark.suite Performance.suite Stress.suite
```

Step 6. $TSR_HOME/conf/db.conf가 자신의 DB 정보와 맞는지 확인한다(Optional)

맞지 않을 경우 맞게 수정한다.

$TSR_HOME/conf/db.conf

```
$> cat $TSR_HOME/conf/db.conf

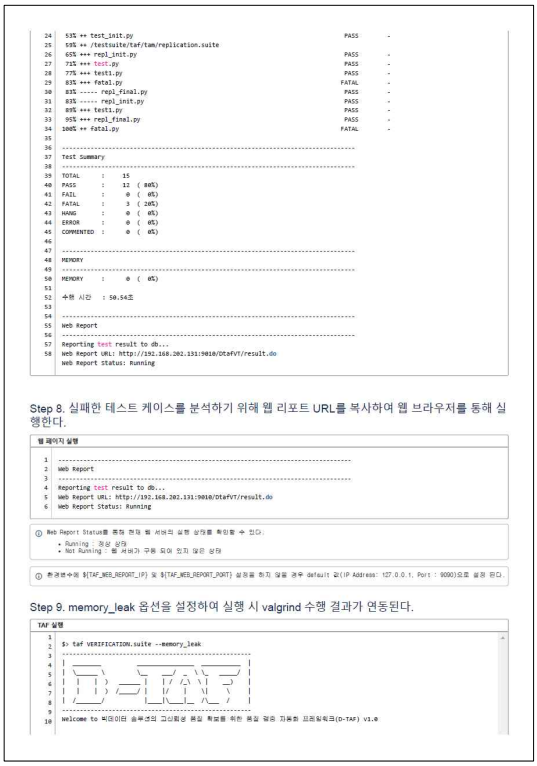
[DEFAULT]
DB_TYPE = Peta
DB_IP = localhost
DB_PORT = 10015
DB_NAME = testdb
DB_FARM_PATH = ~/db_data
DB_USER = petasql
DB_PASSWD = petasql
```

Step 7. 테스트슈트를 선택하여 실행한다.

TAF 실행

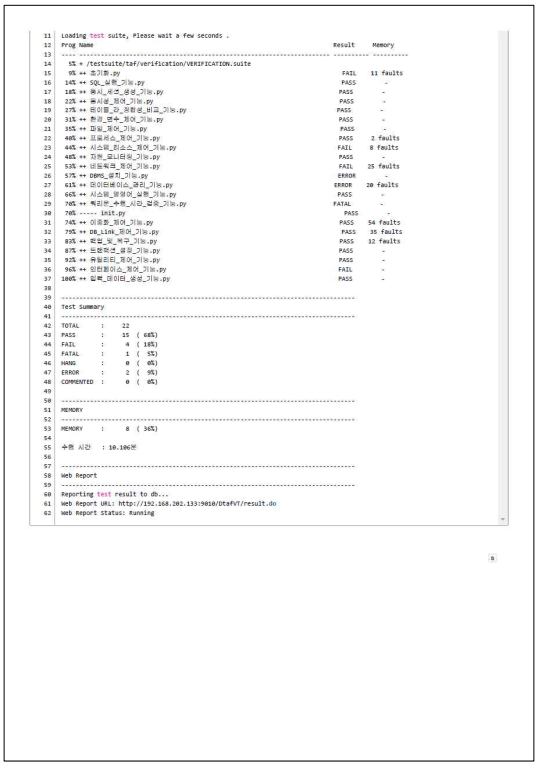
```
$> taf Root.suite
-------------------------------------------------------------
 _____          _____   _____   _____
|_____\    /\  |_____| |_____| |_____|
|       \  /  \ |     | |     | |     |
|       | /____\|     | |     | |     |
|_____ /  |    ||     | |     | |_____|
-------------------------------------------------------------
Welcome to 빅데이터 솔루션의 그리드형 품질 확보를 위한 품질 검증 자동화 프레임워크(D-TAF) v1.0

Loading test suite, Please wait a few seconds .
Prog Name                                                    Result     Memory
------------------------------------------------------------ ---------- ----------
  0% * /testsuite/taf/tbm/TAF.suite
 12% ++ initdb.py                                            PASS       -
 18% ++ diff.py                                              PASS       -
 24% ++ autocommit.py                                        PASS       -
 30% ++ utillity_test.py                                     PASS       -
 36% ++ server_restart.py                                    PASS       -
 41% ++ fatal.py                                             FATAL      -
 42% ----- initdb.py                                         PASS       -
 48% ++ select_diff.py                                       PASS       -
```



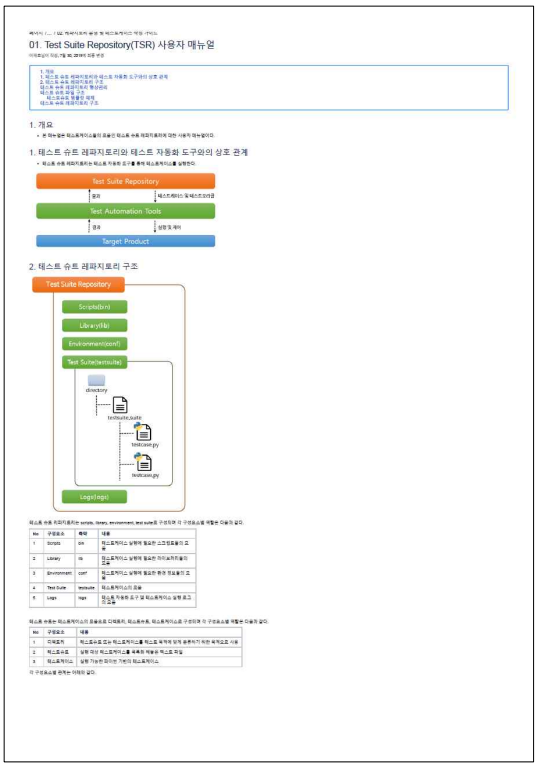
Step 8. 실패한 테스트 케이스를 분석하기 위해 웹 리포트 URL를 복사하여 웹 브라우저를 동해 실행한다.

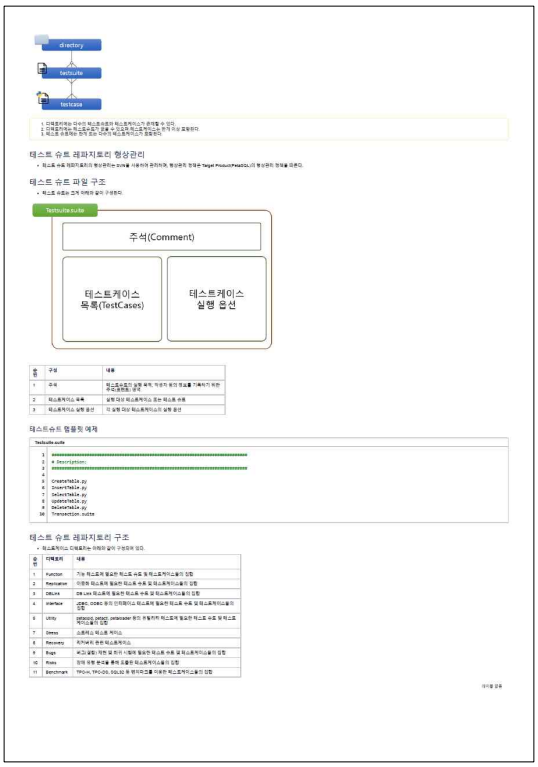
Step 9. memory_leak 옵션을 설정하여 실행 시 valgrind 수행 결과가 연동된다.



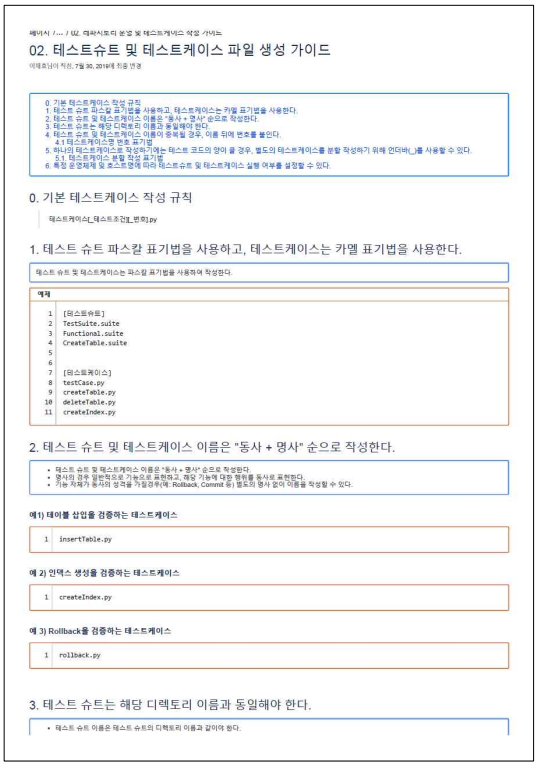
(2) 레파지토리 운영 및 테스트케이스 작성 가이드

- 테스트케이스들의 모음인 테스트 슈트 및 테스트 슈트 레파지토리 구조와 테스트케이스 작성 방법에 대한 가이드 문서를 작성함





국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



페이지 / ... / 02. 테스트케이스 작성 및 테스트케이스 작성 가이드

02. 테스트슈트 및 테스트케이스 파일 생성 가이드

이재호님이 작성, 7월 30, 2019에 최종 변경

0. 기본 테스트케이스 작성 규칙
1. 테스트 슈트 파스칼 표기법을 사용하고, 테스트케이스는 카멜 표기법을 사용한다.
2. 테스트 슈트 및 테스트케이스 이름은 "동사 + 명사" 순으로 작성한다.
3. 테스트 슈트는 해당 디렉토리 이름과 동일해야 한다.
4. 테스트 슈트 및 테스트케이스 이름이 중복될 경우, 이름 뒤에 번호를 붙인다.
   4.1 테스트케이스명 번호 표기법
5. 하나의 테스트케이스로 작성하기에는 테스트 코드의 양이 클 경우, 별도의 테스트케이스를 분할 작성하기 위해 언더바(_)를 사용할 수 있다.
   5.1. 테스트케이스 분할 작성 표기법
6. 특정 운영체제 및 호스트명에 따라 테스트슈트 및 테스트케이스 실행 여부를 설정할 수 있다.

0. 기본 테스트케이스 작성 규칙

테스트케이스_테스트조건_번호.py

1. 테스트 슈트 파스칼 표기법을 사용하고, 테스트케이스는 카멜 표기법을 사용한다.

테스트 슈트 및 테스트케이스는 파스칼 표기법을 사용하여 작성한다.

예제

```
[테스트슈트]
TestSuite.suite
Functional.suite
CreateTable.suite


[테스트케이스]
testCase.py
createTable.py
deleteTable.py
createIndex.py
```

2. 테스트 슈트 및 테스트케이스 이름은 "동사 + 명사" 순으로 작성한다.

- 테스트 슈트 및 테스트케이스 이름은 "동사 + 명사" 순으로 작성한다.
- 명사의 경우 일반적으로 기능으로 표현하고, 해당 기능에 대한 행위를 동사로 표현한다.
- 기능 자체가 동사의 성격을 가질경우(예: Rollback, Commit 등) 별도의 명사 없이 이름을 작성할 수 있다.

예1) 테이블 삽입을 검증하는 테스트케이스

```
insertTable.py
```

예 2) 인덱스 생성을 검증하는 테스트케이스

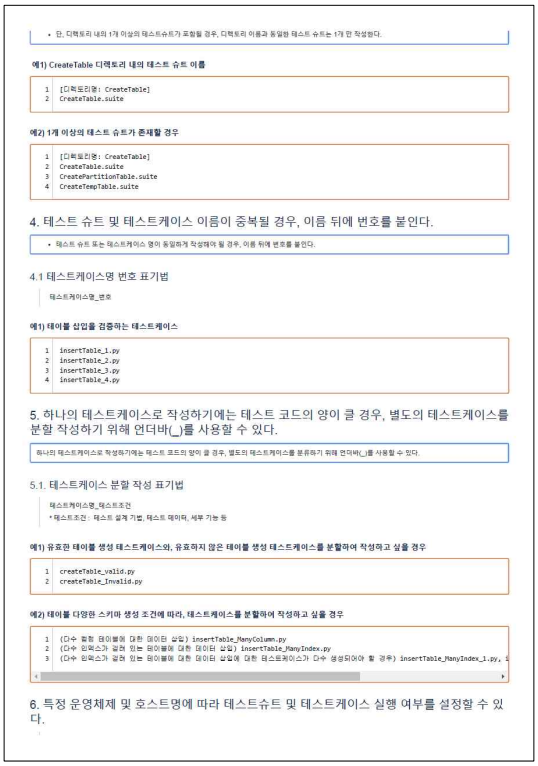
```
createIndex.py
```

예 3) Rollback을 검증하는 테스트케이스

```
rollback.py
```

3. 테스트 슈트는 해당 디렉토리 이름과 동일해야 한다.

- 테스트 슈트 이름은 테스트 슈트의 디렉토리 이름과 같아야 한다.



- 단, 디렉토리 내의 1개 이상의 테스트슈트가 포함될 경우, 디렉토리 이름과 동일한 테스트 슈트는 1개 만 작성한다.

예1) CreateTable 디렉토리 내의 테스트 슈트 이름

```
[디렉토리명: CreateTable]
CreateTable.suite
```

예2) 1개 이상의 테스트 슈트가 존재할 경우

```
[디렉토리명: CreateTable]
CreateTable.suite
CreatePartitionTable.suite
CreateTempTable.suite
```

4. 테스트 슈트 및 테스트케이스 이름이 중복될 경우, 이름 뒤에 번호를 붙인다.

- 테스트 슈트 또는 테스트케이스 명이 동일하게 작성돼야 될 경우, 이름 뒤에 번호를 붙인다.

4.1 테스트케이스명 번호 표기법

테스트케이스명_번호

예1) 테이블 삽입을 검증하는 테스트케이스

```
insertTable_1.py
insertTable_2.py
insertTable_3.py
insertTable_4.py
```

5. 하나의 테스트케이스로 작성하기에는 테스트 코드의 양이 클 경우, 별도의 테스트케이스를 분할 작성하기 위해 언더바(_)를 사용할 수 있다.

하나의 테스트케이스로 작성하기에는 테스트 코드의 양이 클 경우, 별도의 테스트케이스를 분류하기 위해 언더바(_)를 사용할 수 있다.

5.1. 테스트케이스 분할 작성 표기법

테스트케이스명_테스트조건

* 테스트조건 : 테스트 설계 기법, 테스트 메이저, 세부 기능 등

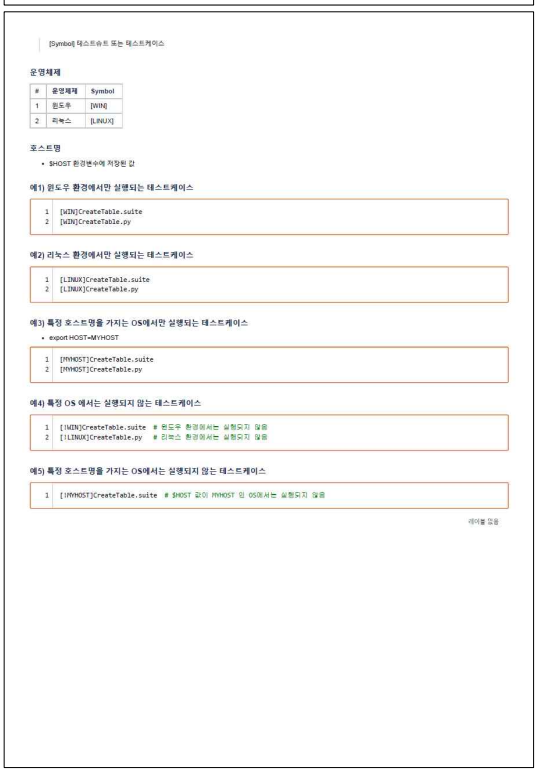
예1) 유효한 테이블 생성 테스트케이스와, 유효하지 않은 테이블 생성 테스트케이스를 분할하여 작성하고 싶을 경우

```
createTable_valid.py
createTable_Invalid.py
```

예2) 테이블 다양한 스키마 생성 조건에 따라, 테스트케이스를 분할하여 작성하고 싶을 경우

```
(다수 컬럼 테이블에 대한 데이터 삽입) insertTable_ManyColumn.py
(다수 인덱스가 걸려 있는 테이블에 대한 데이터 삽입) insertTable_ManyIndex.py
(다수 인덱스가 걸려 있는 테이블에 대한 데이터 삽입에 대한 테스트케이스가 다수 생성되어야 할 경우) insertTable_ManyIndex_1.py, i
```

6. 특정 운영체제 및 호스트명에 따라 테스트슈트 및 테스트케이스 실행 여부를 설정할 수 있다.



[Symbol] 테스트슈트 또는 테스트케이스

운영체제

#	운영체제	Symbol
1	윈도우	[WIN]
2	리눅스	[LINUX]

호스트명

- $HOST 환경변수에 저장된 값

예1) 윈도우 환경에서만 실행되는 테스트케이스

```
[WIN]CreateTable.suite
[WIN]CreateTable.py
```

예2) 리눅스 환경에서만 실행되는 테스트케이스

```
[LINUX]CreateTable.suite
[LINUX]CreateTable.py
```

예3) 특정 호스트명을 가지는 OS에서만 실행되는 테스트케이스

- export HOST=MYHOST

```
[MYHOST]CreateTable.suite
[MYHOST]CreateTable.py
```

예4) 특정 OS 에서는 실행되지 않는 테스트케이스

```
[!WIN]CreateTable.suite  # 윈도우 환경에서는 실행되지 않음
[!LINUX]CreateTable.py   # 리눅스 환경에서는 실행되지 않음
```

예5) 특정 호스트명을 가지는 OS에서는 실행되지 않는 테스트케이스

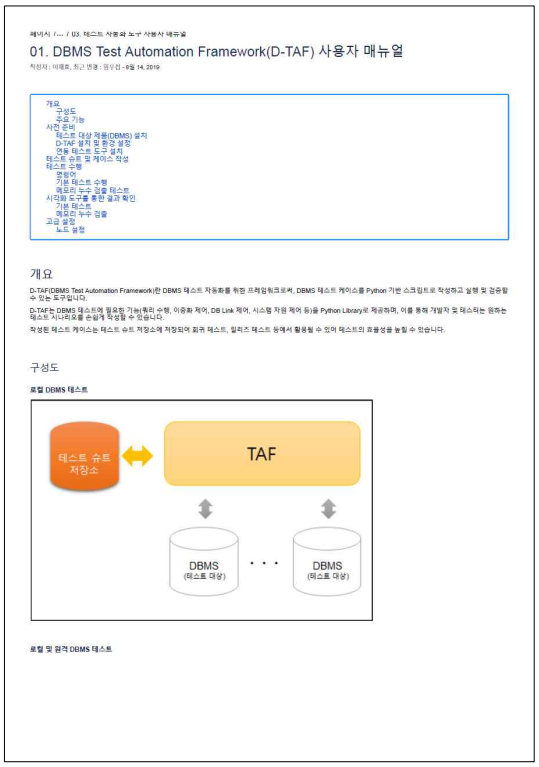
```
[!MYHOST]CreateTable.suite  # $HOST 값이 MYHOST 인 OS에서는 실행되지 않음
```

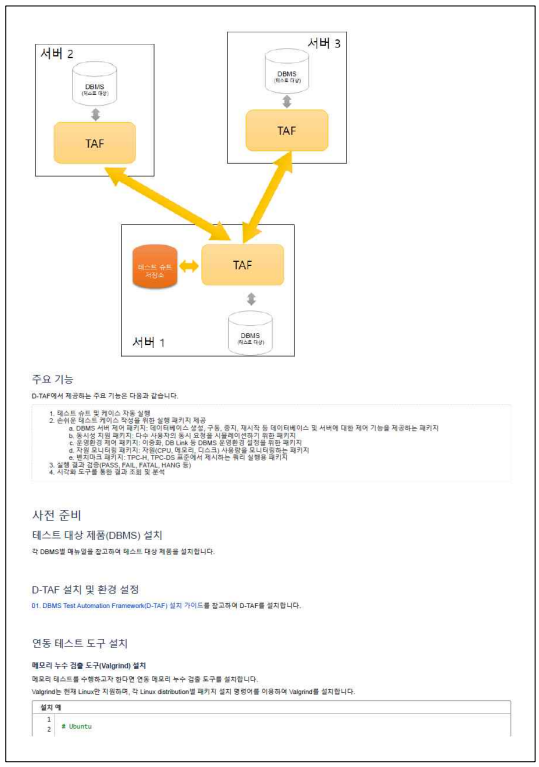
데이블 없음

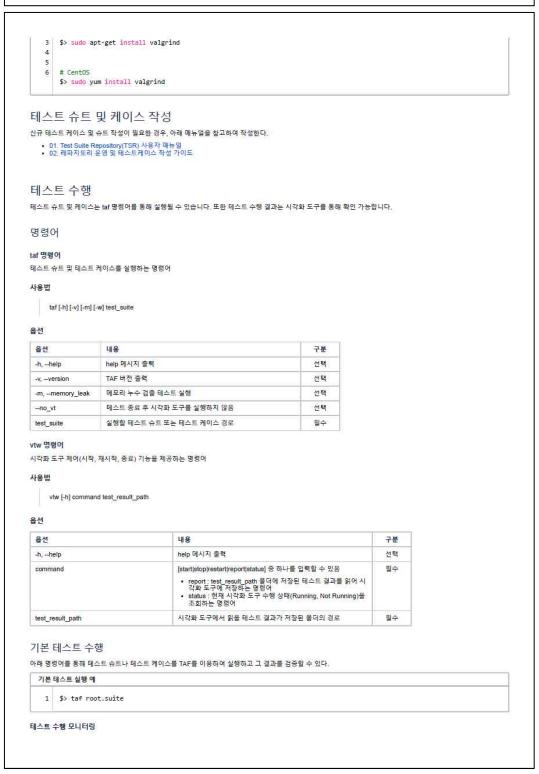
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

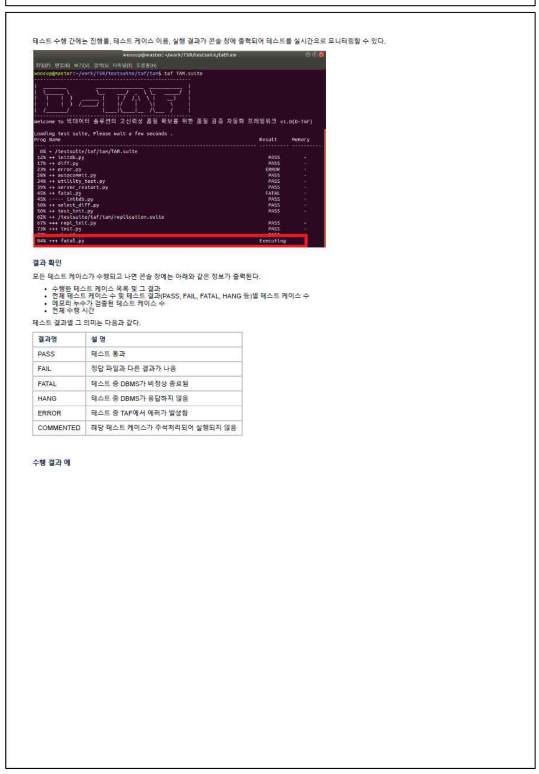
(3) 테스트 자동화 도구 사용자 매뉴얼

- 빅데이터 분석용 DBMS 품질 검증 자동화 프레임워크 실행 방법(테스트 수행 방법, 시각화 도구를 통한 결과 확인 방법 등), API Reference 및 메모리 누수 검출 필터에 대한 가이드 문서를 작성함









국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



메모리 누수 검출 테스트

> 메모리 누수(memory leak) 현상은 컴퓨터 프로그램이 필요하지 않은 메모리를 계속 점유하고 있는 현상이다. 할당된 메모리를 사용한 다음 반환하지 않는 것이 누적되면 메모리가 낭비된다. - 위키피디아

위 설명과 같이 메모리 누수가 DBMS 운영 간 발생할 경우 DBMS 서버가 사용하는 메모리 양은 지속적으로 증가되고 심각할 경우 메모리 부족으로 서비스 제공이 불가능해질 수 있다.

이와 같이 메모리 누수를 검출하고 찾기 위해, D-TAF는 오픈소스 메모리 누수 검출 도구인 'valgrind'를 이용하여 메모리 누수 검출 기능을 제공한다. (Valgrind에 대해서는 여기를 참조: 링크 준비중)

메모리 누수 검출 옵션을 통한 실행

Valgrind와 연동을 위해 D-TAF는 "memory_leak" 옵션을 제공한다. 아래와 같이 실행할 경우, D-TAF는 DBMS 서버를 Valgrind와 연동하여 수행한다.

환경변수

```
1  $> taf root.suite --memory_leak
```

단, DBMS 초기화, 시작, 재시작을 taf에서 제공하는 initdb(), start(), restart() 함수를 통해서 수행해야 한다.

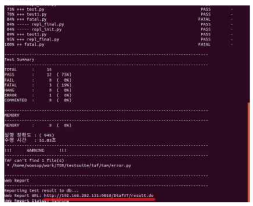


시각화 도구를 통한 결과 확인

테스트 수행이 완료되면 시각화 도구가 자동으로 실행되며 시각화 도구에 접속 가능한 URL이 콘솔창에 출력된다.
사용자는 해당 URL을 복사한 후 웹 브라우저를 통해 결과 확인이 가능하다.
(시각화 도구 사용법은 05. 시각화 도구 사용자 매뉴얼 참조)

기본 테스트

접속 URL 예



시각화 도구 실행 예



메모리 누수 검출

시각화 도구 접속 URL은 기본 테스트시와 동일하게 출력이 되며, 사용자는 웹 브라우저를 통해 접속 가능하다.

메모리 누수 검출 로그 확인

자세한 메모리 누수 검출 로그는 시각화 도구의 "MEM" 컬럼을 통해 확인이 가능하다.



"MEM" 컬럼의 아이콘 클릭 시, 아래와 같은 메모리 누수 검출 로그가 출력된다.

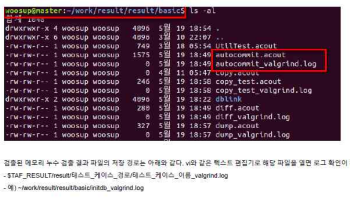


한편 시각화 도구 이용이 불가능할 상황이라면 아래와 같이 메모리 누수 검출 로그 파일에 직접 접근하여 확인할 수 있다.

파일을 통해 메모리 누수 검출 로그 확인하기...

메모리 누수 검출 로그 파일 저장 경로

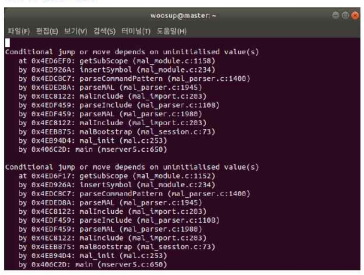
각 테스트케이스별 검출된 메모리 누수 로그는 아래 그림과 같이 결과 파일이 저장되는 경로에 같이 남는다.



검출된 메모리 누수 검출 결과 파일의 저장 경로는 아래와 같다. vi와 같은 텍스트 편집기로 해당 파일을 열면 로그 확인이 가능하다.

- $TAF_RESULT/result/테스트_케이스_경로/테스트_케이스_이름_valgrind.log
- 예) ~/work/result/result/basic/initdb_valgrind.log

메모리 누수 검출 로그 파일 예



고급 설정

TAF 고급 설정에 대해서는 아래 링크를 클릭하세요.

TAF 고급 설정 보기...

노드 설정

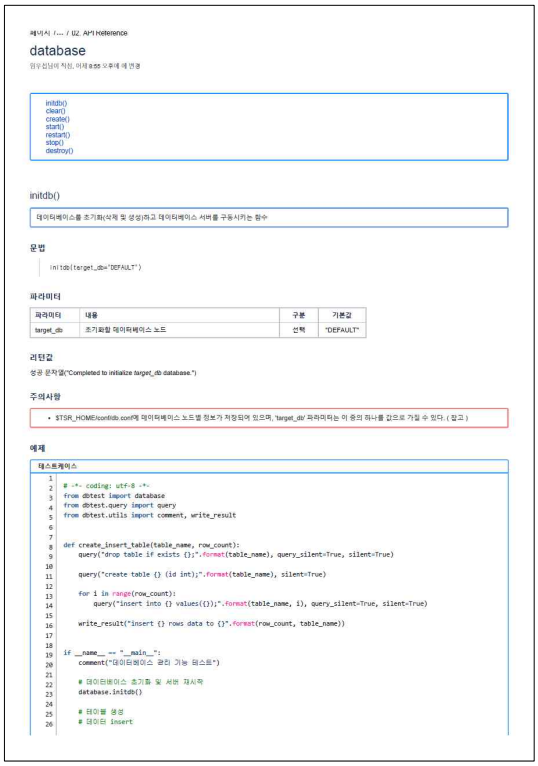
원격 서버에 위치한 DBMS를 테스트하기 위해서는 해당 서버 접속 정보를 설정해주어야 한다.

접속 정보 설정 파일은 $TSR_HOME/conf/db.conf에 위치하며, 아래 환경 변수가 설정되어야 한다.

환경 변수명	설 명
DB_IP	원격 서버의 IP 또는 호스트 네임
TAF_PORT_NO	원격 서버에서 실행 중인 TAF의 포트 번호

- $TSR_HOME/conf/db.conf 파일을 통해 사용자는 노드별로 필요한 환경 변수를 설정할 수 있다.
- "DEFAULT" 노드의 환경 변수는 다른 모든 노드에 상속된다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



페이지 /... / UZ. API Reference

database

임무 신님의 직원, 이제 database 오류에 에 변경

initdb()
clear()
create()
start()
restart()
stop()
destroy()

initdb()

데이터베이스를 초기화(삭제 및 생성)하고 데이터베이스 서버를 구동시키는 함수

문법

```
initdb(target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
target_db	초기화할 데이터베이스 노드	선택	"DEFAULT"

리턴값

성공 문자열("Completed to initialize target_db database.")

주의사항

- $TSR_HOME/conf/db.conf에 데이터베이스 노드별 정보가 저장되어 있으며, 'target_db' 파라미터는 이 중의 하나를 값으로 가질 수 있다. (참고)

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.query import query
from dbtest.utils import comment, write_result


def create_insert_table(table_name, row_count):
    query("drop table if exists {};".format(table_name), query_silent=True, silent=True)

    query("create table {} (id int);".format(table_name), silent=True)

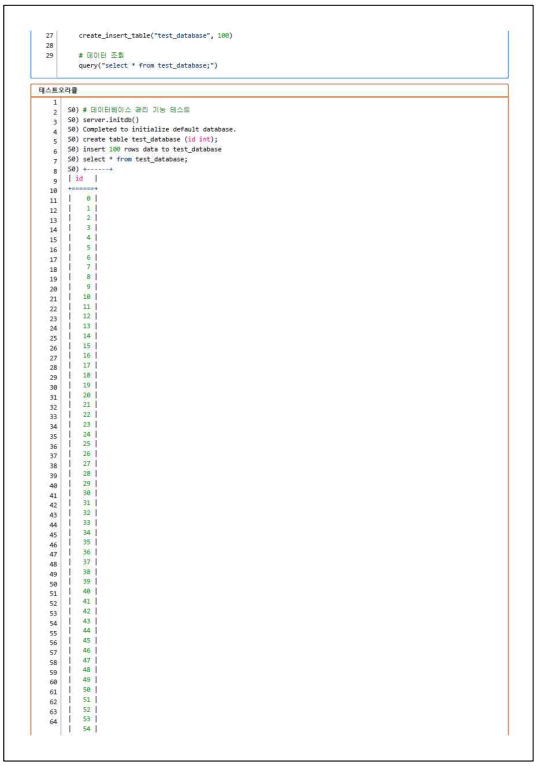
    for i in range(row_count):
        query("insert into {} values({});".format(table_name, i), query_silent=True, silent=True)

    write_result("insert {} rows data to {}".format(row_count, table_name))


if __name__ == "__main__":
    comment("데이터베이스 관리 기능 테스트")

    # 데이터베이스 초기화 및 서버 재시작
    database.initdb()

    # 테이블 생성
    # 데이터 insert
```

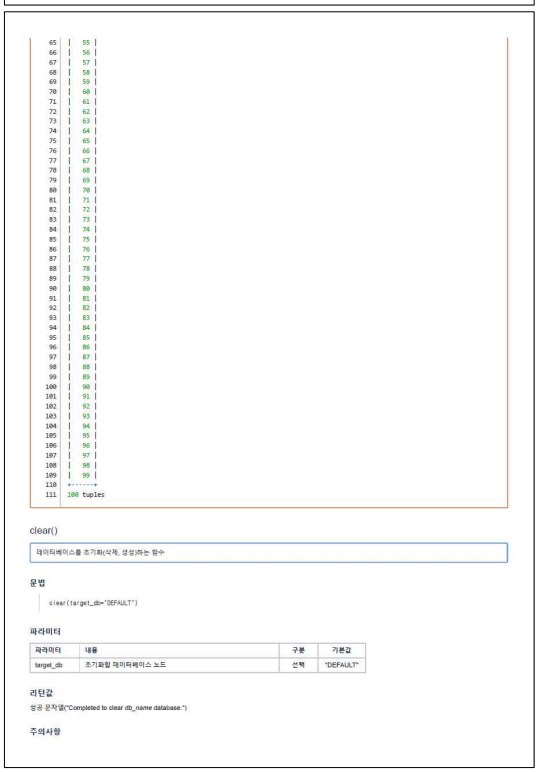


```
    create_insert_table("test_database", 100)

    # 데이터 조회
    query("select * from test_database;")
```

테스트오라클

```
S0) # 데이터베이스 관리 기능 테스트
S0) server.initdb()
S0) Completed to initialize default database.
S0) create table test_database (id int);
S0) insert 100 rows data to test_database
S0) select * from test_database;
S0) +-------+
| id    |
+=======+
|     0 |
|     1 |
|     2 |
|     3 |
|     4 |
|     5 |
|     6 |
|     7 |
|     8 |
|     9 |
|    10 |
|    11 |
|    12 |
|    13 |
|    14 |
|    15 |
|    16 |
|    17 |
|    18 |
|    19 |
|    20 |
|    21 |
|    22 |
|    23 |
|    24 |
|    25 |
|    26 |
|    27 |
|    28 |
|    29 |
|    30 |
|    31 |
|    32 |
|    33 |
|    34 |
|    35 |
|    36 |
|    37 |
|    38 |
|    39 |
|    40 |
|    41 |
|    42 |
|    43 |
|    44 |
|    45 |
|    46 |
|    47 |
|    48 |
|    49 |
|    50 |
|    51 |
|    52 |
|    53 |
|    54 |
```



```
|    55 |
|    56 |
|    57 |
|    58 |
|    59 |
|    60 |
|    61 |
|    62 |
|    63 |
|    64 |
|    65 |
|    66 |
|    67 |
|    68 |
|    69 |
|    70 |
|    71 |
|    72 |
|    73 |
|    74 |
|    75 |
|    76 |
|    77 |
|    78 |
|    79 |
|    80 |
|    81 |
|    82 |
|    83 |
|    84 |
|    85 |
|    86 |
|    87 |
|    88 |
|    89 |
|    90 |
|    91 |
|    92 |
|    93 |
|    94 |
|    95 |
|    96 |
|    97 |
|    98 |
|    99 |
+-------+
100 tuples
```

clear()

데이터베이스를 초기화(삭제, 생성)하는 함수

문법

```
clear(target_db='DEFAULT')
```

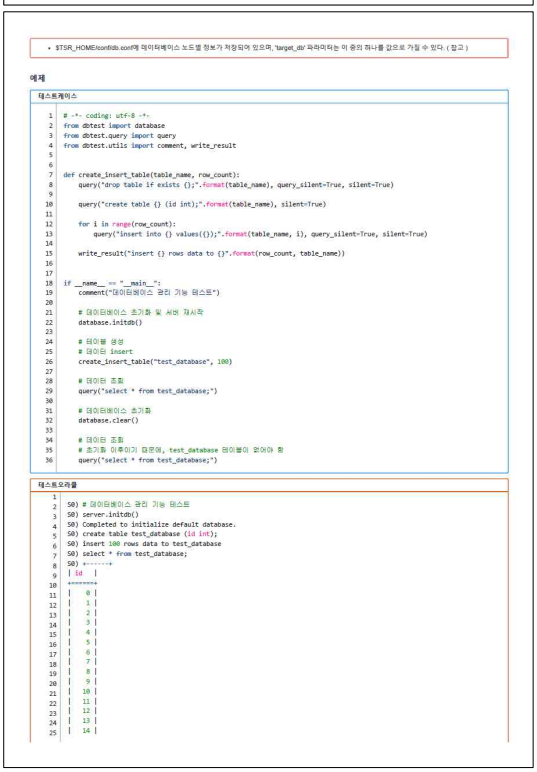
파라미터

파라미터	내용	구분	기본값
target_db	초기화할 데이터베이스 노드	선택	"DEFAULT"

리턴값

성공 문자열("Completed to clear db_name database.")

주의사항



- $TSR_HOME/conf/db.conf에 데이터베이스 노드별 정보가 저장되어 있으며, 'target_db' 파라미터는 이 중의 하나를 값으로 가질 수 있다. (참고)

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.query import query
from dbtest.utils import comment, write_result


def create_insert_table(table_name, row_count):
    query("drop table if exists {};".format(table_name), query_silent=True, silent=True)

    query("create table {} (id int);".format(table_name), silent=True)

    for i in range(row_count):
        query("insert into {} values({});".format(table_name, i), query_silent=True, silent=True)

    write_result("insert {} rows data to {}".format(row_count, table_name))


if __name__ == "__main__":
    comment("데이터베이스 관리 기능 테스트")

    # 데이터베이스 초기화 및 서버 재시작
    database.initdb()

    # 테이블 생성
    # 데이터 insert
    create_insert_table("test_database", 100)

    # 데이터 조회
    query("select * from test_database;")

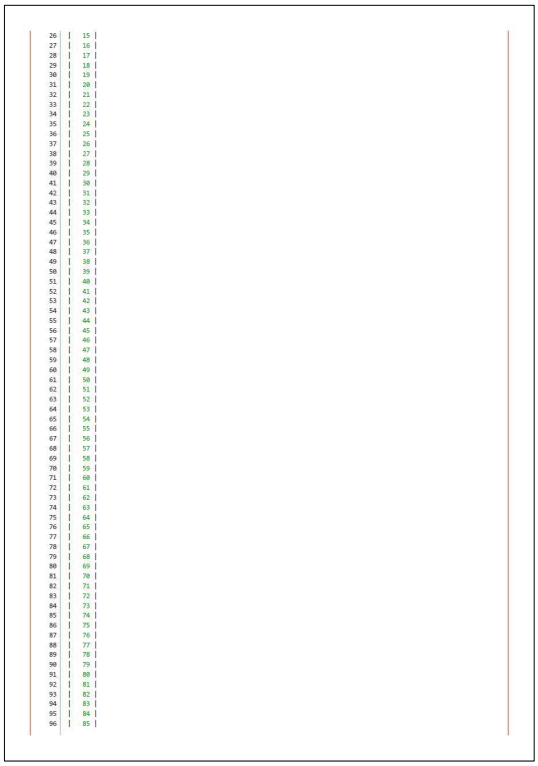
    # 데이터베이스 초기화
    database.clear()

    # 데이터 조회
    # 초기화 하였기 때문에, test_database 테이블이 없어야 함
    query("select * from test_database;")
```

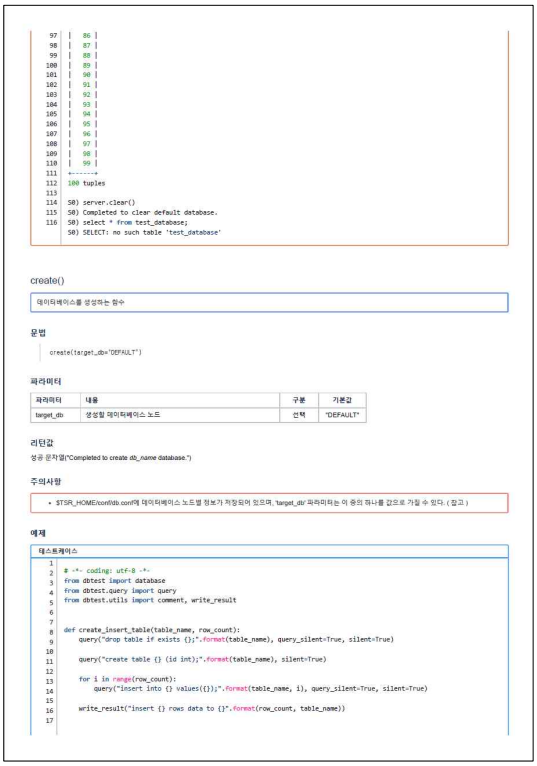
테스트오라클

```
S0) # 데이터베이스 관리 기능 테스트
S0) server.initdb()
S0) Completed to initialize default database.
S0) create table test_database (id int);
S0) insert 100 rows data to test_database
S0) select * from test_database;
S0) +-------+
| id    |
+=======+
|     0 |
|     1 |
|     2 |
|     3 |
|     4 |
|     5 |
|     6 |
|     7 |
|     8 |
|     9 |
|    10 |
|    11 |
|    12 |
|    13 |
|    14 |
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



```
|  15 |
|  16 |
|  17 |
|  18 |
|  19 |
|  20 |
|  21 |
|  22 |
|  23 |
|  24 |
|  25 |
|  26 |
|  27 |
|  28 |
|  29 |
|  30 |
|  31 |
|  32 |
|  33 |
|  34 |
|  35 |
|  36 |
|  37 |
|  38 |
|  39 |
|  40 |
|  41 |
|  42 |
|  43 |
|  44 |
|  45 |
|  46 |
|  47 |
|  48 |
|  49 |
|  50 |
|  51 |
|  52 |
|  53 |
|  54 |
|  55 |
|  56 |
|  57 |
|  58 |
|  59 |
|  60 |
|  61 |
|  62 |
|  63 |
|  64 |
|  65 |
|  66 |
|  67 |
|  68 |
|  69 |
|  70 |
|  71 |
|  72 |
|  73 |
|  74 |
|  75 |
|  76 |
|  77 |
|  78 |
|  79 |
|  80 |
|  81 |
|  82 |
|  83 |
|  84 |
|  85 |
```



```
|  86 |
|  87 |
|  88 |
|  89 |
|  90 |
|  91 |
|  92 |
|  93 |
|  94 |
|  95 |
|  96 |
|  97 |
|  98 |
|  99 |
+-------+
100 tuples

50) server.clear()
50) Completed to clear default database.
50) select * from test_database;
50) SELECT: no such table 'test_database'
```

create()

데이터베이스를 생성하는 함수

문법

```
create(target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
target_db	생성할 데이터베이스 노드	선택	"DEFAULT"

리턴값

성공 문자열("Completed to create *db_name* database.")

주의사항

- $TSR_HOME/conf/db.conf에 데이터베이스 노드별 정보가 저장되어 있으며, 'target_db' 파라미터는 이 중의 하나를 값으로 가질 수 있다. (참고)

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.query import query
from dbtest.utils import comment, write_result

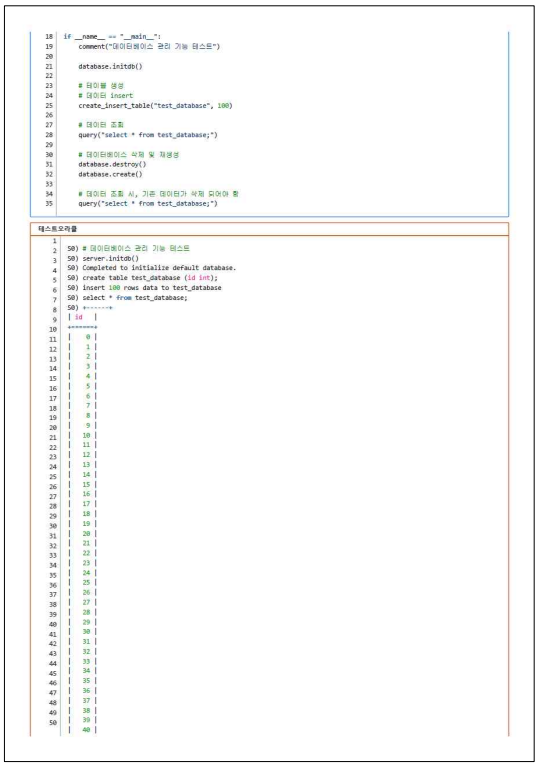

def create_insert_table(table_name, row_count):
    query("drop table if exists {};".format(table_name), query_silent=True, silent=True)

    query("create table {} (id int);".format(table_name), silent=True)

    for i in range(row_count):
        query("insert into {} values({});".format(table_name, i), query_silent=True, silent=True)

    write_result("insert {} rows data to {}".format(row_count, table_name))

```



```
if __name__ == "__main__":
    comment("데이터베이스 관리 기능 테스트")

    database.initdb()

    # 테이블 생성
    # 데이터 insert
    create_insert_table("test_database", 100)

    # 데이터 조회
    query("select * from test_database;")

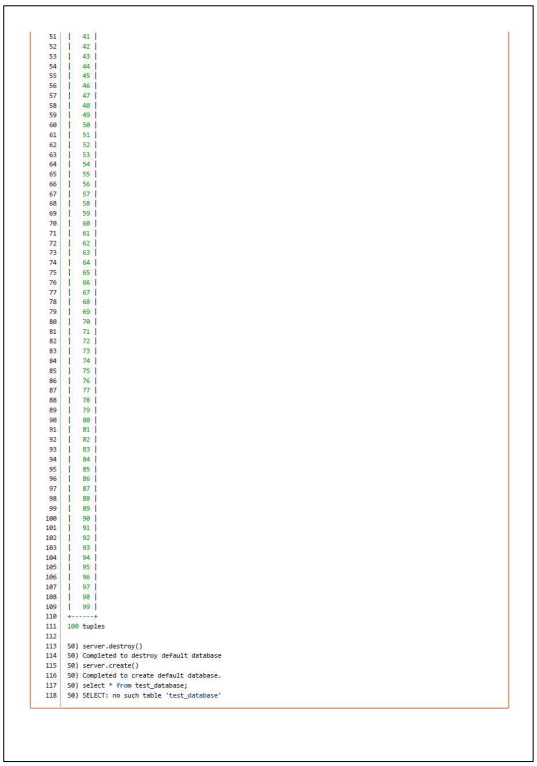
    # 데이터베이스 삭제 및 재생성
    database.destroy()
    database.create()

    # 데이터 조회 시, 기존 데이터가 삭제 되어야 함
    query("select * from test_database;")
```

테스트오라클

```

50) # 데이터베이스 관리 기능 테스트
50) server.initdb()
50) Completed to initialize default database.
50) create table test_database (id int);
50) Insert 100 rows data to test_database
50) select * from test_database;
50) +------+
| id   |
+======+
|    0 |
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
|   11 |
|   12 |
|   13 |
|   14 |
|   15 |
|   16 |
|   17 |
|   18 |
|   19 |
|   20 |
|   21 |
|   22 |
|   23 |
|   24 |
|   25 |
|   26 |
|   27 |
|   28 |
|   29 |
|   30 |
|   31 |
|   32 |
|   33 |
|   34 |
|   35 |
|   36 |
|   37 |
|   38 |
|   39 |
|   40 |
```



```
|  41 |
|  42 |
|  43 |
|  44 |
|  45 |
|  46 |
|  47 |
|  48 |
|  49 |
|  50 |
|  51 |
|  52 |
|  53 |
|  54 |
|  55 |
|  56 |
|  57 |
|  58 |
|  59 |
|  60 |
|  61 |
|  62 |
|  63 |
|  64 |
|  65 |
|  66 |
|  67 |
|  68 |
|  69 |
|  70 |
|  71 |
|  72 |
|  73 |
|  74 |
|  75 |
|  76 |
|  77 |
|  78 |
|  79 |
|  80 |
|  81 |
|  82 |
|  83 |
|  84 |
|  85 |
|  86 |
|  87 |
|  88 |
|  89 |
|  90 |
|  91 |
|  92 |
|  93 |
|  94 |
|  95 |
|  96 |
|  97 |
|  98 |
|  99 |
+-------+
100 tuples

50) server.destroy()
50) Completed to destroy default database
50) server.create()
50) Completed to create default database.
50) select * from test_database;
50) SELECT: no such table 'test_database'
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

start()

데이터베이스 서버를 구동시킨다.

문법

```
start([target_db="DEFAULT"])
```

파라미터

파라미터	내용	구분	기본값
target_db	구동할 데이터베이스 노드	선택	"DEFAULT"

리턴값

성공 문자열("Completed to start server")

주의사항

- $TSRI_HOME/conf/db.conf에 데이터베이스 노드별 정보가 저장되어 있으며, 'target_db' 파라미터는 이 중의 하나를 값으로 가질 수 있다. (필고)

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.process import get_pid
from dbtest.query import query
from dbtest.utils import comment, write_result



def create_insert_table(table_name, row_count):
    query("drop table if exists {};".format(table_name), query_silent=True, silent=True)

    query("create table {} (id int);".format(table_name), silent=True)

    for i in range(row_count):
        query("insert into {} values({});".format(table_name, i), query_silent=True, silent=True)

    write_result("insert {} rows data to {}".format(row_count, table_name))

if __name__ == "__main__":
    comment("데이터베이스 관리 기능 테스트")

    # 데이터베이스 초기화 및 서버 재시작
    database.initdb()


    # 테이블 생성
    # 데이터 insert
    create_insert_table("test_database", 100)

    # 데이터 조회
    query("select * from test_database;")

    # 데이터서버 중지
    database.stop()

    # 데이터 서버 시작
    database.start()

    # 데이터 조회
    query("select * from test_database;")
```

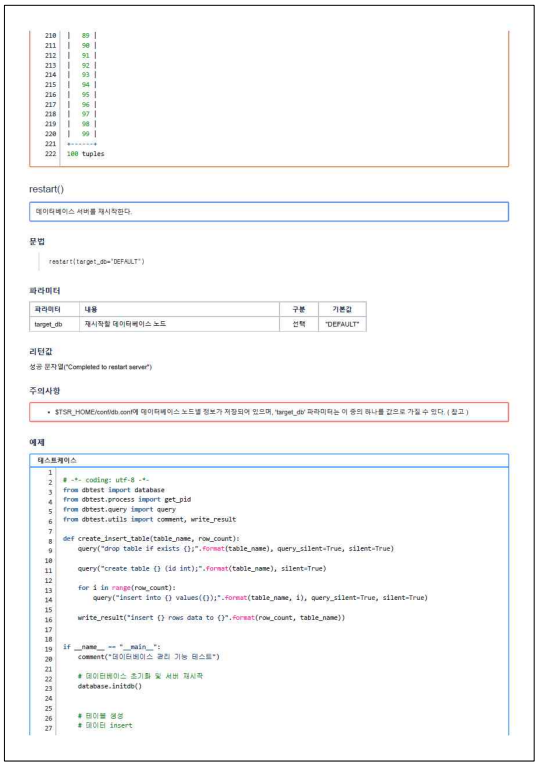
테스트오라클

```

50) # 데이터베이스 관리 기능 테스트
50) server.initdb()
50) Completed to initialize default database.
50) create table test_database (id int);
50) insert 100 rows data to test_database
50) select * from test_database;
50) +-------+
| id    |
+=======+
|     0 |
|     1 |
|     2 |
|     3 |
|     4 |
|     5 |
|     6 |
|     7 |
|     8 |
|     9 |
|    10 |
|    11 |
|    12 |
|    13 |
|    14 |
|    15 |
|    16 |
|    17 |
|    18 |
|    19 |
|    20 |
|    21 |
|    22 |
|    23 |
|    24 |
|    25 |
|    26 |
|    27 |
|    28 |
|    29 |
|    30 |
|    31 |
|    32 |
|    33 |
|    34 |
|    35 |
|    36 |
|    37 |
|    38 |
|    39 |
|    40 |
|    41 |
|    42 |
|    43 |
|    44 |
|    45 |
|    46 |
|    47 |
|    48 |
|    49 |
|    50 |
|    51 |
|    52 |
|    53 |
|    54 |
|    55 |
|    56 |
|    57 |
|    58 |
|    59 |
|    60 |
|    61 |
|    62 |
|    63 |
|    64 |
|    65 |
|    66 |
|    67 |
|    68 |
|    69 |
|    70 |
|    71 |
|    72 |
|    73 |
|    74 |
|    75 |
|    76 |
|    77 |
|    78 |
|    79 |
|    80 |
|    81 |
|    82 |
|    83 |
|    84 |
|    85 |
|    86 |
|    87 |
|    88 |
|    89 |
|    90 |
|    91 |
|    92 |
|    93 |
|    94 |
|    95 |
|    96 |
|    97 |
|    98 |
|    99 |
+-------+
100 tuples

50) server.stop()
50) Completed to stop server
50) server.start()
50) Completed to start server
50) select * from test_database;
50) +-------+
| id    |
+=======+
|     0 |
|     1 |
|     2 |
|     3 |
|     4 |
|     5 |
|     6 |
|     7 |
|     8 |
|     9 |
|    10 |
|    11 |
|    12 |
|    13 |
|    14 |
|    15 |
|    16 |
|    17 |
|    18 |
|    19 |
|    20 |
|    21 |
|    22 |
|    23 |
|    24 |
|    25 |
|    26 |
|    27 |
|    28 |
|    29 |
|    30 |
|    31 |
|    32 |
|    33 |
|    34 |
|    35 |
|    36 |
|    37 |
|    38 |
|    39 |
|    40 |
|    41 |
|    42 |
|    43 |
|    44 |
|    45 |
|    46 |
|    47 |
|    48 |
|    49 |
|    50 |
|    51 |
|    52 |
|    53 |
|    54 |
|    55 |
|    56 |
|    57 |
|    58 |
|    59 |
|    60 |
|    61 |
|    62 |
|    63 |
|    64 |
|    65 |
|    66 |
|    67 |
|    68 |
|    69 |
|    70 |
|    71 |
|    72 |
|    73 |
|    74 |
|    75 |
|    76 |
|    77 |
|    78 |
|    79 |
|    80 |
|    81 |
|    82 |
|    83 |
|    84 |
|    85 |
|    86 |
|    87 |
|    88 |
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



```
| 89 |
| 90 |
| 91 |
| 92 |
| 93 |
| 94 |
| 95 |
| 96 |
| 97 |
| 98 |
| 99 |
+------+
100 tuples
```

restart()

데이터베이스 서버를 재시작한다.

문법

```
restart([target_db='DEFAULT'])
```

파라미터

파라미터	내용	구분	기본값
target_db	재시작할 데이터베이스 노드	선택	"DEFAULT"

리턴값

성공 문자열("Completed to restart server")

주의사항

- $TSR_HOME/conf/db.conf에 데이터베이스 노드별 정보가 저장되어 있으며, 'target_db' 파라미터는 이 중의 하나를 값으로 가질 수 있다. (참고)

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.process import get_pid
from dbtest.query import query
from dbtest.utils import comment, write_result

def create_insert_table(table_name, row_count):
    query("drop table if exists {};".format(table_name), query_silent=True, silent=True)

    query("create table {} (id int);".format(table_name), silent=True)

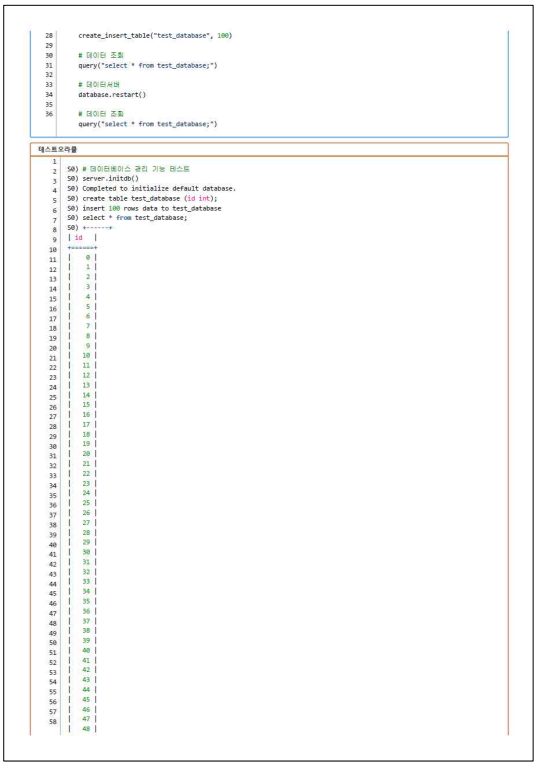
    for i in range(row_count):
        query("insert into {} values({});".format(table_name, i), query_silent=True, silent=True)

    write_result("insert {} rows data to {}".format(row_count, table_name))


if __name__ == "__main__":
    comment("데이터베이스 관리 기능 테스트")

    # 데이터베이스 초기화 및 서버 재시작
    database.initdb()


    # 테이블 생성
    # 데이터 insert
```



```
    create_insert_table("test_database", 100)

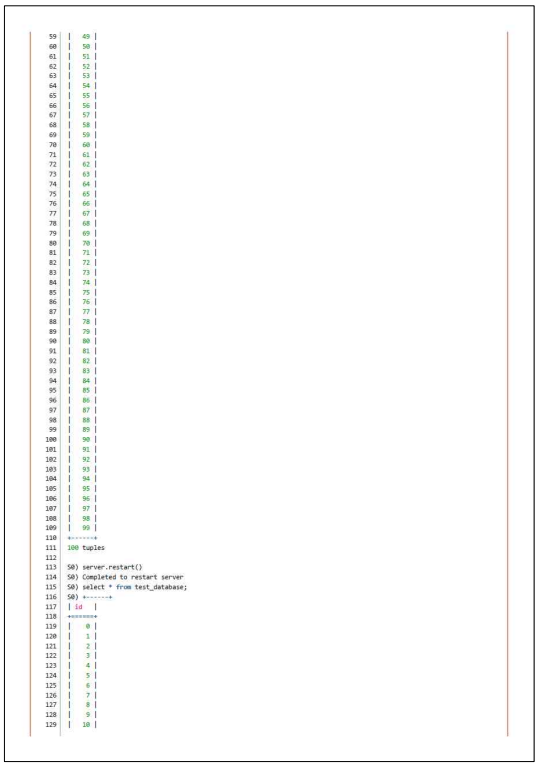
    # 데이터 조회
    query("select * from test_database;")

    # 데이터서버
    database.restart()

    # 데이터 조회
    query("select * from test_database;")
```

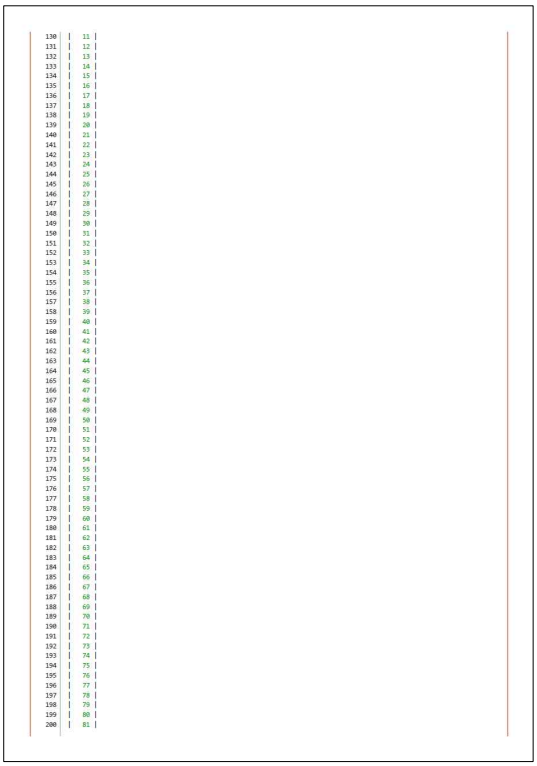
테스트오라클

```
50) # 데이터베이스 관리 기능 테스트
50) server.initdb()
50) Completed to initialize default database.
50) create table test_database (id int);
50) insert 100 rows data to test_database
50) select * from test_database;
50) +------+
| id   |
+======+
|    0 |
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
|   11 |
|   12 |
|   13 |
|   14 |
|   15 |
|   16 |
|   17 |
|   18 |
|   19 |
|   20 |
|   21 |
|   22 |
|   23 |
|   24 |
|   25 |
|   26 |
|   27 |
|   28 |
|   29 |
|   30 |
|   31 |
|   32 |
|   33 |
|   34 |
|   35 |
|   36 |
|   37 |
|   38 |
|   39 |
|   40 |
|   41 |
|   42 |
|   43 |
|   44 |
|   45 |
|   46 |
|   47 |
|   48 |
```



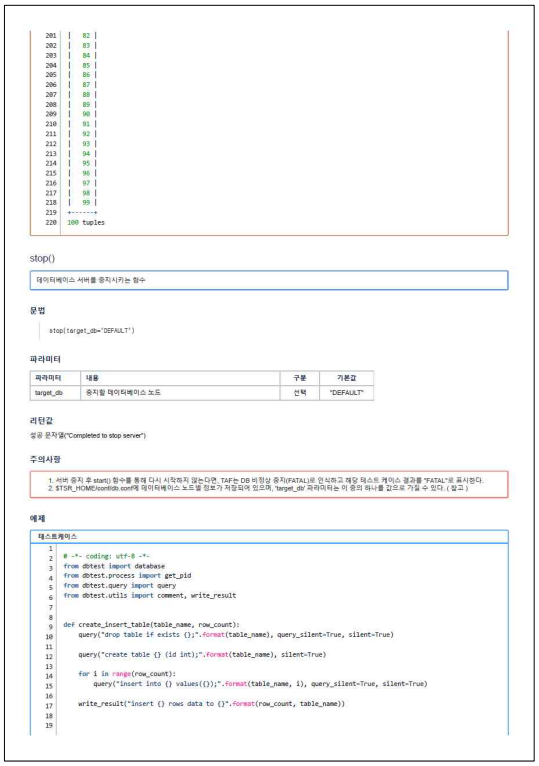
```
|   49 |
|   50 |
|   51 |
|   52 |
|   53 |
|   54 |
|   55 |
|   56 |
|   57 |
|   58 |
|   59 |
|   60 |
|   61 |
|   62 |
|   63 |
|   64 |
|   65 |
|   66 |
|   67 |
|   68 |
|   69 |
|   70 |
|   71 |
|   72 |
|   73 |
|   74 |
|   75 |
|   76 |
|   77 |
|   78 |
|   79 |
|   80 |
|   81 |
|   82 |
|   83 |
|   84 |
|   85 |
|   86 |
|   87 |
|   88 |
|   89 |
|   90 |
|   91 |
|   92 |
|   93 |
|   94 |
|   95 |
|   96 |
|   97 |
|   98 |
|   99 |
+------+
100 tuples

50) server.restart()
50) Completed to restart server
50) select * from test_database;
50) +------+
| id   |
+======+
|    0 |
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
```



```
|   11 |
|   12 |
|   13 |
|   14 |
|   15 |
|   16 |
|   17 |
|   18 |
|   19 |
|   20 |
|   21 |
|   22 |
|   23 |
|   24 |
|   25 |
|   26 |
|   27 |
|   28 |
|   29 |
|   30 |
|   31 |
|   32 |
|   33 |
|   34 |
|   35 |
|   36 |
|   37 |
|   38 |
|   39 |
|   40 |
|   41 |
|   42 |
|   43 |
|   44 |
|   45 |
|   46 |
|   47 |
|   48 |
|   49 |
|   50 |
|   51 |
|   52 |
|   53 |
|   54 |
|   55 |
|   56 |
|   57 |
|   58 |
|   59 |
|   60 |
|   61 |
|   62 |
|   63 |
|   64 |
|   65 |
|   66 |
|   67 |
|   68 |
|   69 |
|   70 |
|   71 |
|   72 |
|   73 |
|   74 |
|   75 |
|   76 |
|   77 |
|   78 |
|   79 |
|   80 |
|   81 |
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



```
|  82 |
|  83 |
|  84 |
|  85 |
|  86 |
|  87 |
|  88 |
|  89 |
|  90 |
|  91 |
|  92 |
|  93 |
|  94 |
|  95 |
|  96 |
|  97 |
|  98 |
|  99 |
+------+
100 tuples
```

stop()

데이터베이스 서버를 중지시키는 함수

문법

```
stop(target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
target_db	중지할 데이터베이스 노드	선택	"DEFAULT"

리턴값

성공 문자열("Completed to stop server")

주의사항

1. 서버 중지 후 start() 함수를 통해 다시 시작하지 않는다면, TAF는 DB 비정상 중지(FATAL)로 인식하고 해당 테스트 케이스 결과를 "FATAL"로 표시한다.
2. $TSR_HOME/conf/db.conf에 데이터베이스 노드를 정보가 저장되어 있으며, "target_db" 파라미터는 이 중의 하나를 값으로 가질 수 있다. (필요)

예제

테스트케이스

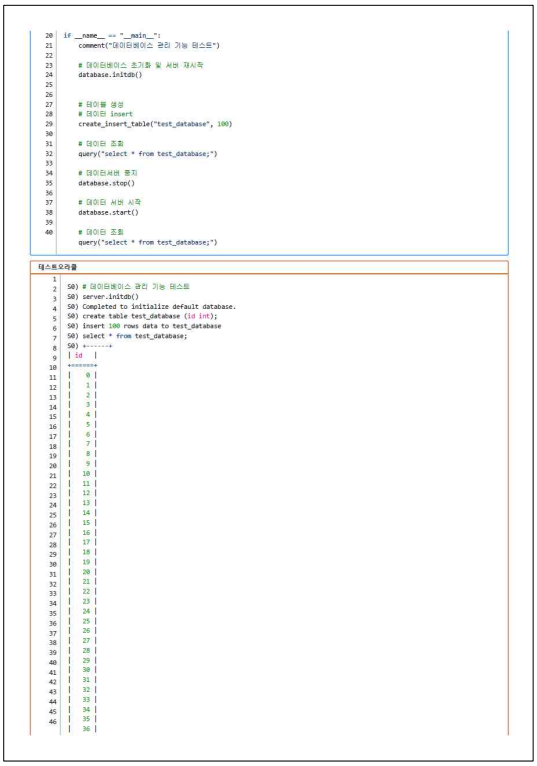
```
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.process import get_pid
from dbtest.query import query
from dbtest.utils import comment, write_result


def create_insert_table(table_name, row_count):
    query("drop table if exists {};".format(table_name), query_silent=True, silent=True)

    query("create table {} (id int);".format(table_name), silent=True)

    for i in range(row_count):
        query("insert into {} values({});".format(table_name, i), query_silent=True, silent=True)

    write_result("insert {} rows data to {}".format(row_count, table_name))
```



```
if __name__ == "__main__":
    comment("데이터베이스 관리 기능 테스트")

    # 데이터베이스 초기화 및 서버 재시작
    database.initdb()


    # 테이블 생성
    # 데이터 insert
    create_insert_table("test_database", 100)

    # 데이터 조회
    query("select * from test_database;")

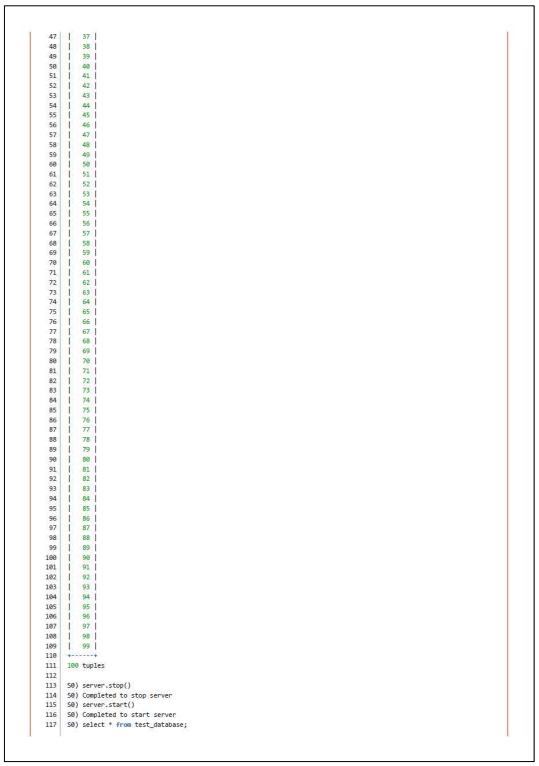
    # 데이터서버 중지
    database.stop()

    # 데이터 서버 시작
    database.start()

    # 데이터 조회
    query("select * from test_database;")
```

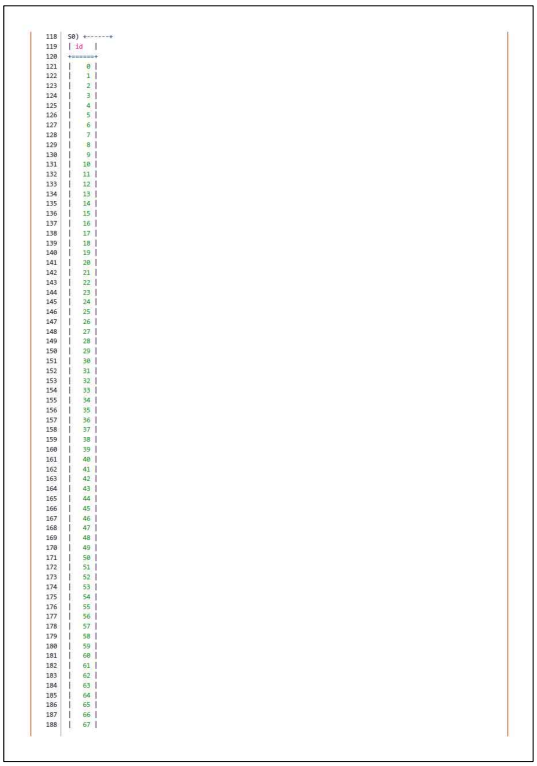
테스트오라클

```
50) # 데이터베이스 관리 기능 테스트
50) server.initdb()
50) Completed to initialize default database.
50) create table test_database (id int);
50) insert 100 rows data to test_database
50) select * from test_database;
50) +------+
| id   |
+======+
|    0 |
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
|   11 |
|   12 |
|   13 |
|   14 |
|   15 |
|   16 |
|   17 |
|   18 |
|   19 |
|   20 |
|   21 |
|   22 |
|   23 |
|   24 |
|   25 |
|   26 |
|   27 |
|   28 |
|   29 |
|   30 |
|   31 |
|   32 |
|   33 |
|   34 |
|   35 |
|   36 |
```



```
|  37 |
|  38 |
|  39 |
|  40 |
|  41 |
|  42 |
|  43 |
|  44 |
|  45 |
|  46 |
|  47 |
|  48 |
|  49 |
|  50 |
|  51 |
|  52 |
|  53 |
|  54 |
|  55 |
|  56 |
|  57 |
|  58 |
|  59 |
|  60 |
|  61 |
|  62 |
|  63 |
|  64 |
|  65 |
|  66 |
|  67 |
|  68 |
|  69 |
|  70 |
|  71 |
|  72 |
|  73 |
|  74 |
|  75 |
|  76 |
|  77 |
|  78 |
|  79 |
|  80 |
|  81 |
|  82 |
|  83 |
|  84 |
|  85 |
|  86 |
|  87 |
|  88 |
|  89 |
|  90 |
|  91 |
|  92 |
|  93 |
|  94 |
|  95 |
|  96 |
|  97 |
|  98 |
|  99 |
+------+
100 tuples

50) server.stop()
50) Completed to stop server
50) server.start()
50) Completed to start server
50) select * from test_database;
```



```
50) +------+
| id   |
+======+
|    0 |
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
|   11 |
|   12 |
|   13 |
|   14 |
|   15 |
|   16 |
|   17 |
|   18 |
|   19 |
|   20 |
|   21 |
|   22 |
|   23 |
|   24 |
|   25 |
|   26 |
|   27 |
|   28 |
|   29 |
|   30 |
|   31 |
|   32 |
|   33 |
|   34 |
|   35 |
|   36 |
|   37 |
|   38 |
|   39 |
|   40 |
|   41 |
|   42 |
|   43 |
|   44 |
|   45 |
|   46 |
|   47 |
|   48 |
|   49 |
|   50 |
|   51 |
|   52 |
|   53 |
|   54 |
|   55 |
|   56 |
|   57 |
|   58 |
|   59 |
|   60 |
|   61 |
|   62 |
|   63 |
|   64 |
|   65 |
|   66 |
|   67 |
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
| 189 |  68 |
| 190 |  69 |
| 191 |  70 |
| 192 |  71 |
| 193 |  72 |
| 194 |  73 |
| 195 |  74 |
| 196 |  75 |
| 197 |  76 |
| 198 |  77 |
| 199 |  78 |
| 200 |  79 |
| 201 |  80 |
| 202 |  81 |
| 203 |  82 |
| 204 |  83 |
| 205 |  84 |
| 206 |  85 |
| 207 |  86 |
| 208 |  87 |
| 209 |  88 |
| 210 |  89 |
| 211 |  90 |
| 212 |  91 |
| 213 |  92 |
| 214 |  93 |
| 215 |  94 |
| 216 |  95 |
| 217 |  96 |
| 218 |  97 |
| 219 |  98 |
| 220 |  99 |
| 221 | +-----+
| 222 | 100 tuples
```

destroy()

데이터베이스를 삭제하는 함수

문법

```
destroy(target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
target_db	초기화할 데이터베이스 노드	선택	"DEFAULT"

리턴값

성공 문자열(Completed to destroy db_name database)

주의사항

- $TSR_HOME/conf/db.conf에 데이터베이스 노드별 정보가 저장되어 있으며, 'target_db' 파라미터는 이 중의 하나를 값으로 가질 수 있다. (참고)

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.query import query
from dbtest.utils import comment, write_result
```

```

def create_insert_table(table_name, row_count):
    query("drop table if exists {};".format(table_name), query_silent=True, silent=True)

    query("create table {} (id int);".format(table_name), silent=True)

    for i in range(row_count):
        query("insert into {} values({});".format(table_name, i), query_silent=True, silent=True)

    write_result("insert {} rows data to {}".format(row_count, table_name))


if __name__ == "__main__":
    comment("데이터베이스 관리 기능 테스트")

    # 데이터베이스 초기화 및 서버 재시작
    database.initdb()

    # 테이블 생성
    # 데이터 insert
    create_insert_table("test_database", 100)

    # 데이터 조회
    query("select * from test_databasej")

    # 데이터베이스 초기화
    database.destroy()
```

테스트오라클

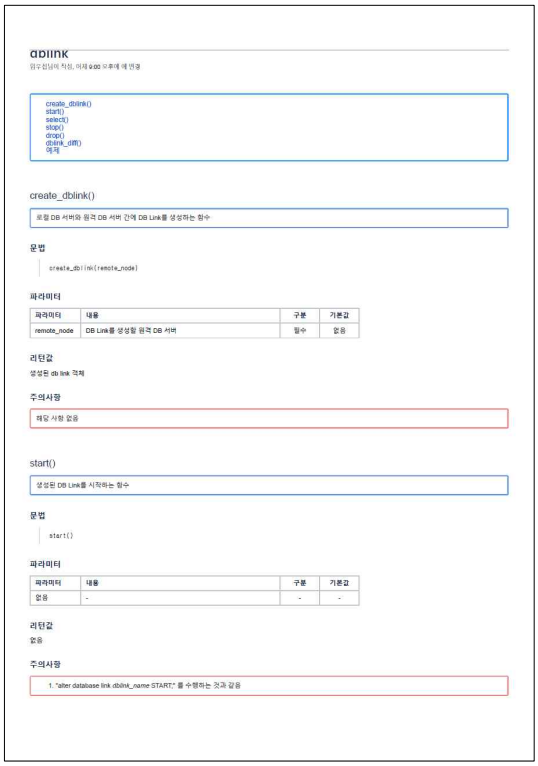
```
50) # 데이터베이스 관리 기능 테스트
50) server.initdb()
50) Completed to initialize default database.
50) create table test_database (id int);
50) insert 100 rows data to test_database
50) select * from test_database;
50) +------+
| id   |
+======+
|    0 |
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
|   11 |
|   12 |
|   13 |
|   14 |
|   15 |
|   16 |
|   17 |
|   18 |
|   19 |
|   20 |
|   21 |
|   22 |
|   23 |
|   24 |
|   25 |
|   26 |
|   27 |
|   28 |
|   29 |
|   30 |
```

```
| 41 |  31 |
| 42 |  32 |
| 43 |  33 |
| 44 |  34 |
| 45 |  35 |
| 46 |  36 |
| 47 |  37 |
| 48 |  38 |
| 49 |  39 |
| 50 |  40 |
| 51 |  41 |
| 52 |  42 |
| 53 |  43 |
| 54 |  44 |
| 55 |  45 |
| 56 |  46 |
| 57 |  47 |
| 58 |  48 |
| 59 |  49 |
| 60 |  50 |
| 61 |  51 |
| 62 |  52 |
| 63 |  53 |
| 64 |  54 |
| 65 |  55 |
| 66 |  56 |
| 67 |  57 |
| 68 |  58 |
| 69 |  59 |
| 70 |  60 |
| 71 |  61 |
| 72 |  62 |
| 73 |  63 |
| 74 |  64 |
| 75 |  65 |
| 76 |  66 |
| 77 |  67 |
| 78 |  68 |
| 79 |  69 |
| 80 |  70 |
| 81 |  71 |
| 82 |  72 |
| 83 |  73 |
| 84 |  74 |
| 85 |  75 |
| 86 |  76 |
| 87 |  77 |
| 88 |  78 |
| 89 |  79 |
| 90 |  80 |
| 91 |  81 |
| 92 |  82 |
| 93 |  83 |
| 94 |  84 |
| 95 |  85 |
| 96 |  86 |
| 97 |  87 |
| 98 |  88 |
| 99 |  89 |
| 100 |  90 |
| 101 |  91 |
| 102 |  92 |
| 103 |  93 |
| 104 |  94 |
| 105 |  95 |
| 106 |  96 |
| 107 |  97 |
| 108 |  98 |
| 109 |  99 |
| 110 | +-----+
| 111 | 100 tuples
```

```
| 112 |
| 113 | 50) server.destroy()
| 114 | 50) Completed to destroy default database
```

레이블 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



dblink

임우섭님의 작성, 마지막 edit 오후에 에 연결

create_dblink()
start()
select()
stop()
drop()
dblink_diff()
예제

create_dblink()

로컬 DB 서버와 원격 DB 서버 간에 DB Link를 생성하는 함수

문법

```
create_dblink(remote_node)
```

파라미터

파라미터	내용	구분	기본값
remote_node	DB Link를 생성할 원격 DB 서버	필수	없음

리턴값

생성된 db link 객체

주의사항

해당 사항 없음

start()

생성된 DB Link를 시작하는 함수

문법

```
start()
```

파라미터

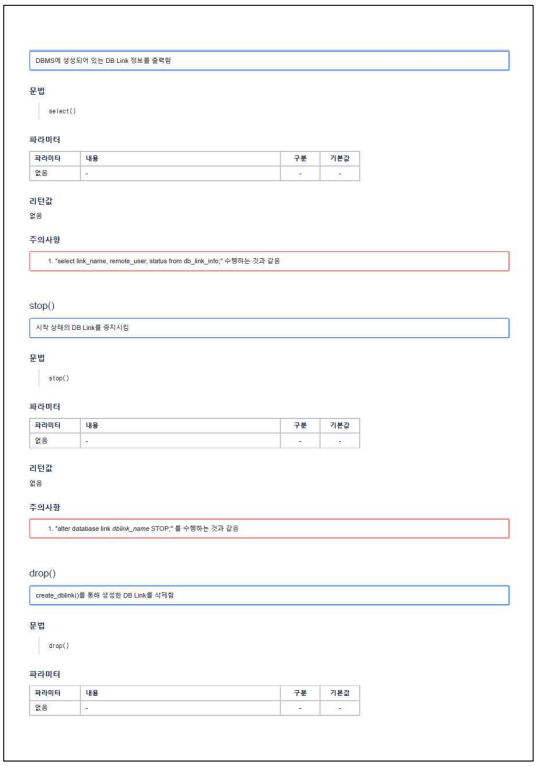
파라미터	내용	구분	기본값
없음	-	-	-

리턴값

없음

주의사항

1. "alter database link *dblink_name* START;" 를 수행하는 것과 같음



DBMS에 생성되어 있는 DB Link 정보를 출력함

문법

```
select()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

없음

주의사항

1. "select link_name, remote_user, status from db_link_info;" 수행하는 것과 같음

stop()

시작 상태의 DB Link를 중지시킴

문법

```
stop()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

없음

주의사항

1. "alter database link *dblink_name* STOP;" 를 수행하는 것과 같음

drop()

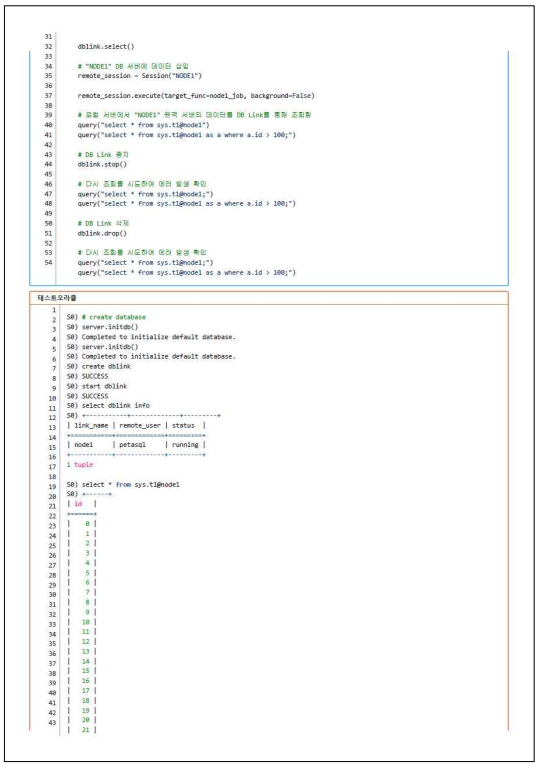
create_dblink()를 통해 생성된 DB Link를 삭제함

문법

```
drop()
```

파라미터

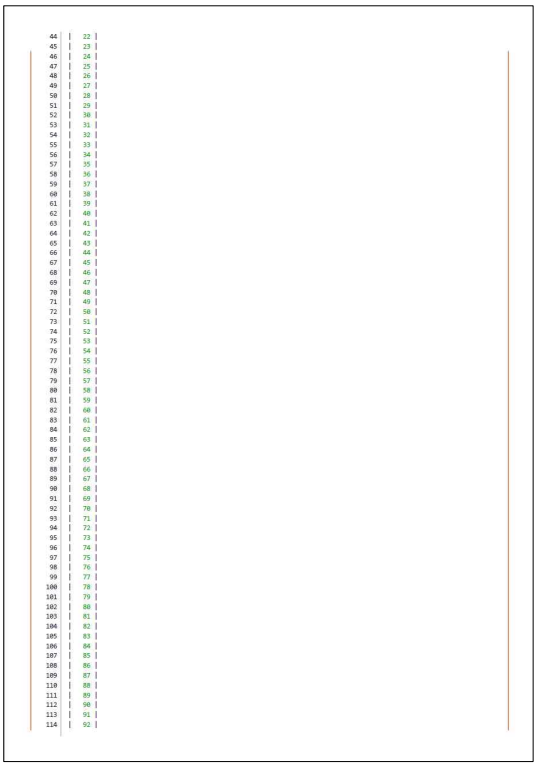
파라미터	내용	구분	기본값
없음	-	-	-



```
31
32      dblink.select()
33
34      # "NODE1" DB 서버에 데이터 삽입
35      remote_session = Session("NODE1")
36
37      remote_session.execute(target_func=node1_job, background=False)
38
39      # 로컬 서버에서 "NODE1" 원격 서버의 데이터를 DB Link를 통해 조회함
40      query("select * from sys.t1@node1")
41      query("select * from sys.t1@node1 as a where a.id > 100;")
42
43      # DB Link 중지
44      dblink.stop()
45
46      # 다시 조회를 시도하여 에러 발생 확인
47      query("select * from sys.t1@node1;")
48      query("select * from sys.t1@node1 as a where a.id > 100;")
49
50      # DB Link 삭제
51      dblink.drop()
52
53      # 다시 조회를 시도하여 에러 발생 확인
54      query("select * from sys.t1@node1;")
        query("select * from sys.t1@node1 as a where a.id > 100;")
```

테스트오라클

```
1
2   SQ) # create database
3   SQ) server.initdb()
4   SQ) Completed to initialize default database.
5   SQ) server.initdb()
6   SQ) Completed to initialize default database.
7   SQ) create dblink
8   SQ) SUCCESS
9   SQ) start dblink
10  SQ) SUCCESS
11  SQ) select dblink info
12  SQ) +-----------+-------------+---------+
13  | link_name | remote_user | status  |
14  +===========+=============+=========+
15  | node1     | petasql     | running |
16  +-----------+-------------+---------+
17  1 tuple
18
19  SQ) select * from sys.t1@node1
20  SQ) +------+
21  | id   |
22  +======+
23  |    0 |
24  |    1 |
25  |    2 |
26  |    3 |
27  |    4 |
28  |    5 |
29  |    6 |
30  |    7 |
31  |    8 |
32  |    9 |
33  |   10 |
34  |   11 |
35  |   12 |
36  |   13 |
37  |   14 |
38  |   15 |
39  |   16 |
40  |   17 |
41  |   18 |
42  |   19 |
43  |   20 |
    |   21 |
```



```
44  |   22 |
45  |   23 |
46  |   24 |
47  |   25 |
48  |   26 |
49  |   27 |
50  |   28 |
51  |   29 |
52  |   30 |
53  |   31 |
54  |   32 |
55  |   33 |
56  |   34 |
57  |   35 |
58  |   36 |
59  |   37 |
60  |   38 |
61  |   39 |
62  |   40 |
63  |   41 |
64  |   42 |
65  |   43 |
66  |   44 |
67  |   45 |
68  |   46 |
69  |   47 |
70  |   48 |
71  |   49 |
72  |   50 |
73  |   51 |
74  |   52 |
75  |   53 |
76  |   54 |
77  |   55 |
78  |   56 |
79  |   57 |
80  |   58 |
81  |   59 |
82  |   60 |
83  |   61 |
84  |   62 |
85  |   63 |
86  |   64 |
87  |   65 |
88  |   66 |
89  |   67 |
90  |   68 |
91  |   69 |
92  |   70 |
93  |   71 |
94  |   72 |
95  |   73 |
96  |   74 |
97  |   75 |
98  |   76 |
99  |   77 |
100 |   78 |
101 |   79 |
102 |   80 |
103 |   81 |
104 |   82 |
105 |   83 |
106 |   84 |
107 |   85 |
108 |   86 |
109 |   87 |
110 |   88 |
111 |   89 |
112 |   90 |
113 |   91 |
114 |   92 |
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
|  93 |
|  94 |
|  95 |
|  96 |
|  97 |
|  98 |
|  99 |
+-------+
100 tuples

S0) select * from sys.t1@node1 as a where a.id > 100;
S0) +-----+
| id |
+=====+
+-----+
0 tuples

S0) stop dblink
S0) SUCCESS
S0) select * from sys.t1@node1;
S0) DATABASE LINK: database link 'node1' is not running

S0) select * from sys.t1@node1 as a where a.id > 100;
S0) DATABASE LINK: database link 'node1' is not running

S0) drop dblink
S0) operation successful

S0) select * from sys.t1@node1;
S0) DATABASE LINK: no such database link 'node1'

S0) select * from sys.t1@node1 as a where a.id > 100;
S0) DATABASE LINK: no such database link 'node1'

S1) drop table t1 if exists;
S1) create table t1 (id int);
S1) insert 100 rows to t1 table
```

레이블 없음

environment

임무컴퓨터 작성, 이제 gcs 모두에 에 연결

set_env()
get_env()
rm_env()
예제

set_env()

환경 변수를 설정하는 함수

문법

```
set_env(name, value, target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
name	환경변수 이름	필수	없음
value	설정할 값	필수	없음
target_db	환경 변수를 적용할 데이터베이스 노드	선택	

리턴값

"SUCCESS" / "FAILURE"

주의사항

해당 사항 없음

get_env()

환경 변수의 값을 얻어오는 함수

문법

```
get_env(name, target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
name	환경변수 이름	필수	없음
target_db	환경 변수 값을 읽어올 데이터베이스 노드	선택	

리턴값

환경 변수 값

주의사항

해당 사항 없음

rm_env()

설정된 환경 변수를 제거하는 함수

문법

```
rm_env(name, target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
name	환경변수 이름	필수	없음
target_db	적용할 데이터베이스 노드	선택	

리턴값

"SUCCESS" / "FAILURE"

주의사항

해당 사항 없음

예제

테스트케이스

```python
# -*- coding: utf-8 -*-

from dbtest.environment import set_env
from dbtest.system import system
from dbtest.utils import comment, write_result, isWindows

if __name__ == "__main__":
    comment("test set env")

    write_result(set_env("TEST_ENV", "TTATEST"))

    if isWindows():
        write_result(system("echo %TEST_ENV%"))
    else:
        write_result(system("echo ${TEST_ENV}"))
```

테스트오라클

```
S0) # test set env
S0) SUCCESS
S0) TTATEST
```

레이블 없음

```
7 : SUCCESS
1
8 : SUCCESS
1
9 : SUCCESS
1
10 : SUCCESS
1
11 : SUCCESS
1
12 : SUCCESS
1
13 : SUCCESS
1
14 : SUCCESS
1
15 : SUCCESS
1
16 : SUCCESS
1
17 : SUCCESS
1
18 : SUCCESS
1
19 : SUCCESS
1

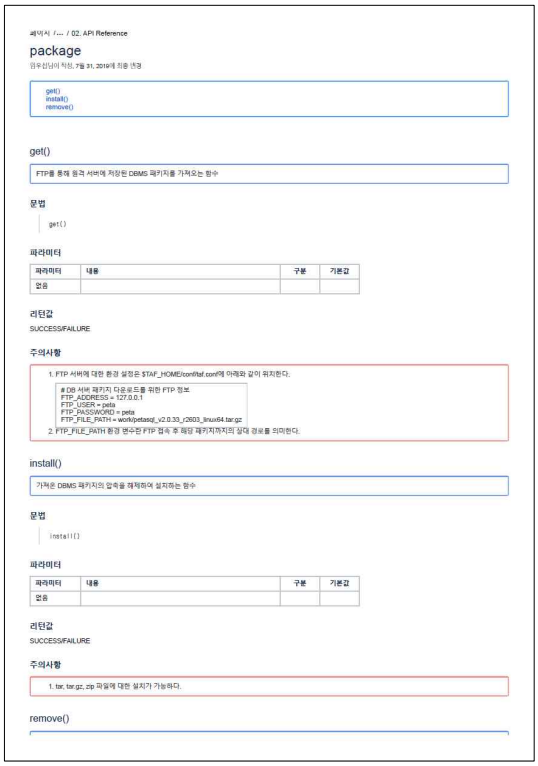
(중략)

9988 : SUCCESS
1
9989 : SUCCESS
1
9990 : SUCCESS
1
9991 : SUCCESS
1
9992 : SUCCESS
1
9993 : SUCCESS
1
9994 : SUCCESS
1
9995 : SUCCESS
1
9996 : SUCCESS
1
9997 : SUCCESS
1
9998 : SUCCESS
1
9999 : SUCCESS
----------------------------------------
Result : Test Success!
----------------------------------------

S0) SUCCESS
```

레이블 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



페이지 /... / 02. API Reference

package

임우십님이 작성, 7월 31, 2019에 최종 변경

get()
install()
remove()

get()

FTP를 통해 원격 서버에 저장된 DBMS 패키지를 가져오는 함수

문법

```
get()
```

파라미터

파라미터	내용	구분	기본값
없음			

리턴값

SUCCESS/FAILURE

주의사항

1. FTP 서버에 대한 환경 설정은 $TAF_HOME/conf/taf.conf에 아래와 같이 위치한다.

```
# DB 서버 패키지 다운로드를 위한 FTP 정보
FTP_ADDRESS = 127.0.0.1
FTP_USER = peta
FTP_PASSWORD = peta
FTP_FILE_PATH = work/petasql_v2.0.33_r2603_linux64.tar.gz
```

2. FTP_FILE_PATH 환경 변수는 FTP 접속 후 해당 패키지까지의 상대 경로를 의미한다.

install()

가져온 DBMS 패키지의 압축을 해제하여 설치하는 함수

문법

```
install()
```

파라미터

파라미터	내용	구분	기본값
없음			

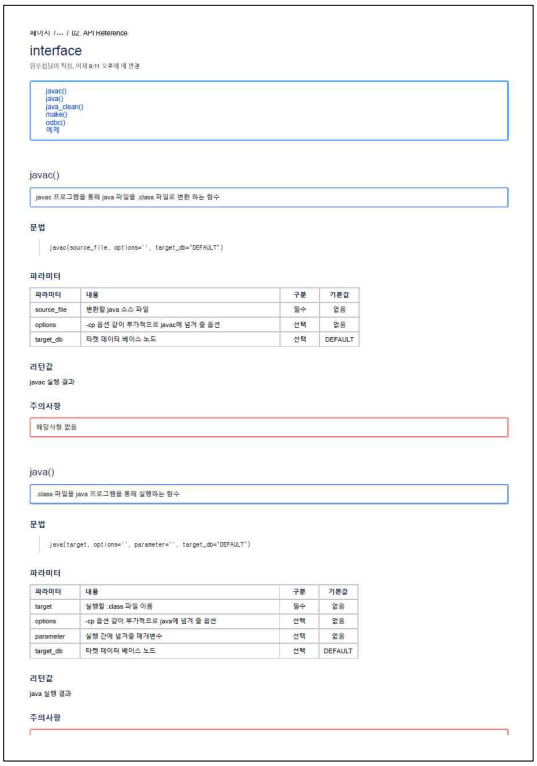
리턴값

SUCCESS/FAILURE

주의사항

1. tar, tar.gz, zip 파일에 대한 설치가 가능하다.

remove()



페이지 /... / 02. API Reference

interface

임우십님이 작성, 이제 9:11 오후에 에 변경

javac()
java()
java_clean()
make()
odbc()
예제

javac()

javac 프로그램을 통해 java 파일을 .class 파일로 변환 하는 함수

문법

```
javac(source_file, options='', target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
source_file	변환할 java 소스 파일	필수	없음
options	-cp 옵션 같이 부가적으로 javac에 넘겨 줄 옵션	선택	없음
target_db	타겟 데이타 베이스 노드	선택	DEFAULT

리턴값

javac 실행 결과

주의사항

해당사항 없음

java()

.class 파일을 java 프로그램을 통해 실행하는 함수

문법

```
java(target, options='', parameter='', target_db='DEFAULT')
```

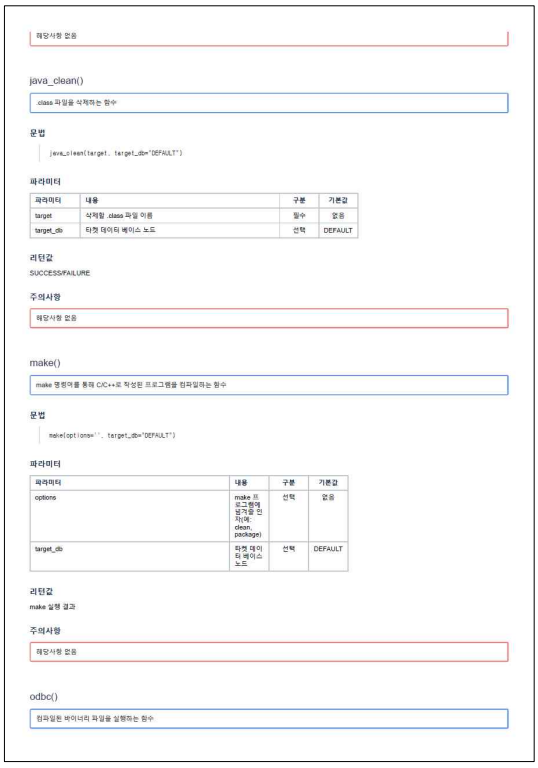
파라미터

파라미터	내용	구분	기본값
target	실행할 .class 파일 이름	필수	없음
options	-cp 옵션 같이 부가적으로 java에 넘겨 줄 옵션	선택	없음
parameter	실행 간에 넘겨줄 매개변수	선택	없음
target_db	타겟 데이타 베이스 노드	선택	DEFAULT

리턴값

java 실행 결과

주의사항



해당사항 없음

java_clean()

.class 파일을 삭제하는 함수

문법

```
java_clean(target, target_db="DEFAULT")
```

파라미터

파라미터	내용	구분	기본값
target	삭제할 .class 파일 이름	필수	없음
target_db	타겟 데이타 베이스 노드	선택	DEFAULT

리턴값

SUCCESS/FAILURE

주의사항

해당사항 없음

make()

make 명령어를 통해 C/C++로 작성된 프로그램을 컴파일하는 함수

문법

```
make(options='', target_db="DEFAULT")
```

파라미터

파라미터	내용	구분	기본값
options	make 프로그램에 넘겨줄 인자(예: clean, package)	선택	없음
target_db	타겟 데이타 베이스 노드	선택	DEFAULT

리턴값

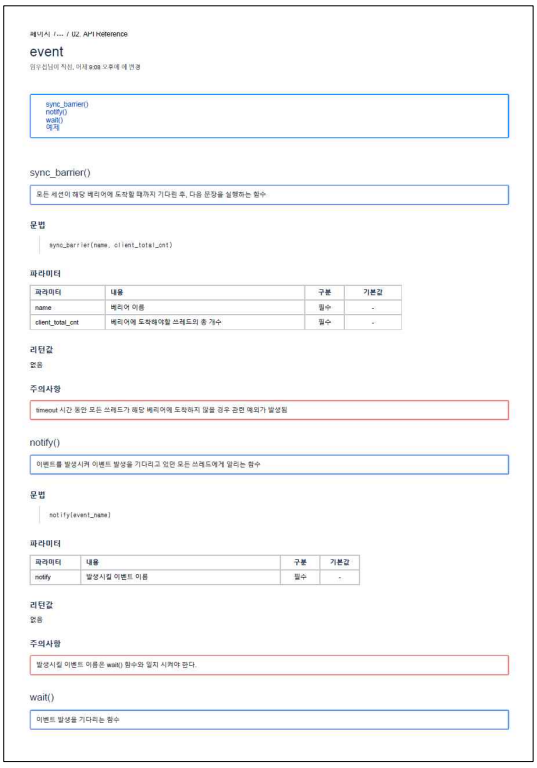
make 실행 결과

주의사항

해당사항 없음

odbc()

컴파일된 바이너리 파일을 실행하는 함수



페이지 /... / 02. API Reference

event

임우십님이 작성, 이제 9:03 오후에 에 변경

sync_barrier()
notify()
wait()
예제

sync_barrier()

모든 세션이 해당 배리어에 도착할 때까지 기다린 후, 다음 문장을 실행하는 함수

문법

```
sync_barrier(name, client_total_cnt)
```

파라미터

파라미터	내용	구분	기본값
name	배리어 이름	필수	-
client_total_cnt	배리어에 도착해야할 쓰레드의 총 개수	필수	-

리턴값

없음

주의사항

timeout 시간 동안 모든 쓰레드가 해당 배리어에 도착하지 않을 경우 관련 예외가 발생됨

notify()

이벤트를 발생시켜 이벤트 발생을 기다리고 있던 모든 쓰레드에게 알리는 함수

문법

```
notify(event_name)
```

파라미터

파라미터	내용	구분	기본값
notify	발생시킬 이벤트 이름	필수	-

리턴값

없음

주의사항

발생시킬 이벤트 이름은 wait() 함수와 일치 시켜야 한다.

wait()

이벤트 발생을 기다리는 함수

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

문법

```
wait(event_name, timeout=TAF_EVENT_TIMEOUT)
```

파라미터

파라미터	내용	구분	기본값
event_name	대기할 이벤트 이름	필수	-
timeout	대기 타임아웃 타임아웃 발생시, Error 문구가 결과 파일에 기록됨	선택	TAF_EVENT_TIMEOUT 환경 변수값

리턴값

없음

주의사항

대기할 이벤트 이름은 notify()와 일치시켜야 한다.

예제

테스트케이스

```python
# -*- coding: utf-8 -*-

from time import sleep

from dbtest.event import sync_barrier
from dbtest.query import query
from dbtest.session import Session


def client1():
    query("drop table if exists t1 CASCADE;")

    query("create table t1 (id int);")

    query("drop table t1;")


def client2():
    query("create table t1 (id int);")
    sleep(1)

    sync_barrier("S1", 2)

    # query("drop table t1;")


def client3():
    sync_barrier("S1", 2)

    for i in range(10):
        query("insert into t1 values(" + str(i) + ");")


if __name__ == "__main__":
    s = Session("DEFAULT")

    s.execute(target_func=client1)

    s.wait()
    print("wait 1 thread")

    s.execute(target_func=client1, session_cnt=10, background=False)

    s = Session()

    s.execute(target_func=client1)

    s.wait()

    # 비동기 execute가 여러번 수행될 때
    s = Session("DEFAULT")
    s.execute(target_func=client1)
    s.execute(target_func=client1, session_cnt=3)
    s.execute(target_func=client1)

    s.wait()

    s = Session()
    s.execute(sql_query="create table t1 (id int);")

    s.wait()

    s = Session("DEFAULT")
    s.execute(sql_query="create table Repl (id int);")
    s.execute(sql_query="insert into Repl values(1)")
    s.execute(sql_query="select * from t1;", session_cnt=3)
    s.execute(sql_query="select * from Repl;")
    s.execute(sql_query="select * from t3;", session_cnt=2)

    s.wait()

    s = Session()
    s.execute(target_func=client2)
    s.execute(target_func=client3)

    s.wait()
```

테스트오라클

```
S1) drop table if exists t1 CASCADE;
S1) operation successful

S1) create table t1 (id int);
S1) operation successful

S1) drop table t1;
S1) operation successful

S2) drop table if exists t1 CASCADE;
S2) operation successful

S2) create table t1 (id int);
S2) operation successful

S2) drop table t1;
S2) DROP TABLE: no such table 't1'

S3) drop table if exists t1 CASCADE;
S3) operation successful

S3) create table t1 (id int);
S3) operation successful

S3) drop table t1;
S3) DROP TABLE: no such table 't1'

S4) drop table if exists t1 CASCADE;
S4) operation successful

S4) create table t1 (id int);
S4) operation successful

S4) drop table t1;
S4) operation successful

S5) drop table if exists t1 CASCADE;
S5) operation successful

S5) create table t1 (id int);
S5) operation successful

S5) drop table t1;
S5) operation successful

S6) drop table if exists t1 CASCADE;
S6) operation successful

S6) create table t1 (id int);
S6) operation successful

S6) drop table t1;
S6) operation successful

S7) drop table if exists t1 CASCADE;
S7) operation successful

S7) create table t1 (id int);
S7) operation successful

S7) drop table t1;
S7) operation successful

S8) drop table if exists t1 CASCADE;
S8) operation successful

S8) create table t1 (id int);
S8) operation successful

S8) drop table t1;
S8) operation successful

S9) drop table if exists t1 CASCADE;
S9) operation successful

S9) create table t1 (id int);
S9) operation successful

S9) drop table t1;
S9) operation successful

S10) drop table if exists t1 CASCADE;
S10) operation successful

S10) create table t1 (id int);
S10) CREATE TABLE: name 't1' already in use

S10) drop table t1;
S10) operation successful

S11) drop table if exists t1 CASCADE;
S11) operation successful

S11) create table t1 (id int);
S11) operation successful

S11) drop table t1;
S11) operation successful

S12) drop table if exists t1 CASCADE;
S12) operation successful

S12) create table t1 (id int);
S12) operation successful

S12) drop table t1;
S12) operation successful

S13) drop table if exists t1 CASCADE;
S13) operation successful

S13) create table t1 (id int);
S13) operation successful

S13) drop table t1;
S13) operation successful

S14) drop table if exists t1 CASCADE;
S14) operation successful

S14) create table t1 (id int);
S14) operation successful

S14) drop table t1;
S14) operation successful

S15) drop table if exists t1 CASCADE;
S15) operation successful

S15) create table t1 (id int);
S15) operation successful

S15) drop table t1;
S15) operation successful

S16) drop table if exists t1 CASCADE;
S16) operation successful

S16) create table t1 (id int);
S16) operation successful

S16) drop table t1;
S16) operation successful

S17) drop table if exists t1 CASCADE;
S17) operation successful

S17) create table t1 (id int);
S17) operation successful

S17) drop table t1;
S17) operation successful

S18) create table t1 (id int);
S18) operation successful

S19) create table Repl (id int);
S19) operation successful

S20) insert into Repl values(1)
S20) 1 affected row

S21) select * from t1;
S21) +----+
| id |
++===++
+----+
0 tuples

S22) select * from t1;
S22) +----+
| id |
++===++
+----+
0 tuples

S23) select * from t1;
S23) +----+
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
| id |
+----+
+----+
0 tuples

524) select * from Rep1;
524) +----+
| id |
+----+
+----+
0 tuples

525) select * from t3;
525) SELECT: no such table 't3'

526) select * from t3;
526) SELECT: no such table 't3'

527) create table t1 (id int);
527) CREATE TABLE: name 't1' already in use

528) insert into t1 values(0);
528) 1 affected row

528) insert into t1 values(1);
528) 1 affected row

528) insert into t1 values(2);
528) 1 affected row

528) insert into t1 values(3);
528) 1 affected row

528) insert into t1 values(4);
528) 1 affected row

528) insert into t1 values(5);
528) 1 affected row

528) insert into t1 values(6);
528) 1 affected row

528) insert into t1 values(7);
528) 1 affected row

528) insert into t1 values(8);
528) 1 affected row

528) insert into t1 values(9);
528) 1 affected row
```

레이블 없음

문법

```
odbc(binary_name, parameter='', target_db='DEFAULT')
```

파라미터

파라미터	내용	구분	기본값
binary_name	실행할 바이너리 파일의 이름	필수	없음
parameter	바이너리 파일에 넘겨줄 매개 변수	선택	없음
target_db	타겟 데이터 베이스 노드	선택	DEFAULT

리턴값

바이너리 파일 실행 결과

주의사항

해당사항 없음

예제

테스트케이스

```
#-*-coding:utf-8

import sys
import os

from dbtest.query import *
from dbtest.utils import *
from dbtest.interface import *

"""
 # Description : 하나의 Connection 객체에 1000개의 Statemnet 객체 생성이 가능한지 확인
 # Scenario : SUCCESS 및 10라는 결과 값이 출력되면 성공
"""

if __name__ == '__main__':

    file_name = 'apiload'

    javac(file_name + '.java', options=' -cp $TSR_HOME/testsuite/Interface/JDBC/lib/*:.')

    java(file_name, options=' -cp $TSR_HOME/testsuite/Interface/JDBC/lib/*:. ')

    java_clean(file_name)
```

테스트오라클

```
S0)
S0) 1
0 : SUCCESS
1
1 : SUCCESS
1
2 : SUCCESS
1
3 : SUCCESS
1
4 : SUCCESS
1
5 : SUCCESS
1
6 : SUCCESS
1
```

가전은 DBMS 패키지를 삭제하는 함수

문법

```
remove()
```

파라미터

파라미터	내용	구분	기본값
없음			

리턴값

SUCCESS/FAILURE

주의사항

1. 해당 사항 없음

예제

테스트케이스

```
# -*- coding: utf-8 -*-
# Copyright (C) 2019 TTA - All Rights Reserved
# Unauthorized copying of this file, via any medium is strictly prohibited
# Proprietary and confidential

from dbtest import database
from dbtest.database import remove_dbms_home
from dbtest.package import get, install, remove
from dbtest.query import query

if __name__ == "__main__":
    # 쿼리인면 petaql관 삭제
    remove_dbms_home()

    get()

    install()

    # 다운 받았던 패키지 삭제
    remove()

    database.initdb()

    query("create table get_dbms ( c1 int);")

    query("insert into get_dbms values(1);")

    query("select * from get_dbms;")
```

테스트오라클

```
S0) server.remove_dbms_home()
S0) Completed to remove dbms home directory.
S0) package.get()
S0) SUCCESS
S0) package.install()
S0) SUCCESS
S0) package.remove()
S0) SUCCESS
S0) server.initdb()
S0) Completed to initialize default database.
```

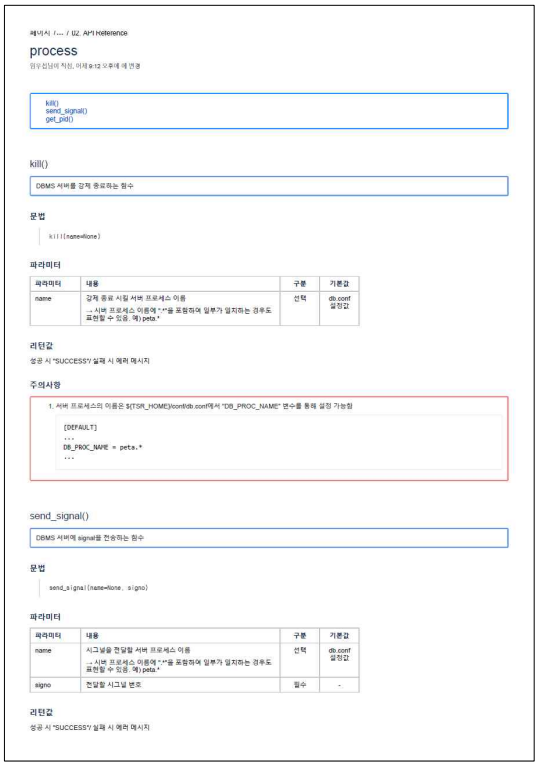
```
S0) create table get_dbms ( c1 int);
S0) operation successful

S0) insert into get_dbms values(1);
S0) 1 affected row

S0) select * from get_dbms;
S0) +------+
| c1   |
+======+
|    1 |
+------+
1 tuple
```

레이블 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



베이시 / ... / 02. API Reference

process

임우섭님께 작성, 이재호님 오후에 에 변경

kill()
send_signal()
get_pid()

kill()

DBMS 서버를 강제 종료하는 함수

문법

```
kill(name=None)
```

파라미터

파라미터	내용	구분	기본값
name	강제 종료 시킬 서버 프로세스 이름 → 서버 프로세스 이름에 "."을 포함하여 일부가 일치하는 경우도 표현할 수 있음. 예) peta.*	선택	db.conf 설정값

리턴값

성공 시 "SUCCESS" 실패 시 에러 메시지

주의사항

1. 서버 프로세스의 이름은 ${TSR_HOME}/conf/db.conf에서 "DB_PROC_NAME" 변수를 통해 설정 가능함

```
[DEFAULT]
...
DB_PROC_NAME = peta.*
...
```

send_signal()

DBMS 서버에 signal을 전송하는 함수

문법

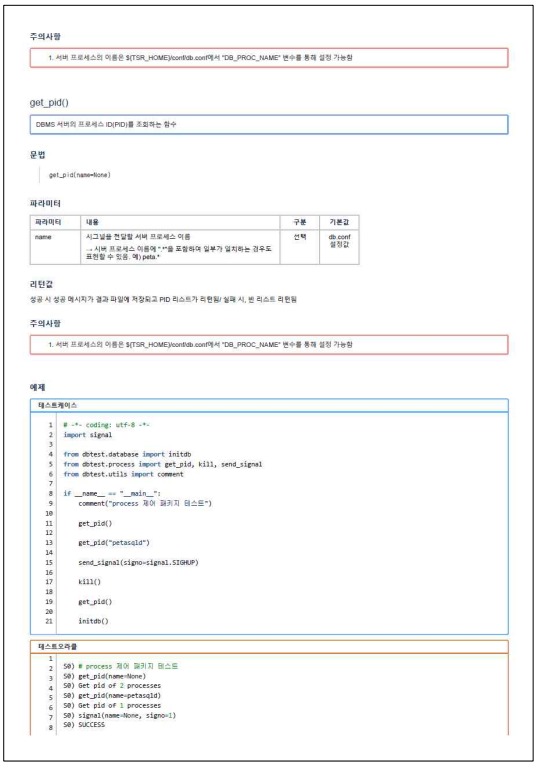
```
send_signal(name=None, signo)
```

파라미터

파라미터	내용	구분	기본값
name	시그널을 전달할 서버 프로세스 이름 → 서버 프로세스 이름에 "."을 포함하여 일부가 일치하는 경우도 표현할 수 있음. 예) peta.*	선택	db.conf 설정값
signo	전달할 시그널 번호	필수	-

리턴값

성공 시 "SUCCESS" 실패 시 에러 메시지



주의사항

1. 서버 프로세스의 이름은 ${TSR_HOME}/conf/db.conf에서 "DB_PROC_NAME" 변수를 통해 설정 가능함

get_pid()

DBMS 서버의 프로세스 ID(PID)를 조회하는 함수

문법

```
get_pid(name=None)
```

파라미터

파라미터	내용	구분	기본값
name	시그널을 전달할 서버 프로세스 이름 → 서버 프로세스 이름에 "."을 포함하여 일부가 일치하는 경우도 표현할 수 있음. 예) peta.*	선택	db.conf 설정값

리턴값

성공 시 성공 메시지가 결과 파일에 저장되고 PID 리스트가 리턴됨/ 실패 시, 빈 리스트 리턴됨

주의사항

1. 서버 프로세스의 이름은 ${TSR_HOME}/conf/db.conf에서 "DB_PROC_NAME" 변수를 통해 설정 가능함

예제

테스트케이스

```python
# -*- coding: utf-8 -*-
import signal

from dbtest.database import initdb
from dbtest.process import get_pid, kill, send_signal
from dbtest.utils import comment

if __name__ == "__main__":
    comment("process 제어 파키지 테스트")

    get_pid()

    get_pid("petasqld")

    send_signal(signo=signal.SIGHUP)

    kill()

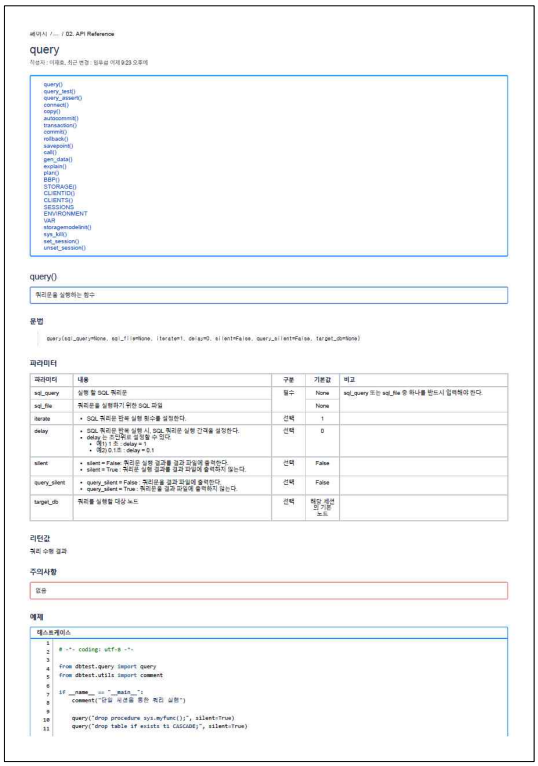
    get_pid()

    initdb()
```

테스트오라클

```
S0) # process 제어 패키지 테스트
S0) get_pid(name=None)
S0) Get pid of 2 processes
S0) get_pid(name-petasqld)
S0) Get pid of 1 processes
S0) signal(name-None, signo=1)
S0) SUCCESS
S0) kill(name-None)
S0) SUCCESS
S0) get_pid(name-None)
S0) Get pid of 0 processes
S0) server.initdb()
S0) Completed to initialize default database.
```

레이블 없음



베이시 / ... / 02. API Reference

query

작성자: 이재호, 최근 변경 : 임우섭 이재호23 오후에

query()
query_text()
query_assert()
connect()
copy()
autocommit()
transaction()
commit()
rollback()
savepoint()
call()
get_data()
explain()
ERP()
ERP()
STORAGE()
CLIENT(D)
CLIENT(D)
SESSIONS
ENVIRONMENT
VAR
storagemodelink()
sys_MS()
set_session()
unset_session()

query()

쿼리문을 실행하는 함수

문법

```
query(sql_query=None, sql_file=None, iterate=1, delay=0, silent=False, query_silent=False, target_dbnode)
```

파라미터

파라미터	내용	구분	기본값	비고
sql_query	실행 할 SQL 쿼리문	필수	None	sql_query 또는 sql_file 중 하나를 반드시 입력해야 한다.
sql_file	쿼리문을 실행하기 위한 SQL 파일		None	
iterate	• SQL 쿼리문 반복 실행 회수를 설정한다.	선택	1	
delay	• SQL 쿼리문 반복 실행 시 SQL 쿼리문 실행 간격을 설정한다. • delay 는 소수점까지 설정할 수 있다. - 예)1.5 초 : delay = 1 - 예)0.1 초 : delay = 0.1	선택	0	
silent	• silent = False: 쿼리문 실행 결과를 결과 파일에 출력한다. • silent = True : 쿼리문 실행 결과를 결과 파일에 출력하지 않는다.	선택	False	
query_silent	• query_silent = False : 쿼리문을 결과 파일에 출력한다. • query_silent = True : 쿼리문을 결과 파일에 출력하지 않는다.	선택	False	
target_db	쿼리를 실행할 대상 노드	선택	현재 세션의 "기본" 노드	

리턴값

쿼리 수행 결과

주의사항

없음

예제

테스트케이스

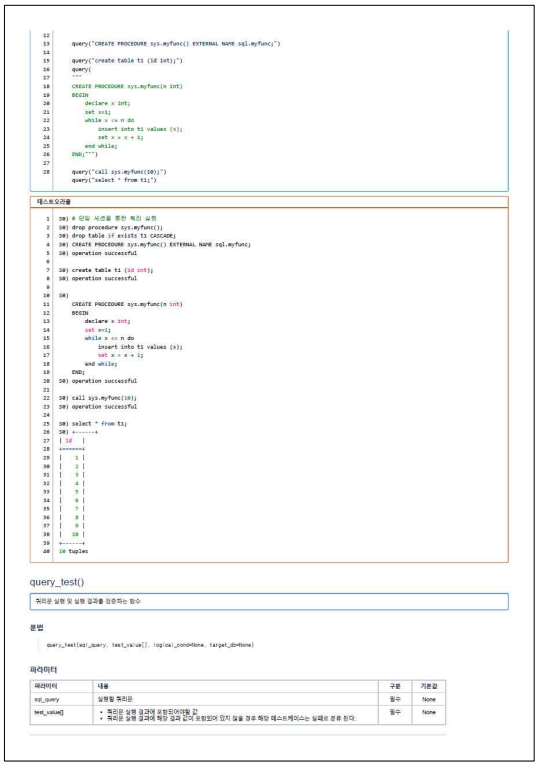
```python
# -*- coding: utf-8 -*-

from dbtest.query import query
from dbtest.utils import comment

if __name__ == "__main__":
    comment("압입 시간을 통한 쿼리 실행")

    query("drop procedure sys.myfunc();", silent=True)
    query("drop table if exists t1 CASCADE)", silent=True)
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



```
    query("CREATE PROCEDURE sys.myfunc() EXTERNAL NAME sql.myfunc;")

    query("create table t1 (id int);")
    query(
    """
    CREATE PROCEDURE sys.myfunc(n int)
    BEGIN
        declare x int;
        set x=1;
        while x <= n do
            insert into t1 values (x);
            set x = x + 1;
        end while;
    END;""")

    query("call sys.myfunc(10);")
    query("select * from t1;")
```

테스트오리클

```
50) # 단일 시스템 콘솔 확인 실행
50) drop procedure sys.myfunc();
50) drop table if exists t1 CASCADE;
50) CREATE PROCEDURE sys.myfunc() EXTERNAL NAME sql.myfunc;
50) operation successful

50) create table t1 (id int);
50) operation successful

50)
    CREATE PROCEDURE sys.myfunc(n int)
    BEGIN
        declare x int;
        set x=1;
        while x <= n do
            insert into t1 values (x);
            set x = x + 1;
        end while;
    END;
50) operation successful

50) call sys.myfunc(10);
50) operation successful

50) select * from t1;
50) +------+
| id   |
+======+
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
+------+
10 tuples
```

query_test()

쿼리문 실행 및 실행 결과를 검증하는 함수

문법

```
query_test(sql_query, test_value[], logical_cond=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
sql_query	실행할 쿼리문	필수	None
test_value[]	• 쿼리문 실행 결과에 포함되어야할 값 • 쿼리문 실행 결과에 해당 결과 값이 포함되어 있지 않을 경우 해당 테스트케이스는 실패로 분류 된다.	필수	None



파라미터	내용	구분	기본값
logical_cond	• logical_cond="AND" : test_value 리스트의 모든 값이 쿼리문 결과에 포함되어야 True를 리턴한다. • logical_cond="OR" : test_value 리스트 중 하나라도 쿼리문 결과에 포함되면 True를 리턴한다.	선택	"AND"
target_db	쿼리를 실행할 대상 노드	선택	해당 세션의 기본 노드

리턴값

True/False

주의사항

없음

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest.query import query
from dbtest.utils import comment

if __name__ == "__main__":
    comment("단일 시스템 콘솔 확인 실행")

    query("drop procedure sys.myfunc()", silent=True)
    query("drop table if exists t1 CASCADE", silent=True)

    query("CREATE PROCEDURE sys.myfunc() EXTERNAL NAME sql.myfunc;")

    query("create table t1 (id int);")
    query(
    """
    CREATE PROCEDURE sys.myfunc(n int)
    BEGIN
        declare x int;
        set x=1;
        while x <= n do
            insert into t1 values (x);
            set x = x + 1;
        end while;
    END;""")

    query("call sys.myfunc(10);")
    query("select * from t1;")
```

테스트오리클

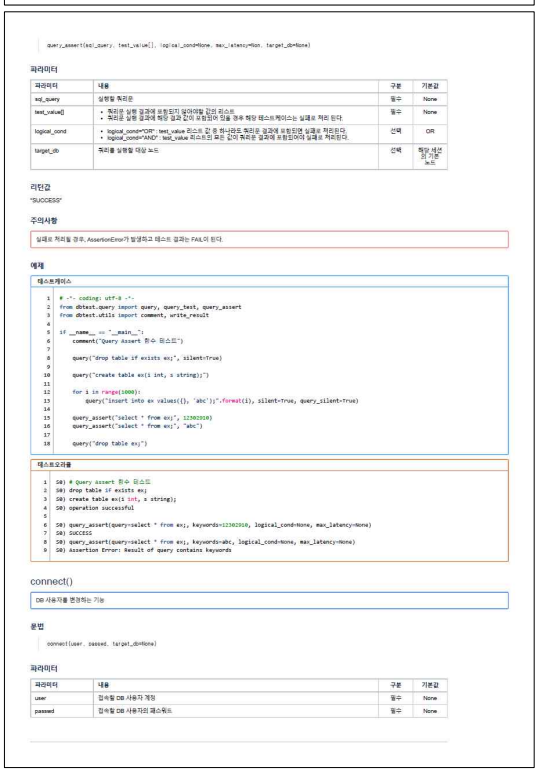
```
50) # Query test 함수 테스트
50) drop table if exists ex;
50) create table ex(i int, s string);
50) operation successful

50) query_test(query=select * from ex, keywords=abc, logical_cond=None)
50) SUCCESS
50) True
50) query_test(query=select * from ex, keywords=12302010, logical_cond=None)
50) SUCCESS
50) False
50) query_test(query=select count(*) from ex, keywords=1000, logical_cond=None)
50) SUCCESS
50) True
50) drop table ex;
50) operation successful
```

query_assert()

쿼리문 실행 및 실행 결과를 검증하는 함수
(query_test()와 달리 test_value가 쿼리 실행 결과에 있을 경우 테스트 케이스는 실패한다.)

문법



```
query_assert(sql_query, test_value[], logical_cond=None, max_latency=Non, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
sql_query	실행할 쿼리문	필수	None
test_value[]	• 쿼리문 실행 결과에 포함되지 않아야할 값의 리스트 • 쿼리문 실행 결과에 해당 결과 값이 포함되어 있을 경우 해당 테스트케이스는 실패로 처리 된다.	필수	None
logical_cond	• logical_cond="OR" : test_value 리스트 값 중 하나라도 쿼리문 결과에 포함되면 실패로 처리한다. • logical_cond="AND" : test_value 리스트의 모든 값이 쿼리문 결과에 포함되어야 실패로 처리한다.	선택	OR
target_db	쿼리를 실행할 대상 노드	선택	해당 세션의 기본 노드

리턴값

"SUCCESS"

주의사항

실패로 처리될 경우, AssertionError가 발생하고 테스트 결과는 FAIL이 된다.

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest.query import query, query_test, query_assert
from dbtest.utils import comment, write_result

if __name__ == "__main__":
    comment("Query Assert 함수 테스트")

    query("drop table if exists ex;", silent=True)

    query("create table ex(i int, s string);")

    for i in range(1000):
        query("insert into ex values({}, 'abc')".format(i), silent=True, query_silent=True)

    query_assert("select * from ex;", 12302010)
    query_assert("select * from ex;", "abc")

    query("drop table ex;")
```

테스트오리클

```
50) # Query assert 함수 테스트
50) drop table if exists ex;
50) create table ex(i int, s string);
50) operation successful

50) query_assert(query=select * from ex;, keywords=12302010, logical_cond=None, max_latency=None)
50) SUCCESS
50) query_assert(query=select * from ex;, keywords=abc, logical_cond=None, max_latency=None)
50) Assertion Error: Result of query contains keywords
```

connect()

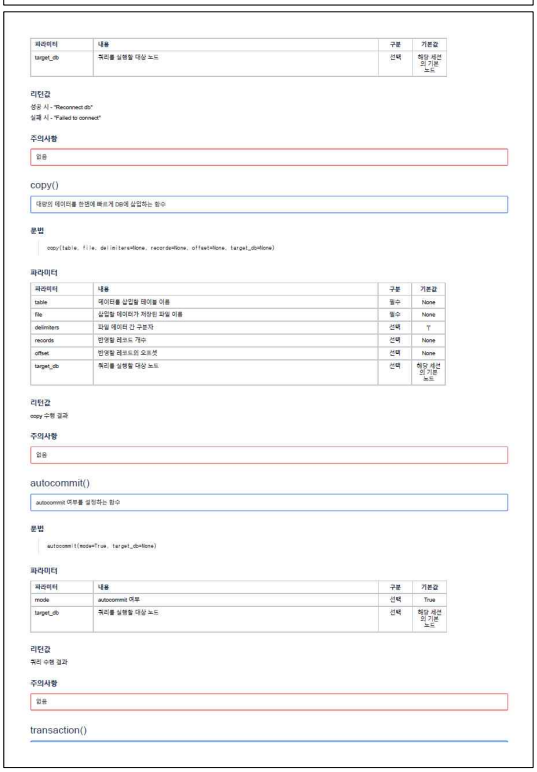
DB 사용자를 변경하는 기능

문법

```
connect(user, passwd, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
user	접속할 DB 사용자 계정	필수	None
passwd	접속할 DB 사용자의 패스워드	필수	None



파라미터	내용	구분	기본값
target_db	쿼리를 실행할 대상 노드	선택	해당 세션의 기본 노드

리턴값

성공 시 - "Reconnect db"
실패 시 - "Failed to connect"

주의사항

없음

copy()

대량의 데이터를 한번에 빠르게 DB에 삽입하는 함수

문법

```
copy(table, file, delimiters=None, records=None, offset=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
table	데이터를 삽입할 테이블 이름	필수	None
file	삽입할 데이터가 저장된 파일 이름	필수	None
delimiters	파일 데이터 간 구분자	선택	'\'
records	한번에 레코드 개수	선택	None
offset	한번에 레코드 오프셋	선택	None
target_db	쿼리를 실행할 대상 노드	선택	해당 세션의 기본 노드

리턴값

copy 수행 결과

주의사항

없음

autocommit()

autocommit 여부를 설정하는 함수

문법

```
autocommit(mode=True, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
mode	autocommit 여부	선택	True
target_db	쿼리를 실행할 대상 노드	선택	해당 세션의 기본 노드

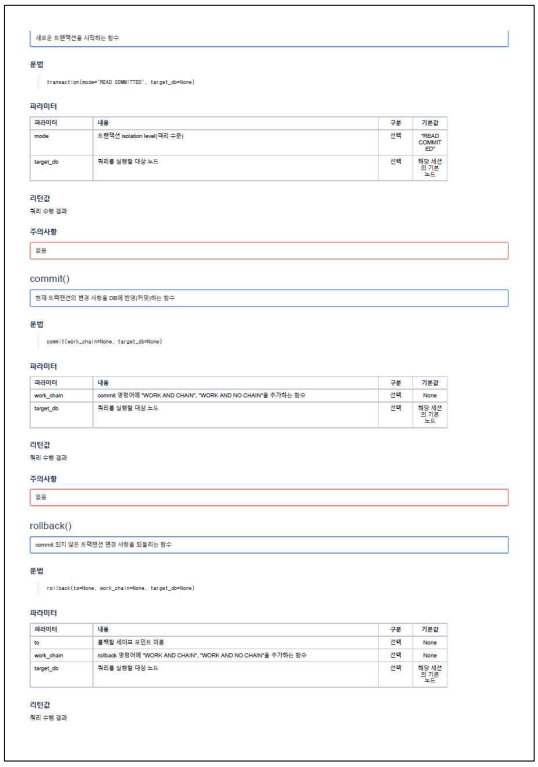
리턴값

쿼리 수행 결과

주의사항

없음

transaction()



새로운 트랜잭션을 시작하는 함수

문법

```
transaction(mode='READ COMMITTED', target_db=None)
```

파라미터

파라미터	내용	구분	기본값
mode	트랜잭션 isolation level(격리 수준)	선택	"READ COMMITTED"
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

쿼리 수행 결과

주의사항

없음

commit()

현재 트랜잭션의 변경 사항을 DBMS 반영(커밋)하는 함수

문법

```
commit(work_chain=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
work_chain	commit 명령어에 "WORK AND CHAIN", "WORK AND NO CHAIN"을 추가하는 함수	선택	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

쿼리 수행 결과

주의사항

없음

rollback()

commit 되기 앞선 트랜잭션 변경 사항을 되돌리는 함수

문법

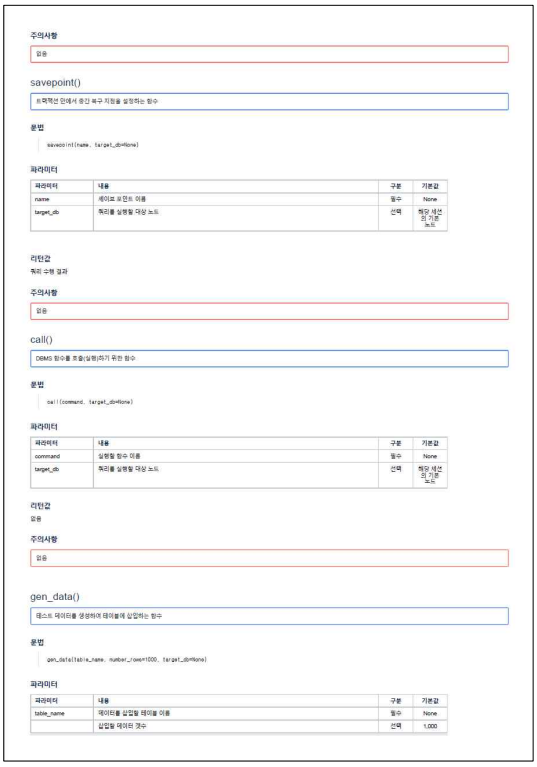
```
rollback(to=None, work_chain=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
to	롤백할 세이브 포인트 이름	선택	None
work_chain	rollback 명령어에 "WORK AND CHAIN", "WORK AND NO CHAIN"을 추가하는 함수	선택	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

쿼리 수행 결과



주의사항

없음

savepoint()

트랜잭션 안에서 중간 복구 지점을 설정하는 함수

문법

```
savepoint(name, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
name	세이브 포인트 이름	필수	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

쿼리 수행 결과

주의사항

없음

call()

DBMS 함수를 호출(실행)하기 위한 함수

문법

```
call(command, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
command	실행할 함수 이름	필수	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

없음

주의사항

없음

gen_data()

테스트 데이터를 생성하여 테이블에 삽입하는 함수

문법

```
gen_data(table_name, number_rows=1000, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
table_name	데이터를 삽입할 테이블 이름	필수	None
	삽입할 데이터 갯수	선택	1,000



파라미터	내용	구분	기본값
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

없음

주의사항

없음

예제

테스트 python

```
# -*- coding: utf-8 -*-
from dbtest.query import gen_data, query

if __name__ == "__main__":
    query("drop table if exists aaa;", silent=True)
    query("drop table if exists bbb;", silent=True)

    query(sql_query="create table aaa ( c1 VARCHAR(4000), c2 int, c3 numeric(10, 5), c4 boolean);")
    query(sql_query="create table bbb ( c1 CHAR(4000), c2 smallint, c3 float);")

    gen_data("aaa")
    gen_data("bbb")

    query(sql_query="select * from aaa limit 100;")
    query(sql_query="select * from bbb limit 100;")

    query("drop table aaa;")
    query("drop table bbb;")
```

테스트 결과를

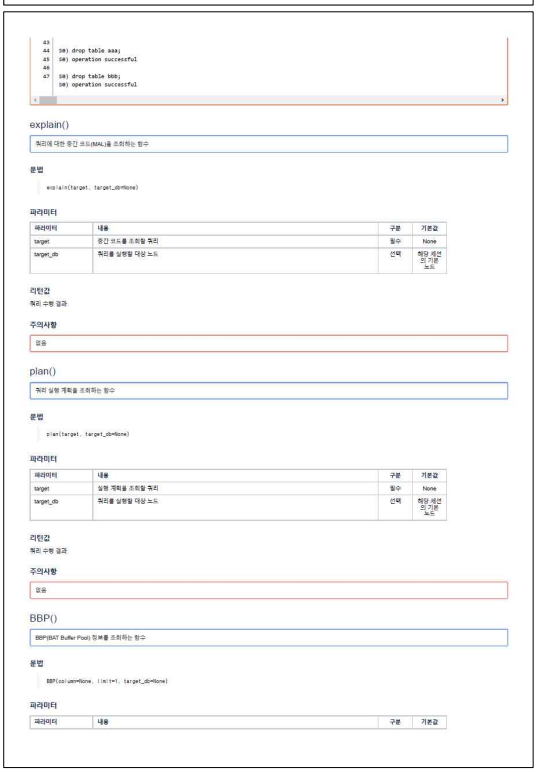
```
50) drop table if exists aaa;
50) drop table if exists bbb;
50) create table aaa ( c1 VARCHAR(4000), c2 int, c3 numeric(10, 5), c4 boolean);
50) operation successful

50) create table bbb ( c1 CHAR(4000), c2 smallint, c3 float);
50) operation successful

50) D-TAP complete to insert 1000 rows to aaa table
50) D-TAP complete to insert 1000 rows to bbb table
50) select * from aaa limit 100;
50) +----------
| c1
+==========
| U97d993hQgB5A01lfk6D447+e07L4pr85G4C7jgB1489aYQ1108uLrF9nB0K1ZtXPx81A0397haI88TjE39tP5HAD1161u07fND2Z2KhOE461348Pa31/6iI9j/Ca1974p315bvy
| 0c188H2611P4b6E8a33e1WEgKrbfb6Y7RC35Dp3neeeEn3J6dfaF3uJ12ynkJF9n38c52JdI6vrfC53h4Bl1rbf8iPBgrbicuB0Bk2zE84Q49h
| D973k239QadVv9IBbZYNL9Y7HY+H3QL/QD29I5AQZVivf5k54A8543bDOHK2H7sA77Sm0o6H4aRlTb6A4i47mP3Ua14I4e75luW07dCgR7O74tz29a7l7Vv96AhBeZ1X
| 3kx4v4B4BF9Hjuv4B3354FHPl7B9I8RvZ6BuN4X3723fBlH89HgP1tEN3590fBlaB8a5fPmm69mOI8FSndfkx17BH8JrZH76GA0lD8uZaAvQl2I5DsO2fGe6fBvH9MZH3G
+----------
100 tuples

50) select * from bbb limit 100;
50) +----------
| c1
+==========
| G3m07B9Q3S4Q3BUi4v2ZfuSiQfI13jWU40igP1uu8l55fy35NM93vJNiRN6B0q5mC6y7eD3D26uB9T8eE3oMu3wx3Uu5TF2Ou659Q1O3EcnZJFv9l04FBEOF2NFQ3q7457918N
| U8B639bU3N617uN6yQc2I4OaZeMJeW7BE3n8jF7KTU2eI98AI1iET4OKQNmuB8BKyuWl3i1tHN4M7vBz7AyqmbsvEd4I2AUx3D7NOM7Ui8cIfvSiv3mUi4Fq5fU1F7a
| 3Q615356597Z0X34c5W5dTybiyJup6MWkK6eS1hP69UZ1I88gzE576e4vI2J7qBi4fNJC038Bf2M2M5Z3qd67W7S9vVv447Z0Y5IApCJezMi82y20adk2sak7a3bW0M9
| 6yO3oGa6SfmK3oI2f937Fy9883CWrp3ecH2tD85IJB9W0lf2A8Ua6s777bB8ID1dggergSa66J1By3UQv9oDAaZy2i5N6g6bWfiFdxpm387Tgw767rEiF9H0Jp42Jd1RdPFvi1B
| 9RuHtf6452JW2W6i5y93bT7cHIjH3IBE2bR52I5189TSI6k0595vrQjQF3S0Hc2SR3497bfuWB5hTU7y5kBHRlJNi1BN2RjBZT5uIaBnBxbTTE66GHRmdf3U7dETgRfF36RBlmwcf
| 3Ke4v4B4BF9Hjuv4B3354FHPl7B9I8RvZ6BuN4X3723fBlH89HgP1tEN3590fBlaB8a5fPmm69mOI8FSndfkx17BH8JrZH76GA0lD8uZaAvQl2I5DsO2fGe6fBvH9MZH3G
+----------
100 tuples

| c1
+==========
| 6Bc3e2534Bh4i1bK51LRrB6097wM3zMSIyJUSQC58y7r1eN6iPV126aUl7eBbEl2a6J5l1727tf5d55j437C5B5N09Fe4wB09yu93546hfkJ2qKLRAbN9o2eiPFggJ7Sl
| 9T1RnKBwSl0eiJ4D947WB3N8DjDQ95JN2DB24Jq7ijRaP7S7P0NMTJaqN5BwH9H2l5N6R3hM9Aa4crffEgPUaLqRf7SLvPrJH3hOT32J2BfFrQPx6NvTqCiSJ
| 8Z16kG6N9r5T2UE7sJ8B8Gk1E0NkEQBT2MB5lRTBBXFC7BS9BmEk7RQ5Efj8vlBuQ6fLaVvOu85S2B43uT4WHrE9lcM7hJ25duT5Ue4Ga4557AwBQ7B6uS1JuZ
| W8T6U7IR4YvA2S0Z9dP7eZIB9OuVkD+4VZ0c2DKG89Tw92QB524RL4NPnDP7BK6TBaI0qJ9a7599T12RvyMxI8iZB4AeaNBCUm59aAIIVHgwB8IBDG4Mlk9sIp3YBC
| 7BB1555OMCh04Q15677y9eZz9511586gKDYRLuOBm3KhSaMv6317452ge9f8P3uZt72dH73OB5ou1477ZB13w8BBB0dWq59481kFiB67h4320HC7R34k6SPvlP19vlfHVQTO
| I74EDkRQ8Z9QdC7kARhMsMlK7O4ISN9DAIMCnPlkN78iQ1eL1ZbW4EiI9H5N8W0W7y57hR5OB11WiK9b43gTDN59v92N2RyYVjDn82N6u4WTeW183Ju347F56B214vAuN
| bKEDyKr447kM7W7bZ7fL3eZb4WrwE4SKaWDBflaDbYB0TDA6bEz9Y16SFt6z7Z7gTeBQKpN0S15Ca7kBCoLpyzfTBcFVB2rqFidCJ6VoTzqGlIfuSJ5I8W7aIrPWHxN
+----------
100 tuples
```



```
43
44  50) drop table aaa;
45  50) operation successful
46
47  50) drop table bbb;
    50) operation successful
```

explain()

쿼리에 대한 중간 코드(MAL)를 조회하는 함수

문법

```
explain(target, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
target	중간 코드를 조회할 쿼리	필수	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

쿼리 수행 결과

주의사항

없음

plan()

쿼리 실행 계획을 조회하는 함수

문법

```
plan(target, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
target	실행 계획을 조회할 쿼리	필수	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값

쿼리 수행 결과

주의사항

없음

BBP()

BBP(BAT Buffer Pool) 정보를 조회하는 함수

문법

```
BBP(column=None, like=*, target_db=None)
```

파라미터

파라미터	내용	구분	기본값

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

파라미터	내용	구분	기본값
column	조회할 정보 • id, name, htype, type, count, where 등의 조회 가능	선택	*
limit	최대 몇 개의 row를 조회할 것인지	선택	1
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
쿼리 수행 결과

주의사항
없음

STORAGE()

각 컬럼의 물리적인 저장정보를 조회하는 함수

문법

```
STORAGE(get_list=None, schema=None, table=None, column=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
get_list	조회할 저장 정보 종류 • schema, table, column, type 등을 입력 가능	선택	*
schema	컬럼 정보를 조회할 스키마 이름. 해당 스키마에 속한 컬럼의 정보만 조회됨	선택	None
table	컬럼 정보를 조회할 테이블 이름. 해당 테이블에 속한 컬럼의 정보만 조회됨	선택	None
column	정보를 조회할 컬럼 이름	선택	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
쿼리 수행 결과

주의사항
없음

CLIENTID()

현재 세션의 ID를 조회하는 함수

문법

```
CLIENTID(target_db=None)
```

파라미터

파라미터	내용	구분	기본값
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
없음

주의사항
없음

파라미터	내용	구분	기본값
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
쿼리 수행 결과

주의사항
없음

VAR

현재 세션의 변수 목록을 조회하는 함수

문법

```
VAR(target_db=None)
```

파라미터

파라미터	내용	구분	기본값
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
쿼리 수행 결과

주의사항
없음

storagemodelinit()

storagemodelinput 테이블에 각 테이블의 컬럼별 상세정보와 외부참조여부를 새로 저장하는 함수

문법

```
storagemodelinit(target_db=None)
```

파라미터

파라미터	내용	구분	기본값
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
없음

주의사항
없음

sys_kill()

현재 실행중인 세션을 강제로 종료시키는 함수

문법

```
sys_kill(id, target_db=None)
```

파라미터

CLIENTS()

현재 PetaSQL server에 접속해 있는 client 정보를 출력하는 함수

문법

```
CLIENTS(column=None, where=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
column	조회할 정보 • id, user, ip, app 등 SYS.CLIENTS()의 각 컬럼의 입력 가능함	선택	*
where	조회 조건	선택	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
쿼리 수행 결과

주의사항
없음

SESSIONS

현재 PetaSQL server에 접속해 있는 세션 정보를 출력하는 함수

문법

```
SESSIONS(column=None, where=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
column	조회할 정보 • user, login, sessiontimeout 등 SYS.SESSIONS()의 각 컬럼이 입력 가능함	선택	*
where	조회 조건	선택	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
쿼리 수행 결과

주의사항
없음

ENVIRONMENT

PetaSQL의 환경변수 정보를 출력하는 함수

문법

```
ENVIRONMENT(column=None, where=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
column	조회할 정보 • name, value 입력 가능함	선택	*
where	조회 조건	선택	None

파라미터	내용	구분	기본값
id	강제 종료한 세션의 ID	필수	None
target_db	쿼리를 실행할 대상 노드	선택	현장 세션의 기본 노드

리턴값
쿼리 수행 결과

주의사항
id의 값은 SESSIONS() 함수를 통해서 얻을 수 있다.

set_session()

현재 세션이 연결된 DBMS를 다른 노드의 DBMS로 변경하는 함수

문법

```
set_session(new_db)
```

파라미터

파라미터	내용	구분	기본값
new_db	새로 연결할 DBMS가 설치된 노드 이름	필수	None

리턴값
없음

주의사항
new_db 값은 $(TSR_HOME)/conf/db.conf 를 참조하여 입력해야 한다.

unset_session()

set_session()의 변경 사항을 되돌리는 함수

문법

```
unset_session()
```

파라미터

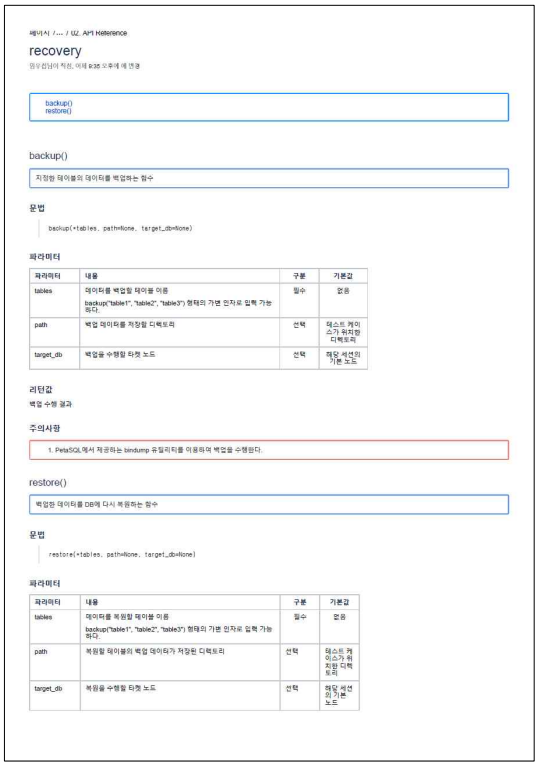
파라미터	내용	구분	기본값
없음			

리턴값
없음

주의사항
없음

데이블 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



레버시 /... / UZ. API Reference

recovery

임무컴님이 작성, 마지막 오후에 변경

backup()
restore()

backup()

지정한 테이블의 데이터를 백업하는 함수

문법

```
backup(*tables, path=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
tables	데이터를 백업할 테이블 이름 backup("table1", "table2", "table3") 형태의 가변 인자로 입력 가능하다.	필수	없음
path	백업 데이터를 저장할 디렉토리	선택	테스트 케이스가 위치한 디렉토리
target_db	백업을 수행할 타겟 노드	선택	해당 세션의 기본 노드

리턴값

백업 수행 결과

주의사항

1. PetaSQL에서 제공하는 bindump 유틸리티를 이용하여 백업을 수행한다.

restore()

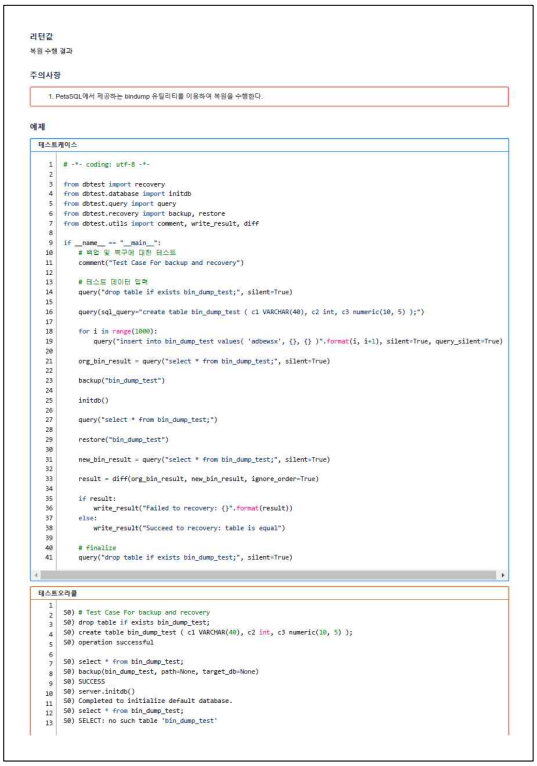
백업한 데이터를 DB에 다시 복원하는 함수

문법

```
restore(*tables, path=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
tables	데이터를 복원할 테이블 이름 backup("table1", "table2", "table3") 형태의 가변 인자로 입력 가능하다.	필수	없음
path	복원할 데이블의 백업 데이터가 저장된 디렉토리	선택	테스트 케이스가 위치한 디렉토리
target_db	복원을 수행할 타겟 노드	선택	해당 세션의 기본 노드



리턴값

복원 수행 결과

주의사항

1. PetaSQL에서 제공하는 bindump 유틸리티를 이용하여 복원을 수행한다.

예제

테스트케이스

```
# -*- coding: utf-8 -*-

from dbtest import recovery
from dbtest.database import initdb
from dbtest.query import query
from dbtest.recovery import backup, restore
from dbtest.utils import comment, write_result, diff

if __name__ == "__main__":
    # 백업 및 복구에 대한 테스트
    comment("Test Case For backup and recovery")

    # 테스트 데이터 입력
    query("drop table if exists bin_dump_test;", silent=True)

    query(sql_query="create table bin_dump_test ( c1 VARCHAR(40), c2 int, c3 numeric(10, 5) );")

    for i in range(1000):
        query("insert into bin_dump_test values( 'adbewsx', {}, {} )".format(i, i+1), silent=True, query_silent=True)

    org_bin_result = query("select * from bin_dump_test;", silent=True)

    backup("bin_dump_test")

    initdb()

    query("select * from bin_dump_test;")

    restore("bin_dump_test")

    new_bin_result = query("select * from bin_dump_test;", silent=True)

    result = diff(org_bin_result, new_bin_result, ignore_order=True)

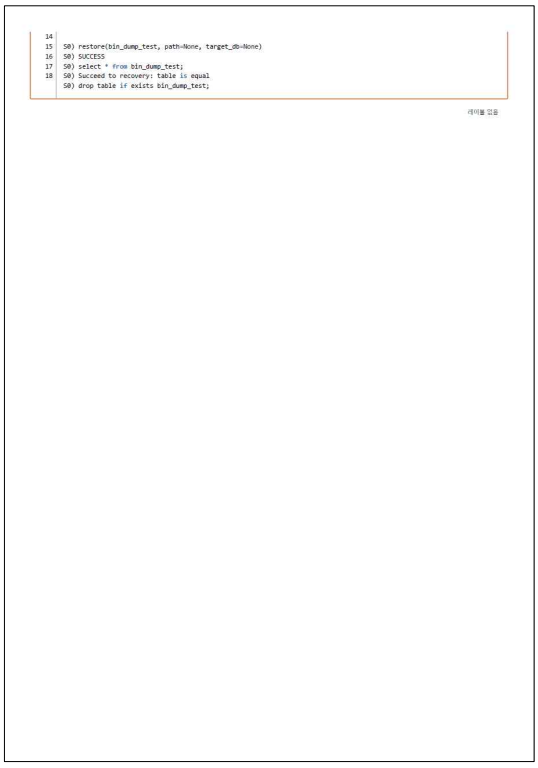
    if result:
        write_result("Failed to recovery: {}".format(result))
    else:
        write_result("Succeed to recovery: table is equal")

    # finalize
    query("drop table if exists bin_dump_test;", silent=True)
```

테스트오라클

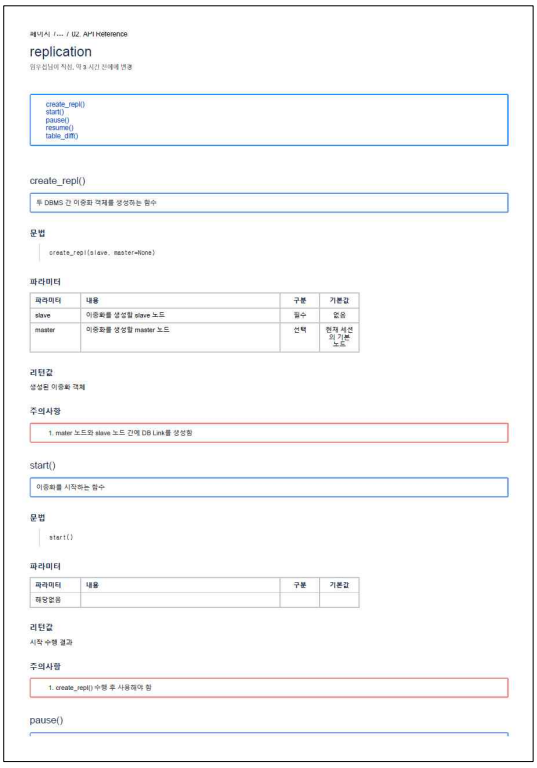
```
S0) # Test Case For backup and recovery
S0) drop table if exists bin_dump_test;
S0) create table bin_dump_test ( c1 VARCHAR(40), c2 int, c3 numeric(10, 5) );
S0) operation successful

S0) select * from bin_dump_test;
S0) backup(bin_dump_test, path=None, target_db=None)
S0) SUCCESS
S0) server.initdb()
S0) Completed to initialize default database.
S0) select * from bin_dump_test;
S0) SELECT: no such table 'bin_dump_test'
```



```
S0) restore(bin_dump_test, path=None, target_db=None)
S0) SUCCESS
S0) select * from bin_dump_test;
S0) Succeed to recovery: table is equal
S0) drop table if exists bin_dump_test;
```

레이블 없음



레버시 /... / UZ. API Reference

replication

임무컴님이 작성, 마지막 오후에 변경

create_repl()
start()
pause()
resume()
table_diff()

create_repl()

두 DBMS 간 이중화 객체를 생성하는 함수

문법

```
create_repl(slave, master=None)
```

파라미터

파라미터	내용	구분	기본값
slave	이중화를 생성할 slave 노드	필수	없음
master	이중화를 생성할 master 노드	선택	현재 세션의 기본 노드

리턴값

생성된 이중화 객체

주의사항

1. mater 노드와 slave 노드 간에 DB Link를 생성함

start()

이중화를 시작하는 함수

문법

```
start()
```

파라미터

파라미터	내용	구분	기본값
해당없음			

리턴값

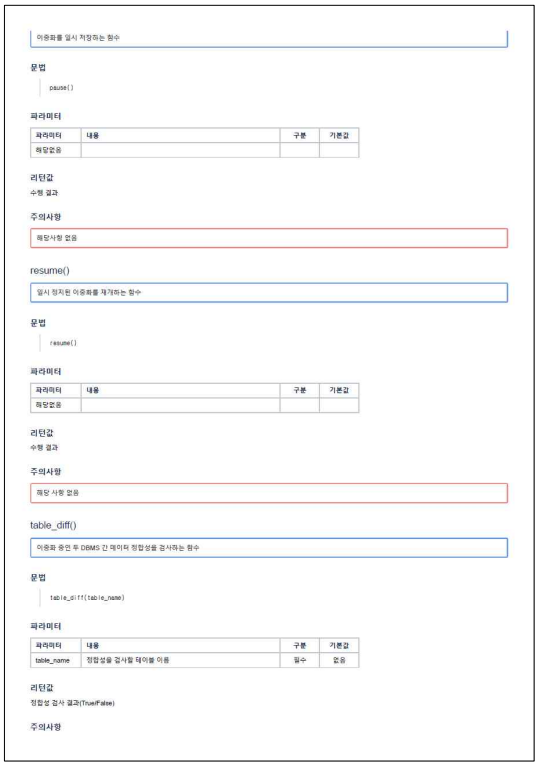
시작 수행 결과

주의사항

1. create_repl() 수행 후 사용해야 함

pause()

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



이중화를 일시 저장하는 함수

문법

```
pause()
```

파라미터

파라미터	내용	구분	기본값
해당없음			

리턴값

수행 결과

주의사항

해당사항 없음

resume()

일시 정지된 이중화를 재개하는 함수

문법

```
resume()
```

파라미터

파라미터	내용	구분	기본값
해당없음			

리턴값

수행 결과

주의사항

해당 사항 없음

table_diff()

이중화 쌍인 두 DBMS 간 데이터 정합성을 검사하는 함수

문법

```
table_diff(table_name)
```

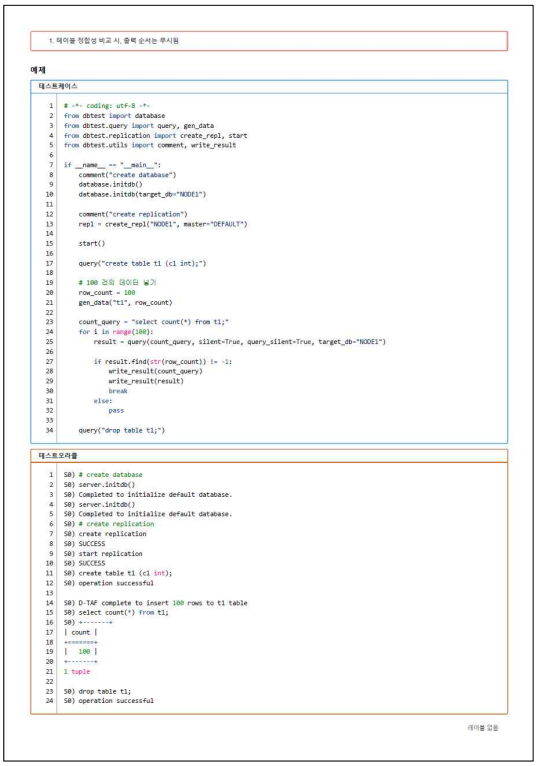
파라미터

파라미터	내용	구분	기본값
table_name	정합성을 검사할 테이블 이름	필수	없음

리턴값

정합성 검사 결과(True/False)

주의사항



1. 테이블 정합성 비교 시, 출력 순서는 무시됨

예제

테스트케이스

```python
# -*- coding: utf-8 -*-
from dbtest import database
from dbtest.query import query, gen_data
from dbtest.replication import create_repl, start
from dbtest.utils import comment, write_result

if __name__ == "__main__":
    comment("create database")
    database.initdb()
    database.initdb(target_db="NODE1")

    comment("create replication")
    repl = create_repl("NODE1", master="DEFAULT")

    start()

    query("create table t1 (c1 int);")

    # 100 건의 데이터 넣기
    row_count = 100
    gen_data("t1", row_count)

    count_query = "select count(*) from t1;"
    for i in range(100):
        result = query(count_query, silent=True, query_silent=True, target_db="NODE1")

        if result.find(str(row_count)) != -1:
            write_result(count_query)
            write_result(result)
            break
        else:
            pass

    query("drop table t1;")
```

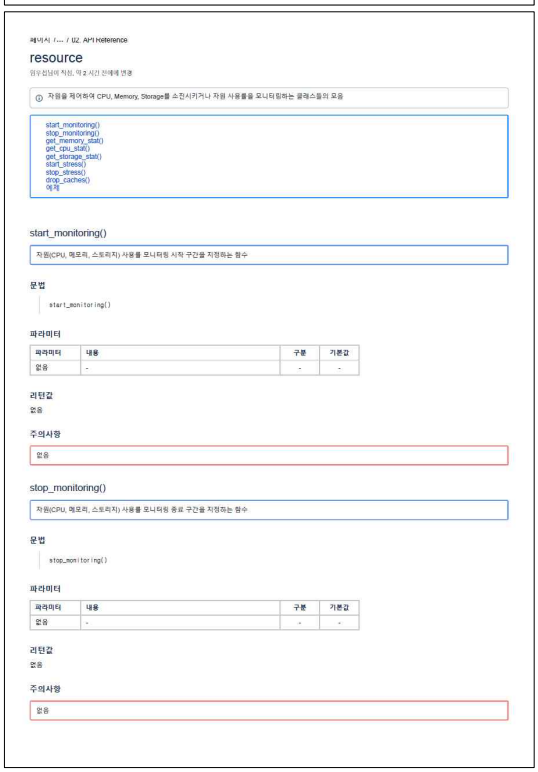
테스트오라클

```
S0) # create database
S0) server.initdb()
S0) Completed to initialize default database.
S0) server.initdb()
S0) Completed to initialize default database.
S0) # create replication
S0) create replication
S0) SUCCESS
S0) start replication
S0) SUCCESS
S0) create table t1 (c1 int);
S0) operation successful

S0) D-TAF complete to insert 100 rows to t1 table
S0) select count(*) from t1;
S0) +-------+
| count |
+=======+
|   100 |
+-------+
1 tuple

S0) drop table t1;
S0) operation successful
```

데이블 없음



페이지 시 / ... / 02. API Reference

resource

임우섭님이 작성, 약 2 시간 전에에 변경

ⓘ 자원을 제어하여 CPU, Memory, Storage를 소진시키거나 자원 사용률을 모니터링하는 클래스들의 모음

start_monitoring()
stop_monitoring()
get_memory_stat()
get_cpu_stat()
get_storage_stat()
start_stress()
stop_stress()
drop_caches()
예제

start_monitoring()

자원(CPU, 메모리, 스토리지) 사용률 모니터링 시작 구간을 지정하는 함수

문법

```
start_monitoring()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

없음

주의사항

없음

stop_monitoring()

자원(CPU, 메모리, 스토리지) 사용률 모니터링 종료 구간을 지정하는 함수

문법

```
stop_monitoring()
```

파라미터

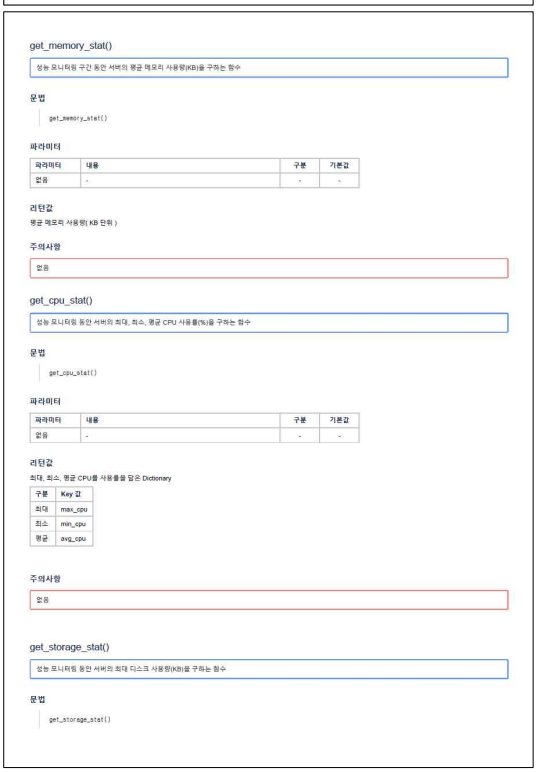
파라미터	내용	구분	기본값
없음	-	-	-

리턴값

없음

주의사항

없음



get_memory_stat()

성능 모니터링 구간 동안 서버의 평균 메모리 사용량(KB)을 구하는 함수

문법

```
get_memory_stat()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

평균 메모리 사용량(KB 단위)

주의사항

없음

get_cpu_stat()

성능 모니터링 동안 서버의 최대, 최소, 평균 CPU 사용률(%)을 구하는 함수

문법

```
get_cpu_stat()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

최대, 최소, 평균 CPU를 사용률을 담은 Dictionary

구분	Key 값
최대	max_cpu
최소	min_cpu
평균	avg_cpu

주의사항

없음

get_storage_stat()

성능 모니터링 동안 서버의 최대 디스크 사용량(Kb)을 구하는 함수

문법

```
get_storage_stat()
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

최대 디스크 사용량(KB)

주의사항

없음

start_stress()

자원(CPU, 메모리, 스토리지)이 소진된 상황을 재현하는 함수

문법

```
start_stress(target_db=None, cpu=None, memory=None, storage=None)
```

파라미터

파라미터	내용	구분	기본값
target_db	자원 소진 상황을 재현할 타겟 노드	선택	해당 세션의 기본 노드
cpu	CPU 자원 소진 목표치 (만약 "0" 이 입력되었다면, CPU 사용률이 0이 될때까지 소진시킴)	선택	없음
memory	메모리 자원 소진 목표치 (G, GB, M, MB, K, KB 사용 가능) (만약 "1024MB"가 입력되었다면 1,024MB가 남을 때까지 자원을 소진 시킴)	선택	없음
storage	스토리지 자원 소진 목표치 (G, GB, M, MB, K, KB 사용 가능) (만약 "1024MB"가 입력되었다면 1,024MB가 남을 때까지 자원을 소진 시킴)	선택	없음

리턴값

없음

주의사항

1. 자원 소진 목표치와 실제 소진된 양은 다소 차이가 있을 수 있음
2. 테스트 종료 시 stop_stress를 꼭 수행해야함

stop_stress()

자원 소진을 중단하고 원래 상태로 복귀함

문법

```
stop_stress()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

없음

주의사항

1. 테스트 종료 시 stop_stress를 꼭 수행해야함

drop_caches()

OS의 cache를 비우는 함수

문법

```
drop_caches()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

리턴값

없음

주의사항

1. 리눅스에서만 사용 가능한 함수임
2. "sync && echo 3 > /proc/sys/vm/drop_caches" 를 수행하는 것과 같음

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from time import sleep

from dbtest.resource import *
from dbtest.utils import write_result

if __name__ == "__main__":
    # 부하 발생 모니터링
    start_monitoring()

    sleep(5)

    cpu_usage_before = get_cpu_stat()

    start_stress(cpu='0')

    sleep(60)

    stop_stress()

    cpu_usage_after = get_cpu_stat()

    stop_monitoring()

    # max 사용량이 100이 되었다면 PASS
    if cpu_usage_after['max_cpu'] == 100 and cpu_usage_after['avg_cpu'] > cpu_usage_before['avg_cpu']:
        write_result("SUCCESS")
    else:
        write_result("FAILURE")
```

테스트로그

```
50) start_monitoring()
50) SUCCESS
50) get_cpu_stat()
50) start_stress()
50) SUCCESS
50) stop_stress()
50) SUCCESS
50) get_cpu_stat()
50) stop_monitoring()
50) SUCCESS
50) SUCCESS
```

레이블 없음

메서시 /... / UZ. API Reference

session

임무 입님에 지친, 약 2 시간 전에에 변경

Session() - 생성자
execute()
wait()
예제

Session() - 생성자

테스트 DB와 연결된 세션을 생성하는 함수

문법

```
Session(target_db=None)
```

파라미터

파라미터	내용	구분	기본값
target_db	연결하고자 하는 테스트 DB	선택	해당 세션의 연결기본 노드

리턴값

없음

주의사항

테스트프레임워크 환경 설정 파일($TAF_HOME/conf/db.conf)에 name에 해당하는 DB 서버의 설정이 정의되어 있어야 함다.(참고 링크)

execute()

동기, 비동기 세션을 통해 테스트 함수를 수행하는 함수

문법

```
.ssor/on.execute(target_func=None, func_args=None, sql_query='', background=True, session_cnt=1)
```

파라미터

파라미터	내용	구분	기본값
target_func	각 세션이 수행할 쿼리문, 동기화문, 제어문 등으로 구성한 테스트 함수	필수	없음
sql_query	각 세션이 수행할 쿼리문		없음
func_args	target_func에 넘길 매개변수 리스트	선택	매개변수 없음
background	동기 실행, 비동기 실행 여부 설정 • True : 비동기 실행, 세션이 종료를 기다리지 않음 • False : 동기 실행, 세션이 종료될 때까지 테스트 케이스 실행이 중지됨	선택	비동기(True)
session_cnt	실행할 세션의 개수. 동시에 같은 일을 하는 여러 개의 세션 생성 가능	선택	1

리턴값

없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

주의사항

비동기 세션의 경우, wait() 함수를 통해 수행 종료를 대기해야 한다.

wait()

비동기 세션이 종료될 때까지 대기하는 함수

문법

```
session.wait()
```

파라미터

파라미터	내용	구분	기본값
없음	-	-	-

주의사항

없음

예제

테스트케이스

```python
# -*- coding: utf-8 -*-

from time import sleep

from dbtest.event import sync_barrier
from dbtest.query import query
from dbtest.session import Session


def client1():
    query("drop table if exists t1 CASCADE;")

    query("create table t1 (id int);")

    query("drop table t1;")


def client2():
    query("create table t1 (id int);")
    sleep(1)

    sync_barrier("S1", 2)

    # query("drop table t1;")


def client3():
    sync_barrier("S1", 2)

    for i in range(10):
        query("insert into t1 values(" + str(i) + ");")


if __name__ == "__main__":
    s = Session("DEFAULT")

    s.execute(target_func=client1)

    s.wait()
    print("wait 1 thread")
```

```python
    s.execute(target_func=client1, session_cnt=10, background=False)

    s = Session()

    s.execute(target_func=client1)

    s.wait()

    # 비동기 execute가 여러번 수행될 경
    s = Session("DEFAULT")
    s.execute(target_func=client1)
    s.execute(target_func=client1, session_cnt=3)
    s.execute(target_func=client1)

    s.wait()

    s = Session()
    s.execute(sql_query="create table t1 (id int);")

    s.wait()

    s = Session("DEFAULT")
    s.execute(sql_query="create table Repl (id int);")
    s.execute(sql_query="insert into Repl values(1)")
    s.execute(sql_query="select * from t1;", session_cnt=3)
    s.execute(sql_query="select * from Repl;")
    s.execute(sql_query="select * from t3;", session_cnt=2)

    s.wait()

    s = Session()
    s.execute(target_func=client2)
    s.execute(target_func=client3)

    s.wait()
```

테스트오라클

```
S1) drop table if exists t1 CASCADE;
S1) operation successful

S1) create table t1 (id int);
S1) operation successful

S1) drop table t1;
S1) operation successful

S2) drop table if exists t1 CASCADE;
S2) operation successful

S2) create table t1 (id int);
S2) operation successful

S2) drop table t1;
S2) DROP TABLE: no such table 't1'

S3) drop table if exists t1 CASCADE;
S3) operation successful

S3) create table t1 (id int);
S3) operation successful

S3) drop table t1;
S3) DROP TABLE: no such table 't1'

S4) drop table if exists t1 CASCADE;
S4) operation successful

S4) create table t1 (id int);
S4) operation successful
```

```
S4) drop table t1;
S4) operation successful

S5) drop table if exists t1 CASCADE;
S5) operation successful

S5) create table t1 (id int);
S5) operation successful

S5) drop table t1;
S5) operation successful

S6) drop table if exists t1 CASCADE;
S6) operation successful

S6) create table t1 (id int);
S6) operation successful

S6) drop table t1;
S6) operation successful

S7) drop table if exists t1 CASCADE;
S7) operation successful

S7) create table t1 (id int);
S7) operation successful

S7) drop table t1;
S7) operation successful

S8) drop table if exists t1 CASCADE;
S8) operation successful

S8) create table t1 (id int);
S8) operation successful

S8) drop table t1;
S8) operation successful

S9) drop table if exists t1 CASCADE;
S9) operation successful

S9) create table t1 (id int);
S9) operation successful

S9) drop table t1;
S9) operation successful

S10) drop table if exists t1 CASCADE;
S10) operation successful

S10) create table t1 (id int);
S10) CREATE TABLE: name 't1' already in use

S10) drop table t1;
S10) operation successful

S11) drop table if exists t1 CASCADE;
S11) operation successful

S11) create table t1 (id int);
S11) operation successful

S11) drop table t1;
S11) operation successful

S12) drop table if exists t1 CASCADE;
S12) operation successful

S12) create table t1 (id int);
S12) operation successful
```

```

S12) drop table t1;
S12) operation successful

S13) drop table if exists t1 CASCADE;
S13) operation successful

S13) create table t1 (id int);
S13) operation successful

S13) drop table t1;
S13) operation successful

S14) drop table if exists t1 CASCADE;
S14) operation successful

S14) create table t1 (id int);
S14) operation successful

S14) drop table t1;
S14) operation successful

S15) drop table if exists t1 CASCADE;
S15) operation successful

S15) create table t1 (id int);
S15) operation successful

S15) drop table t1;
S15) operation successful

S16) drop table if exists t1 CASCADE;
S16) operation successful

S16) create table t1 (id int);
S16) operation successful

S16) drop table t1;
S16) operation successful

S17) drop table if exists t1 CASCADE;
S17) operation successful

S17) create table t1 (id int);
S17) operation successful

S17) drop table t1;
S17) operation successful

S18) create table t1 (id int);
S18) operation successful

S19) create table Repl (id int);
S19) operation successful

S20) insert into Repl values(1)
S20) 1 affected row

S21) select * from t1;
S21) +----+
| id |
+====+
+----+
0 tuples

S22) select * from t1;
S22) +----+
| id |
+====+
+----+
0 tuples
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



```
523) select * from t1;
523) +-----+
| id |
+=====+
+-----+
0 tuples

524) select * from Repl;
524) +-----+
| id |
+=====+
+-----+
0 tuples

525) select * from t3;
525) SELECT: no such table 't3'

526) select * from t3;
526) SELECT: no such table 't3'

527) create table t1 (id int);
527) CREATE TABLE: name 't1' already in use

528) insert into t1 values(0);
528) 1 affected row

528) insert into t1 values(1);
528) 1 affected row

528) insert into t1 values(2);
528) 1 affected row

528) insert into t1 values(3);
528) 1 affected row

528) insert into t1 values(4);
528) 1 affected row

528) insert into t1 values(5);
528) 1 affected row

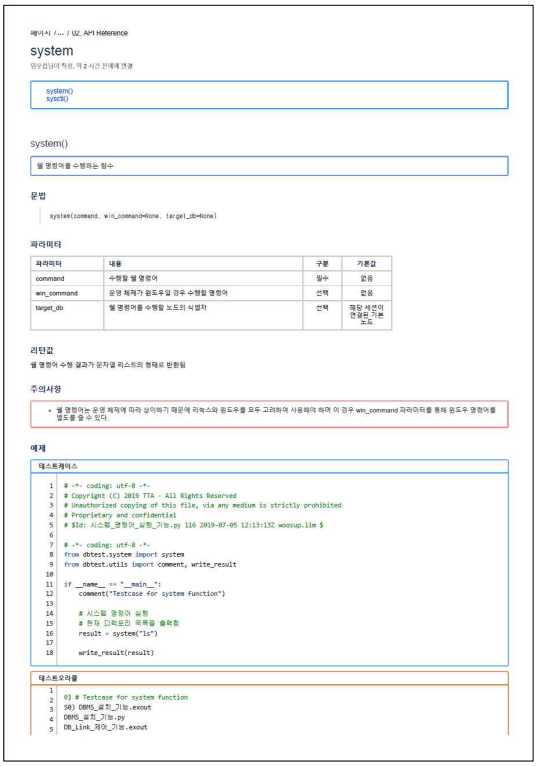
528) insert into t1 values(6);
528) 1 affected row

528) insert into t1 values(7);
528) 1 affected row

528) insert into t1 values(8);
528) 1 affected row

528) insert into t1 values(9);
528) 1 affected row
```

레이블 없음



메뉴 / ... / 02. API Reference

system

명령셸(shell)에서 실행

```
system()
sysctl()
```

system()

웹 명령어를 수행하는 함수

문법

```
system(command, win_command=None, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
command	수행할 웹 명령어	필수	없음
win_command	운영 체제가 윈도우일 경우 수행할 명령어	선택	없음
target_db	웹 명령어를 수행할 노드의 식별자	선택	해당 세션이 연결된 기본 노드

리턴값

웹 명령어 수행 결과가 문자열 리스트의 형태로 반환됨

주의사항

- 웹 명령어는 운영 체제에 따라 상이하기 때문에 리눅스와 윈도우를 모두 고려하여 사용해야 하며 이 경우 win_command 파라미터를 통해 윈도우 명령어를 별도로 줄 수 있다.

예제

테스트케이스

```
# -*- coding: utf-8 -*-
# Copyright (C) 2019 TTA - All Rights Reserved
# Unauthorized copying of this file, via any medium is strictly prohibited
# Proprietary and confidential
# $Id: 시스템_명령어_실행_기능.py 116 2019-07-05 12:13:13Z woosup.lim $

# -*- coding: utf-8 -*-
from dbtest.system import system
from dbtest.utils import comment, write_result

if __name__ == "__main__":
    comment("Testcase for system function")

    # 시스템 명령어 실행
    # 현재 디렉토리 목록을 출력함
    result = system("ls")

    write_result(result)
```

테스트오라클

```
0) # Testcase for system function
50) DBMS_설치_기능.exout
DBMS_설치_기능.py
DB_Link_제어_기능.exout
```



```
DB_Link_제어_기능.py
SQL_실행_기능.exout
SQL_실행_기능.py
VERIFICATION.suite
__pycache__
bin_dump_test.wvz
common.py
connectionLoad.class
connectionLoad.java
invalidApi.java
lib
src
네트워크_제어_기능.exout
네트워크_제어_기능.py
데이터베이스_관리_기능.exout
데이터베이스_관리_기능.py
동시_세션_생성_기능.exout
동시_세션_생성_기능.py
동시성_제어_기능.exout
동시성_제어_기능.py
백업_및_복구_기능.exout
백업_및_복구_기능.py
시스템_리소스_제어_기능.exout
시스템_리소스_제어_기능.py
시스템_명령어_실행_기능.exout
시스템_명령어_실행_기능.py
유틸리티_제어_기능.exout
유틸리티_제어_기능.py
이중화_제어_기능.exout
이중화_제어_기능.py
인터페이스_제어_기능.exout
인터페이스_제어_기능.py
임펙_데이터_생성_기능.exout
임펙_데이터_생성_기능.py
자원_모니터링_기능.exout
자원_모니터링_기능.py
초기화.exout
초기화.py
쿼리문_수행_시간_측정_기능.exout
쿼리문_수행_시간_측정_기능.py
테이블_간_정합성_비교_기능.exout
테이블_간_정합성_비교_기능.py
트랜잭션_설정_기능.exout
트랜잭션_설정_기능.py
파일_제어_기능.exout
파일_제어_기능.py
프로세스_제어_기능.exout
프로세스_제어_기능.py
환경_변수_제어_기능.exout
환경_변수_제어_기능.py
```

sysctl()

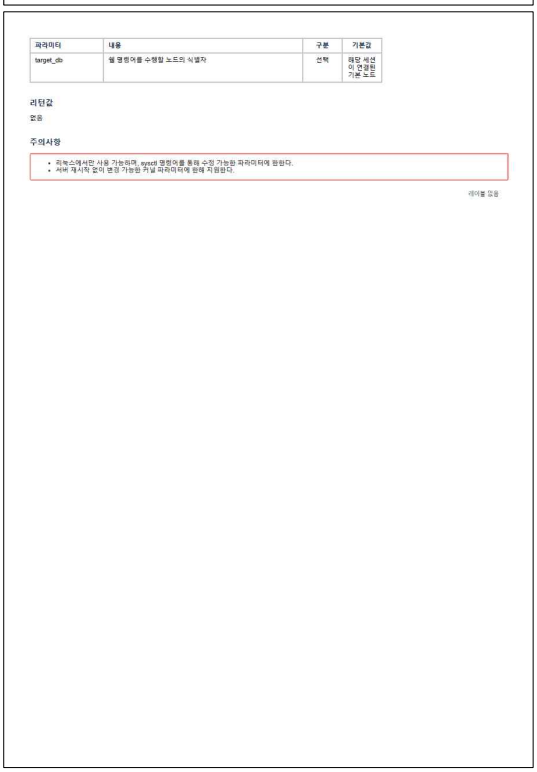
리눅스 커널 파라미터를 변경하는 함수

문법

```
sysctl(parameter_name, value, target_db=None)
```

파라미터

파라미터	내용	구분	기본값
parameter_name	커널 파라미터 이름	필수	없음
value	변경 값	필수	없음



파라미터	내용	구분	기본값
target_db	웹 명령어를 수행할 노드의 식별자	선택	해당 세션이 연결된 기본 노드

리턴값

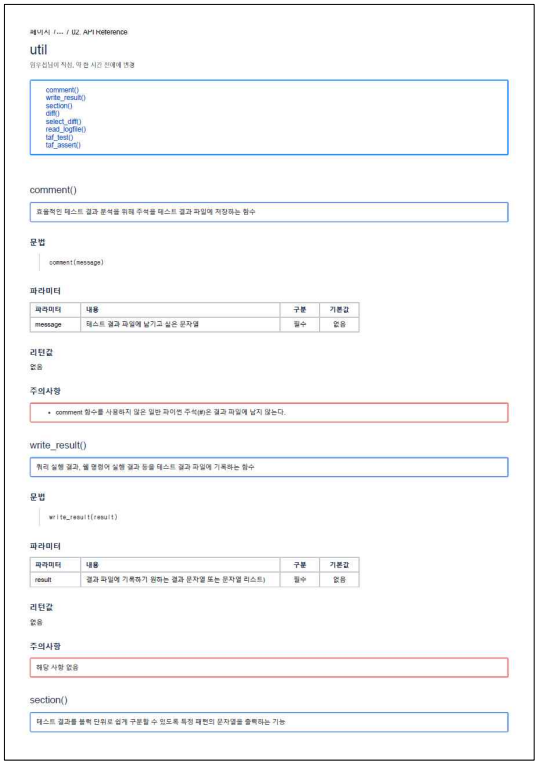
없음

주의사항

- 리눅스에서만 사용 가능하며, sysctl 명령어를 통해 수정 가능한 파라미터에 한함.
- 서버 재시작 없이 변경 가능한 커널 파라미터에 한해 지원함.

레이블 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



페이지 /... / UI2. API reference

util

팀우선님께 작성, 약 한 시간 전에에 변경

comment()
write_result()
section()
diff()
select_diff()
read_logfile()
taf_test()
taf_assert()

comment()

효율적인 테스트 결과 분석을 위해 주석을 테스트 결과 파일에 저장하는 함수

문법

```
comment(message)
```

파라미터

파라미터	내용	구분	기본값
message	테스트 결과 파일에 남기고 싶은 문자열	필수	없음

리턴값

없음

주의사항

- comment 함수를 사용하지 않은 일반 라이언 주석(#)은 결과 파일에 남지 않는다.

write_result()

쿼리 실행 결과, 쉘 명령어 실행 결과 등을 테스트 결과 파일에 기록하는 함수

문법

```
write_result(result)
```

파라미터

파라미터	내용	구분	기본값
result	결과 파일에 기록하기 원하는 결과 문자열 또는 문자열 리스트)	필수	없음

리턴값

없음

주의사항

해당 사항 없음

section()

테스트 결과를 블록 단위로 쉽게 구분할 수 있도록 특정 패턴의 문자열을 출력하는 기능



문법

```
section(message)
```

파라미터

파라미터	내용	구분	기본값
message	테스트 결과 파일에 남기고 싶은 문자열	필수	없음

리턴값

없음

주의사항

해당 사항 없음

예제

테스트케이스

```
# -*-coding:utf-8
from dbtest.query import query
from dbtest.utils import *

"""
 # Description : 1만건 insert 테스트
 # Scenario : N/A
"""


if __name__ == '__main__':

    section("Initialization")

    query("DROP TABLE INSERTTEST;", silent=True)

    query("CREATE TABLE INSERTTEST(  B_INT int NOT NULL AUTO_INCREMENT PRIMARY KEY, B_CHAR10 char(10) default 'TTA', B_

    query("CREATE INDEX IDX_INSERTTEST_INT_CHAR10_CHAR100 USING HASH ON INSERTTEST(B_INT, B_CHAR10, B_CHAR100);")

    query("CREATE INDEX IDX_INSERTTEST_INT_SMALLINT_BIGINT USING HASH ON INSERTTEST(B_INT, B_SMALLINT, B_BIGINT);")

    query("CREATE INDEX IDX_INSERTTEST_CHAR1000_VARCHAR10_VARCHAR100 USING HASH ON INSERTTEST(B_CHAR1000, B_VARCHAR10,

    section("TEST")

    for i in range(10000) :
        query("INSERT INTO INSERTTEST(B_CHAR100, B_CHAR1000, B_VARCHAR10, B_VARCHAR100, B_VARCHAR1000, B_TEXT, B_STRIN

    section("SELECT LIMIT 10")

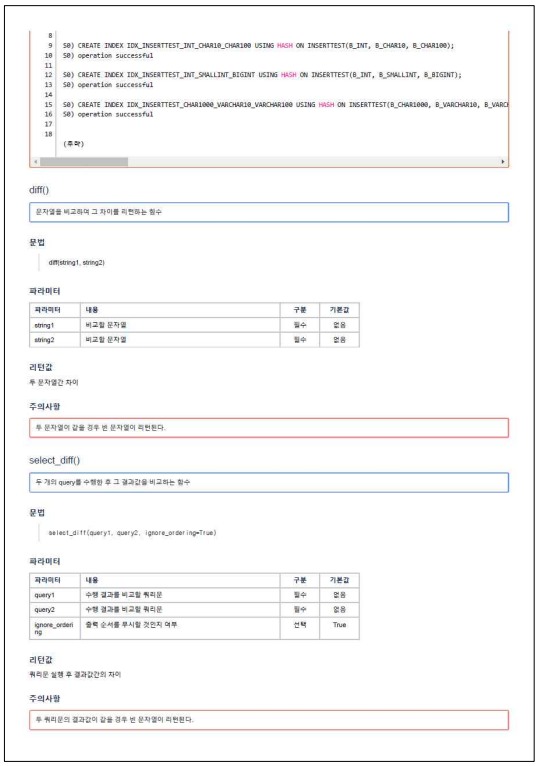
    query("SELECT * FROM INSERTTEST LIMIT 10;")

    section("Finalization")

    query("DROP TABLE INSERTTEST;");
```

테스트오라클

```
S0) ########################################################################
S0) # Initialization
S0) ########################################################################
S0) DROP TABLE INSERTTEST;
S0) CREATE TABLE INSERTTEST(  B_INT int NOT NULL AUTO_INCREMENT PRIMARY KEY, B_CHAR10 char(10) default 'TTA', B_CHAR10
S0) operation successful
```



```
S0) CREATE INDEX IDX_INSERTTEST_INT_CHAR10_CHAR100 USING HASH ON INSERTTEST(B_INT, B_CHAR10, B_CHAR100);
S0) operation successful

S0) CREATE INDEX IDX_INSERTTEST_INT_SMALLINT_BIGINT USING HASH ON INSERTTEST(B_INT, B_SMALLINT, B_BIGINT);
S0) operation successful

S0) CREATE INDEX IDX_INSERTTEST_CHAR1000_VARCHAR10_VARCHAR100 USING HASH ON INSERTTEST(B_CHAR1000, B_VARCHAR10, B_VARCH
S0) operation successful


(후략)
```

diff()

문자열을 비교하여 그 차이를 리턴하는 함수

문법

```
diff(string1, string2)
```

파라미터

파라미터	내용	구분	기본값
string1	비교할 문자열	필수	없음
string2	비교할 문자열	필수	없음

리턴값

두 문자열간 차이

주의사항

두 문자열이 같을 경우 빈 문자열이 리턴된다.

select_diff()

두 개의 query를 수행한 후 그 결과값을 비교하는 함수

문법

```
select_diff(query1, query2, ignore_ordering=True)
```

파라미터

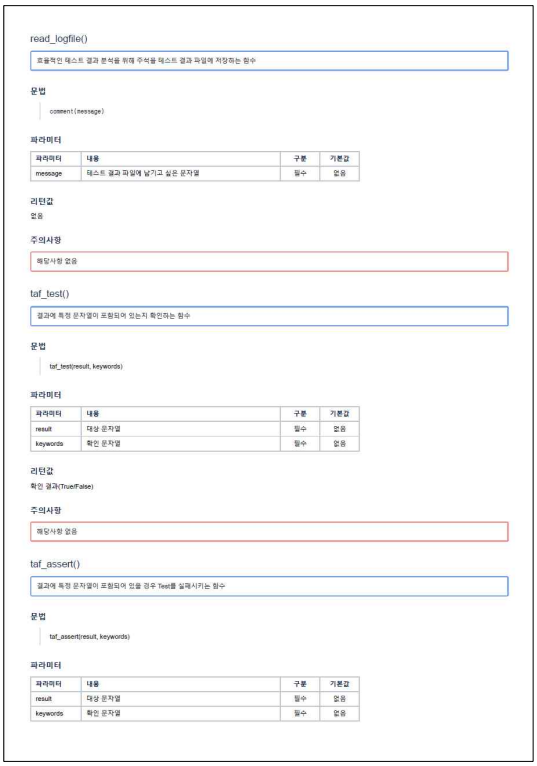
파라미터	내용	구분	기본값
query1	수행 결과를 비교할 쿼리문	필수	없음
query2	수행 결과를 비교할 쿼리문	필수	없음
ignore_ordering	출력 순서를 무시할 것인지 여부	선택	True

리턴값

쿼리문 실행 후 결과값간의 차이

주의사항

두 쿼리문의 결과값이 같을 경우 빈 문자열이 리턴된다.



read_logfile()

효율적인 테스트 결과 분석을 위해 주석을 테스트 결과 파일에 저장하는 함수

문법

```
comment(message)
```

파라미터

파라미터	내용	구분	기본값
message	테스트 결과 파일에 남기고 싶은 문자열	필수	없음

리턴값

없음

주의사항

해당사항 없음

taf_test()

결과에 특정 문자열이 포함되어 있는지 확인하는 함수

문법

```
taf_test(result, keywords)
```

파라미터

파라미터	내용	구분	기본값
result	대상 문자열	필수	없음
keywords	확인 문자열	필수	없음

리턴값

확인 결과(True/False)

주의사항

해당사항 없음

taf_assert()

결과에 특정 문자열이 포함되어 있을 경우 Test를 실패시키는 함수

문법

```
taf_assert(result, keywords)
```

파라미터

파라미터	내용	구분	기본값
result	대상 문자열	필수	없음
keywords	확인 문자열	필수	없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

리턴값

없음

주의사항

해당 사항 없음

레이블 없음

utility

임무 라이어 처리, 이 또 시간 전에에 변경

- psql()
- send_wait()
- quit()
- kill()
- client()
- prepare()
- exec()
- petadump()
- bindump()
- petaloader()

psql()

petasql 유틸리티를 실행하는 함수

문법

```
psql(extra_args=None, conn_db=None)
```

파라미터

파라미터	내용	구분	기본값
extra_args	추가로 입력할 매개 변수 • 기본적으로 아래 필수 매개변수가 입력됨 • -h ${DB_IP} -p ${DB_PORT} -u ${DB_USER} -P ${DB_PASSWD} -d ${DB_NAME} -i	선택	없음
conn_db	petasql 유틸리티를 통해 접속할 DB 서버	선택	해당 세션의 기본 노드

리턴값

없음

주의사항

해당 사항 없음

예제

테스트케이스

```
# -*- coding: utf-8 -*-
from dbtest import database, dblink
from dbtest.dblink import get_dblink_name
from dbtest.query import query
from dbtest.session import Session
from dbtest.utils import comment, write_result


def model_5gb():
    query("drop table t1 if exists;", silent=True)

    query("create table t1 (id int);", silent=True)

    for i in range(100):
        query("insert into t1 values({})".format(i), query_silent=True, silent=True)

    write_result("insert 100 rows to t1 table")


if __name__ == "__main__":
    comment("create database")
    database.initdb()
    database.initdb(target_db="NODE1")

    dblink.create_dblink("NODE1")

    dblink.start()

    dblink.select()

    remote_session = Session("NODE1")

    remote_session.execute(target_func=model_5gb, background=False)

    query("select * from sys.t1@{};".format(get_dblink_name()))
    query("select * from sys.t1@{} as a where a.id > 100;".format(get_dblink_name()))

    dblink.stop()

    query("select * from sys.t1@{};".format(get_dblink_name()))
    query("select * from sys.t1@{} as a where a.id > 100;".format(get_dblink_name()))

    dblink.drop()

    query("select * from sys.t1@{};".format(get_dblink_name()))
    query("select * from sys.t1@{} as a where a.id > 100;".format(get_dblink_name()))
```

테스트오라클

```
S0) # init database
S0) server.initdb()
S0) Completed to initialize default database.
S0) select * from t1
S0) SELECT: no such table 't1'
```

send_wait()

실행된 petasql 유틸리티에 명령어를 전송하는 기능

문법

```
send_wait(line)
```

파라미터

파라미터	내용	구분	기본값
line	전송할 명령어	필수	없음

리턴값

명령어 수행 결과

주의사항

해당 사항 없음

quit()

실행한 petasql 유틸리티를 종료하는 기능

문법

```
quit()
```

파라미터

파라미터	내용	구분	기본값
해당 없음			

리턴값

없음

주의사항

해당 사항 없음

kill()

실행한 petasql 유틸리티를 강제 종료하는 기능

문법

```
kill()
```

파라미터

파라미터	내용	구분	기본값
해당 없음			

리턴값

없음

주의사항

해당 사항 없음

client()

petasql 유틸리티를 실행하는 함수

문법

```
client(extra_args=None, conn_db=None)
```

파라미터

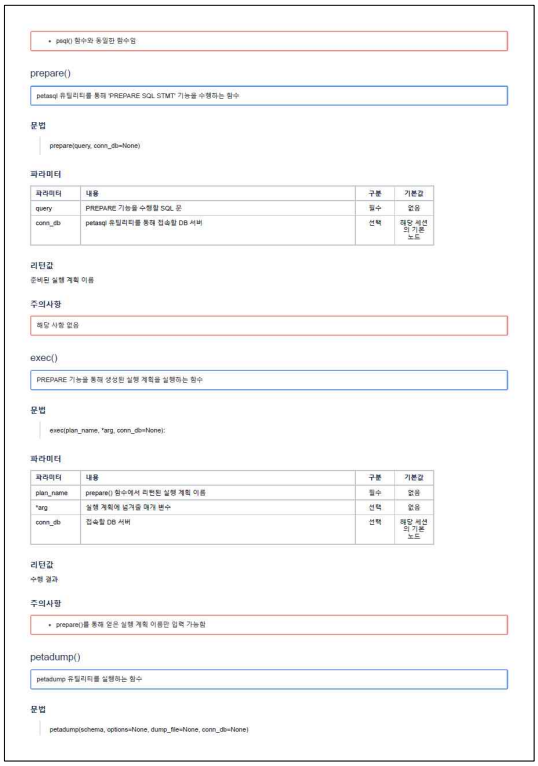
파라미터	내용	구분	기본값
extra_args	추가로 입력할 매개 변수 • 기본적으로 아래 필수 매개변수가 입력됨 • -h ${DB_IP} -p ${DB_PORT} -u ${DB_USER} -P ${DB_PASSWD} -d ${DB_NAME} -i	선택	없음
conn_db	접속할 DB 서버	선택	해당 세션의 기본 노드

리턴값

없음

주의사항

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



- psql() 함수와 동일한 함수임

prepare()

petasql 유틸리티를 통해 'PREPARE SQL STMT' 기능을 수행하는 함수

문법

```
prepare(query, conn_db=None)
```

파라미터

파라미터	내용	구분	기본값
query	PREPARE 기능을 수행할 SQL 문	필수	없음
conn_db	petasql 유틸리티를 통해 접속할 DB 서버	선택	해당 세션의 기본 노드

리턴값

준비된 실행 계획 이름

주의사항

해당 사항 없음

exec()

PREPARE 기능을 통해 생성된 실행 계획을 실행하는 함수

문법

```
exec(plan_name, *arg, conn_db=None):
```

파라미터

파라미터	내용	구분	기본값
plan_name	prepare() 함수에서 리턴된 실행 계획 이름	필수	없음
*arg	실행 계획에 넘겨줄 매개 변수	선택	없음
conn_db	접속할 DB 서버	선택	해당 세션의 기본 노드

리턴값

수행 결과

주의사항

- prepare()를 통해 얻은 실행 계획 이름만 입력 가능함

petadump()

petadump 유틸리티를 실행하는 함수

문법

```
petadump(schema, options=None, dump_file=None, conn_db=None)
```

파라미터

파라미터	내용	구분	기본값
schema	덤프를 수행할 스키마 이름	필수	없음
options	petadump에 전달할 옵션	선택	없음
dump_file	덤프 파일 저장 경로	선택	없음
conn_db	접속할 DB 서버	선택	해당 세션의 기본 노드

리턴값

petadump 실행 결과

주의사항

해당 사항 없음

bindump()

bindump 유틸리티를 실행하는 함수

문법

```
bindump(*table, options=None, mode='-b', path=None, conn_db=None)
```

파라미터

파라미터	내용	구분	기본값
table	dump를 수행할 테이블 명	필수	없음
options	bindump에 전달할 옵션	선택	없음
mode	bindump 수행 모드	선택	백업 모드
path	백업 파일 경로	선택	테스트 케이스와 same 디렉토리
conn_db	접속할 DB 서버	선택	해당 세션의 기본 노드

리턴값

수행 결과

주의사항

해당 사항 없음

petaloader()

petaloader 유틸리티를 실행하는 함수

문법

```
petaloader(control_file, extra_args=None, conn_db=None)
```

파라미터

파라미터	내용	구분	기본값
control_file	컨트롤 파일 경로	필수	없음
extra_args	추가 매개변수 • -h ${DB_IP} -U ${DB_USER} -p ${DB_PORT} -u ${DB_USER} -P ${DB_PASSWD} 는 기본으로 전달됨	선택	없음
conn_db	petasql 유틸리티를 통해 접속할 DB 서버	선택	해당 세션의 기본 노드

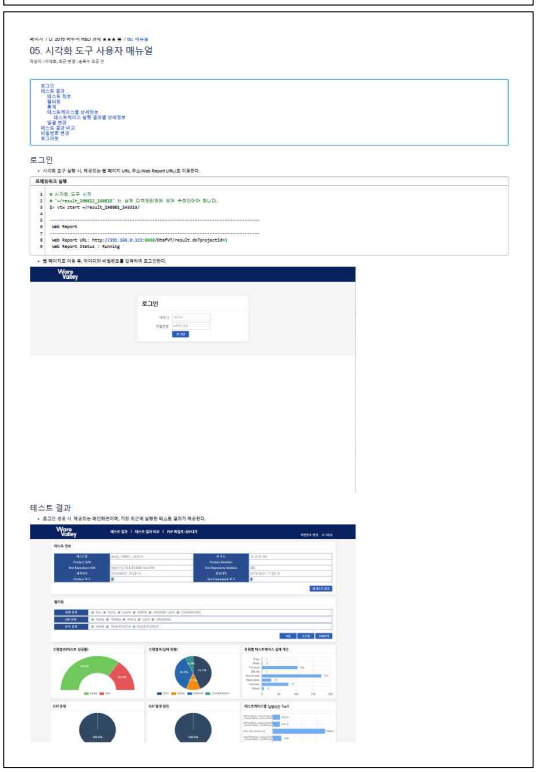
리턴값

수행 결과

주의사항

해당 사항 없음

테이블 없음

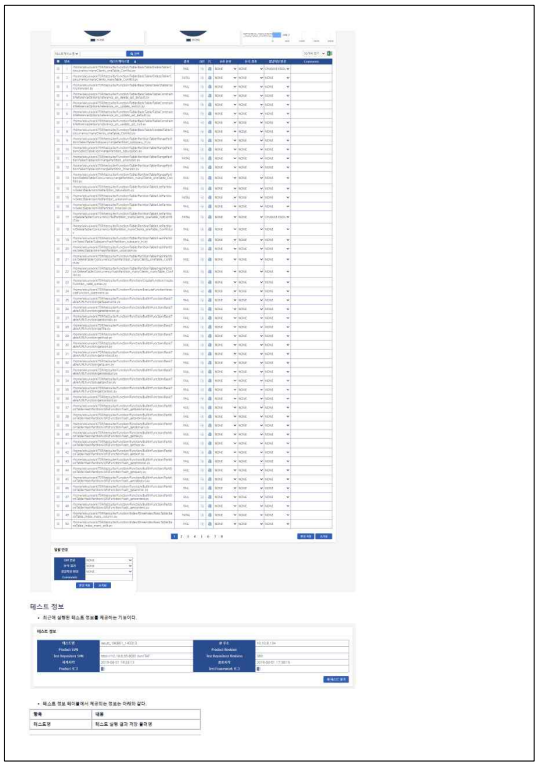


05. 시각화 도구 사용자 매뉴얼

로그인

테스트 결과

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

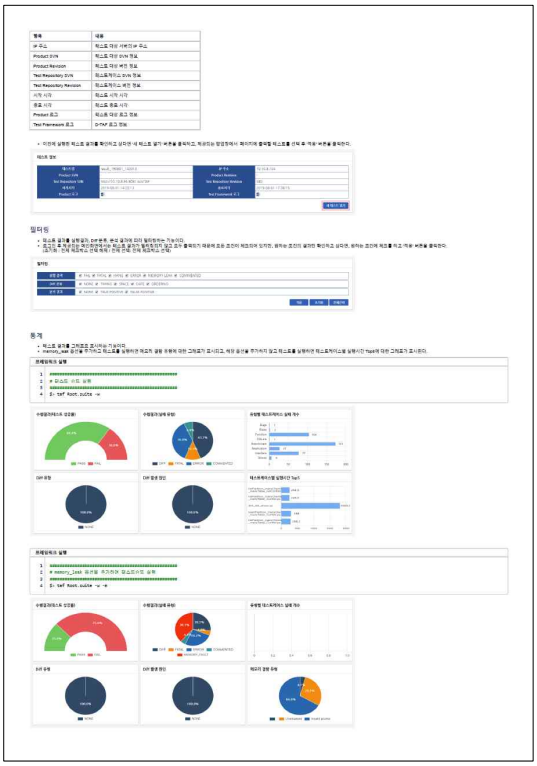


테스트 정보

- 리그에 실행한 테스트 정보를 제공하는 기능이다.

- 테스트 정보 제작을 위해 제공하는 정보는 아래와 같다.

항목	내용
테스트명	테스트 실행 결과 저장 폴더명



필터링

- 테스트 결과를 실행 결과, Diff 분류, 분석 결과에 따라 필터링하는 기능이다.
- 테스트 결과 화면상의 특정 조건에서는 테스트 결과가 필터링되지 않고 추가 필터링이 적용되는 경우도 있으며, 원하는 조건이 설정이 확인하고 싶다면, 원하는 조건에 체크를 하고 '적용' 버튼을 클릭한다.
(참고로, 인덱 체크하고 선택 해제 안해 인덱 체크하고 선택)

동계

- 테스트 결과를 그래프로 표시하는 기능이다.
- memory_leak 결과를 추가하고 테스트를 실행하면 제공하 실행 완료에 대한 그래프가 표시되고, 해당 실행을 추가하지 않고 테스트를 실행하면 테스트하지고 실행시간 Topdll에 대한 그래프가 표시된다.

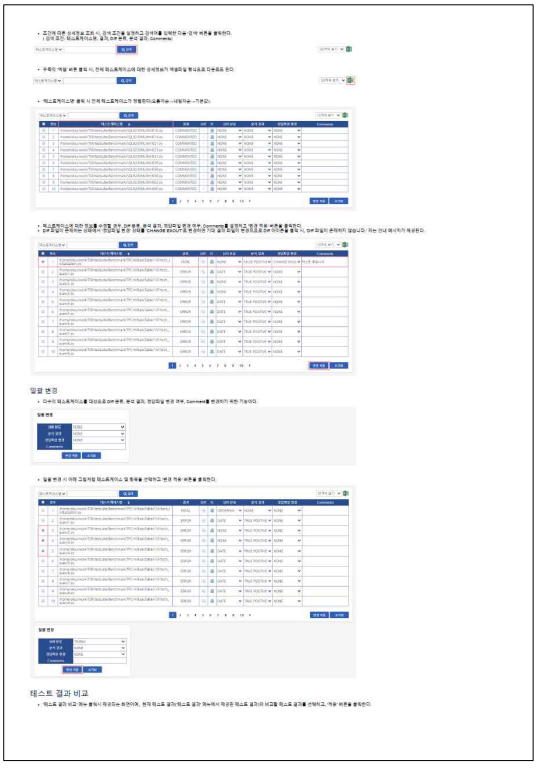
테스트케이스별 상세정보

- 테스트케이스별 상세정보를 제공하는 기능이며, 각각의 테스트케이스에 대한 분석, 각종 실행의 설정과 Diff 테스트케이스 종료 결과를 수 있다.
- 분석 결과는 분류하기 위해 분석 기준으로 설정하면 분류가 결정되며, 각각 상황 해석하기 달라진다.

테스트케이스 실행 결과별 상세정보

실행 결과	설명	Diff 판정 유무	Diff 파일이 존재하지 않는 경우
PASS	기대 결과와 실행 결과가 동일함	X	
FAIL	기대 결과와 실행 결과가 다름	O	• 정답파일을 변경하는 경우
FATAL	테스트 실행 도중 Crash가 감지 종료됨	O	• 정답파일을 변경하는 경우
ERROR	• 테스트 실행 도중 Hang가 감지 종료됨 • 프로그램 종료코드 0이 아닌 테스트케이스가 실행되지 않음	X	• 문제가 발생하는 경우에 한해 테스트케이스가 실행되지 않음 • 정답파일을 변경하는 경우
COMMENTED	테스트케이스가 주석처리되어 실행되지 않음	X	-





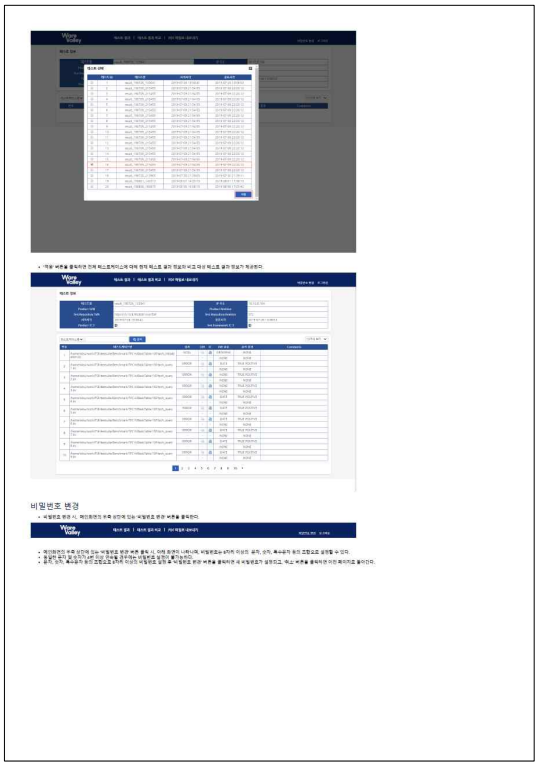
일괄 변경

- Diff가 테스트케이스를 대상으로 Diff 분류, 분석 결과, 정답파일 변경 여부, Comment를 변경하기 위한 기능이다.

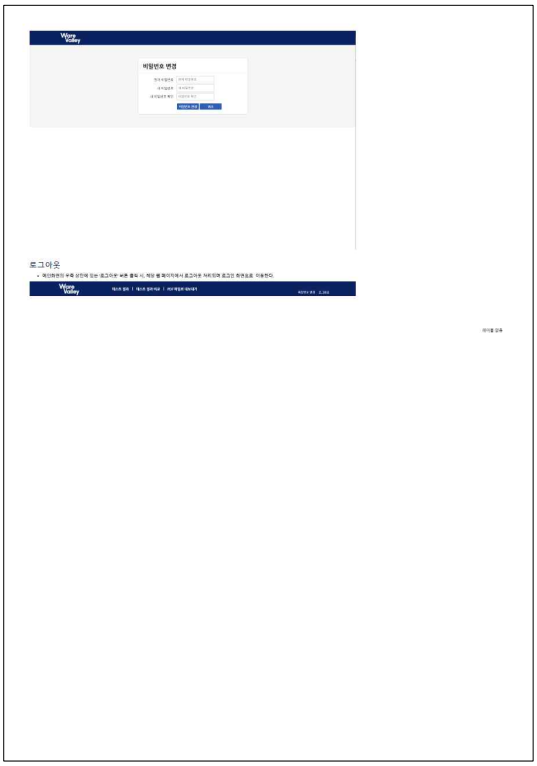
테스트 결과 비교

- 테스트 결과 비교 기능 클릭시 제공하는 화면이며, 현재 테스트 결과와 비교하고 싶은 테스트 결과의 비교할 테스트 결과를 선택하고, '적용' 버튼을 클릭한다.

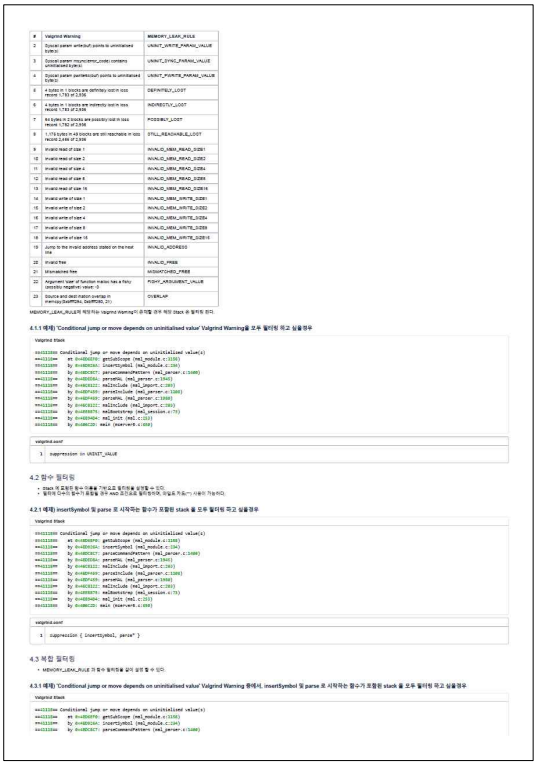
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

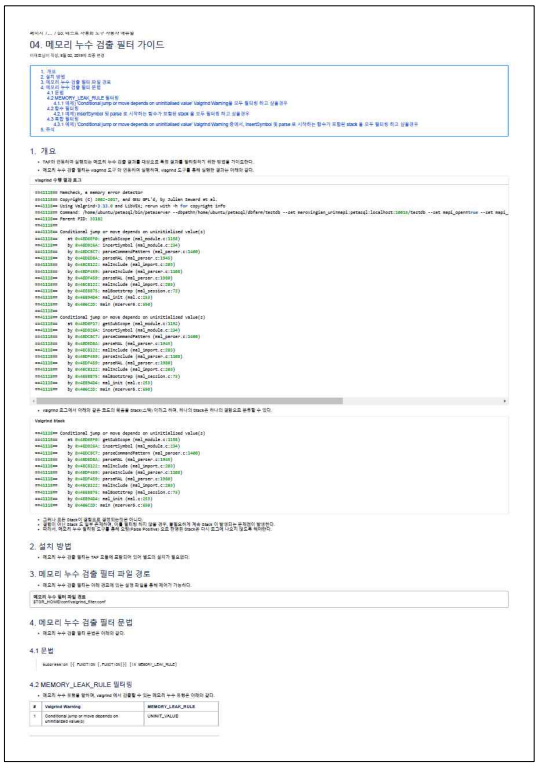


비밀번호 변경



로그아웃





국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



(4) 트라블슈팅(Troubleshooting) 가이드

- 빅데이터 분석용 DBMS 품질 검증 자동화 프레임워크 사용 도중 발생할 수 있는 오류의 증상과 해결 방법에 대한 가이드 문서를 작성함





국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



작업 같은 경우 아래 사항을 확인해보시기 바랍니다.

TAF 로그 파일을 통한 원인 분석

아래 로그 파일에 에러 메시지가 남아 있을 수 있습니다. 해당 파일을 확인해보시기 바랍니다.

로그 종류	경로	비고
실행 실패 로그	~/smurf_1000 14_1122541/logs/dataset.log	결과 폴더의 이름은 'result_날짜_시간' 형식으로 수정되어야 한다.
nodeworker 실행 로그	~/.taf/worker.log	-

DBMS 수동 실행을 통한 원인 분석

본실행시 작업 DBMS 정확히를 수행하여 문제의 원인을 모을 수도 있습니다. install은 주어진 정확하는 아래와 같습니다.

구분	정확식
DB Farm 생성	petalcopt create $(PETA_HOME)/dbfarm/testdb
DBMS 시작	petalcopt start $(PETA_HOME)/dbfarm/testdb
DBMS 포트 변경	petalcopt set.confvar(PETA_PORT_NO):$(PETA_HOME)/dbfarm/testdb
DB 제거	petact -b $(PETA_PORT_NO) destroy -f testdb
DB 생성	petact -b $(PETA_PORT_NO) create testdb petact -b $(PETA_PORT_NO) rmosur testdb

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

라. 품질 검증 자동화 프레임워크를 이용한 빅데이터 솔루션에 대한 품질 검증 서비스 제공

(1) 품질 검증 자동화 프레임워크를 활용한 시스템 테스트

- 품질 검증 자동화 프레임워크를 이용하여 빅데이터 분석용 DBMS(PetaSQL)에 대한 품질 검증을 진행함
- 핵심 요소별, 품질 특성별로 작성된 약 1,200여개의 테스트케이스 및 테스트 스크립트를 이용하여 테스트를 진행함

순번	시험 항목	세부 시험 항목	설명
1	SQL 쿼리문	SQL (Non-Partitioned Table)	Non-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
2		SQL (Hash-Partitioned Table)	Hash-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
3		SQL (List-Partitioned Table)	List-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
4		SQL (Range-Partitioned Table)	Range-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
5		SQL (Temp Table)	Temp Table 에서의 SQL 쿼리문(DML, DDL, DCL) 기능 시험
6		트랜잭션	트랜잭션(Read Committed, Serialization) 기능 시험
7		사용자 및 권한	사용자 및 권한 기능 시험
8		뷰	뷰 기능 시험
9		내장함수	내장함수(AVG, SUM, COUNT 등) 기능 시험
10		함수	사용자 정의 함수 기능 시험
11		프로시저	프로시저 기능 시험
12		데이터 타입	char, int, numeric 등의 데이터 타입 기능 시험
13		인덱스	인덱스 기능 시험
14		옵티마이저	옵티마이저(Explain, Plan) 기능 시험
15	이중화	-	이중화 기능 시험

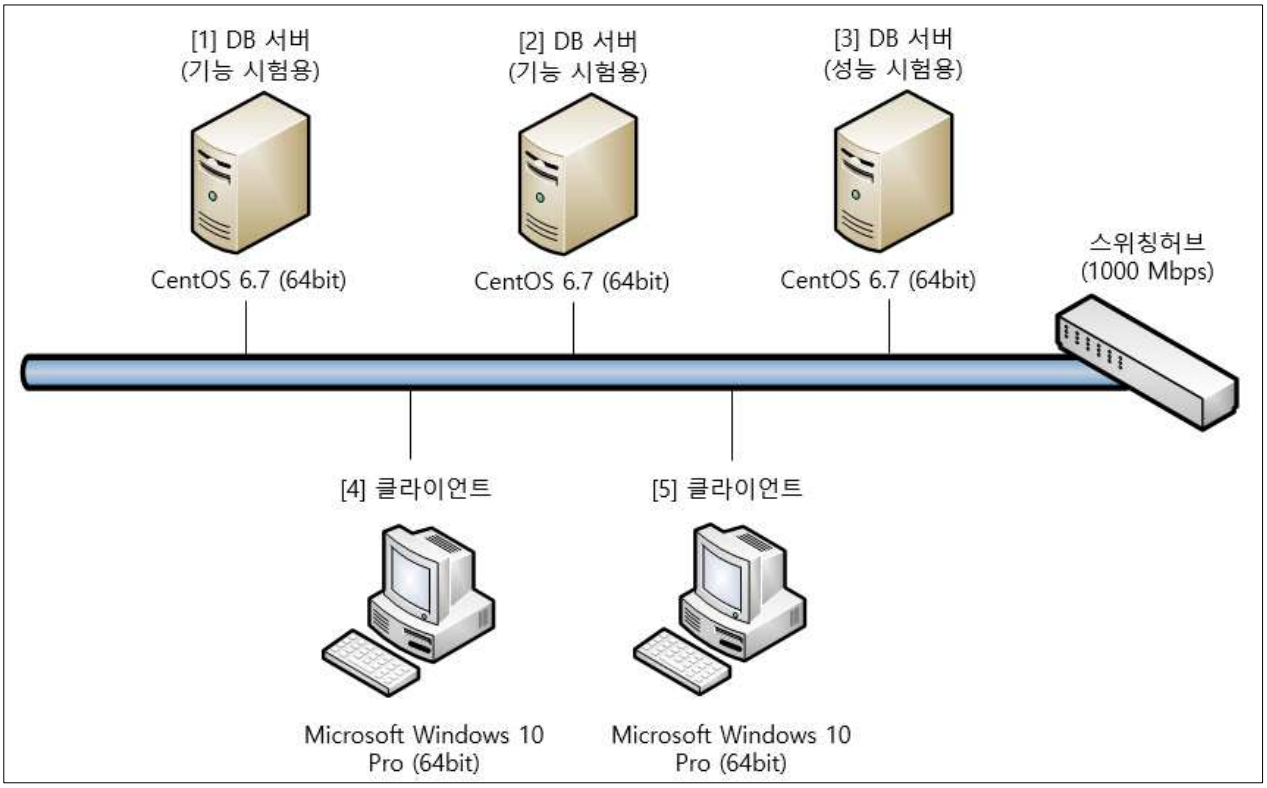
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	시험 항목	세부 시험 항목	설명
16	DB Link	-	DB Link 기능 시험
17	인터페이스	ODBC	ODBC API 시험
18		JDBC	JDBC API 시험
19	유틸리티	-	유틸리티 기능 시험
20	자원 한계 상황	-	자원(스토리지 등)이 부족한 상황에서의 기능 시험
21	스트레스	-	다수 클라이언트 환경에서의 트랜잭션 부하 시험
22	리커버리	-	백업 및 복구 시험
23	성능	-	TPC-H 100SF 성능 시험
24	SQL92	-	SQL92 표준 쿼리문 기능 시험
25	TPC-H	-	TPC-H 시험
26	TPC-DS	-	TPC-DS 시험
27	메모리 관리	-	메모리 오류(메모리 누수, 초기화 되지 않은 변수 사용 등) 검출 시험

[표 19] 품질 검증 자동화 프레임워크를 이용한 DBMS 시험항목

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 시험에 필요한 시험 환경 및 세부사항은 아래와 같이 구성하였음



[그림 61] 시험환경구성도

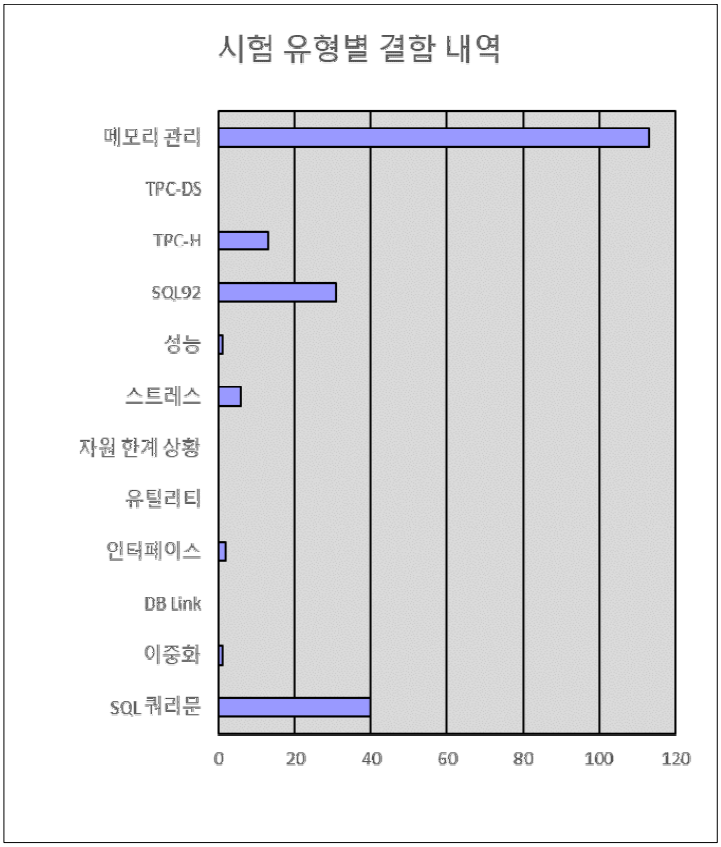
No	Role	OS	CPU	Mem	Storage	Pre-Requisite
1	DB 서버	CentOS 6.7 (64bit)	Intel® Xeon™ E5-2620@ 2.10 GHz	64GB	1TB	- OpenJDK 11.0.3 - Valgrind 3.8.0 - gcc 4.4.7 - unixODBC 2.2.14
2	DB 서버	CentOS 6.7 (64bit)	Intel® Xeon™ E5-2650@ 2.10 GHz	64GB	1TB	- OpenJDK 11.0.3 - Valgrind 3.8.0 - gcc 4.4.7 - unixODBC 2.2.14
3	DB 서버	CentOS 6.7 (64bit)	Intel® Xeon™ E7-8860@ 2.19 GHz	256GB	1TB	- OpenJDK 11.0.3 - Valgrind 3.8.0 - gcc 4.4.7 - unixODBC 2.2.14
4	클라이언트	Microsoft Windows 10 Pro (64bit)	Intel® Core™ i7-3770 CPU @ 3.40GHz	8GB	1TB	- putty 0.72
5	클라이언트	Microsoft Windows 10 Pro (64bit)	Intel® Core™ i7-3770 CPU @ 3.40GHz	8GB	1TB	- putty 0.72

[표 20] 세부사양

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 품질 검증 자동화 프레임워크를 이용한 빅데이터 솔루션 시험 결과 총 207개의 결함이 발생되었으며, SQL 쿼리문 실행 및 메모리 누수 오류가 다수 발견되었음

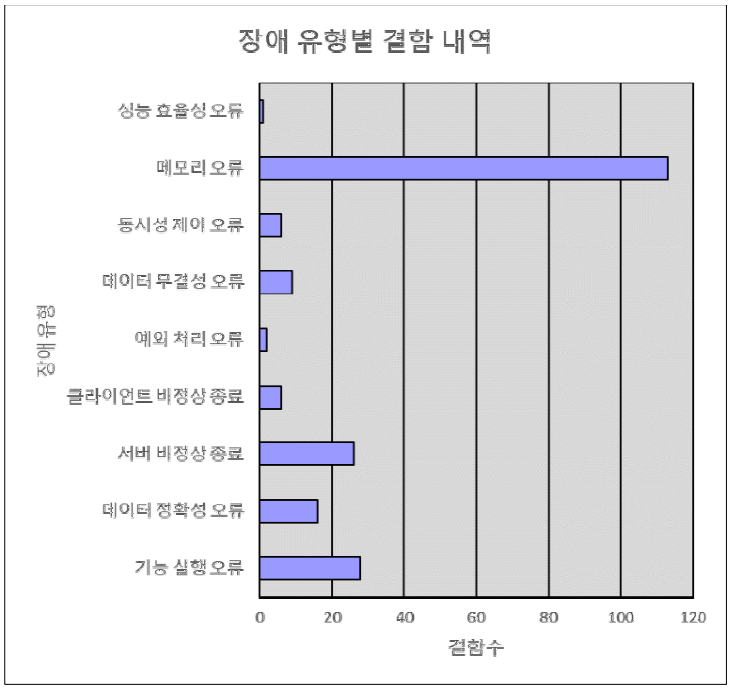
시험 유형	결함수
SQL 쿼리문	40
이중화	1
DB Link	0
인터페이스	2
유틸리티	0
자원 한계 상황	0
스트레스	6
리커버리	0
성능	1
SQL92	31
TPC-H	13
TPC-DS	0
메모리 관리	113
계	**207**



[그림 62] 시험 유형별 결함 내역

- 장애 유형별 결함 내역을 분석한 결과 서버 비정상 종료, 클라이언트 비정상 종료 등 제품 신뢰성에 영향을 미치는 심각한 결함이 다수 발견되었음

장애 유형	결함수
기능 실행 오류	28
데이터 정확성 오류	16
서버 비정상 종료	26
클라이언트 비정상 종료	6
예외 처리 오류	2
데이터 무결성 오류	9
동시성 제어 오류	6
메모리 오류	113
성능 효율성 오류	1
계	**207**



[그림 63] 장애 유형별 결함 내역

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	결함 요약	결함 설명
1	Natural Join 기능 오류	NATURAL JOIN SELECT 결과가 잘못 출력됨 (테스트케이스) $TSR_HOME/testsuite/Function/Table/PartitionTable/RangePartition/SelectTable/Join/rangePartition_naturalJoin.py
2	SEQUENCE 삭제 기능 오류	sequence를 갖는 temp table 삭제 시, sequence 가 삭제되지 않음 (테스트케이스) $TSR_HOME/testsuite/Function/Table/TempTable/CreateTable/temp_createTable.py $TSR_HOME/testsuite/Function/Table/TempTable/InsertTable/temp_insertTable.py $TSR_HOME/testsuite/Function/Table/TempTable/SelectTable/temp_selectTable.py $TSR_HOME/testsuite/Function/Table/TempTable/UpdateTable/temp_updateTable.py $TSR_HOME/testsuite/Function/Table/TempTable/DeleteTable/temp_deleteTable.py $TSR_HOME/testsuite/Function/Table/TempTable/TableConstraint/temp_tableConstraint.py $TSR_HOME/testsuite/Function/Table/ComplexTableJoin/Basic_TempTableJoin/basic_tempTableJoin.py $TSR_HOME/testsuite/Function/Table/ComplexTableJoin/Temp_PartitionTableJoin/temp_hashpartitionTableJoin.py $TSR_HOME/testsuite/Function/Table/ComplexTableJoin/Temp_PartitionTableJoin/temp_listPartitionTableJoin.py $TSR_HOME/testsuite/Function/Table/ComplexTableJoin/Temp_PartitionTableJoin/temp_rangePartitionTableJoin.py
3	Index 조건 검색 기능 오류	index 걸린 컬럼에 select where <= value 조건 입력 시, 값이 조회되지 않음 (테스트케이스) $TSR_HOME/testsuite/Function/Index/BtreeIndex/BasicTable/basicTable_index_data_skew.py
4	like 기능 오류	'_'를 이용한 like 기능이 동작하지 않음 (쿼리문) SELECT CITY FROM STAFF WHERE EMPNAME LIKE 'yan___%'; (테스트케이스) $TSR_HOME/testsuite/Benchmark/SQL92/Schema/Schema.suite $TSR_HOME/testsuite/Benchmark/SQL92/Load/Load.suite $TSR_HOME/testsuite/Benchmark/SQL92/DML/dml039.py
5	클라이언트 비정상 종료	50개의 세션으로 DELETE 후 다시 INSERT 및 DELETE 쿼리문 실행 시, FATAL : DB Client was dead 에러 발생 후 클라이언트가 비정상 종료됨 (테스트케이스) $TSR_HOME/testsuite/Function/Table/BasicTable/DeleteTable/Concurrency/manyClients_oneTable_Confilct.py

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	결함 요약	결함 설명
6	클라이언트 비정상 종료	100개의 세션으로 DML 및 DDL 쿼리문 반복 실행 시, FATAL : DB Client was dead 에러 발생 후 클라이언트가 비정상 종료됨 (테스트케이스) $TSR_HOME/testsuite/Stress/dml_ddl_stress.py
7	서버 비정상 종료	autocommit(off) 상태에서 UNION JOIN 구문이 포함된 SELECT 쿼리문 실행 시, 서버가 비정상 종료됨 (테스트케이스) $TSR_HOME/testsuite/Function/Table/BasicTable/SelectTable/Join/unionJoin.py
8	서버 비정상 종료	IN 구문이 포함된 SELECT 쿼리문 실행 시, 서버가 비정상 종료됨 (테스트케이스) $TSR_HOME/testsuite/Function/Table/PartitionTable/HashPartition/SelectTable/Subquery/hashPartition_subquery_in.py
9	서버 비정상 종료	[Index>BtreeIndex>PartitionTable] Partition 테이블에 index 생성 시, 서버 비정상 종료 및 재시작 되지 않음 <동일결함> [Index>HashIndex>PartitionTable] (테스트케이스) $TSR_HOME/testsuite/Function/Index/BtreeIndex/PartitionTable/PartitionTable.suite $TSR_HOME/testsuite/Function/Index/HashIndex/PartitionTable/PartitionTable.suite
10	서버 비정상 종료	파티션 테이블을 생성하여 TPC-H 데이터 삽입 후, TPC-H 10번 쿼리 실행 시, 서버가 비정상 종료함 (테스트케이스) $TSR_HOME/testsuite/Benchmark/TPC-H/PartitionTable/HashPartitionedTable/1SF/tpch_initialization.py $TSR_HOME/testsuite/Benchmark/TPC-H/PartitionTable/HashPartitionedTable/1SF/tpch_query10.py
11	서버 비정상 종료	SQL92 테스트케이스(dml121) 실행 시, 서버가 비정상 종료함 (테스트케이스) $TSR_HOME/testsuite/Benchmark/SQL92/Schema/Schema.suite $TSR_HOME/testsuite/Benchmark/SQL92/Load/Load.suite $TSR_HOME/testsuite/Benchmark/SQL92/DML/dml121.py

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	결함 요약	결함 설명
12	메모리 오류	rangePartition_unionJoin 테스트 수행 시, 메모리 누수(4건)가 발생함 [메모리 누수 유형] - Invalid read of size 4 - Conditional jump or move depends on uninitialised value(s) - Syscall param write(buf) points to uninitialised byte(s) (테스트케이스) $TSR_HOME/testsuite/Function/Table/PartitionTable/RangePartition/SelectTable/Join/rangePartition_unionJoin.py (메모리 오류 로그) Invalid read of size 4 at 0x10068535: table_ref (rel_select.c:2152) by 0x1006CD77: query_exp_optname (rel_select.c:6863) by 0x10068586: table_ref (rel_select.c:2257) by 0x1006790D: rel_subquery (rel_select.c:6496) by 0x1006C956: rel_selects (rel_select.c:6931) by 0xFFCDB8D: sql_symbol2relation (sql.c:155) by 0xFFE5F44: SQLparser (sql_scenario.c:3617) by 0x4CBC259: runScenarioBody (mal_scenario.c:564) by 0x4CBC3BE: runScenario (mal_scenario.c:597) by 0x4CBCC4B: MSserveClient (mal_session.c:671) by 0x550AF8A: thread_starter (gdk_system.c:803) by 0x3AE7807AA0: start_thread (in /lib64/libpthread-2.12.so) Address 0x4 is not stack'd, malloc'd or (recently) free'd at 0x10068535: table_ref (rel_select.c:2152) by 0x1006CD77: query_exp_optname (rel_select.c:6863) by 0x10068586: table_ref (rel_select.c:2257) by 0x1006790D: rel_subquery (rel_select.c:6496) by 0x1006C956: rel_selects (rel_select.c:6931) by 0xFFCDB8D: sql_symbol2relation (sql.c:155) by 0xFFE5F44: SQLparser (sql_scenario.c:3617) by 0x4CBC259: runScenarioBody (mal_scenario.c:564) by 0x4CBC3BE: runScenario (mal_scenario.c:597) by 0x4CBCC4B: MSserveClient (mal_session.c:671) by 0x550AF8A: thread_starter (gdk_system.c:803) by 0x3AE7807AA0: start_thread (in /lib64/libpthread-2.12.so) Conditional jump or move depends on uninitialised value(s) at 0x4CA8E70: getSubScope (mal_module.c:1158) by 0x4CAB1EA: insertSymbol (mal_module.c:234) by 0x4CAE847: parseCommandPattern (mal_parser.c:1400) by 0x4CB0D0A: parseMAL (mal_parser.c:1945) by 0x4C9A0A2: malInclude (mal_import.c:203) by 0x4CB13D9: parseMAL (mal_parser.c:1108) by 0x4C9A0A2: malInclude (mal_import.c:203) by 0x4CBD7E5: malBootstrap (mal_session.c:73) by 0x4C8B464: mal_init (mal.c:253) by 0x406BFD: main (mserver5.c:650) Conditional jump or move depends on uninitialised value(s) at 0x4CA8E97: getSubScope (mal_module.c:1152)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

순번	결함 요약	결함 설명
		by 0x4CAB1EA: insertSymbol (mal_module.c:234) by 0x4CAE847: parseCommandPattern (mal_parser.c:1400) by 0x4CB0D0A: parseMAL (mal_parser.c:1945) by 0x4C9A0A2: malInclude (mal_import.c:203) by 0x4CB13D9: parseMAL (mal_parser.c:1108) by 0x4C9A0A2: malInclude (mal_import.c:203) by 0x4CBD7E5: malBootstrap (mal_session.c:73) by 0x4C8B464: mal_init (mal.c:253) by 0x406BFD: main (mserver5.c:650) Syscall param write(buf) points to uninitialised byte(s) at 0x3AE780E7CD: ??? (in /lib64/libpthread-2.12.so) by 0x54F017F: GDKsave (gdk_storage.c:413) by 0x542ED33: HEAPsave_intern (gdk_heap.c:835) by 0x54F2454: BATsave (gdk_storage.c:724) by 0x5428F74: BBPsync (gdk_bbp.c:3762) by 0x541CA64: TMsubcommit_list (gdk_tm.c:198) by 0x550ECDD: bm_subcommit (gdk_logger.c:1295) by 0x5513D0E: logger_new (gdk_logger.c:1636) by 0x5514A65: logger_create (gdk_logger.c:1872) by 0x10CC6B05: bl_create (bat_logger.c:258) by 0x10CBC381: store_init (store.c:1848) by 0x10C58339: mvc_init (sql_mvc.c:51) Address 0xe8f9fd1 is 8,257 bytes inside a block of size 20,504 alloc'd at 0x4A06C20: realloc (vg_replace_malloc.c:662) by 0x54AE5CC: GDKreallocmax (gdk_utils.c:864) by 0x5430279: HEAPextend (gdk_heap.c:302) by 0x54AF55A: strPut (gdk_atoms.c:1147) by 0x5505A03: BUNappend (gdk_bat.c:1438) by 0x54AC1F1: xdb_load_catalog (gdk_utils.c:1230) by 0x5513FE5: logger_new (gdk_logger.c:1561) by 0x5514A65: logger_create (gdk_logger.c:1872) by 0x10CC6B05: bl_create (bat_logger.c:258) by 0x10CBC381: store_init (store.c:1848) by 0x10C58339: mvc_init (sql_mvc.c:51) by 0x10BE1654: SQLprelude (sql_scenario.c:374) at 0x10C68535: table_ref (rel_select.c:2152) by 0x10C6CD77: query_exp_optname (rel_select.c:6863) by 0x10C68586: table_ref (rel_select.c:2257) by 0x10C6790D: rel_subquery (rel_select.c:6496) by 0x10C6C956: rel_selects (rel_select.c:6931) by 0x10BCDB8D: sql_symbol2relation (sql.c:155) by 0x10BE5F44: SQLparser (sql_scenario.c:3617) by 0x4CBC259: runScenarioBody (mal_scenario.c:564) by 0x4CBC3BE: runScenario (mal_scenario.c:597) by 0x4CBCC4B: MSserveClient (mal_session.c:671) by 0x550AF8A: thread_starter (gdk_system.c:803) by 0x3AE7807AA0: start_thread (in /lib64/libpthread-2.12.so)

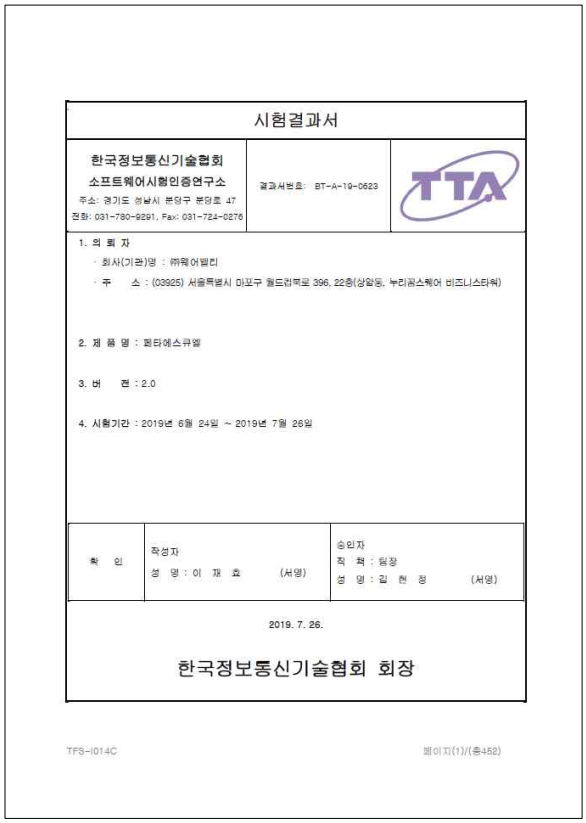
[표 21] 품질 검증 자동화 프레임워크를 이용한 시스템 테스트 주요 결함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(2) 품질 개선을 위한 컨설팅 보고서 제공

- 품질 검증 자동화 프레임워크를 이용한 빅데이터 솔루션 품질 개선을 위한 컨설팅 보고서를 작성하여 주관기관인 ㈜웨어밸리에 제공함

< 컨설팅 보고서 주요 내용>



시험결과서

한국정보통신기술협회 소프트웨어시험인증연구소 주소: 경기도 성남시 분당구 분당로 47 전화: 031-780-9291, Fax: 031-724-0276	결과서번호: BT-A-19-0623	TTA

1. 의 뢰 자
 · 회사(기관)명 : ㈜웨어밸리
 · 주 소 : (03925) 서울특별시 마포구 월드컵북로 396, 22층(상암동, 누리꿈스퀘어 비즈니스타워)

2. 제 품 명 : 페타에스큐엘

3. 버 전 : 2.0

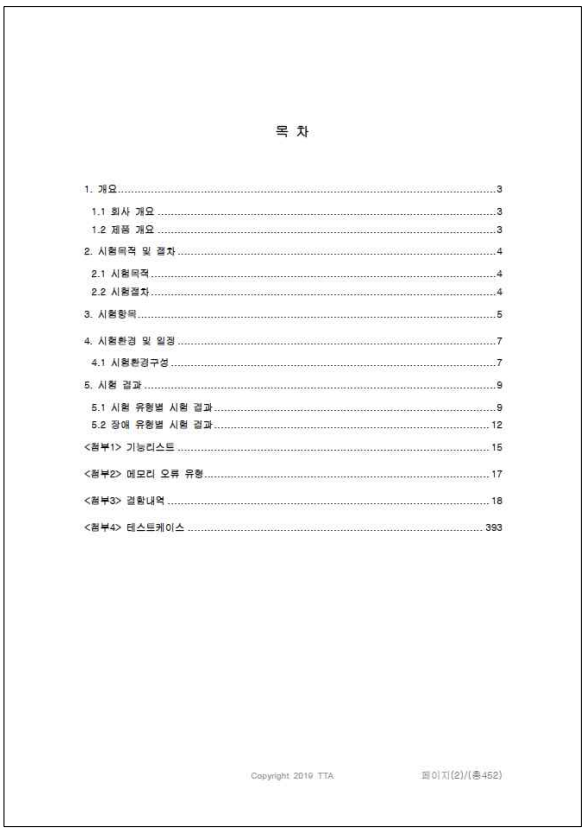
4. 시험기간 : 2019년 6월 24일 ~ 2019년 7월 26일

확 인	작성자 성 명 : 이 재 효 (서명)	승인자 직 책 : 팀장 성 명 : 김 현 정 (서명)

2019. 7. 26.

한국정보통신기술협회 회장

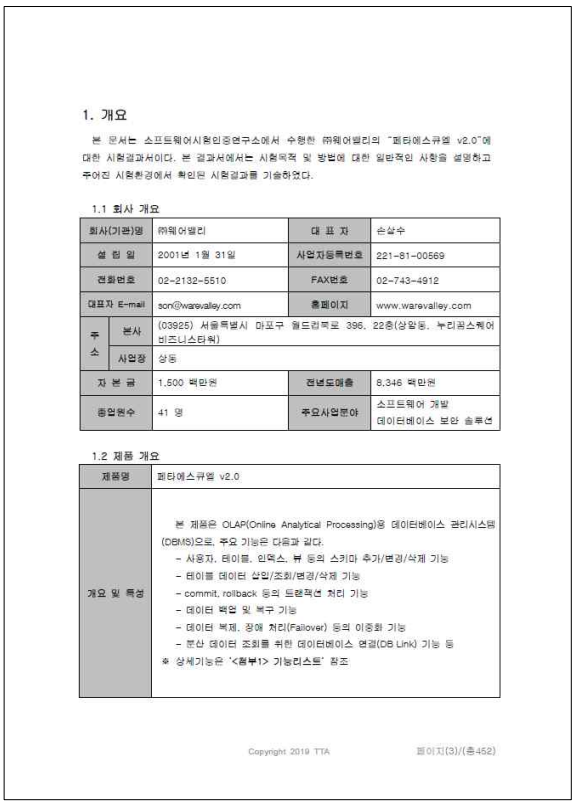
TFS-I014C 페이지(1)/(총452)



목 차

1. 개요 3
1.1 회사 개요 3
1.2 제품 개요 3
2. 시험목적 및 절차 4
2.1 시험목적 4
2.2 시험절차 4
3. 시험항목 5
4. 시험환경 및 일정 7
4.1 시험환경구성 7
5. 시험 결과 9
5.1 시험 유형별 시험 결과 9
5.2 장애 유형별 시험 결과 12
<첨부1> 기능리스트 15
<첨부2> 메모리 오류 유형 17
<첨부3> 결함내역 18
<첨부4> 테스트케이스 393

Copyright 2019 TTA 페이지(2)/(총452)



1. 개요

본 문서는 소프트웨어시험인증연구소에서 수행한 ㈜웨어밸리의 "페타에스큐엘 v2.0"에 대한 시험결과서이다. 본 결과서에서는 시험목적 및 방법에 대한 일반적인 사항을 설명하고 주어진 시험환경에서 확인된 시험결과를 기술하였다.

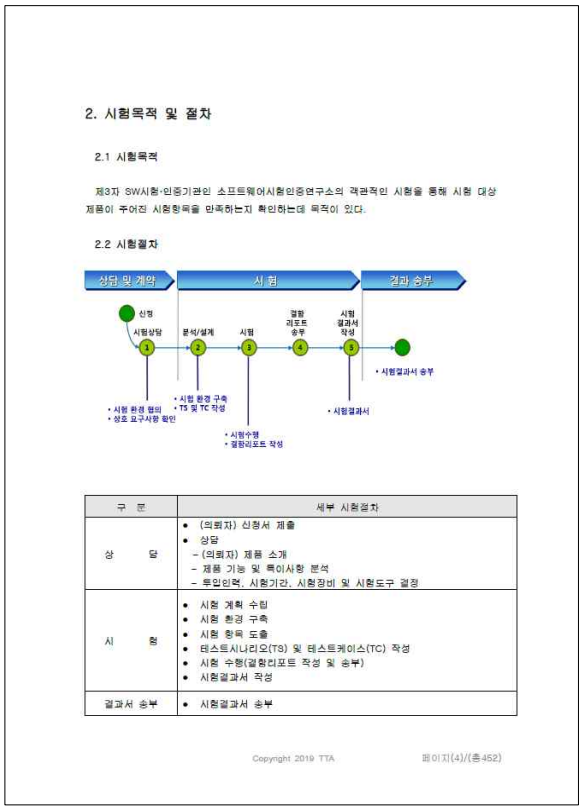
1.1 회사 개요

회사(기관)명	㈜웨어밸리	대 표 자	손삼수
설 립 일	2001년 1월 31일	사업자등록번호	221-81-00569
전화번호	02-2132-5510	FAX번호	02-743-4912
대표자 E-mail	son@warevalley.com	홈페이지	www.warevalley.com
주소 본사	(03925) 서울특별시 마포구 월드컵북로 396, 22층(상암동, 누리꿈스퀘어 비즈니스타워)		
주소 사업장	상동		
자 본 금	1,500 백만원	전년도매출	8,346 백만원
종업원수	41 명	주요사업분야	소프트웨어 개발 데이터베이스 보안 솔루션

1.2 제품 개요

제품명	페타에스큐엘 v2.0
개요 및 특성	본 제품은 OLAP(Online Analytical Processing)용 데이터베이스 관리시스템(DBMS)으로, 주요 기능은 다음과 같다. - 사용자, 테이블, 인덱스, 뷰 등의 스키마 추가/변경/삭제 기능 - 테이블 데이터 삽입/조회/변경/삭제 기능 - commit, rollback 등의 트랜잭션 처리 기능 - 데이터 백업 및 복구 기능 - 데이터 복제, 장애 처리(Failover) 등의 이중화 기능 - 분산 데이터 조회를 위한 데이터베이스 연결(DB Link) 기능 등 ※ 상세기능은 '<첨부1> 기능리스트' 참조

Copyright 2019 TTA 페이지(3)/(총452)



2. 시험목적 및 절차

2.1 시험목적

제3자 SW시험·인증기관인 소프트웨어시험인증연구소의 객관적인 시험을 통해 시험 대상 제품이 주어진 시험항목을 만족하는지 확인하는데 목적이 있다.

2.2 시험절차

구 분	세부 시험절차
상 담	• (의뢰자) 신청서 제출 • 상담 – (의뢰자) 제품 소개 – 제품 기능 및 특이사항 분석 – 투입인력, 시험기간, 시험장비 및 시험도구 결정
시 험	• 시험 계획 수립 • 시험 환경 구축 • 시험 항목 도출 • 테스트시나리오(TS) 및 테스트케이스(TC) 작성 • 시험 수행(결함리포트 작성 및 송부) • 시험결과서 작성
결과서 송부	• 시험결과서 송부

Copyright 2019 TTA 페이지(4)/(총452)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 시험항목

'빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 v1.0'을 실행하여 시험 대상 제품인 '페타에스큐엘 v2.0'을 시험하였으며 시험 항목은 다음과 같다.

순번	시험 항목	세부 시험 항목	설명
1	SQL 쿼리문	SQL (Non-Partitioned Table)	Non-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
2		SQL (Hash-Partitioned Table)	Hash-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
3		SQL (List-Partitioned Table)	List-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
4		SQL (Range-Partitioned Table)	Range-Partitioned Table 에서의 SQL 쿼리문 (DML, DDL, DCL) 기능 시험
5		SQL (Temp Table)	Temp Table 에서의 SQL 쿼리문(DML, DDL, DCL) 기능 시험
6		트랜잭션	트랜잭션(Read Committed, Serialization) 기능 시험
7		사용자 및 권한	사용자 및 권한 기능 시험
8		뷰	뷰 기능 시험
9		내장함수	내장함수(AVG, SUM, COUNT 등) 기능 시험
10		함수	사용자 정의 함수 기능 시험
11		프로시저	프로시저 기능 시험
12		데이터 타입	char, int, numeric 등의 데이터 타입 기능 시험
13		인덱스	인덱스 기능 시험
14		옵티마이저	옵티마이저(Explain, Plan) 기능 시험

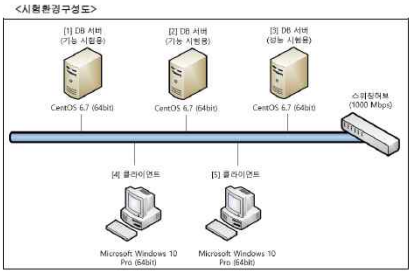
순번	시험 항목	세부 시험 항목	설명
15	이중화	-	이중화 기능 시험
16	DB Link	-	DB Link 기능 시험
17	인터페이스	ODBC	ODBC API 시험
18		JDBC	JDBC API 시험
19	유틸리티	-	유틸리티 기능 시험
20	자원 한계 상황	-	자원(스토리지 등)이 부족한 상황에서의 기능 시험
21	스트레스	-	다수 클라이언트 환경에서의 트랜잭션 부하 시험
22	리커버리	-	백업 및 복구 시험
23	성능	-	TPC-H 100SF 성능 시험
24	SQL92	-	SQL92 표준 쿼리문 기능 시험
25	TPC-H	-	TPC-H 시험
26	TPC-DS	-	TPC-DS 시험
27	메모리 관리	-	메모리 오류(메모리 누수, 초기화 되지 않은 변수 사용 등) 검출 시험

※ 각 시험항목에 대한 테스트케이스는 '<붙부4> 테스트케이스' 참조

4. 시험환경 및 일정

4.1 시험환경구성

<시험환경구성도>



<DB 서버>
- [1]~[3]번 DB 서버에 설치한 프로그램
  - 시험 대상 제품(서버 모듈)
  - JAVA: OpenJDK 11.0.3
  - 메모리 누수 검출 도구: Valgrind 3.8.0
  - 컴파일러: gcc 4.4.7
  - ODBC: unixODBC 2.2.14

<클라이언트>
- [4]~[5]번 클라이언트에 설치한 프로그램
  - 시험 대상 제품(클라이언트 모듈)
  - 터미널 에뮬레이터: putty 0.72

<품질 검증 자동화 도구>
- 빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 v1.0 (D-TAF v1.0)
  - [1]~[3]번 DB서버에 설치함

<네트워크>
- 1000 Mbps 스위칭 허브를 사용함

<세부사양>

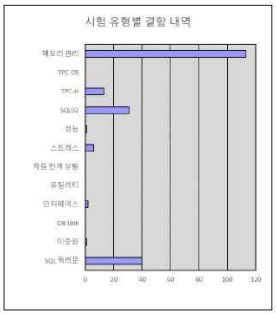
No	Role	OS	CPU	Mem	Storage	Pre-Requisite
1	DB 서버	CentOS 6.7 (64bit)	Intel® Xeon™ E5-2620@ 2.10 GHz	64GB	1TB	- OpenJDK 11.0.3 - Valgrind 3.8.0 - gcc 4.4.7 - unixODBC 2.2.14
2	DB 서버	CentOS 6.7 (64bit)	Intel® Xeon™ E5-2650@ 2.10 GHz	64GB	1TB	- OpenJDK 11.0.3 - Valgrind 3.8.0 - gcc 4.4.7 - unixODBC 2.2.14
3	DB 서버	CentOS 6.7 (64bit)	Intel® Xeon™ E7-8860@ 2.19 GHz	256GB	1TB	- OpenJDK 11.0.3 - Valgrind 3.8.0 - gcc 4.4.7 - unixODBC 2.2.14
4	클라이언트	Microsoft Windows 10 Pro (64bit)	Intel® Core™ i7-3770 CPU @ 3.40GHz	8GB	1TB	- putty 0.72
5	클라이언트	Microsoft Windows 10 Pro (64bit)	Intel® Core™ i7-3770 CPU @ 3.40GHz	8GB	1TB	- putty 0.72

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

5. 시험 결과

5.1 시험 유형별 시험 결과

시험 유형	결함수
SQL 쿼리문	40
이중화	1
DB Link	0
인터페이스	2
유틸리티	0
자원 한계 상황	0
스트레스	6
리커버리	0
성능	1
SQL92	31
TPC-H	13
TPC-DS	0
메모리 관리	113
계	207



< SQL >
- Non-Partitioned 테이블에 대한 SQL쿼리문 실행은 전반적으로 정상 동작하지만, Partitioned 테이블의 경우 데이터 불일치 비정상 종료 등의 결함이 다수 발생되어 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 이중화 >
- 이중화를 설정한 상태에서 DDL, DCL 쿼리문 실행은 전반적으로 정상 동작하지만, Master 서버에서 데이터 업데이트 시, Slave 서버에 업데이트 결과가 반영되지 않는 결함이 발생하여 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< DB Link>
- DB Link 설정, 원격 데이터 조회 등의 DB Link 기능이 정상 동작함

< 인터페이스 >
- 기능 전반적으로 정상 동작하지만, ODBC의 경우 일부 API의 SQLSTATE 정보에 오류가 발생하여 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 유틸리티 >
- PetaSQL에서 제공하는 유틸리티(petadump, petasql 등)의 기능이 정상 동작함

< 자원 한계 상황 >
- 스토리지 용량이 부족한 상황에서 DDL 및 DML 쿼리문 실행 시, '스토리지 용량 부족'에 대한 에러 메시지를 제공함

< 스트레스 >
- 100개 이상의 클라이언트가 동시에 DDL 및 DML 쿼리문 실행 시, 해당 기능이 정상 동작함

< 성능 >
- TPC-H 데이터(100GB) 산입 후, TPC-H 5번 쿼리문 실행 시, 산입된 데이터 스토리지 용량 보다 10배 이상의 스토리지 용량(1TB)이 비정상적으로 요구되는 성능 오류가 발생하여 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< SQL92 >
- 전반적으로 SQL92 표준에서 제공하는 쿼리문이 정상적으로 동작하지만, 일부 쿼리문 실행 시, 서버가 비정상 종료하거나, 무결성 오류 등의 결함이 발생하여 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< TPC-H >
- 전반적으로 TPC-H에서 제공하는 쿼리문이 정상적으로 동작하지만, 4개 이상의 클라이언트가 TPC-H 쿼리를 동시에 실행 시, 서버가 비정상 종료하는 등의 결함이 발생하여 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

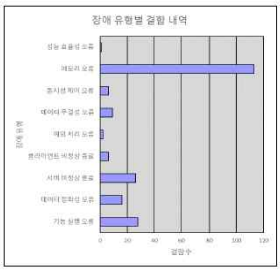
< TPC-DS >
- TPC-DS에서 제공하는 쿼리문이 정상적으로 동작함

< 메모리 관리 >
- 초기화 되지 않은 변수 사용, 잘못된 메모리 참조 등의 메모리 오류가 다수 발생하여 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

5.2 장애 유형별 시험 결과

장애 유형	결함수
기능 실행 오류	28
데이터 정확성 오류	16
서버 비정상 종료	26
클라이언트 비정상 종료	6
예외 처리 오류	2
데이터 무결성 오류	9
동시성 제어 오류	6
메모리 오류	113
성능 효율성 오류	1
계	207



< 기능 실행 오류 >
- 다음과 같은 기능 실행 오류가 발생하여 보완이 요구됨
  - sequence를 갖는 temp table 삭제 시, 해당 sequence가 삭제되지 않음
  - SQL92 'TIDES' 사용자 계정의 데이터 산입 시, 오류 메시지(TypeException:user.a_1[12]:' calc.date' undefined in: _29:any := calc.date(_28:ing) program contains errors)가 발생하고 데이터 산입에 실패함
  - '_'를 이용한 like 기능이 동작하지 않음 등

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 데이터 정확성 오류 >
- 다음과 같은 기능 실행 오류가 발생하여 보완이 요구됨
  - NATURAL JOIN SELECT 결과가 잘못 출력됨
  - DROP 쿼리문에 대한 PLAN 실행 시, PLAN 값이 잘못 출력됨
  - INNER JOIN SELECT 쿼리문 결과가 잘못 출력됨 등

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

< 서버 비정상 종료 >

- 다음과 같은 서버 비정상 종료가 발생하여 보완이 요구됨
  - IN 구문이 포함된 SELECT 쿼리문 실행 시, 서버가 비정상 종료함
  - UNION JOIN 구문이 포함된 SELECT 쿼리문 실행 시, 서버가 비정상 종료함
  - Partition 테이블에 index 생성 시, 서버 비정상 종료 및 서버가 재구동 되지 않음
  - DROP USER 쿼리문 실행 시, 오류메시지(GDKerror:!ERROR: BATdel: BUN does not occur.)가 출력되고, 서버 비정상 종료함
  - TPC-H Throughtput 실행 시, 서버가 비정상 종료함
  - 파티션 테이블을 생성하여 TPC-H 데이터 삽입 후 TPC-H 2번 쿼리 실행 시, 서버가 비정상 종료함
  - autocommit(off) 상태에서 UNION JOIN 구문이 포함된 SELECT 쿼리문 실행 시, 서버가 비정상 종료함 등

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 클라이언트 비정상 종료 >

- 다음과 같은 클라이언트 비정상 종료가 발생하여 보완이 요구됨
  - 특정 DELETE 쿼리문 실행 시, 클라이언트가 비정상 종료함
  - 특정 UPDATE 쿼리문 실행 시, 클라이언트가 비정상 종료함 등

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 예외 처리 오류 >

- 다음과 같은 예외 처리 오류가 발생하여 보완이 요구됨
  - REAL 타입의 범위를 벗어나는 값(예: 9,999,999,999) 입력이 가능함
  - 시퀀스 옵션(MAXVALUE, MINVALUE 등)의 타입을 INTEGER로 설정할 경우, INTEGER 범위를 벗어나는 값(예: 9,999,999,999) 입력이 가능함

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 데이터 무결성 오류 >

- 다음과 같은 데이터 무결성 오류가 발생하여 보완이 요구됨
  - SQL92의 테스트케이스(cdr009.py, cdr011.py, cdr015.py 등) 실행 시, 외래키 제약조건으로 인해 업데이트에 실패 해야 하지만, 오류메시지가 발생하고 업데이트가 가능함
  - 컬럼 제약사항이 not null 인경우 null 값 삽입에 실패해야 하지만, null 값 삽입이 가능함 등

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 동시성 제어 오류 >

- 다음과 같은 동시성 제어 오류가 발생하여 보완이 요구됨
  - 100개의 클라이언트가 동시에 INSERT 쿼리문 실행 시, 클라이언트가 비정상 종료함
  - 100개의 클라이언트가 ALTER 쿼리문 실행 시, 클라이언트가 비정상 종료함
  - 100개의 클라이언트가 DELETE 쿼리문 실행 시, 클라이언트가 비정상 종료함 등

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 메모리 오류 >

- 다음과 같은 메모리 오류가 발생하여 보완이 요구됨
  - 파티션 테이블에 대한 DML 실행 시, 메모리 오류가 발생함
  - 인덱스 생성 시, 메모리 오류가 발생함
  - DELETE 쿼리문 실행 시, 메모리 오류가 발생함
  - TPC-DS 쿼리문 실행 시, 메모리 오류가 발생함
  - TPC-H 데이터 삽입 시, 메모리 오류가 발생함
  - 100개의 클라이언트가 동시에 CREATE 및 DROP 쿼리문 실행 시, 메모리 오류가 발생함 등

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

< 성능 효율성 오류 >

- TPC-H 데이터(100GB) 삽입 후, TPC-H 5번 쿼리문 실행 시, 삽입된 데이터 스토리지 용량보다 10배 이상의 스토리지 용량(1TB)이 비정상적으로 요구되는 성능 효율성 오류가 발생하여 보완이 요구됨

※ 세부 결함내역은 '<첨부3> 결함내역' 참조

<첨부1> 기능리스트

대분류	중분류	소분류	기능설명
클라이언트	-	-	데이터베이스에 접속하여 SQL문을 실행하는 기능
데이터베이스 관리	데이터베이스	create/start/stop/destroy	데이터베이스 생성/구동/중지/삭제하는 기능
데이터 정의 언어(DDL)	사용자	create/alter/drop	사용자/테이블/인덱스(btree, hash)/뷰/제약조건에 대한 DDL 문을 생성/변경/삭제하는 기능 ※ DDL : Data Definition Language
	테이블		
	인덱스		
	뷰		
	제약조건		
	권한	grant/revoke	select, insert, update 등에 대한 SQL 실행 권한을 사용자에게 부여/철회하는 기능
데이터 조작 언어(DML)	테이블	insert/select/update/delete	데이터를 삽입/조회/변경/삭제하는 기능 ※ DML : Data Manipulation Language
데이터 제어 언어(DCL)	트랜잭션	read commited/serializable	트랜잭션 격리수준을 설정하는 기능
		commit/rollback	DML 실행을 통해 조작된 데이터를 데이터베이스에 반영하거나, DML 실행 이전 상태로 되돌리는 기능 ※ DCL : Data Control Language
저장 프로시저	프로시저	create/call/drop	프로시저 및 함수를 생성/실행/삭제하는 기능
	함수		
내장 함수	통계 함수	avg, count, max, min, sum 등 10개	데이터를 집계(avg, count, max, min, sum 등)하는 함수
	수치 연산 함수	sign, abs, log, cos, sqrt 등 24개	데이터를 수치 연산(sign, abs, log, cos, sqrt 등) 하는 함수
	분석 함수	rank over, median, dense_rank over 등 4개	데이터를 분석(rank_over, median, dense_rank over 등)하는 함수
	문자열 함수	upper, lower, length, lpad, rpad 등 29개	문자열을 처리(upper, lower, length, lpad, rpad 등)하는 함수
	URL 주소 처리 함수	getbasename, getcontent 등 12개	URL 주소를 처리(getbasename, getcontent 등)하는 함수
	날짜 및 시간 함수	sysdate, week 등 16개	날짜 및 시간을 처리(sysdate, week, local_timezone 등)하는 함수

대분류	중분류	소분류	기능설명
내장 함수	변환 함수	date_to_str, str_to_date 등 5개	데이터 타입을 변환(date_to_str, str_to_date 등)하는 함수
	system 함수	storage, clientid, clients, sessions 등 8개	데이터베이스의 정보(storage, clientid, clients, sessions 등)를 출력하는 함수
백업 및 복구 관리	백업	-	데이터를 백업하는 기능
	복구	-	데이터를 복구하는 기능
감사 기록	-	-	데이터베이스 구동시작, 내부 오류 등의 정보를 로그 파일로 기록하는 기능
이중화 (Replication)	복제	-	데이터를 동종 제품의 데이터베이스에 복제하는 기능
	생성/구동/중지/삭제		복제 서비스를 생성/구동/중지/삭제하는 기능
DB Link	조회	-	분산된 데이터를 조회하기 위해 동종 데이터베이스를 연결하는 기능
	생성/구동/삭제	-	DB Link를 생성/구동/삭제하는 기능
인터페이스	JDBC	-	자바(JAVA)에서 시험대상제품에 접속할 수 있도록 지원하는 API ※ JDBC: Java Database Connectivity
	ODBC	-	ODBC 드라이버를 통해 시험대상제품에 접속할 수 있도록 지원하는 API ※ ODBC: Open Database Connectivity

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<첨부2> 메모리 오류 유형

메모리 오류 유형	설명
Conditional jump or move depends on uninitialized value(s)	초기화 되지 않은 변수 사용시 발생하는 메모리 오류
Syscall param func() points to uninitialised byte(s)	초기화 되지 않은 변수(또는 구조체)를 함수의 파라미터로 사용하여 실행 시 발생하는 메모리 오류
Blocks are indirectly lost in loss record	메모리 누수가 발생하여, 해당 메모리 참조 또는 해제를 할 수 없는 메모리 오류
Blocks are definitely lost in loss record	
Blocks are possibly lost in loss record	
Blocks are still reachable in loss record	
Invalid read of size	잘못된 메모리를 참조하려고 할 때 발생하는 메모리 오류
Invalid write of size	잘못된 메모리를 대상으로 쓰기연산 시 발생하는 메모리 오류
Jump to the invalid address stated on the next line	유효하지 않은 상태(corruption)메모리를 참조하려고 할 때 발생하는 메모리 오류
Invalid free	잘못된 메모리를 대상으로 메모리 해제 시도 시 발생하는 메모리 오류
Mismatched free	잘못된 방법으로 메모리를 해제할 때 발생하는 메모리 오류
Argument 'size' of function malloc has a fishy	malloc 함수의 파라미터 값이 음수일 경우 발생하는 메모리 오류
Source and dest ination overlap in memcpy	memcpy 함수 실행 시, 소스(source)와 타겟(target) 메모리 주소가 overlap 될때 발생하는 메모리 오류

<첨부3> 결함내역

- 최종 결함내역 -

순번	시험환경 OS	결함요약	결함 정도	결함 설명
1	시험환경 모든 OS	SET DEFAULT 액션 기능 오류	H	DB에만 삭제시 디폴트 값으로 넣어야 하지만 기능이 정상적으로 동작하지 않음 (테스트케이스) $TSR_HOME/testsuite/Function/Table/BasicTable/TableConstraint/ReferenceOptions/reference_on_delete_set_default.py
2	시험환경 모든OS	SET DEFAULT 액션 기능 오류	H	디폴트 값으로 값이 set 되어 하지만 기능이 정상적으로 동작하지 않음 (테스트케이스) $TSR_HOME/testsuite/Function/Table/BasicTable/TableConstraint/ReferenceOptions/reference_on_update_set_default.py
3	시험환경 모든OS	RESTRICT 액션 기능 오류	H	참조 키를 현재 테이블에서 참조하고 있을 경우, 변경이 실행되어 하지만 기능이 정상적으로 동작하지 않음 (테스트케이스) $TSR_HOME/testsuite/Function/Table/BasicTable/TableConstraint/ReferenceOptions/reference_on_update_restrict.py
4	시험환경 모든OS	SET NULL 액션 기능 오류	H	참조 제약사항이 not null 인경우 null 값이 반영되지 않아야 하지만 기능이 정상적으로 동작하지 않음 (테스트케이스) $TSR_HOME/testsuite/Function/Table/BasicTable/TableConstraint/ReferenceOptions/reference_on_update_set_null.py
5	시험환경 모든OS	Natural Join 기능 오류	H	NATURAL JOIN SELECT 결과가 잘못 출력됨 (테스트케이스) $TSR_HOME/testsuite/Function/Table/PartitionTable/RangePartition/SelectTable/Join/rangePartition_naturalJoin.py
6	시험환경 모든OS	Inner Join 기능 오류	H	INNER JOIN SELECT 결과가 잘못 출력됨 (테스트케이스) $TSR_HOME/testsuite/Function/Table/PartitionTable/RangePartition/SelectTable/Join/rangePartition_innerJoin.py
7	시험환경 모든OS	Natural Join 기능 오류	H	NATURAL JOIN 결과가 잘못 출력됨 (테스트케이스) $TSR_HOME/testsuite/Function/Table/PartitionTable/ListPartition/SelectTable/Join/listPartition_naturalJoin.py

순번	시험환경 OS	결함요약	결함 정도	결함 설명
				by 0x407AA5: logListener (merovingian.c:200) by 0x3AE7807AA0: start_thread (in /lib64/libpthread-2.12.so) by 0x3AE74EBC4C: clone (in /lib64/libc-2.12.so) Address 0x683f561 is 1 byte inside a block of size 7 free'd at 0x4A05430: free (vg_replace_malloc.c:446) by 0x407A9E: logListener (merovingian.c:199) by 0x3AE7807AA0: start_thread (in /lib64/libpthread-2.12.so) by 0x3AE74EBC4C: clone (in /lib64/libc-2.12.so) Invalid read of size 1 at 0x4A08D21: memcpy (mc_replace_strmem.c:882) by 0x407AA5: logListener (merovingian.c:200) by 0x3AE7807AA0: start_thread (in /lib64/libpthread-2.12.so) by 0x3AE74EBC4C: clone (in /lib64/libc-2.12.so) Address 0x6835f56 is 6 bytes inside a block of size 7 free'd at 0x4A05430: free (vg_replace_malloc.c:446) by 0x407A9E: logListener (merovingian.c:199) by 0x3AE7807AA0: start_thread (in /lib64/libpthread-2.12.so) by 0x3AE74EBC4C: clone (in /lib64/libc-2.12.so)
204	시험환경 모든OS	메모리 오류	H	ster_stress 테스트 수행 시, 메모리 누수(10건)가 발생함 [메모리 누수 유형] - Conditional jump or move depends on uninitialised value(s) - Syscall param write(buf) points to uninitialised byte(s) - Invalid read of size 1 - Syscall param mq=nc(start) points to uninitialised byte(s) (테스트케이스) $TSR_HOME/testsuite/Stress/ster_stress.py (메모리 오류 로그) Conditional jump or move depends on uninitialised value(s) at 0x4CA8E70: getSubScope (maL_module.c:1168) by 0x4CAB1EA: insertSymbol (maL_module.c:234) by 0x4CAEB47: parseCommandPattern (maL_parser.c:1400) by 0x4CB0D0A: parseMAL (maL_parser.c:1945) by 0x4C9A0A2: malnclude (maL_import.c:203) by 0x4CB13D9: parseMAL (maL_parser.c:1108) by 0x4C9A0A2: malnclude (maL_import.c:203) by 0x4CBD7E5: malBootstrap (maL_session.c:73) by 0x4CB8A64: maLint (mal.c:253) by 0x406BFD: main (mserver5.c:650) Conditional jump or move depends on uninitialised value(s) at 0x4CA8697: getSubScope (maL_module.c:1152) by 0x4CAB1EA: insertSymbol (maL_module.c:234) by 0x4CAEB47: parseCommandPattern (maL_parser.c:1400) by 0x4CB0D0A: parseMAL (maL_parser.c:1945) by 0x4C9A0A2: malnclude (maL_import.c:203) by 0x4CB13D9: parseMAL (maL_parser.c:1108) by 0x4C9A0A2: malnclude (maL_import.c:203)

<첨부4> 테스트케이스

번호	구분	테스트케이스 스크립트	테스트 시나리오
1	SQL 쿼리문	createTable_basic.py	테이블이 생성되는지 확인
2		createTable_manyColumn.py	2000 개 컬럼을 포함하는 테이블이 생성되는지 확인
3		createTable_manyTypes.py	다수 타입(25 개)를 포함하는 테이블이 생성되는지 확인
4		insertTable_basic.py	테이블에 1 만건의 데이터가 삽입되는지 확인
5		selectTable_basic.py	테이블의 데이터가 조회되는지 확인
6		projection_manyProjection.py	50 개 이상의 projection 결과가 조회되는지 확인
7		projection_BigDataProjection.py	10000 바이트 이상의 긴 값이를 가지는 결과가 조회되는지 확인
8		projection_manyRows.py	1000 개 이상의 결과가 조회되는지 확인
9		projection_inlineView.py	인라인 뷰를 이용하여 결과가 조회되는지 확인
10		projection_alias.py	alias 를 이용하여 결과가 조회되는지 확인
11		projection_nested_function.py	nested(예: sum(avg(c1)) 되는 내장 함수를 이용하여 결과가 조회되는지 확인
12		predicate_null.py	조건절에 IS NULL, IS NOT NULL 를 이용하여 결과가 조회되는지 확인
13		predicate_like.py	조건절에 Like 를 이용하여 결과가 조회되는지 확인
14		predicate_comparision.py	조건절에 비교 연산자를 이용하여 결과가 조회되는지 확인
15		pridicate_builin_function.py	조건절에 내장 함수를 이용하여 결과가 조회되는지 확인
16		subquery_in.py	IN 구문을 이용하여 결과가 조회되는지 확인
17		subquery_exists.py	EXIST 구문을 이용하여 결과가 조회되는지 확인
18		subquery_any_some.py	ANY, SOME 구문을 이용하여 결과가 조회되는지 확인
19		subquery_all.py	ALL 구문을 이용하여 결과가 조회되는지 확인
20		subquery_many.py	2 개 이상의 서브쿼리를 이용하여 결과가 조회되는지 확인
21		basicJoin.py	JOIN 를 이용하여 결과가 조회되는지 확인
22		crossJoin.py	CROSS JOIN 를 이용하여 결과가 조회되는지 확인
23		naturalJoin.py	NATURAL JOIN 를 이용하여 결과가 조회되는지 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 2 절 연구개발 추진 체계(각 기관/기업별 역할 및 추진 내역)

1. 기관별 역할

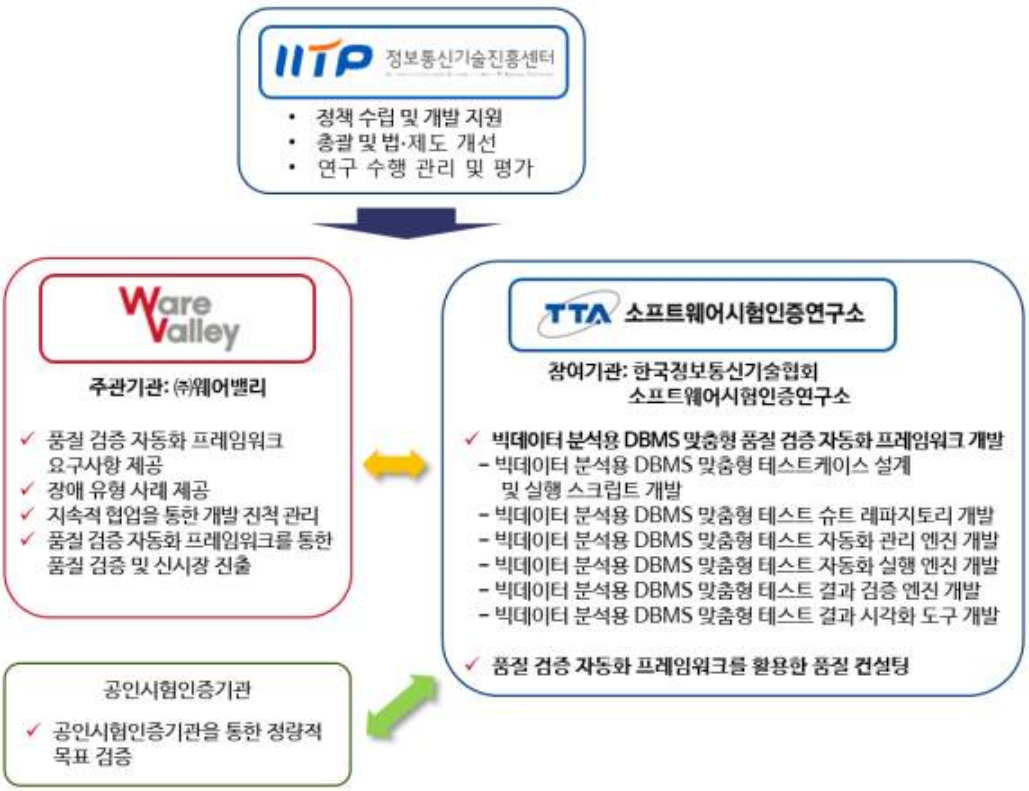
가. ㈜웨어밸리(주관기관)

① 품질 검증 자동화 프레임워크 요구사항 제공
② 장애 유형 사례 제공
③ 지속적 협업을 통한 갭랍 진척 관리
④ 품질 검증 자동화 프레임워크를 통한 품질 검증 및 신시장 진출

나. 한국정보통신기술협회(참여기관)

① 빅데이터 분석용 DBMS 맞춤형 품질 검증 자동화 프레임워크 개발
- 빅데이터 분석용 DBMS 맞춤형 테스트케이스 설계 및 설계 스크립트 개발
- 빅데이터 분석용 DBMS 맞춤형 테스트 슈트 레파지토리 개발
- 빅데이터 분석용 DBMS 맞춤형 테스트 자동화 관리 엔진 개발
- 빅데이터 분석용 DBMS 맞춤형 테스트 결과 검증 엔진 개발
- 빅데이터 분석용 DBMS 맞춤형 테스트 결과 시각화 도구 개발

② 품질 검증 자동화 프레임워크를 활용한 컨설팅



[그림 64] 연구개발 추진 체계

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 연구개발 추진내역



[그림 65] 참여기관(한국정보통신기술협회) 연구개발 추진 내역

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 3 절 시장현황 및 사업화 전망

1. 시장 현황

가. 빅데이터 분석 솔루션 시장 현황

(1) 국내 빅데이터 시장 규모

국내 빅데이터 시장은 기업의 빅데이터에 대한 인식 호전과 매출 1,000억 원 이상 중견·대기업의 투자 증가, 정부의 강력한 빅데이터 산업 육성의지에 따라 전년 대비 31.1% 성장한 3,439.6억 원을 기록했다.

특히 정부·공공 투자가 전년 대비 43.1% 성장한 998.6억 원으로 크게 확대되며 시장을 견인했다. 시장 영역별로 정부·공공시장은 빅데이터 산업활성화 법안 등을 기반으로 공공기관에 빅데이터 도입·활용에 대한 적극적인 권고로 인해 43.1% 성장한998.6억 원을 기록했다.

반면 민간시장은 시장 사이클 관점에서 도입 단계에 해당된다. 성장 잠재력은 높은 편이나 아직 금융, 통신 등의 일부 대기업에 한정된수요로 인해 공공시장보다는 낮은 26.8% 성장세로 2,441억 원을 기록했다.

제품별 국내 빅데이터 시장은 서버, 스토리지, 네트워크 등 인프라 투자에 55.7%(1,915.2억 원)가 집중돼 있다. 소프트웨어(23.5%, 809.3억 원), 서비스(20.8%, 715.1억 원) 투자는 선진시장에 비해 비중이 낮은 상황이다. 그러나 그동안 인프라 투자 중심에서 탈피해 빅데이터 분석 시스템 구축이 증가하면서 관련 소프트웨어와 서비스 매출이 각각 34%, 40%로 빠르게 성장하고 있다.

구분	시장 규모(억 원)	비중(%)
서버	736.5	21.4
스토리지	911.3	26.5
네트워크	267.4	7.8
소프트웨어	809.3	23.5
서비스	715.1	20.8
합계	3,439.6	100.0

[그림 66] 국내 빅데이터 시장 규모

공급자 유형별 국내 빅데이터 시장은 국내 SI 기업이 44.5%(1,530.6억 원)로 시장을 선도하는 가운데 해외 IT 서비스 기업 28.1%(966.5억 원), 해외 빅데이터 전문기업 14.9%(512.5억 원), 국

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

내 빅데이터 전문기업 12.5%(430억 원) 순으로 집계됐다. 특히 2018년 빅데이터 시장은 공공기관 중심의 투자가 활발하게 전개돼 공공 소프트웨어 구축사업 중심으로 영업을 강화한 국내 기업의 매출 비중이 2017년 48.4%에서 2018년 57.0%로 확대됐다.

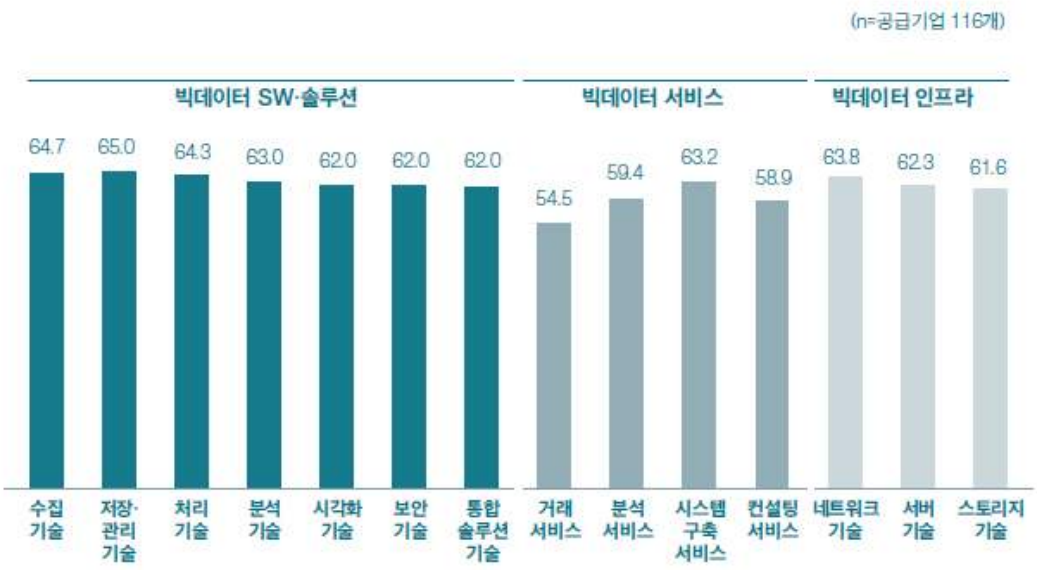
구분	시장 규모(억 원)	비중(%)
국내 SI	1,530.6	44.5
해외 IT 서비스	966.5	28.1
국내 빅데이터 전문기업	430.0	12.5
해외 빅데이터 전문기업	512.5	14.9
합계	3,439.6	100.0

[그림 67] 국내 빅데이터 시장의 기업 유형

(2) 국내외 빅데이터 기술수준 비교

국내 빅데이터 기술수준은 선진국 대비 열세를 면치 못하고 있으나 분야별로 빅데이터 SW·솔루션과 인프라 분야는 평균 이상을 상회하며 상대적으로 경쟁력을 확보하고 있는 것으로 조사됐다.

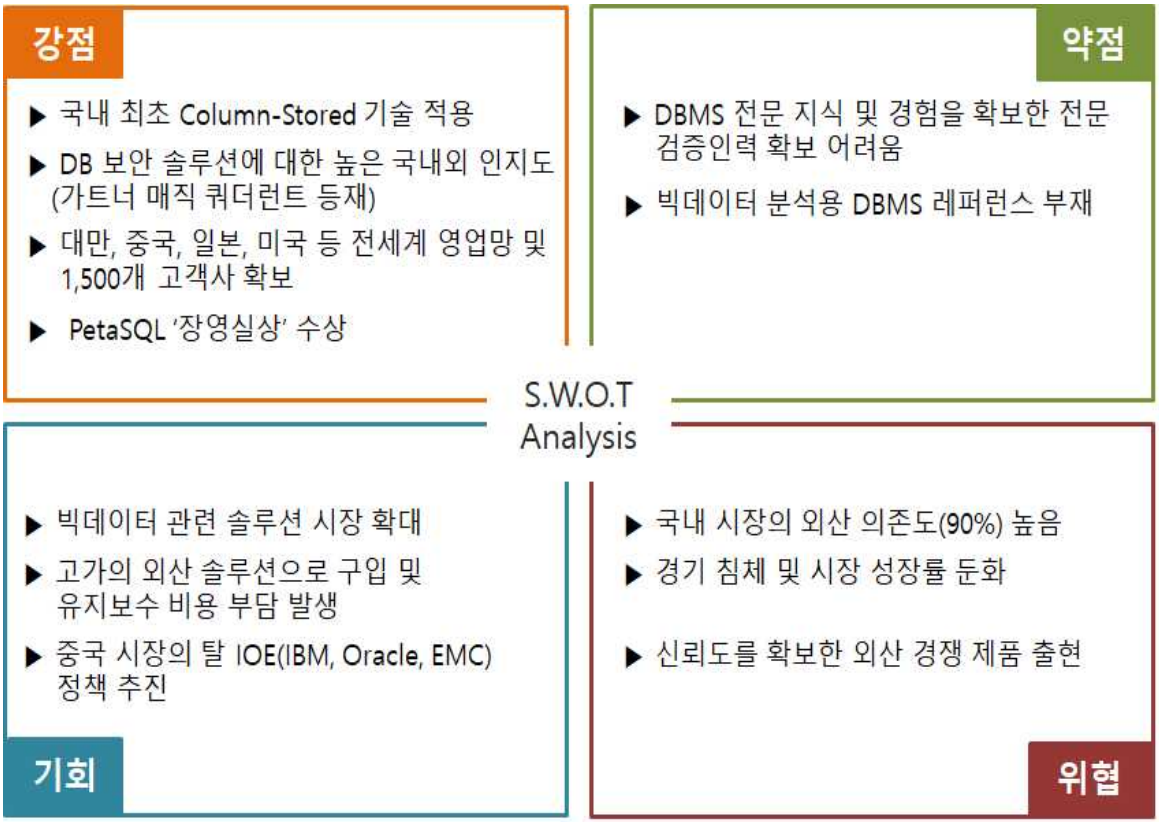
반면 서비스 분야는 여전히 취약한 것으로 나타났다. 즉 빅데이터 저장·관리기술, 수집기술, 처리기술 및 관련 인프라 구축 기술력은 상대적으로 높은 경쟁력을 확보하고 있으나 빅데이터 유통체계 부재와 데이터 기반의 응용 서비스 경험 부족에 따라 거래 서비스, 컨설팅 서비스, 분석 서비스 등의 빅데이터 서비스 분야는 경쟁력이 미흡한 것으로 조사됐다.



[그림 68] 국내 빅데이터 기술 수준

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

나. 사업화를 위한 SWOT 분석



[그림 69] SWOT 분석

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 사업화 전망

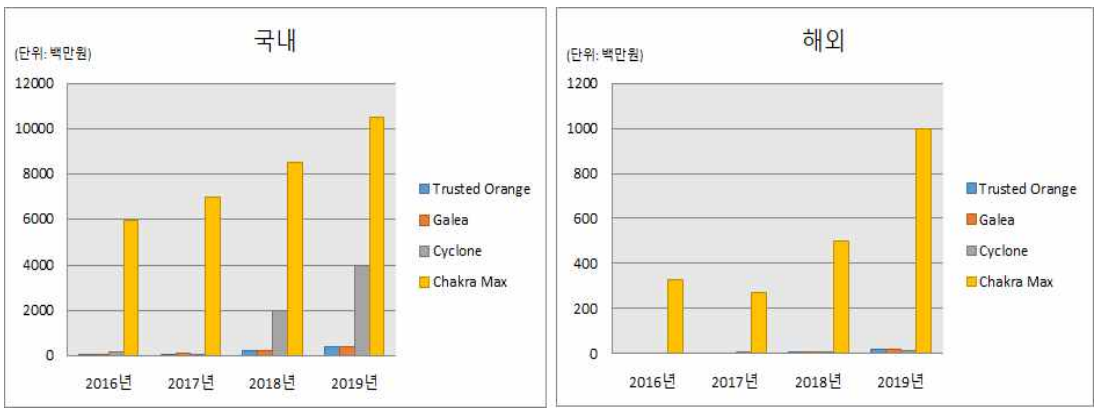
가. 단계별 계획

(1) 1단계 : 레파지토리용 DBMS로 활용하여 레퍼런스 확보

① ㈜웨어밸리는 Chakra Max, Cyclone 등 DB 보안 제품을 국내 뿐만 아니라 해외 시장에 판매하는 기업으로 해당 제품의 레파지토리 DB를 MySQL, 오라클 등에서 PetaSQL 제품으로 전환하여 레퍼런스 확보

DB 솔루션	제품명	설명
데이터 보안 진단	Cyclone	데이터보안진단 / PC, Server, DB 개인정보 Scan / 취약점 분석
DB 접근통제	Chakra Max	Enterprise급 규모의 DB 및 Server 접근 통제/감사
DB 암호화	Galea	Column-Level (Plug-in, API) 방식의 DB 암호화
DB 작업결재	Chakra Max	DB Tool, DB 종류에 상관없는 강력한 DB 작업결재
	TRUSTED Orange	Oracle - Orange 사용자 기반의 강력한 DB 작업결재

[그림 70] ㈜웨어밸리(주관기업) 보안 솔루션



[그림 71] 국내외 매출 현황 및 예상 매출

② 국내 시장에서 MySQL, 오라클 등을 레파지토리 DB로 사용하고 있는 상황으로 MySQL의 안정성의 우려, 오라클의 구입비용 부담 등의 이유로 주관기업의 DBMS로 대체 가능

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(2) 2단계: 빅데이터 분석용 DBMS로 활용

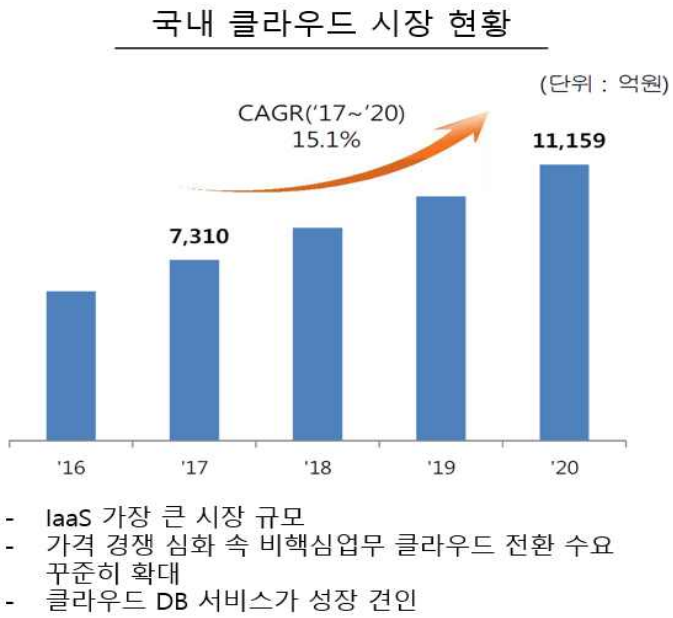
① 대용량 데이터 동기화 및 복제 시스템(Lora)과 연동할 수 있는 빅데이터 분석용 DBMS(PetaSQL)로 판매 추진



[그림 72] Business Intelligence용 PetaSQL

(3) 3단계: 클라우드 기반 빅데이터 분석용 DBMS로 활용

① 클라우드 마켓플레이스 등록 및 CSP(Cloud Service Provider) 제휴를 통해 국내 클라우드 시장으로 빅데이터 분석용 DBMS 판매 영역을 확대하여 DBaaS(Database as a Service) 형태로 제공



[그림 73] 국내 클라우드 시장 현황

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

다. 사업화 전략

(1) 1단계: 레파지토리용 DBMS 판매

① 전세계 1,500사 이상 고객사에 공급되고 있는 DB 보안 솔루션(Chakra Max)의 레파지토리 DB로 사용하여 국내외 시장에 판매. ㈜웨어밸리의 솔루션(Cyclone, Galea, Trusted Orange) 뿐만 아니라 타사의 솔루션의 레파지토리 DB로 판매 확대

(2) 2단계 : 빅데이터 분석용 DBMS 판매

① 탈(脫) IOE(IBM, Oracle, EMC) 정책이 추진중인 중국 시장을 타겟으로 빅데이터 분석 솔루션 판매를 우선적으로 공략하고 중국 시장 레퍼런스 확보를 통해 국내 시장 진출

(3) 3단계: 클라우드 기반 빅데이터 분석용 DBMS

① DB 보안 솔루션을 클라우드 서비스로 제공할 계획으로 해당 솔루션과 연계하여 클라우드 기반 레파지토리 DB로 판매 영역 확대

② 더 나아가 클라우드 기반 빅데이터 분석용 DBMS으로 판매 및 서비스 제공

③ 국내/해외 시장 확보 - 글로벌 SW 공급 총판

- (국내) 국내 협력사와의 협력 및 총판, 60여개 리셀러 활용하여 시장 확대
- (해외) 현지 법인 및 각 나라별 파트너사 활용



[그림 74] ㈜웨어밸리의 주요 총판·협력사



[그림 75] ㈜웨어밸리의 해외 진출 계획

☞ **㈜웨어밸리는 국내 시장 뿐만 아니라 해외 시장 진출을 시도하고 있는 기업으로, 본 기술개발 결과를 통해 빅데이터 분석솔루션의 신뢰성 및 안정성을 확보한 후 단계별 사업화 전략을 통해 신시장 진출 전망이 매우 좋음**

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 4 절 사업비 사용현황

1. 비목별 총괄표

(단위: 천원)

구 분 / 비 목	계획		사용금액		집행률(%)		비고
	현금	현물	현금	현물	현금	현물	
1. 직접비	363,810	126,667	362,329	126,665	99.6	99.9	
1.1 인건비	269,555	126,667	268,644	126,665	99.7	99.9	
1.2 학생인건비							
1.3 연구장비· 재료비	6,667		6,667		100		
1.4 연구활동비	45,143		44,747		99.1		
1.5 연구과제추진비	11,154		11,087		99.4		
1.6 연구수당	31,291		31,184		99.7		
1.7 위탁연구개발비							
2. 간접비	42,857		42,857		100		
합 계	406,667	126,667	405,186	126,665	99.6	99.9	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 참여연구원

소속 기관명	구분	성명	부서명 (직급)	참여시작일 / 참여종료일	참여 개월수	참여율(%)	비고
㈜웨어밸리	연구 책임자	김인호	기술연구소 (이사)	2018. 08. 01. / 2019. 07. 31.	12개월	30.00	
	참여 연구원	박덕수	기술연구소 (이사)	2018. 08. 01. / 2018. 09. 30.	2개월	15.00	퇴사
	참여 연구원	정선희	품질관리 (수석)	2018. 08. 01. / 2019. 07. 31.	12개월	40.00	
	참여 연구원	김재석	품질관리 (수석)	2018. 08. 01. / 2019. 07. 31.	12개월	50.00	
	참여 연구원	장효진	품질관리 (전임)	2018. 08. 01. / 2019. 07. 31.	12개월	60.00	
	참여 연구원	목선	기술연구소 (책임)	2018. 08. 01. / 2019. 07. 31.	12개월	36.68	
	참여 연구원	박상은	기술연구소 (책임)	2018. 08. 01. / 2019. 07. 31.	12개월	15.00	
	참여 연구원	정일석	기술연구소 (책임)	2018. 08. 01. / 2018. 08. 31.	1개월	24.00	육아 휴직
한국정보통신기술협회	연구 책임자	김현정	SW시험 인증연구소 (팀장)	2018. 08. 01. / 2019. 07. 31.	12개월	60	
	참여 연구원	구정회	SW시험 인증연구소 (수석)	2018. 08. 01. / 2019. 07. 31.	12개월	70	
	참여 연구원	이재효	SW시험 인증연구소 (책임)	2018. 08. 01. / 2019. 07. 31.	12개월	70	
	참여 연구원	임우섭	SW시험 인증연구소 (책임)	2018. 08. 01. / 2019. 07. 31.	12개월	70	
	참여 연구원	송옥수	SW시험 인증연구소 (전임)	2018. 08. 01. / 2019. 07. 31.	12개월	60	
	참여 연구원	이윤선	SW시험 인증연구소 (선임)	2018. 08. 01. / 2019. 07. 31.	12개월	60	

3. 위탁 및 용역과제

해당사항 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 5 절 연구개발결과의 활용계획

1. 품질검증 자동화 프레임워크를 적용하여 품질 경쟁력 확보

- 경쟁 제품들과의 기술 경쟁력이 있으나 낮은 안전성과 신뢰도로 인해 보안제품 전용 DBMS 및 빅데이터 분석 솔루션 시장 진출에 큰 어려움에 직면해 있었고,
- 또 검증을 위한 전문적인 시스템과 인력이 크게 부족한 상황에서 사업화를 위해 제품의 품질 확보가 절실한 상황
- 테스트 프레임워크를 적용하여 짧은 시간에 제품에 대한 안전성 및 신뢰도를 확보할 수 있었고,
- 이를 통해 품질 경쟁력을 확보하여 보안 제품 전용 DBMS에서 빅데이터 분석용 DBMS 시장에 진출하고, 더 나아가 외산 솔루션이 장악하고 있는 OLAP 시장으로 진출 확대 예정

2. 자사 제품의 자동화 테스트에 활용

- 품질 자동화 프레임워크를 높은 신뢰도가 요구되는 자사 제품들, 특히 DB 접근 제어(Chakra Max), DB 암호화 (Galea), DB 작업 결재(Trusted Orange), 개인정보 접속 이력 관리(Grifas), DBMS간 실시간 동기화(Lora) 제품에 순차 적용하고, 이를 통해 제품의 품질 경쟁력을 향상시켜 시장 점유율 확대

3. 테스트 서비스(TaaS)로서 제공

- 마이그레이션 같은 작업 시 테스트 프레임워크를 통해 기능 및 신뢰성을 사전에 미리 셀프로 검증할 수 있도록 제공하고, 어플리케이션 및 클라우드 사용자들을 위해 TaaS (PetaSQL Testing as a Service) 형태로 제공

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

【부록 1】 해외논문

D-TAF: Test Automation Framework Compatible with various DBMS

Woosup Lim, Hyunchong Kim, Jaehyo Lee, Oksu Song
Software Testing & Certification Laboratory
Telecommunications Technology Association
Republic of Korea
woosup.lim@tta.or.kr, hchkim@tta.or.kr, jaehyo@tta.or.kr, donut92@tta.or.kr

Abstract—**To guarantee high quality, it is required to retest DBMS thoroughly whenever any changes occur such as operating system updates and configuration changes. Because retesting requires repetitive jobs, development companies start to consider test automation to improve productivity. Moreover, a special tool is also required because it is difficult to test manually some functions of DBMS.**

Although there are test automation tools such as MySQL Test Framework, Testgres and so on, these frameworks can only be used for particular DBMS. For an example, MySQL Test Framework can only be used for MySQL. Because DBMS testing is completely different from GUI based testing, it is impossible to use GUI based test automation tools such as Selenium when testing DBMS. For this reason, there are needs of the test automation framework for DBMS, which can be easily customizable so that it can be used for other DBMS.

In this paper, we propose the framework for DBMS which will be actually used in DBMS development companies and can also be used for various kinds of DBMS. We propose our design and implementation approaches on the framework for DBMS.

Keywords-test automation framework;DBMS;testing tools

I. INTRODUCTION

As a system software, reliability is the highest priority for DBMS. DBMS is a complicated system, so small changes, even a line of code, affect others which occur unexpected error. It is required to test repetitively whenever any changes occur such as bug patches, updates, configuration changes and so on. In addition, it is difficult to perform testing manually without assistance of a simulation software in case of particular functions such as concurrency and replication. For an example, it is hard to make simultaneous requests to DBMS manually when testing the concurrency of DBMS. Due to the complexity of DBMS, it is difficult to reproduce the situation where defects occur, which requires recording every step in order to find out the root causes of defects.

These are the reasons why DBMS companies or open source foundations have own test frameworks such as MySQL Test Framework, Testgres and so on which are described in Chapter 2. Unfortunately, these frameworks can only be used for particular DBMS. For an example, MySQL Test Framework can only be used for MySQL. For these reasons, there are necessities of the test automation framework which can be easily customizable and compatible with any DBMS so that it can be used for various DBMS.

In this paper, we propose the test automation framework to support various DBMS. We concentrate on the compatibility for designing and implementing the framework. Because this framework will be used a DBMS company, *called WareVally*, we gather requirements from real users. We share user requirements and design considerations for the framework in Chapter 3.

Next, we explain the proposed test automation framework in Chapter 4. The framework consists of 1) Test Suite Repository, 2) Automation Management Engine, 3) Test Execution Engine and 4) Visualization Tool. We explain how the framework is designed and implemented in detail.

We review the strength and limitation of the framework in Chapter 5. Finally, we conclude this paper and share future works in Chapter 6.

II. CURRENT TEST AUTOMATION FRAMEWORKS

There are several test automation frameworks for MySQL, PostgreSQL, Oracle and so on. The followings are the currently used frameworks.

A. *MySQL Test Framework*

MySQL Test Framework [1] is the test automation tool for MySQL. It provides its own programming language to write test cases. MySQL is distributed with the framework including test suites. Users such as DBAs or developers can automate DBMS testing by using this framework. The framework consists of several programs for SQL testing, API testing, load testing, unit testing and so on. However, it is hard to customize the framework for other DBMS, because it uses its own interface, which cannot be used for other DBMS.

B. *Testgres*

Testgres [2] is the test automation tool for PostgreSQL. Test cases can be written in Python, so we can use Python library. It provides functions including node control, query test, database backup, replication and so on. It is hard to use for other DBMS because it requires the high effort to change.

C. *Oracle Real Application Testing*

Real Application Testing (RAT) [3] is for Oracle. The RAT can be used for functional testing and load testing in operational environments. Users can reproduce error situations in operation mode and verify SQL execution plans and statistic data. It is implemented for Oracle, which means it cannot be used for others.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

D. QAGen

[4] proposes QAGen which is a query-aware test database generator. QAGen analyzes all possible usage cases of the given queries and generates test data for them. The test data helps to define the input of test cases for functional testing.

E. SQL Query Generation Tool

[5] proposes SQL Query Generation Tool to generate queries based on the SQL syntax and schema, which helps to define possible queries for SQL testing.

III. CONSIDERATIONS ON THE FRAMEWORK

A. Necessity of Test Automation Framework

Because DBMS requires high reliability, companies or open source foundations provide the test automation framework for their DBMS. The following are the reasons why the test automation frameworks are necessary:

- According to [6], test automation is 2,400 times more productive than manual testing. It allows testers to minimize a time-consuming job and improve productivity.
- The full testing and impact analysis are required whenever any changes occur because minor changes or bug fix could lead to system failures. For an example, misuse of memory related functions could cause buffer overflow.
- It is impossible to perform testing manually. It is because there are various kinds of test cases for SQL query test when considering a variety of operating systems where DBMS is installed, combination of WHERE clause and so on.
- It enables to test particular functions, which are difficult to test manually, such as concurrency and replication.
- DBMS is a complicated system in which modules are executed together. This is one of main reasons why it is difficult to find the root cause of error. For this reason, the framework is required to provide a function which records every execution step.

B. The DBMS Test Automation Framework Requirements

In this paper, we propose the test automation framework for DBMS. This research has been started to help a development company, *called WareVally*, who wants to automate testing for DBMS. The requirements of the company are as follows:

- The framework should proceed without interruption or stop even though DBMS is unexpectedly terminated or stopped.
- It should support the concurrency testing such as multi-session and transaction control such as start, commit, rollback and so on.
- It should change the configuration of test environments such as stand-alone, replication, database link and so on.
- It should change the setting of operating systems, for example, the maximum number of open files, which could affects malfunction or performance degradation of DBMS.
- It should monitor system resources to find out performance problems such as memory leak, response time delay and so on.
- It should test functionality under limited resources such as lack of memory.
- It should make exceptional situations such as system power failure and network disconnection under replication mode.

IV. TEST AUTOMATION FRAMEWORK

A. Framework Configuration

In this paper, we propose the test automation framework for DBMS. The framework consists of 1) Test Suite Repository, 2) Automation Management Engine, 3) Test Execution Engine and 4) Visualization Tool. The architecture of the proposed framework is shown in Fig. 1.

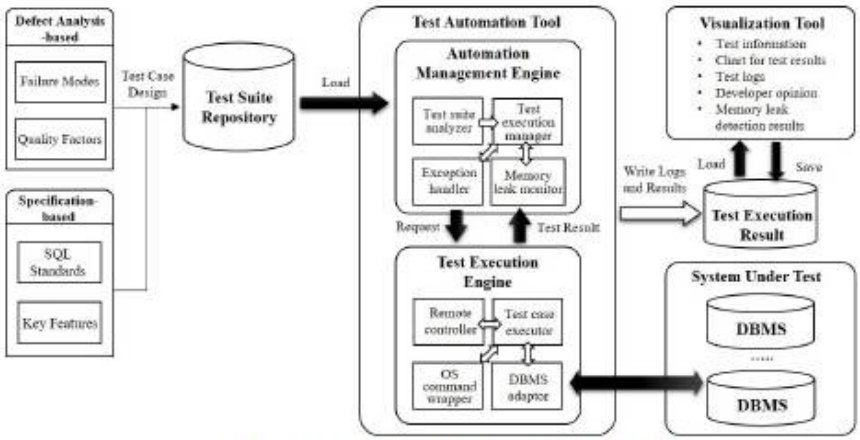


Fig. 1. Architecture of the proposed test automation framework

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1) Test Suite Repository

The test suite repository is the repository that stores and manages a set of test suites. Each test suite consists of several test cases. The test cases are written in Python and a test case is saved as a python module. The test execution engine executes a test suite which consists of several test suites or test cases.

2) Automation Management Engine

The automation management engine controls the test case execution such as loading the test case, determining execution order, reporting execution results and so on. The engine also includes an exception handler to support non-stop testing.

3) Test Execution Engine

The framework determines 'pass' or 'fail' for each test case by comparison between expected and actual results. If the actual result is same as the expected result, the framework determines that the test case is 'passed'. If not, it determines that it is 'failed'.

4) Visualization Tool

The visualization tool provides graphical statistics about test results using various kinds of charts. In addition, it gives the detail information about 'failed' test case and memory leak.

B. Design Approaches on the Framework

We design the test automation framework which is customizable and compatible so that the framework can be used for various kinds of DBMS. In addition, we consider maintainability of test cases. The followings are our design approaches.

1) Test Suite Repository

We make a set of test cases as a group, called 'test suite'. We classify the test cases based on the test objectives as follows:

- *Functions:* table, index, view, transaction, data types, user and role, procedures, functions, optimization and so on.
- *Quality Characteristics:* functional suitability, performance efficiency, compatibility, usability, reliability and so on [7].
- *SQL-92 Standard:* DDL, DML and DCL, transaction, connection and session management, dynamic SQL, and so on.
- *Failure Mode:* limited resource, boundary values, unexpected network disconnection, shutdown and so on.
- *Benchmark:* TPC-H, TPC-DS, and so on [8].

Fig. 2 shows the directory structures for test suites. One test suite is mapped into one directory and the name of test suite is same as the directory name. To support the ease of maintenance, the configuration information for each test suite is written in a text file. It provides the simple way to add new test suites by creating a directory and adding text files in this directory.

As a tree structure, there is a root test suite which include several test suites. If the root node is designated with start command, for an example,'taf root.suite', all test cases are executed. If one of nodes is selected, selected test cases are executed.

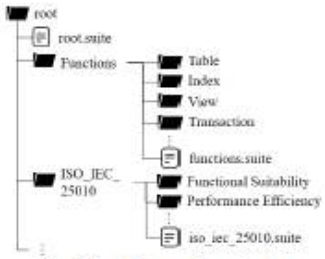


Fig. 2. Test suite repository structure

It is important to perform configuration management for test cases because the test cases are evolving as DBMS is updated. There are different kinds of binary codes even for one DBMS because of the diversity of customer requirements, operating environments and so on. There is a necessity to manage a test suite for each binary code. For these reasons, we use a configuration management tool for the test suites. The test case change history is displayed in a visualization tool by inter-operating with the configuration management tool.

Users should define the expected results for each test case in a text file, which are managed in the test suite repository.

2) Automation Management Engine (AME)

The automation management engine is to load and execute a unit of the test suite. The test cases are executed in a designated order. If there is dependency between more than two test cases, the precedent test case is executed prior to the others. Some of the test cases can be used as the precondition that should be executed in advance. Examples of precondition are related to database link setting, replication configuration setting, and so on. Users can define test case dependency like this.

- dblink.suite --init=create_link.py --fini=destroy_link.py

During testing, DBMS could be terminated or stopped unexpectedly due to its defects or failures. As soon as the DBMS is stopped, the framework is stopped. To support non-stop testing, we design a recovery function to handle the unexpected exception. The recovery function asks Test Execution Engine (TEE) for data backup and then performs database initialization, DBMS restart and so on. Once the DBMS is restarted, the framework automatically resumes the test at the point which the test was stopped. Table 1 shows the backup files when DBMS is stopped.

TABLE I. BACKUP FILES

Backup files	Description
Core dump files	core dump files to analyze memory status in the situation that DBMS is unexpectedly stopped
Stack trace files	stack trace files to analyze call history of processes and find out steps and queries causing DBMS stop
DBMS log files	log files such as operation log and error log to find out functions which are needed to be restarted

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3) Test Execution Engine (TEE)

a) Module Reusability

During testing, same modules are executed repetitively such as table creation, data insertion, server restarts and so on. We design these kinds of modules as common modules which are frequently called by other test cases. The common modules are as follows:

- *DBMS control package:* a package for controlling a DB server such as database creation, DBMS shutdown, server restart and so on.
- *Concurrency package:* a package for making concurrent requests.
- *Environment package:* a package for DBMS configuration setting such as replication, database link and so on.
- *OS setting package:* a package for OS configuration setting such as kernel parameters, environment variables.
- *Monitoring package:* a package for monitoring system resources such as CPU, memory, disk space and so on.
- *Backup and restore package:* a package for executing backup and restore functions of DBMS and verifying backup data integrity.
- *Utility package:* a package for controlling the DBMS utilities such as a CLI client program, Data Load/Export program and so on.
- *Interface package:* a package for compiling and executing JDBC, ODBC and Embedded SQL.
- *Threshold test package:* a package for executing functions under lack of system resources.
- *Data generation package:* a package for generating test data with various kinds of data types such as integer, float and character. This package produces test data according to the data type and size that user define.
- *Exception test package:* a package for making exceptional situations such as network disconnection.
- *Benchmark package:* a package for performing benchmark test such as TPC-H, TPC-DS, and so on.

b) Compatibility

To support various kinds of DBMS, our design consideration of the execution engine are as follows:

- *Interface:* ODBC is used to provide interface between DBMS and the proposed framework.
- *Queries:* query definition functions is provided to resolve the issue that SQL syntax is different from DBMS to DBMS.
- *Wrapper:* wrapper functions are provided to resolve the issue that commands are different from DBMS to DBMS.

c) Test Result Decision and Exception Detection

After test cases are executed, the test execution engine compares actual results with expected results and then records test results for each test case. The engine also makes a record for the *'failed'* test cases so that testers can check the reasons why the actual result is different from the expected one. The engine compares the two results as text basis. There would be possibility to report wrong results due to text format difference, a blank character and so on. To resolve these issues, we use regular expressions like this.

- TAF_IGNORE_KEYWORD = '\([\d.]+m?s\)$'

During testing, DBMS could be terminated or stopped unexpectedly. We design an exception detection mechanism, as shown in Fig. 3.

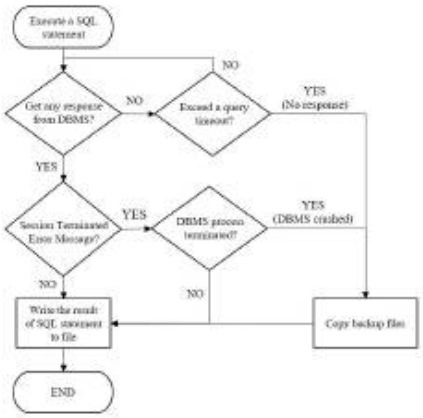


Fig. 3. Exception detection mechanism

If the engine doesn't get any response from DBMS within a specified time, the engine determines that it is an exceptional situation needed to handle it. The engine records a log file, requests DBMS restart and skips the test cases which occurred the exception.

d) Detecting memory leak

Because DBMS should be executed for a long period, it should use system resource efficiently such as CPU, memory, disk space and so on. Because a small size of memory leak leads to the DBMS stop, it is very important for the engine to check the memory usage during testing DBMS. For this reason, we make the engine to interoperate with Valgrind [9] which is commonly used for monitoring memory leak. In addition, the engine reports memory leak which is displayed in the visualization tool.

e) Test logs and Execution results

The engine makes log for every step during executing a test case. In addition, we make records detail information that is used to analyze the root cause of *'failed'* test cases. The logs include the followings:

- the total number of test case, the number of 'failed' or 'passed', memory leak status
- test case execution time and test results
- the list of failed test cases
- test case execution log, SQL execution results and so on
- diagnostic information to find out the root cause of system failures such as unexpected DBMS stop and so on

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4) Visualization Tool (VT)

The visualization tool provides the following information:

- *Test information:* general information about test execution such as the name of operating system, the version and configuration of DBMS, test suites and so on.
- *Test logs:* detail logs for failed test cases.
- *Developer Opinion:* analysis opinion on failed test cases written by developers.
- *Memory leak detection results:* detail information about memory leak and related test cases.

C. Implementation Approaches on the Framework

We consider the implementation of the framework with practical ways. The followings are our implementation approaches.

1) Test Suite Repository (TSR)

The repository stores test cases, test library and expected results. All kinds of data are saved in the file format.

a) Test Case Format

To support the ease of repository maintenance, test cases and suites are saved in a file format instead of using DBMS. For configuration management, The files are managed and controlled by SVN(Subversion) or Git.

b) Test Library

We use Python language to write test cases for learnability. The repository has various kinds of packages such as DBMS control package, concurrency package and so on. When writing test cases, these packages can be easily imported in test cases because we write the test case in Python. It allows users to use methods of the packages. An example of test case written in Python is shown in Fig. 4.

```
01 # -*- coding: utf-8 -*-

02 from dbtest.query import query
03 from dbtest.utils import comment
04
05 if __name__ == "__main__":
06   comment("Execute SQL statement on one session")
07
08   query("drop procedure sys.myfunc();", silent=True)
09   query("drop table if exists t1 CASCADE;", silent=True)
10
11   query("CREATE PROCEDURE sys.myfunc()
           EXTERNAL NAME sql.myfunc;")
12
13   query("create table t1 (id int);")
14   query(
15   """
16     CREATE PROCEDURE sys.myfunc(n int)
17     BEGIN
18       declare x int;
19       set x=1;
20         while x <= n do
21           insert into t1 values (x);
22           set x = x + 1;
23       end while;
24     END;""")
25
26   query("call sys.myfunc(10);", silent=True, commit=True)
27   query("select * from t1;")
```

Fig. 4. An example of a test case

In the packages, there are various kinds of methods which are callable in the test cases. The method parameters are used to define precondition or input data for test cases. The parameter can be SQL statements, the number of repetition, execution interval and so on. These parameters can be used for input data for test cases.

c) Test Case Grouping

A set of test cases are grouped into a test suite. For an example, we can define a test suite named *replication.suite* having several test cases which are logically similar. The execution engine executes either a test suite or test case. This kind of approach makes it easy to define the test scope by designating the name of a test suite or test case.

```
########################################
#        Test Suite For Replication Testing        #
########################################

create_table_on_replication.py
insert_on_replication.py
select_on_replication.py
update_on_replication.py
drop_table_on_replication.py
create_truncate_many_time.py
fail_over_when_master_shutdown.py
operate_when_slave_shutdown.py
```

Fig. 5. An example of test case grouping

2) Automation Management Engine (AME)

The automation management engine (AME) organizes test execution. It requests for the test execution to a test execution engine (TEE) and receives the response of test results from TEE.

a) Test Case Execution Order

The name of test cases are listed up in a test suite file. The test case sequence in the list affects the sequence of the test case execution. For an example, test cases are written in the sequence A, B, C, the engine executes A, B, C.

Users can define an initialization and a finalization method. An initialization method is executed before a test suite and a finalization method is executed after the test suite. In a test suite file, we can add the option in the file for initialization and finalization.

If we want to designate initialization and finalization, we just add a method name as a follow:

- 'replication.suite --init=create.py --fini=destroy.py'

create.py is executed for initializing replication.suite and *destroy.py* is executed for finalizing the suite.

AME makes a request to the execution engine for executing designated test suites with a specified order, and then receives test results including pass/fail, exception and error. When AME detects the exception of DBMS, it initializes DBMS and resumes the testing. Because the way to detect exceptions are different from DBMS to DBMS, it is needed to be customized based on the types of DBMS.

b) Test Resumption

We implement initialization mechanisms for several DBMS such as MySQL, PetaSQL [10] and so on. We also

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

provide the interface to modify the mechanisms for other DBMS.

c) Momory Leak Detection

There is a memory monitoring tool, *called Valgrind,* which detects memory leak and makes records in a log file. When using this tool, there are difficulties in locating the test case causing memory leak. To resolve this problem, AME provides the function to find out the related test case and maps the memory leak logs on the related test cases.

3) Test Execution Engine (TEE)

Test Execution Engine (TEE) executes the test cases according to AME's requests.

a) Compatibility for DBMS

We use ODBC to connect to various kinds of DBMS and implement the functions to modify SQL statements so that the statements can be executed for different kinds of DBMS. Because the configurations of ODBC are different from DBMS to DBMS, we define a config file, *called taf.conf,* in order to define data sources of ODBC.

b) Compatibility for OS

Because DBMS can be installed in various kinds of operating systems (OS), the way to control DBMS, such as executing and killing process, is different from OS to OS. Especially, there are big differences between Windows and Unix/Linux. This is one of the difficulties to support compatibility. To support this, the framework is implemented to test DBMS installed in any platforms.

Test libraries in TSR can be interpreted by Python interpreter because they are written in Python. Because Python is a platform independent language, the test libraries can also be executed in any platforms.

c) Remote Control

To prepare test environments for particular functions such as replication and database link, TEE should be able to control DBMS and operating system remotely. For example, TEE needs to control the operating system, in which DBMS is installed, to monitor system resources. For this reason, we implement remotely controllable TEE.

4) Visualization Tool (VT)

We implement a web based visualization tool. The tool loads test result files and displays statistics graphically in the web pages. Users can analyze failed test cases by comparing the difference between expected and actual results. The tool also displays the detection results of memory leak and related test cases, which allows to analyze the root cause of error.

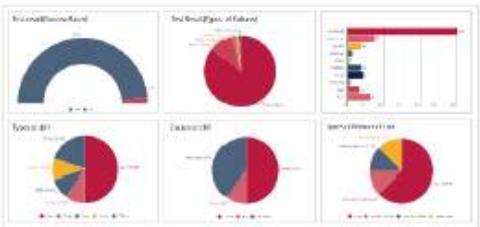


Fig. 6. An example of visualization tool



Fig. 7. An example of execute test suite

V. STRENGTH AND LIMITATION OF THE FRAMEWORK

In this paper, we propose the test automation framework for DBMS. Strength and limitation of the framework are as follows.

A. Strength of the Framework

1) Sophisticated Verification Mechanism

The proposed framework provides an assertion method, *which is 'assert(actual, expected)',* which compares actual results with expected results. This style of method lets users know the comparing data of each test case explicitly, which also support readability of test cases.

The assertion method can also be used for verifying intermediate results during executing test cases. The example of the assertion method is as follow.

- TAF_ASSERT(result, "operation success")

The framework can support not only the comparison of data types, but also SQL statement comparison. For example, the assertion method verifies that the execution results of the SQL statement include a particular keyword. The example of the assertion method for the SQL statement is as follow.

- QUERY_ASSERT("select count(*) from emp;", 26)

Data integrity is the key for DBMS reliability. Therefore, it is important to verify data correctness and consistency during testing. In addition, there are several functions of DBMS requiring to check data consistency, for examples, replication, database link and so on. For these functions, we provide a comparison method, called 'selelectDiff' to compare two tables. If the two tables are different from each other, it means that there are critical defects in terms of data integrity in these functions.

2) Resuability and Maintaiblity

Test cases are written in Python so that test cases are easily reusable because of Python programming characteristics. A test case can include more than two test cases by using an 'import' keyword. Moreover, users can define a new method to include several test cases with parameters which is used for inputs of test cases. We design the test cases which correspond to a python file.

The framework uses file format for saving test cases, test results, test logs and so on. The reason why we use file format is to support the ease of maintenance. Users don't need

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

to install another DBMS for operating the framework and can edit these files easily without installing editing tools. In addition, we can easily do configuration management with SVN(Subversion) or Git.

3) Visability Support

During development or testing, users make some of code disabled by putting # in test cases on purpose, but they sometimes forget to uncomment later. The commented test cases could report '*pass*' results which would lead to big surprises when deploying products in customer sites. To prevent this mistake, we report commented test cases, *called 'Zombie test case'* to let users recognize them.

According to user requirements, the framework monitors resource usages such as CPU and memory for each test cases to find out the functions which use resource excessively.

Pre-condition is the condition before executing test cases and post-condition is the condition after executing them. As mentioned above, test cases can be used as pre-condition or post-condition. In this case, the framework does not report *'pass' or 'failed'* for these test cases to prevent from misunderstanding test case results.

4) Compatiblity Support

We design the framework which supports various kinds of DBMS. To do this, the framework has the special methods which are a wrapper, query definition and connection interface which are explained in Chapter 4.

DBMS are installed various kinds of operating systems such as Linux, Windows and so on. The test cases are affected by the operating systems in which DBMS are installed. Test cases could not be executed when environments are changed. To resolve this issue, the framework has a config file. The file includes OS accounts, DBMS port, DBMS path and so on.

B. Limitation of the Framework

DBMS includes GUI based utilities such as a SQL query tool and database migration tool. The proposed framework supports CLI interface, not GUI interface. When comparing with other test automation frameworks, they also does not support GUI test automation.

Writing expected results are a time consuming job. Unfortunately, there is no way to automate these kinds of jobs. In the future, we hope that we could resolve this limitation by applying new technologies like AI (Artificial Intelligence).

VI. Conclusion and Future Work

Big data analysis helps to analyze business trends and find out a new opportunity. As the importance of big data analysis is increasing, the needs of a special solution are also increasing. This is the reason why DBMS supports OLAP(On-Line Analytical Processing). Because the DBMS should guarantee integrity and availability in any situations, it is crucial to perform systematic testing when developing DBMS.

The repetitive testing is required whenever any changes occur such as OS changes, version up and so on. To perform DBMS testing efficiently and effectively, test automation is necessary especially for DBMS development companies.

Because the proposed framework is going to be used for real users from the DBMS development company, called *WareVally*, we develop the framework based on real user requirements. We also consider the customizability and compatibility to support various kinds of DBMS. We design the framework maintainable, especially for test cases, because test cases are needed to be added, modified and deleted according to DBMS updates. The framework includes the visualization tool which displays test execution results, memory leak, error logs and so on. In addition, the framework provides the detail information for failed test cases, which enables testers to analyze the root cause.

We've just finished the implementation of the framework. Now we are planning to apply on *PetaSQL*, which is OLAP RDBMS, and then improve the framework based on the user feedbacks. In addition, we will operate the framework together with Continuous Integration (CI) tool to support full automation from code commit to deployment.

Acknowledgment

This work was supported by Institute for Information & communications Technology Promotion (IITP) grant funded by the Korea government (MSIT) [No. 20180011930012002, Development of quality verification automation framework to provide high reliability of bigdata solution].

References

[1] "MySQL: The MySQL Test Framework, Version 2.0", Dev.mysql.com, 2019. [Online]. Available: https://dev.mysql.com/doc/dev/mysql-server/8.0.1/PAGE_MYSQL_TEST_RUN.html. [Accessed: 30- Jan- 2019]

[2] "Testgres documentation — testgres 1.5 documentation", Postgrespro.github.io, 2019. [Online]. Available: https://postgrespro.github.io/testgres. [Accessed: 30- Jan- 2019].

[3] "Oracle Real Application Testing", Oracle.com, 2019. [Online]. Available: https://www.oracle.com/technetwork/database/options/real- application-testing-option-2355173.html. [Accessed: 30- Jan- 2019].

[4] E. Lo, C. Binnig, D. Kossmann, M. Tamer Özsu and W. Hon, "A framework for testing DBMS features", The VLDB Journal, vol. 19, no. 2, pp. 203-230, 2009.

[5] Shadi A. Khalek and Sarfraz Khurshid. Automated SQL Query Generation for Systematic Testing of Database Engines, In ASE'10: Proceedings of the IEEE/ACM international conference on Automated software engineering, pp. 329-332, Antwerp, Belgium, 2010.

[6] D. Graham and M. Fewster, Experiences of test automation. Upper Saddle River, NJ: Addison-Wesley, 2012.

[7] ISO/IEC 25010:2011 Systems and software engineering -- Systems and software Quality Requirements and Evaluation (SQuaRE) -- System and software quality models

[8] "TPC - Benchmarks", Tpc.org, 2019. [Online]. Available: http://www.tpc.org/information/benchmarks.asp. [Accessed: 30- Jan- 2019]

[9] "Valgrind Home", Valgrind.org, 2019. [Online]. Available: http://valgrind.org. [Accessed: 31- Jan- 2019].

[10] "PetaSQL - Database for Analysis", Warevalley, 2019. [Online]. Available: https://www.warevalley.com/eng/solutions/petasql.asp. [Accessed: 31- Jan- 2019]..

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

【부록 2】 국내논문

빅데이터 분석용 DBMS를 위한 테스트 자동화 프레임워크 설계

임우섭°, 송옥수, 김현정
한국정보통신기술협회(TTA)
woosup.lim@tta.or.kr, donut92@tta.or.kr, hchkim@tta.or.kr

Test Automation Framework for OLAP DBMS

Woo Sup Lim°, Oksu Song, Hyunchong Kim
Telecommunications Technology Association

요 약

빅데이터 분석용 DBMS는 대량의 데이터를 처리, 저장 및 분석하는 시스템으로, 단순 결함도 치명적인 장애로 이어질 수 있는 높은 신뢰도가 요구되는 시스템이다. 고신뢰성의 DBMS를 개발하기 위해서는 다양한 테스트 방법론을 적용하고 방대한 테스트케이스를 구축하여 체계적인 테스트를 수행해야 한다. DBMS 제품 특성상 기능이 추가될 때마다 전체 기능에 대한 테스트를 수행해야 하고, 운영체제, 설정파일 등을 변경하는 경우에도 DBMS의 전체 기능에 대한 회귀 테스트는 필수적이다. 동시성 시험 등 수작업으로 테스트가 불가능한 영역이 다수 있기 때문에 시험 자동화에 대한 필요성은 더욱 높아진다.

본 논문에서는 체계적인 테스트 수행을 위해 DBMS 개발기업이 필요로 하는 테스트 자동화 프레임워크를 제안하고자 한다. SW개발기업의 실질적인 활용을 위해 기업의 요구사항을 분석하여 프레임워크를 설계하였으며 본 논문에서는 설계 결과인 구성 요소 및 요소별 설계 내용을 소개하고자 한다.

1. 서 론

한국데이터진흥원 조사 결과[1]에 따르면 빅데이터 솔루션 기업의 국내 기술 수준은 2016년 기준 선진국 대비 약 65.7점, 기술수준 격차는 약 3.1년, 선진 기술 도달 시간은 약 3.4년으로 큰 차이를 보이고 있다. 외산 제품 대비 경쟁력 저하의 주요 원인으로는 국내 제품의 낮은 안정성 및 신뢰성 등으로 분석하고 있다.

빅데이터 분석용 DBMS는 대량의 데이터를 처리, 저장 및 분석하는 시스템으로, 단순 결함도 치명적인 장애로 이어질 수 있기 때문에 신뢰성을 가장 중요하게 고려하고 있다. 고신뢰도를 요구하는 DBMS의 경우, 소스코드를 일부만 수정하여도 전체 시스템에 대한 전수 시험을 수행하고, 매뉴얼 테스트가 불가능한 기술적인 영역이 존재하기 때문에 테스팅 도구 활용은 필수적이다.

소프트웨어를 비롯하여 다양한 산업 분야에서 반복적인 작업을 줄이고 업무의 효과를 높이기 위해 자동화를 추진하고 있다. 다양한 테스팅 영역에서 테스팅이 자동화되고 있지만, 쉽게 변경이 가능한 소프트웨어 특성으로 인해 성능시험도구, GUI 테스팅 등 제한된 영역을 대상으로 테스팅 자동화가 이루어지고 있다.

개발된 제품을 대상으로 일차적인 테스팅이 완료되면 그 이후부터는 반복적인 테스팅 작업이 시작된다. 결함 패치, 기능 업그레이드 등 일부 기능이 변경된 경우에도 제품의 품질을 확보하기 위해서는 전체 기능에 대한 시험이 필수적이다. SW개발기업에서는 이러한 반복적인 테스트를 효과적이고 생산적으로 수행하기 위해 테스팅 자동화 도구를 도입한다. 테스팅 자동화는 반복된 테스팅 작업에 대한 투입 대비 효과를 높일 수 있고 수작업으로 테스팅이 불가능한 기술 영역도 테스트가 가능하도록 도와줄 수 있다. 특히, DBMS, 운영체제 등과 같이 높은 신뢰성을 요구하는 시스템 소프트웨어 분야의 경우, 수작업으로 테스팅이 불가능한 영역이 존재하기 때문에 도구의 필요성은 더욱 높아진다.

DBMS의 복잡성으로 인해 DBMS에서 결함 발생 시, 결함이 발생한 상황을 수작업으로 재현하는 것은 현실적으로 매우 어려운 일이다. 그렇기 때문에 테스트 과정을 실시간으로 로그로 기록하는 등 일련의 작업을 자동화해야한다. DBMS가 다수 사용자의 동시 요청에 대한 제어를 제대로 처리하지 못한다면, 데이터의 무결성이 깨질 수 있는 치명 결함이 발생할 수 있기 때문에 이를 테스트할 수 있는 시험 도구는 테스트의 핵심이 된다. 이러한 배경과 함께 본 논문에서는 DBMS 개발기업의 효과적인 품질 검증을 지원하기 위해 테스팅 자동화 프레임워크 연구개발을 제안하였다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

단순 결함도 치명적인 장애로 이어질 수 있는 시스템 특성상 제품 신뢰성에 대한 기대수준이 매우 높은 상황이다. 고신뢰성의 DBMS를 개발하기 위해서는 다양한 테스트 방법론의 적용은 물론이고 방대한 테스트케이스를 구축하여 완전한 테스트를 수행하여야 한다. DBMS 제품에 기능이 추가될 때마다 전체 기능에 대한 테스트를 수행해야 하고, 운영체제, 설정파일 등을 변경하는 경우에도 DBMS의 전체 기능에 대한 테스트는 필수적이다. 다수 사용자 동시 요청과 같은 테스트케이스는 수작업으로 테스트 수행이 불가능하기 때문에 테스팅 자동화 도구는 DBMS 검증에서는 없어서는 안되는 필수품이라는 사실은 두말할 것도 없다.

본 논문에서는 DBMS 개발기업이 필요로 하는 테스트 자동화 프레임워크를 제안하고자 한다. SW개발기업에서 실질적으로 활용할 수 있도록 기업의 요구사항을 분석하여 프레임워크를 설계하였고, 프레임워크 설계 결과인 구성 요소 및 각 요소별 설계 내용을 소개하고자 한다. 테스트 자동화 프레임워크는 4가지 요소로 구성되고, 각각은 다음과 같다.

① 테스트 슈트 레파지토리
② 자동화 관리 엔진
③ 자동화 실행 엔진
④ 결과 확인 및 분석을 위한 시각화 도구

본 논문의 구성은 다음과 같다.

2장 배경
3장 자동화 프레임워크 필요성 및 설계 고려사항
4장 빅데이터 분석용 DBMS 테스트 자동화 프레임워크
5장 결론 및 향후 계획

2. 배 경

2.1 기존 DBMS 테스트 자동화 도구

고신뢰성이 요구되는 DBMS의 경우 상용 제품뿐만 아니라 오픈소스에서도 테스트 자동화 프레임워크를 활용하고 있다. 대표적인 테스트 자동화 프레임워크는 다음과 같다.

1) MySQL Test Framework

MySQL 서버와 클라이언트 프로그램을 테스트하기 위해 제공되는 MySQL 테스트 프레임워크로[2], 테스트를 위한 별도의 언어를 제공하는 것이 특징이다. 테스트 언어에 따라 작성된 테스트 슈트가 MySQL과 함께 배포되고 있어 MySQL을 대상으로 테스트 수행이 가능하다. 테스트 프레임워크는 다수의 프로그램으로 구성되고 SQL문 테스트, 클라이언트 API 테스트, 부하 테스트, 단위 테스트 등이 가능하다. 하지만 MySQL에 특화되어 있기 때문에 다른 DBMS를 대상으로 테스트를 수행하는 것은 불가능하며 별도의 언어를 가지고 있어 기능 추가 시 개발 비용이 큰 편이다.

2) Testgres

PostgreSQL을 테스트하기 위한 테스트 프레임워크로[3], 테스트케이스가 파이썬 파일로 작성되는 것이 특징이다. 작성된 테스트케이스는 일반 파이썬 파일과 동일하게 파이썬 인터프리터를 통해 실행된다. 파이썬을 이용하기 때문에 파이썬에서 제공하는 제어문과 풍부한 라이브러리를 가져다가 사용할 수 있다는 장점을 갖는다. 하지만, MySQL 테스트 프레임워크와 달리 실행 결과를 검증하는 메커니즘을 제공하지 않아 사용자가 테스트케이스 내에서 검증을 진행해야 하며, 테스트 슈트의 개념이 없이 테스트케이스를 하나씩 실행해야 하는 단점을 갖는다. MySQL과 마찬가지로 다른 DBMS에 대한 테스트는 불가능하다.

3) Oracle Real Application Testing

상용 소프트웨어인 오라클은 DBMS를 테스트할 수 있는 솔루션을 판매하고 있다. 이중 Real Application Testing(RAT) 솔루션[4]은 DB의 운영환경 변경, 버전 업그레이드, 파라미터 변경 등에 따른 전체 기능 테스트 및 부하 테스트가 가능한 솔루션이다. DB Replay 기능을 통해 운영 환경에서의 워크로드를 저장하여 동시성 및 트랜잭션을 테스트 시스템에 재현할 수 있으며, 성능 분석기를 통해 SQL 실행계획 및 통계정보를 검증할 수 있다.

3. 자동화 프레임워크 필요성 및 설계 고려사항

3.1 DBMS 자동화 프레임워크 필요성

고신뢰성이 요구되는 DBMS 특성상 상용 SW 뿐만 아니라 오픈소스에서 테스트 자동화 프레임워크를 함께 제공하고 있다. 테스트 자동화 프레임워크가 활용되고 있는 이유는 다음과 같다.

① [5]에 따르면 데이터베이스 테스트 자동화는 약 2,400배의 능률을 향상시키는 효과가 있다. 자동화를 통해 테스터는 반복적인 회귀 테스트에서 벗어나 새로운 테스트 개발과 도구 개발에 집중할 수 있으며 결과적으로 제품의 품질을 높일 수 있다.

② 환경 설정이나 결함 패치 시에 테스트를 반복적으로 수행해야 하는 상황으로 자동화 도구 없이 수작업으로 하는 것은 비용적으로 효과적이지 않다. 단순 결함 패치도 시스템 장애로 이어질 수 있기 때문에 패치에 대한 영향 분석을 실시해야 하고 이를 위해서는 변경 시마다 전수 테스트를 수행해야 한다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

③ DDL, DML, DCL 등 조합에 따라 수많은 SQL 문이 존재하며 이를 수동 테스트를 통해서 실행 및 검증하는 것은 불가능하다.

④ DBMS에서 제공하는 동시성, 이중화 등 기능의 경우 매뉴얼 테스트가 불가능하다. 예를 들어 동시성 테스트를 위해서는 다수 사용자의 동시 요청을 시뮬레이트해야 하지만 도구 없이는 불가능하다.

⑤ DBMS는 다수의 모듈이 유기적으로 동작하는 복잡한 시스템으로 오류 발생 시 원인 파악이 매우 어렵다. 원인분석을 위해서는 시험 과정을 자동으로 기록하는 자동화 도구가 필수적이다.

⑥ DBMS는 시스템의 자원을 효율적으로 활용해야 하는 시스템 소프트웨어로, 각 기능의 자원(CPU, 메모리, 디스크) 사용률을 분석하기 위해 자원 모니터링 도구와의 연동이 필요하다.

3.2 DBMS 자동화 프레임워크 요구사항

본 논문에서는 중소기업에서 개발한 빅데이터 분석용 DBMS의 품질 고신뢰성 확보를 위해 테스트 자동화 프레임워크 연구 개발을 제안하고자 한다. 해당 기업에게 수집한 요구사항은 아래와 같다.

① 자동화 테스트 실행 시, 선행 테스트 케이스로 인해 DBMS가 비정상 종지되더라도 중단없이 이후 테스트가 수행되어야 한다.

② 동시성 테스트를 위해 다중 세션을 지원하고 트랜잭션 제어(시작, 커밋, 롤백 등)가 가능해야 한다.

③ 다양한 DBMS 운영 환경(단독운영, 이중화, DBLink 등) 구성을 자동화할 수 있어야 한다.

④ 운영 간 발생하는 성능 문제(메모리 누수, 응답 시간 지연 등)를 탐지할 수 있어야 한다.

3.3 프레임워크 설계 고려사항

본 논문은 DBMS 개발 기업의 요구사항을 기반으로 시작되었지만, 다양한 DBMS에 대한 테스트 자동화에 적용할 수 있도록 프레임워크를 설계하였다. 요구사항을 기반으로 프레임워크 설계 시 고려사항은 다음과 같으며 각 고려사항은 [표 1]과 같이 설계 항목에 반영하였다.

① One-Stop 테스트: 한번의 실행으로 DBMS의 전체를 테스트할 수 있도록 자동화. 테스트 과정에서 에러가 발생하여도 중단없이 테스트가 진행되도록 에러 처리

② 다중 세션: 동시에 여러 세션을 생성하여 SQL문을 실행하고 트랜잭션 제어가 가능하도록 기능 지원

③ 운영 환경 구성: 다수 노드에서 DBMS 서버 실행, 이중화 설정, DB Link 설정 등이 가능하도록 관리 기능 제공

④ 성능 모니터링: CPU 사용률, 메모리 사용량, SQL문 수행 시간을 모니터링하고 사용자가 명시한 기준 범위를 벗어날 경우 해당 테스트 케이스를 실패 처리하는 기능 제공

⑤ 테스트 오라클: 테스트 케이스별로 성공 실패를 비교할 수 있는 메커니즘을 제공. 기대 출력과 실제 출력을 비교하여 성공 실패를 결정하고, 공백 문자열, 날짜 차이 등으로 발생할 수 있는 오류에 대한 전처리

⑥ 유지보수 용이성: DBMS 기능이 추가됨에 따라 테스트케이스가 지속적으로 확장될 수 있기 때문에 테스트케이스 추가, 수정 및 삭제가 용이하도록 설계

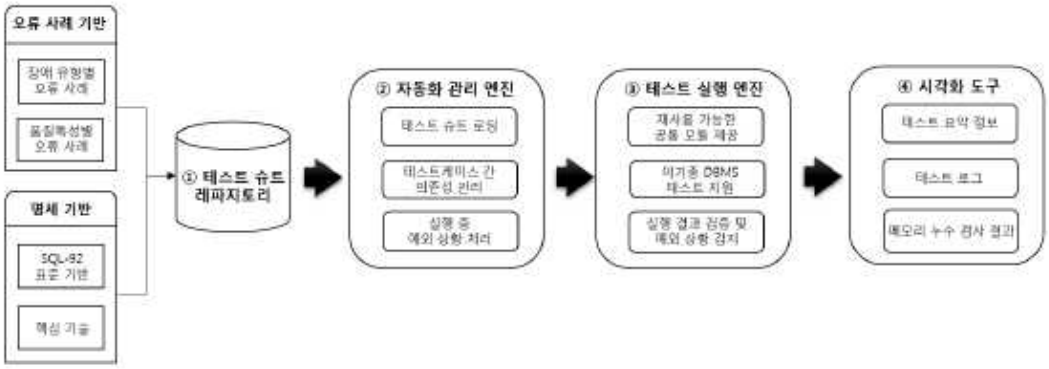
⑦ 형상 관리: DBMS의 형상 변경에 따라 테스트케이스의 형상도 함께 변경되어야 하므로, 자동화 프레임워크와 형상 관리 도구와의 연동 고려

⑧ 의존성 설정: 테스트케이스의 실행 의존성을 고려하여 테스트가 실행될 수 있도록 설계

[표 1] 설계 고려사항별 설계 결과

요구사항 및 설계 고려사항	설계 결과
One-Stop 테스트	자동화 관리 엔진에서 테스트 실행 중 DBMS 비정상 종료와 같은 예외 상황 발생 시, 자동으로 DBMS 서버를 초기화하도록 설계
다중 세션	다중 세션 생성 기능을 제공하는 동시성 지원 패키지를 테스트 실행 엔진에 설계
운영 환경 구성	이중화, DB Link 등의 운영 환경 구성을 자동화 할 수 있는 운영환경 제어 패키지를 테스트 실행 엔진에 설계
성능 모니터링	자원 모니터링 패키지를 테스트 실행 엔진에 설계
테스트 오라클	자동화 관리 엔진에서 실행 결과 검증 모듈을 통해 테스트 오라클 설계
유지보수 용이성	테스트케이스 추가, 수정 및 삭제가 용이하도록 테스트 파일 단위로 테스트 슈트 구조 설계
형상 관리	형상관리도구와 연동을 통해 DBMS 제품 형상별 테스트케이스 관리 및 추적 지원
의존성 설정	테스트 슈트 옵션을 통해 의존성을 설정할 수 있도록 설계

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 1] 빅데이터 분석용 DBMS 테스트 자동화 프레임워크

4. 빅데이터 분석용 DBMS 테스트 자동화 프레임워크

본 논문에서는 [그림 1]과 같은 빅데이터 분석용 DBMS 테스트 자동화 프레임워크를 제안하고자 한다. 자동화 프레임워크는 크게 1) 테스트 슈트 레파지토리, 2) 자동화 관리 엔진, 3)테스트 실행 엔진, 4)시각화 도구로 구성되고 각각에 대한 설명은 다음과 같다.

4.1 테스트자동화 프레임워크 구성

1) 테스트 슈트 레파지토리

테스트 슈트를 저장하고 관리하기 위한 저장소로, 하나의 테스트 슈트는 다수의 테스트케이스로 구성되어 있다. 테스트케이스는 파이썬의 모듈 단위로 작성하고 테스트 슈트는 다수의 파이썬 모듈 파일을 묶어서 실행하도록 레파지토리를 설계하였다. 테스트 슈트 레파지토리에는 기대 출력과 실제 출력을 비교할 수 있도록 기대 출력을 정의한 파일이 포함되어 있다.

2) 자동화 관리 엔진

테스트케이스 실행을 자동화하기 위한 관리 모듈로, 테스트케이스 로딩, 실행 순서 결정, 테스트 실행 모듈에 실행 요청, 실행 결과 기록 등을 수행하는 엔진이다. 또한, 테스트케이스가 중단없이 실행될 수 있도록 예외 처리 기능을 제공한다.

3) 테스트 실행 엔진

테스트케이스를 실행하고 실행 과정을 로그로 기록하는 모듈로, 실행하는 과정에서 비정상 종료와 같은 예외 상황 발생 시 이를 감지한다. 또한, 각 테스트케이스 실행 과정에서 기록된 실제 출력과 레파지토리의 기대 출력을 비교하여 성공/실패를 결정한다.

4) 시각화 도구

테스트 실행 결과(테스트케이스 실행 건수, 실패 및 성공 건수 등)와 실패 테스트케이스에 대한 상세 로그 조회, 메모리 누수 여부 검사 결과 조회 등을 시각적으로 제공한다.

4.2 테스트 자동화 프레임워크 설계

특정 기업의 빅데이터 분석용 DBMS 뿐만 아니라 다른 기업의 제품에도 활용될 수 있도록 확장성과 호환성 있는 테스트 자동화 프레임워크를 설계하였다. 또한, 테스트케이스를 지속적으로 추가 및 관리할 수 있도록 유지보수를 고려하여 시스템을 설계하였다. 프레임워크의 각 구성 요소별 설계 내용은 다음과 같다.

4.2.1 테스트 슈트 레파지토리

1) 테스트 슈트 레파지토리 구조

일반적으로 적용되는 디자인 패턴에 따라 테스트케이스를 디렉토리 기반으로 구조화하였고, 논리적으로 성격이 유사한 테스트케이스를 하나의 테스트 슈트로 그룹핑하였다. DBMS에 대한 체계적인 테스트를 위해 테스트 슈트는 아래와 같이 분류하였고, 각각의 테스트케이스는 파이썬으로 개발하였다.

① 제품 기능: 테이블, 인덱스, 뷰, 트랜잭션, 데이터 타입, 유저 및 권한, 프로시져, 함수, 최적화 등

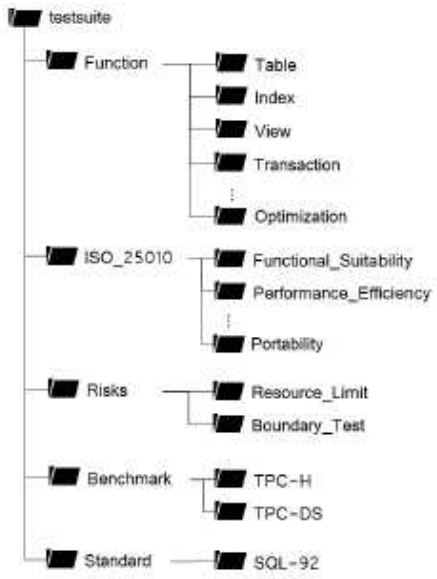
② ISO/IEC 25010 기반 품질 특성[6]: 기능적합성, 성능효율성, 호환성, 사용성, 신뢰성 등

③ SQL-92 표준: 데이터 정의어, 데이터 조작어, 트랜잭션 관리, 커넥션 관리, 세션 관리, 동적 SQL 등

④ 장애유형: 자원 한계 상황, 경계값, 네트워크 절체, 비정상 종료, 비정상 중지 등

⑤ 벤치마크테스트: TPC-C[7], TPC-H[8], TPC-DS[9] 등

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 3] 테스트 슈트 레파지토리 구조 예시

[그림 2]와 같이 테스트 슈트는 디렉토리와 매핑되도록 구성하였고, 테스트 슈트명을 디렉토리명으로 사용하였다. 테스트케이스 추가, 삭제, 수정 등의 관리가 용이하도록 텍스트 파일 단위로 테스트 슈트를 작성하였다. DBMS에 새로운 기능이 추가되는 경우 이에 대한 테스트케이스를 추가해야 한다. 테스트 자동화 프레임워크에서는 디렉토리 및 파일을 추가하는 것만으로 테스트케이스 추가가 가능하도록 유지보수 용이성을 고려하였다.

2) 테스트 슈트명 기반 테스트 실행

테스트를 전체 혹은 부분을 수행할 수 있도록 최상위 슈트가 다수의 슈트를 포함하고, 하나의 슈트는 다수의 테스트케이스를 포함하도록 설계하였다. 슈트를 선택하여 부분 테스트가 가능하도록 하였고, 최상위 슈트를 선택하면 전체 기능에 대한 테스트가 자동으로 실행된다.

3) 테스트케이스 형상 관리

테스트케이스가 지속적으로 변경될 수 있기 때문에 소스코드 형상관리처럼 테스트케이스에 대한 형상관리도 중요하다. 이러한 이유로 프레임워크 설계 시 형상관리 도구와 연동하여 테스트케이스를 관리하고 테스트 변경 이력을 프레임워크의 시각화 도구에서 조회할 수 있도록 설계하였다.

DBMS의 경우에는 고객사별, 버전별, 운영체제별로 다양한 형상이 존재하기 때문에 이에 대한 테스트케이스도 제품의 형상에 맞게 관리되어야 한다. 따라서 SW개발기업에서

```
#################################################
#                  기능 테스트 케이스                  #
#################################################

table.suite
index.suite
view.suite
transaction.suite
optimization.suite
replication.suite  --init=create_repl.py  --fini=destroy_repl.py
dblink.suite  --init=create_dblink.py  --fini=destroy_dblink.py
utility.suite
interface.suite
```

[그림 2] 테스트케이스 의존성 설정 예

사용하는 형상관리도구(Subversion 등)와 연동함으로써 DBMS 제품 형상별 테스트케이스 관리 및 추적이 용이하도록 하였다.

4) 테스트 오라클

테스트 오라클이란 기대 출력과 실제 출력을 비교해주는 메커니즘으로, 방대한 테스트를 수행하는 DBMS 테스트를 위해서는 반드시 구현되어야 할 요소이다. 이를 위해 각 테스트케이스별 기대 출력을 인벤토리로 구축하였고 테스트 실행 결과와 기대 출력을 자동으로 비교할 수 있도록 메커니즘을 설계하였다.

4.2.2 자동화 관리 엔진

1) 테스트 슈트 로딩

테스트 슈트 단위로 테스트가 실행되기 때문에 자동화 관리 엔진이 테스트 슈트를 로딩하고, 해당 슈트에 포함된 테스트케이스를 목록화 하도록 설계하였다.

2) 테스트케이스 간 의존성 관리

다수의 테스트케이스는 순차적으로 실행된다. 두 개 이상의 테스트케이스가 상호 의존 관계에 있을 경우, 선행 테스트케이스를 우선 실행하고 후행 테스트케이스를 나중에 실행하도록 자동화 관리 엔진을 설계하였다. 일부 테스트케이스는 테스트케이스의 사전 조건으로도 활용될 수 있기 때문에 이와 같은 테스트케이스는 의존성 관계에서 선행 테스트케이스로 정의하였다. 예를 들어, 사전조건으로 활용될 수 있는 테스트케이스는 [그림 3]과 같이 이중화 설정, DB Link 설정 등이 될 수 있다.

3) 실행 중 예외 상황 처리

테스트를 수행하다보면 DBMS의 결함으로 DBMS가 비정상 종료 혹은 중지될 수 있다. DBMS가 비정상 종료 및 중지될 경우, 자동화 프레임워크의 테스트도 함께 중단되기 때문에 DBMS의 재구동이 필요하다. DBMS의 비정상 종료 및 중지

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

상황 발생 시, [표 2]과 같이 테스트 정보를 백업하고 데이터베이스를 초기화한 후 서버를 재구동하도록 자동화 관리 모듈을 설계하였다. DBMS가 재구동이 완료되면 중단된 시점부터 테스트를 다시 수행하도록 하였다.

[표 2] 백업 파일 대상

백업 대상	설 명
Core 덤프 파일	메모리에 저장된 프로세스 상태 정보를 백업하여 비정상 종료 시 메모리의 저장 데이터를 분석하기 위해 활용
Stack Trace	프로세스 실행과정을 추적할 수 있도록 콜 스택 정보를 백업하여 비정상 종지를 발생시킨 함수, 쿼리를 분석하기 위해 활용
DBMS 로그 파일	DBMS 운영 로그, 에러 로그 등을 백업하여 예외 상황 발생 시 DBMS의 실행 기능을 파악하기 위해 활용

4.2.3 테스트 실행 엔진

1) 재사용이 가능한 공통 모듈 제공

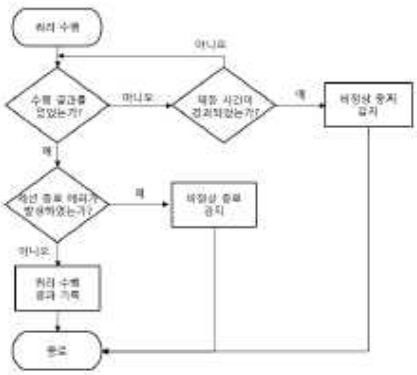
DBMS 테스트 수행 시 테이블 추가, 데이터 삽입, 서버 재구동 등의 작업이 반복적으로 발생한다. 반복적으로 실행되는 작업을 재사용할 수 있도록 공통 작업을 모듈화하였다. 테스트 실행 엔진을 구성하는 공통 모듈은 다음과 같다.

① DBMS 서버 제어 패키지: 데이터베이스 생성, 구동, 중지, 재시작 등 데이터베이스 및 서버에 대한 제어 기능을 제공하는 패키지
② 동시성 지원 패키지: 다수 사용자의 동시 요청을 시뮬레이트하기 위한 패키지
③ 운영환경 제어 패키지: 이중화, DB Link 등 DBMS 운영환경 설정을 위한 패키지
④ 자원 모니터링 패키지: 자원(CPU, 메모리, 디스크) 사용량을 모니터링하는 패키지
⑤ 벤치마크 패키지: TPC-H, TPC-DS 표준에서 제시하는 쿼리 실행용 패키지

2) 이기종 DBMS 테스트 지원

특정 DBMS 뿐만 아니라 다양한 DBMS에 대한 품질 검증을 지원하기 위해 실행 엔진 설계 시, 아래 사항을 고려하였다.

① 다양한 DBMS로 접속이 가능하도록 JDBC 활용
② DBMS별 상이한 쿼리문 지원을 위해 주요 DBMS에 대한 쿼리문 생성 함수 제공



[그림 4] 예외 상황 감지 메커니즘

③ DBMS별 상이한 명령어 지원을 위해 주요 DBMS에 대한 명령어 래퍼 지원

3) 실행 결과 검증 및 예외 상황 감지

테스트케이스가 실행되면 레파지토리의 기대 출력과 테스트 실행 결과를 비교하여 각 테스트별 성공/실패를 결정한다. 또한, 실패된 테스트케이스에 대한 사유를 확인할 수 있도록 로그 조회 기능을 제공한다. 기대 출력과 실제 출력을 테스트로 비교할 경우 날짜 포맷 차이, 공백 등으로 인해 테스트케이스 실행 결과가 '실패'로 잘못 판단될 가능성이 높기 때문에 정규표현식을 사용하여 이를 처리할 수 있도록 하였다.

테스트케이스 실행 중에 DBMS가 비정상 종료 또는 중지될 수 있기 때문에 [그림 4]와 같이 예외 상황 감지 메커니즘을 설계하였다. 테스트케이스 실행 후 일정시간 이내에 DBMS에 응답이 없을 경우 관리 엔진을 호출하여 DBMS를 재구동 시킨다.

4) 메모리 누수 검출을 위한 Valgrind와 연동

DBMS는 장시간 중단없이 구동되어야 하는 시스템 소프트웨어로 작은 메모리 누수도 시스템 장애로 이어질 수 있는 위험이 있다. 그렇기 때문에 DBMS 실행 과정에서 발생되는 메모리 누수를 사전에 검출할 경우 신뢰성 향상에 큰 도움이 된다. 메모리 누수 검출을 위해 많이 사용되는 Valgrind 프로그램[10]을 연동하였고 메모리 누수 상황을 쉽게 파악할 수 있도록 메모리 누수 검출 결과를 테스트케이스와 맵핑하였다.

4.2.4 시각화 도구

테스트 실행 결과를 시각적으로 분석할 수 있도록 시각화 도구를 제공하였고 다음의 정보를 시각적으로 제공한다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

① 테스트 요약 정보: 테스트 수행 정보(수행 환경, 제품 형상, 테스트케이스 형상 등)와 실행 결과에 대한 통계 정보(성공/실패 수, 실패된 테스트케이스 리스트)
② 원인 분석을 위한 테스트 로그: DBMS 로그 및 실패 테스트케이스에 대한 상세 로그
③ 원인 분석 결과 및 공유: 실패 테스트케이스에 대한 원인 분석 결과
④ 메모리 누수 검사 결과: 메모리 누수 상세 내용 및 관련 테스트케이스

5. 결론 및 향후 계획

새로운 비즈니스 기회와 부가가치를 창출하기 위해 빅데이터 활용 중요성이 높아지고 있다. 이에 따라 빅데이터 분석용 DBMS에 대한 시장의 니즈는 매년 증가하고 있고, 다양한 빅데이터 분석용 DBMS 제품들이 출시되고 있다. 이러한 변화에 발 맞추어 국내 기업들도 빅데이터 분석을 위한 DBMS를 출시하고 있으나, DBMS의 복잡한 구조, 다양한 운영 환경, 방대한 기능 등으로 인해 신뢰성 높은 DBMS 개발에 어려움을 겪고 있다.

DBMS와 같이 복잡하고 고신뢰성이 요구되는 시스템 소프트웨어의 경우 테스팅 인력만으로는 체계적인 테스트 수행에 한계가 있다. 이러한 이유로 빅데이터 분석용 DBMS 개발기업에서 TTA에 테스트 자동화 프레임워크 개발을 제안하였다. 본 논문에서 제안한 자동화 프레임워크는 실 사용자의 요구사항을 분석하여 설계하였고, 다양한 DBMS를 대상으로도 활용될 수 있도록 변경성과 확장성을 고려하여 설계하였다. 또한, DBMS에 새로운 기능이 추가될 때마다 테스트케이스도 함께 추가되어야 하므로 테스트케이스 추가, 수정 및 삭제가 용이하도록 프레임워크를 설계하였다. 그리고 테스트 결과를 시각적으로 제공함으로써 DBMS의 결함 발생현황을 직관적으로 확인할 수 있도록 하였고 테스트 케이스의 오류 발생 시 원인 분석이 가능하도록 상세한 로그를 제공하였다.

현재, 본 논문에서 제안한 소프트웨어 아키텍처를 기반으로 테스트 자동화 프레임워크를 개발하고 있다. 프레임워크 개발이 완료되면 DBMS개발기업에 이를 설치하여 빅데이터 분석용 DBMS를 대상으로 시범적용을 수행할 예정이다.

* 이 논문은 2018년도 정부(과학기술정보통신부)의 재원으로 정보통신기술진흥센터의 지원을 받아 수행된 연구임(No. 2018001193001200 2, 빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 개발)

참 고 문 헌

[1] 2017 데이터산업 백서. 한국데이터진흥원
[2] The MySQL Test Framework. Available: dev.mysql.com/doc/dev/mysql-server/8.0.1/PAGE_MYSQL_TEST_RUN.html
[3] Testgres. Available: postgrespro.github.io/testgres
[4] Oracle. "Real Application Testing". Available: www.oracle.com/technetwork/database/options/real-application-testing-option-2355173.html
[5] 도로시 그레이엄, 마크 퓨스터. 소프트웨어 테스트 자동화. 어용구, 김윤명, 황영석, 성찬혁 옮김. 에이콘, 2013.
[6] ISO/IEC 25010:2011 Systems and software engineering -- Systems and software Quality Requirements and Evaluation (SQuaRE) -- System and software quality models
[7] TPC-C. Available: www.tpc.org/tpcc
[8] TPC-H. Available: www.tpc.org/tpch
[9] TPC-DS. Available: www.tpc.org/tpcds
[10] Valgrind Home. Available: www.valgrind.org

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

【부록 3】 공인시험성적서

시 험 성 적 서
(TEST REPORT)

ktl 한국산업기술시험원 Korea Testing Laboratory

성적서 번호 : 19-034887-01-1
Report No.

페이지 (1)/(총 10)
Page of Pages

ktl KOREA TESTING LABORATORY 한국산업기술시험원

1. 의 뢰 자 (Client)

기 관 명 (Name) : 한국정보통신기술협회

주 소 (Address) : 경기도 성남시 분당구 분당로 47 (서현동)

의뢰일자 (Date of Receipt) : 2019. 05. 28.

2. 시험성적서의 용도 (Use of Report) : 제출용

3. 시험대상품목/물질/시료명 (Test Sample)

제 품 명 (Description) : 빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 v1.0

제작회사 (Manufacturer) : 한국정보통신기술협회

모 델 명 (Model Name) : -

제조번호 (Serial Number) : -

기 타 (Remark) : -

4. 시험기간 (Date of Test) : 2019년 06월 11일 - 2019년 07월 23일

5. 시험장소 (Location of Test) :

□ KTL 고정시험실

■ 현장시험 (주소 : 경기도 성남시 분당구 분당로 47)

6. 시험규격/방법 (Test Standard/Method) : 신청자(의뢰자) 제시 방법(기준)

7. 시험결과 (Test Results) : 시험완료

비고(Note) : 1. 이 성적서는 의뢰자가 제출한 시료에 한하며, 법적 및 기타분쟁의 근거 등으로의 사용을 금합니다.
2. 이 성적서는 원본만 유효하며, 임의로 재 가공된 사본 및 전자인쇄본 등은 유효하지 않습니다.
("원본"이란 KTL에서 정해진 검사에 따라 보관절차를 포함시켜 배정하는 모든 성적서를 의미합니다.)
3. 아래의 2D바코드를 스캔하여 성적서의 원본내용 확인이 가능하며, KTL 보관 원본과의 동일성은 고객전용홈페이지(customer.ktl.re.kr)의 "성적서 원본확인"창에서 비교가능 합니다.

확 인 Affirmation	작성자(Tested by)		기술책임자(Technical Manager)	
	성 명(Name): 최현문	(Signature)	성 명(Name): 이승환	(Signature)

2019. 07. 23.

한국산업기술시험원장

서울특별시 구로구 디지털로26길 87 (구로동) (87, Digital-ro 26-gil, Guro-gu, Seoul, KOREA) Tel.02-860-1564 Fax. 02-860-1569

FP204-01-06

※위 마크는 추후 전자확인용 애프 프로그램에서 원본대조시 사용되는 2D고드입니다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

【부록 4】 특허 출원 번호 통지서

관 인 생 략

출 원 번 호 통 지 서

출 원 일 자	2019.02.28
특 기 사 항	심사청구(무) 공개신청(무) 참조번호(DPE192112)
출 원 번 호	10-2019-0024162 (접수번호 1-1-2019-0214309-70)
출원인 명칭	한국정보통신기술협회(2-2001-051521-3)
대 리 인 성 명	특허법인 정안(9-2012-100022-4)
발 명 자 성 명	길현정 이재효 일우섭 송옥수
발 명 의 명 칭	빅데이터 분석용 DBMS를 위한 테스트 자동화 프레임워크 및 테스트 자동화 방법

특 허 청 장

<< 안내 >>

1. 귀하의 출원은 위와 같이 정상적으로 접수되었으며, 이후의 심사 진행상황은 출원번호를 통해 확인하실 수 있습니다.

2. 출원에 따른 수수료는 접수일로부터 다음날까지 동봉된 납입영수증에 성명, 납부자번호 등을 기재하여 가까운 우체국 또는 은행에 납부하여야 합니다.
※ 납부자번호 : 0131(기관코드) + 접수번호

3. 귀하의 주소, 연락처 등의 변경사항이 있을 경우, 즉시 [특허고객번호 정보변경(경정), 정정신고서]를 제출하여야 출원 이후의 각종 통지서를 정상적으로 받을 수 있습니다.
※ 특허로(patent.go.kr) 접속 > 민원서식다운로드 > 특허법 시행규칙 별지 제5호 서식

4. 특허(실용신안등록)출원은 명세서 또는 도면의 보정이 필요한 경우, 등록결정 이전 또는 의견서 제출기간 이내에 출원서에 최초로 첨부된 명세서 또는 도면에 기재된 사항의 범위 안에서 보정할 수 있습니다.

5. 외국으로 출원하고자 하는 경우 PCT 제도(특허·실용신안)나 마드리드 제도(상표)를 이용할 수 있습니다. 국내출원일을 외국에서 인정받고자 하는 경우에는 국내출원일로부터 일정한 기간 내에 외국에 출원하여야 우선권을 인정받을 수 있습니다.
※ 제도 안내 : http://www.kipo.go.kr-특허마당-PCT/마드리드
※ 우선권 인정기간 : 특허·실용신안은 12개월, 상표·디자인은 6개월 이내
※ 미국특허상표청의 선출원을 기초로 우리나라에 우선권주장출원 시, 선출원이 미공개상태이면, 우선일로부터 16개월 이내에 미국특허상표청에 [전자적교환허가서(PTO/SB/39)]를 제출하거나 우리나라에 우선권 증명서류를 제출하여야 합니다.

6. 본 출원사실을 외부에 표시하고자 하는 경우에는 아래와 같이 하여야 하며, 이를 위반할 경우 관련법령에 따라 처벌을 받을 수 있습니다.
※ 특허출원 10-2010-0000000, 상표등록출원 40-2010-0000000

7. 종업원이 직무수행과정에서 개발한 발명을 사용자(기업)가 명확하게 승계하지 않은 경우, 특허법 제62조에 따라 심사단계에서 특허거절결정되거나 특허법 제133조에 따라 등록이후에 특허무효사유가 될 수 있습니다.

8.기타 심사 절차에 관한 사항은 동봉된 안내서를 참조하시기 바랍니다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

【부록 5】 프로그램 등록증



제 C-2019-021009 호

저작권 등록증

1. 저작물의 제호(명칭)	빅데이터 솔루션의 고신뢰성 품질 확보를 위한 품질 검증 자동화 프레임워크 v1.0
2. 저작물의 종류	컴퓨터프로그램저작물>시스템프로그램
3. 저작자 성명(법인명)	한국정보통신기술협회 경기도 성남시 분당구 분당로
4. 생년월일(법인등록번호)	110171-0008282
5. 창작연월일	2019년07월24일
6. 공표연월일	-
7. 등록연월일	2019년07월30일
8. 등록사항	저작자 : 한국정보통신기술협회, 창작 : 2019.07.24

『저작권법』 제53조에 따라 위와 같이 등록되었음을 증명합니다.

2019년 07월 31일

한국저작권위원회

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

주 의

1. 이 보고서는 과학기술정보통신부에서 시행한 정보통신·방송 연구개발사업의 연구보고서입니다.

2. 이 보고서의 내용을 발표하는 때에는 반드시 과학기술정보통신부에서 시행한 정보통신·방송연구개발사업의 연구결과임을 밝혀야 합니다.

3. 국가과학기술 기밀유지에 필요한 내용은 대외적으로 발표 또는 공개하여서는 아니됩니다.